



Escuela Técnica Superior de
Ingeniería Informática

TRABAJO FIN DE GRADO

Sevillaneando

Repositorio del proyecto

github.com/cyncasjua/TFG-cyncasjua

Realizado por

Cynthia Castaño Juan

Para la obtención del título de

Grado en Ingeniería Informática - Ingeniería del Software

Dirigido por

José Antonio Parejo Maestre

En el departamento de

Lenguajes y Sistemas Informáticos

Convocatoria de junio/julio, curso 2025/26

*A todas las personas que me enseñaron que llegar hasta aquí era posible,
y a las que estuvieron a mi lado para que mereciera la pena.*

Agradecimientos

Hay personas sin las cuales este trabajo no habría sido posible, y hay otras sin las cuales el camino no habría tenido el mismo sentido. Este Trabajo de Fin de Grado es el resultado de muchas horas de esfuerzo, dudas, aprendizaje y constancia, pero también de todas las personas que, de una forma u otra, me han acompañado durante esta etapa. A todas ellas van dedicadas estas líneas.

A mi padre, por estar siempre presente. Por cada vez que me preguntó cómo iba el TFG, aunque muchas veces no entendiera del todo la respuesta, y aun así me escuchara con toda la atención del mundo. Gracias por sostenerme en los momentos en los que el cansancio pesaba más que las ganas, por confiar en mí incluso cuando yo dudaba, y por recordarme, con gestos sencillos, que era capaz de llegar hasta aquí. Mucho de lo que soy, de mi manera de esforzarme y de no rendirme, viene de ti.

A **Iana Miranda** y **Macarena Pereira**, por haberse convertido en amigas de verdad durante esta etapa. Gracias por las tardes de estudio, por las conversaciones que empezaban con apuntes y terminaban arreglando la vida, por las risas entre exámenes y por estar también cuando las cosas se hacían cuesta arriba. Haber compartido la carrera con vosotras ha hecho que todo fuese más llevadero, más bonito y mucho más mío. Me llevo no solo los recuerdos de estos años, sino la suerte de haber encontrado a dos amigas que han sabido acompañarme con cariño, paciencia y alegría. Ojalá sigáis estando cerca en todo lo que venga después.

A **Irene González**, **Lucía Torres** e **Ignacio Mora**, por sus aportaciones, su tiempo y su disposición a ayudar a mejorar este proyecto. Gracias por tomaros el interés que os tomasteis, por las ideas que aportasteis y por hacerme ver aspectos que yo sola no habría visto. Vuestra ayuda me permitió mirar el trabajo con más perspectiva y mejorarlo con más criterio.

A **Santia**, por haber compartido conmigo tantos momentos de esta etapa y por haber estado cerca también durante este último tramo. Gracias por las dudas compartidas, por los ánimos en los días complicados y por esas pequeñas victorias que se disfrutaban más cuando alguien entiende lo que ha costado conseguirlas.

Y, de manera muy especial, a mi tutor, **José Antonio Parejo**, por su implicación, su cercanía y su paciencia durante todo el desarrollo de este trabajo. Gracias por aportar siempre ideas que ayudaban a mejorar el proyecto, por responder con rapidez cada duda y por guiarme con claridad cuando necesitaba ordenar el camino. Sus orientaciones han sido fundamentales para que este TFG avanzara con sentido y para convertir una idea inicial en un proyecto del que me siento orgullosa. Este trabajo también es suyo.

Resumen

¿Cuántas veces nos hemos enterado de un concierto, una exposición o una fiesta de barrio cuando ya había pasado? A veces por un cartel que vimos tarde, a veces porque alguien lo subió a sus redes sociales y no preguntamos a tiempo, a veces simplemente porque nadie nos lo contó ni vimos su publicación. Sevilla tiene una vida cultural y festiva extraordinaria, pero su oferta está dispersa: cada evento vive en su propia red social, en su propia web o en el boca a boca de quien ya lo conocía.

Sevillaneando es una aplicación móvil que reúne toda esa oferta en un solo lugar, pero con una diferencia fundamental respecto a un simple directorio de eventos: está pensada para que la propia comunidad la construya y la haga crecer. Cualquier usuario puede publicar un evento: un taller, una quedada, una ruta por el centro... y compartirlo para todo el mundo, o bien crearlo como evento privado con acceso restringido para que solo llegue a quien él decida. Los asistentes pueden chatear entre ellos antes, durante y después del evento, subir fotos en el momento, dejar reseñas y descubrir a otras personas con sus mismos intereses. La aplicación también sugiere planes personalizados según lo que a cada usuario le gusta y cerca suya.

El resultado es una plataforma donde descubrir Sevilla deja de depender de la suerte o de a quién conozcas.

Sevillaneando ha sido desarrollada como Trabajo de Fin de Grado para la obtención del título de Grado en Ingeniería del Software en la Escuela Técnica Superior de Ingeniería Informática de la Universidad de Sevilla. El sistema se ha construido desde cero a lo largo de seis sprints de desarrollo iterativo, gestionados mediante una adaptación de la metodología Scrum, con un esfuerzo real registrado cercano a 320 horas de trabajo. La solución comprende una aplicación móvil multiplataforma (React Native + Expo), un servidor backend (NestJS + PostgreSQL/PostGIS) y múltiples integraciones con servicios externos (Firebase, Cloudinary, OpenStreetMap, Nominatim, Eventbrite, Ticketmaster y Google Gemini AI). La versión final contiene 165 ficheros TypeScript/TSX de producción y 29.281 líneas de código en src, que ascienden a 30.550 líneas si se incluyen las pruebas unitarias del backend. El backlog del proyecto ha dado lugar a más de 30 historias de usuario priorizadas, cubriendo desde la gestión básica de eventos hasta el motor de recomendaciones y el sistema de rutas personalizadas.

Palabras clave: aplicación móvil, eventos culturales, Sevilla, ocio, comunidad, tiempo real, geolocalización, recomendaciones personalizadas, descubrimientos.

Abstract

How many times have we found out about a concert, an exhibition, or a neighbourhood street party after it had already happened? Sometimes because we saw a flyer too late, sometimes because someone posted it on social media and we didn't ask in time, sometimes simply because nobody told us or we missed their post. Seville has an extraordinary cultural and festive life, but its offering is scattered: every event lives on its own social network, its own website, or in the word-of-mouth of those who already knew about it.

Sevillaneando is a mobile application that brings all of that together in one place, but with a fundamental difference from a simple event directory: it is designed so that the community itself builds it and makes it grow. Any user can publish an event — a workshop, a meetup, a walk through the city centre — and share it with everyone, or create it as a private event with restricted access so that only the people they choose can see it. Attendees can chat with each other before, during and after the event, upload photos in the moment, leave reviews, and discover other people who share their interests. The application also suggests personalised plans based on what each user enjoys and what is happening nearby.

The result is a platform where discovering Seville no longer depends on luck or on who you happen to know.

Sevillaneando has been developed as a Final Degree Project for the Bachelor's degree in Software Engineering at the Escuela Técnica Superior de Ingeniería Informática of the University of Seville. The system was built from scratch over six iterative development sprints managed using an adapted Scrum methodology, with an actual recorded effort close to 320 working hours. The solution comprises a cross-platform mobile application (React Native + Expo), a backend server (NestJS + PostgreSQL/PostGIS) and multiple integrations with external services (Firebase, Cloudinary, OpenStreetMap, Nominatim, Eventbrite, Ticketmaster and Google Gemini AI). The final version contains 165 production TypeScript/TSX files and 29,281 lines of code under src, rising to 30,550 lines when backend unit tests are included. The project backlog produced more than 30 prioritised user stories, covering everything from basic event management to the recommendation engine and the personalised route system.

Keywords: mobile application, cultural events, Seville, leisure, community, real time, geolocation, personalised recommendations, discovery.

Índice general

1	Introducción	1
1.1.	Motivación	1
1.2.	Soluciones existentes	2
1.3.	Estructura del documento	3
2	Estudio previo	4
2.1.	Introducción	4
2.2.	Objetivos	4
2.2.1.	Objetivos del proyecto	4
2.2.2.	Objetivos del sistema	5
2.3.	Metodología	6
2.3.1.	Metodología de gestión del proyecto	6
2.3.2.	Metodología de gestión del desarrollo	6
2.3.3.	Metodología de gestión del código fuente	7
2.3.4.	Metodología de seguimiento del tiempo	7
2.4.	Planificación	8
2.4.1.	Gestión de riesgos	8
2.4.2.	Desviaciones respecto a la planificación inicial	10
2.4.3.	Estructura de Desglose del Trabajo inicial	13
2.4.4.	Planificación por sprints	16
2.4.5.	Diagrama de planificación	16
2.5.	Presupuesto	17
2.5.1.	Recursos humanos	17
2.5.2.	Distribución del esfuerzo	18
2.5.3.	Infraestructura y costes indirectos	19
2.5.4.	Coste total del proyecto	19
2.5.5.	Presupuesto estimado	19
2.5.6.	Coste real del proyecto	20
3	Análisis del problema	21
3.1.	Introducción	21
3.2.	Actores del sistema	21
3.2.1.	Visitante	21
3.2.2.	Ciudadano	21
3.2.3.	Organizador	22
3.2.4.	Moderador de contenido	22
3.2.5.	Administrador de la plataforma	22
3.3.	Diagrama de casos de uso	23
3.4.	Requisitos de información	25
3.5.	Requisitos funcionales	28
3.5.1.	Matriz de trazabilidad de requisitos a objetivos	33
3.6.	Diseño de mockups	34

3.6.1.	Vista general de mockups	35
3.7.	Requisitos no funcionales	36
3.8.	Reglas de negocio	37
3.8.1.	Gestión de usuarios y roles	37
3.8.2.	Gestión de eventos	38
3.8.3.	Eventos privados	38
3.8.4.	Participación en eventos	38
3.8.5.	Comunicación y contenido	38
3.8.6.	Valoraciones y reseñas	39
3.8.7.	Recomendaciones y geolocalización	39
3.8.8.	Scraping	39
3.8.9.	Restricciones de integridad de datos	39
3.8.10.	Notificaciones	39
3.9.	Cumplimiento del Reglamento General de Protección de Datos (RGPD)	40
3.9.1.	Datos personales tratados	40
3.9.2.	Finalidad y base legal del tratamiento	40
3.9.3.	Terceros encargados del tratamiento	41
3.9.4.	Derechos de los usuarios	41
3.9.5.	Medidas técnicas y organizativas	41
3.9.6.	Consentimiento explícito en el registro	42
3.10.	Conclusiones	42
4	Diseño de la solución	43
4.1.	Introducción	43
4.2.	Arquitectura del sistema	43
4.2.1.	Aplicación móvil (frontend)	44
4.2.2.	Servidor backend	45
4.2.3.	Base de datos	47
4.2.4.	Servicios externos	47
4.3.	Modelo de datos	50
4.3.1.	Diagrama entidad-relación	51
4.4.	Diseño de la API REST	53
4.4.1.	Versionado	53
4.4.2.	Autenticación y control de acceso	53
4.4.3.	Endpoints principales	54
4.5.	Flujo de interacción del usuario	56
4.6.	Diseño del sistema de moderación	57
4.7.	Flujo de interacción del administrador	59
4.8.	Diseño de la interfaz de usuario	61
4.8.1.	Principios de diseño	61
4.8.2.	Grafo de navegación	61
4.8.3.	Pantallas del sistema	62
4.9.	Diseño del sistema de comunicación en tiempo real	64
4.9.1.	Ciclo de vida de la conexión y los listeners	64
4.10.	Diseño del sistema de recomendaciones	65
4.11.	Patrones de diseño aplicados	66
4.12.	Decisiones de diseño relevantes	67

4.12.1. Roles, permisos y responsabilidades	68
4.12.2. Moderación y ciclo de vida de eventos	69
4.12.3. Comunicación, tiempo real y notificaciones	70
4.12.4. Visibilidad, recurrencia y datos geográficos	72
4.12.5. Automatización y fuentes externas	73
4.12.6. Estado del frontend y experiencia de usuario	76
4.12.7. Seguridad, administración y reglas de acceso	77
4.12.8. Elección de tecnologías e infraestructura	78
4.12.9. Funcionalidades sociales y reglas de dominio	82
4.13. Conclusiones	84
5 Implementación	86
5.1. Introducción	86
5.2. Estructura del proyecto	86
5.3. Implementación del backend	87
5.3.1. Organización modular	87
5.3.2. Persistencia geográfica con TypeORM	87
5.3.3. Scraping de eventos externos	88
5.3.4. Tareas programadas	89
5.3.5. API de eventos	90
5.3.6. Validación de datos	90
5.3.7. Actualización de eventos	90
5.3.8. Control de roles	91
5.3.9. Limitación de velocidad	91
5.3.10. Autenticación en endpoints protegidos	91
5.3.11. Eventos recurrentes	92
5.4. Implementación del frontend	92
5.4.1. Estructura de pantallas	92
5.4.2. Estado global y proveedores	93
5.4.3. Comunicación con la API	94
5.4.4. Gestión de mapas	95
5.4.5. Gestión de imágenes	95
5.4.6. Utilidades de presentación y normalización	95
5.5. Implementación de funcionalidades principales	96
5.5.1. Eventos, administración y moderación	96
5.5.2. Recomendaciones y rutas	98
5.5.3. Usuarios y funciones sociales	99
5.5.4. Comunicación y notificaciones	100
5.6. Carga inicial y mantenimiento de datos	102
5.7. Despliegue y entorno de desarrollo	102
5.8. Conclusiones	102
6 Pruebas	103
6.1. Introducción	103
6.2. Metodología y tipos de pruebas	103
6.2.1. Pruebas unitarias del backend	103
6.2.2. Resumen de pruebas unitarias	107

6.2.3.	Cobertura de código	107
6.2.4.	Pruebas de aceptación	108
6.3.	Entorno de pruebas	108
6.3.1.	Incidencias de compatibilidad con Android	109
6.4.	Plan de pruebas de aceptación	109
6.4.1.	Pruebas de autenticación y perfil	109
6.4.2.	Pruebas de consulta, búsqueda y detalle de eventos	110
6.4.3.	Pruebas de geolocalización y mapa	111
6.4.4.	Pruebas de interacción social: chat, mensajería y valoraciones	111
6.4.5.	Pruebas de recomendaciones y rutas	111
6.4.6.	Pruebas de eventos privados y gestión de eventos creados	112
6.4.7.	Pruebas de administración y moderación	113
6.4.8.	Pruebas de notificaciones	114
6.4.9.	Pruebas complementarias incorporadas en la revisión final	114
6.5.	Resultados de las pruebas de aceptación	115
6.6.	Conclusiones	118
7	Conclusiones	119
7.1.	Introducción	119
7.2.	Objetivos alcanzados	119
7.2.1.	Objetivos del proyecto	119
7.2.2.	Objetivos del sistema	120
7.3.	Resultados obtenidos	121
7.4.	Lecciones aprendidas	122
7.5.	Desviaciones respecto a la planificación	123
7.6.	Consideraciones de seguridad y privacidad	124
7.6.1.	Limitaciones respecto al RGPD	124
7.7.	Trabajo futuro	125
7.7.1.	Mejoras sobre funcionalidades existentes	125
7.7.2.	Nuevas funcionalidades	126
7.8.	Uso de herramientas de inteligencia artificial	128
7.8.1.	Herramientas utilizadas	128
7.8.2.	Alcance del uso	129
7.8.3.	Verificación	129
7.9.	Conclusión final	129
	Bibliografía	130
	Anexos	134
A	Glosario de términos	135
B	Manual de Desarrollo	137
B.1.	Objetivo	137
B.2.	Arquitectura del sistema	137
B.2.1.	Visión general	137
B.2.2.	Cliente móvil	138

B.2.3.	Servidor backend	138
B.2.4.	Base de datos	139
B.2.5.	Servicios externos	139
B.3.	Tecnologías y versiones	139
B.4.	Prerrequisitos	140
B.5.	Obtención del código fuente	140
B.6.	Instalación de dependencias	140
B.7.	Variables de entorno	141
B.7.1.	Backend (apps/backend/.env.development)	141
B.7.2.	Frontend (apps/frontend/.env.development)	142
B.8.	Ejecución local	142
B.8.1.	1. Base de datos	142
B.8.2.	2. Backend	143
B.8.3.	3. Frontend	143
B.9.	Estructura del proyecto	143
B.9.1.	Módulos del backend	144
B.9.2.	Pantallas del frontend	145
B.10.	Ecosistema de desarrollo	145
B.10.1.	Herramientas de implementación	145
B.10.2.	Herramientas de documentación	146
B.10.3.	Herramientas de gestión del proyecto	146
B.11.	Política de gestión del código fuente	146
B.11.1.	Estrategia de ramas	146
B.11.2.	Convención de commits	146
B.12.	Integración continua	147
B.13.	Resolución de problemas comunes	147
B.13.1.	Base de datos	147
B.13.2.	Backend	147
B.13.3.	Frontend	148
B.14.	Referencias	148
C	Manual de despliegue	149
C.1.	Objetivo	149
C.2.	Arquitectura de despliegue	149
C.2.1.	Componentes principales	150
C.3.	Versiones	150
C.4.	Despliegue de la base de datos	150
C.4.1.	Opción recomendada: servicio gestionado	151
C.4.2.	Configuración de SSL	151
C.4.3.	Creación del esquema de base de datos en producción	151
C.4.4.	Activación de Row-Level Security (RLS)	152
C.4.5.	Alternativa: despliegue con Docker	152
C.5.	Despliegue del backend	153
C.5.1.	Construcción local previa	153
C.5.2.	Variables de entorno de producción	153
C.5.3.	Despliegue en Render	154
C.6.	Distribución de la aplicación móvil	155

C.6.1. Variables de entorno del frontend	155
C.6.2. Opción 1: APK para Android	156
C.6.3. Opción 2: Expo Go con túnel	156
C.7. Validación posterior al despliegue	157
C.7.1. Verificación del backend	157
C.7.2. Funcionalidades a validar	157
C.8. Checklist de despliegue	158
C.9. Actualización del sistema	158
C.10. Rollback y recuperación	158
C.10.1. Rollback del backend en Render	159
C.10.2. Rollback de la aplicación móvil	159
C.10.3. Respaldo de la base de datos	159
C.11. Referencias	159
D Manual de usuario	160
D.1. Introducción	160
D.2. Definición de entidades y funcionalidades	161
D.3. Requisitos técnicos	164
D.4. Guía de uso de la aplicación	164
D.4.1. Registro e inicio de sesión (para todos los roles)	165
D.4.2. Gestión del perfil (para todos los roles)	167
D.4.3. Mapa de eventos (para todos los roles)	170
D.4.4. Página principal y filtros (para todos los roles)	172
D.4.5. Detalle de evento	174
D.4.6. Listados de eventos	177
D.4.7. Gestión de administrador (rol de administrador)	180
D.4.8. Recomendaciones y rutas recomendadas (para todos los roles)	184
D.4.9. Rutas creadas por usuarios (para todos los roles)	186
D.4.10. Creación de eventos (rol de usuario)	188
D.4.11. Eventos privados (todos los roles)	191
D.4.12. Gestión de eventos privados (todos los roles)	193
D.4.13. Mis eventos (rol de usuario)	194
D.4.14. Moderación (rol de moderador)	196
D.4.15. Notificaciones (todos los roles)	198
D.4.16. Amistad y mensajería privada (todos los roles)	200
D.4.17. Eventos finalizados	204
D.4.18. Guía de uso integrada	205
D.5. Grafo de navegabilidad	207
D.6. Errores frecuentes	207
E Reporte de horas	209
F Informes de avance y cambios	215
F.1. Informe de avance 1: definición del alcance y análisis inicial	215
F.1.1. Trabajo realizado	215
F.1.2. Cambios y decisiones	215
F.2. Informe de avance 2: requisitos, planificación y diseño conceptual	215

F.2.1.	Trabajo realizado	215
F.2.2.	Cambios y decisiones	216
F.3.	Informe de avance 3: arquitectura e implementación base	216
F.3.1.	Trabajo realizado	216
F.3.2.	Cambios y decisiones	216
F.4.	Informe de avance 4: funcionalidades sociales, mapas y recomendaciones	216
F.4.1.	Trabajo realizado	216
F.4.2.	Cambios y decisiones	217
F.5.	Informe de avance 5: scraping, moderación y despliegue	217
F.5.1.	Trabajo realizado	217
F.5.2.	Cambios y decisiones	217
F.6.	Informe de avance 6: pruebas, validación y cierre	217
F.6.1.	Trabajo realizado	217
F.6.2.	Cambios y decisiones	218
F.7.	Resumen de cambios relevantes	218
G	Declaración de autoría	219
H	Usuarios piloto	220
H.1.	Introducción	220
H.2.	Perfil de los usuarios	220
H.3.	Metodología de recogida de feedback	220
H.4.	Resultados e incidencias detectadas	221
H.4.1.	Problemas de compatibilidad con Android	221
H.4.2.	Sugerencias de mejora visual y de interacción	222
H.4.3.	Ideas de nuevas funcionalidades	222
H.5.	Valoración general de los usuarios	223
H.6.	Conclusiones del proceso piloto	223
I	Actas de reunión	225
Acta de Reunión 1		225
Acta de Reunión 2		228
Acta de Reunión 3		231
Acta de Reunión 4		235
Acta de Reunión 5		239
Acta de Reunión 6		242
Acta de Reunión 7		245
Acta de Reunión 8		249

Índice de figuras

2.1. Estructura de Desglose del Trabajo del proyecto	15
2.2. Diagrama de Gantt – planificación temporal por fases	17
3.1. Diagrama de actores del sistema Sevillaneando	23
3.2. Diagrama de casos de uso (parte 1): acceso, perfil, descubrimiento y gestión de eventos	24
3.3. Diagrama de casos de uso (parte 2): funcionalidades sociales, moderación y administración	25
3.4. Diagrama del modelo conceptual del sistema	28
3.5. Acceso, listado de eventos y detalle de evento	35
3.6. Chat de evento, creación de eventos y mapa	35
3.7. Perfil, menú lateral y rutas recomendadas	36
3.8. Notificaciones, moderación y administración	36
4.1. Vista lógica del sistema Sevillaneando	44
4.2. Flujo de una petición en el frontend desde la acción del usuario hasta la actualización de la pantalla	45
4.3. Cadena de responsabilidades en el backend para una petición autenticada	46
4.4. Flujo de autenticación: desde el inicio de sesión hasta la verificación del token en el backend	47
4.5. Conexión entre tecnologías del sistema en tiempo de ejecución	49
4.6. Vista de despliegue del sistema Sevillaneando	49
4.7. Diagrama entidad-relación del sistema	52
4.8. Diagrama de secuencias del flujo del usuario (registro, consulta, creación y asistencia)	57
4.9. Diagrama de secuencias del flujo del moderador	58
4.10. Diagrama de secuencias del flujo del administrador	60
4.11. Grafo de navegación de la aplicación organizado por rol de acceso	62
4.12. Flujo de comunicación en tiempo real mediante Socket.IO: conexión, autenticación y entrega de mensajes	65
4.13. Entradas y proceso del motor de recomendaciones	66
5.1. Flujo del sistema de scraping: orquestación, fuentes externas y persistencia	89
B.1. Visión general de la arquitectura del sistema	137
C.1. Arquitectura general de despliegue del sistema	149
D.1. Acceso a la aplicación	166
D.2. Acceso y opciones del perfil	168
D.3. Gestión del perfil	169
D.4. Mapa de eventos	171

D.5. Filtros de búsqueda	173
D.6. Pantalla del evento	175
D.7. Interacción dentro del evento	176
D.8. Listado de eventos guardados	178
D.9. Listado de eventos apuntados	178
D.10. Calendario de eventos	179
D.11. Funcionalidades de administración	181
D.12. Panel de estadísticas y scraping	182
D.13. Gestión de categorías	183
D.14. Recomendaciones y rutas	185
D.15. Rutas creadas por usuarios	187
D.16. Creación de eventos	189
D.17. Creación de eventos	190
D.18. Acceso a evento privado	191
D.19. Consulta de evento privado	192
D.20. Gestión de eventos privados	193
D.21. Eventos creados por el usuario	195
D.22. Moderación de eventos	197
D.23. Notificaciones	199
D.24. Amistad y perfil de usuario	201
D.25. Mensajes privados	202
D.26. Consulta de mensajes privados	203
D.27. Evento finalizado	204
D.28. Guía de uso integrada en la aplicación	206
D.29. Grafo de navegabilidad de la aplicación	207

Índice de tablas

1.1. Comparativa de soluciones existentes	3
2.1. Tabla de riesgos identificados	9
2.2. Resumen cuantitativo de desviaciones por área	12
2.4. Estimación del coste base de personal	17
2.5. Costes sociales y coste total de personal	18
2.6. Distribución estimada del esfuerzo y coste base por fase	18
2.7. Infraestructura y costes indirectos	19
2.8. Coste total del proyecto	19
2.9. Partidas adicionales del presupuesto	20
2.10. Presupuesto final del proyecto	20
2.11. Comparativa estimación hipotética vs. coste real	20
3.1. Requisitos de información del sistema	26
3.2. Requisitos funcionales del sistema	29
3.3. Matriz de trazabilidad: requisitos funcionales y objetivos del sistema	34
3.4. Requisitos no funcionales del sistema	37
4.1. Entidades del modelo de datos	50
4.2. Endpoints de la API REST (fuente: OpenAPI 3.0)	54
4.3. Pantallas del sistema y roles que pueden acceder	62
4.4. Patrones de diseño aplicados en el sistema	67
6.4. Resumen de pruebas unitarias del backend	107
6.5. Métricas de cobertura de código del backend	108
A.1. Glosario de términos del proyecto	135
H.1. Completitud de flujos principales por los usuarios piloto (n = 11) . .	221

Índice de extractos de código

5.1. Definición del campo geográfico y de precios en la entidad Event con TypeORM	87
5.2. Construcción conceptual de la ubicación geográfica de un evento . .	90
5.3. Guard de autenticación Firebase en NestJS	91
5.4. Función de parseo de coordenadas geográficas en el frontend	94
5.5. Pesos del sistema de recomendaciones (valores por defecto)	98

1. Introducción

Sevilla es una ciudad que no para. Cada semana hay talleres, conciertos en salas pequeñas, exposiciones temporales, rutas guiadas, mercadillos, festividades de barrio y decenas de actividades que pasan desapercibidas para la mayoría de sus habitantes. No es que no existan: es que nadie las reúne en un mismo sitio. La información cultural de la ciudad llega fragmentada, a través de redes sociales, carteles físicos, webs de organismos públicos desconectadas entre sí o, simplemente, por recomendación de un conocido.

Esta situación no es exclusiva de Sevilla. Según el Informe de Hábitos y Prácticas Culturales del Ministerio de Cultura y Deporte [1], el 43 % de la población española declara no asistir a actividades culturales por falta de información sobre la oferta disponible en su entorno. En ciudades con una programación cultural densa y descentralizada como Sevilla, con más de 700.000 habitantes y una agenda que combina patrimonio histórico, cultura popular y escena emergente, esta fragmentación informativa tiene un impacto directo en la visibilidad de los eventos más pequeños y en la participación ciudadana.

La consecuencia es una brecha real: los eventos grandes tienen visibilidad, pero los pequeños y emergentes, a menudo los más interesantes, mueren sin audiencia. Al mismo tiempo, mucha gente busca planes sin saber qué tiene a diez minutos de casa, y descubre lo que se perdió días después, al verlo por redes sociales o leer un cartel ya caducado.

Más allá del impacto social, existe también una dimensión económica de peso. El ocio y la cultura constituyen un sector de enorme relevancia para la economía española: según datos del INE, los hogares españoles destinan en torno al 9 % de su gasto total al ocio, cultura y espectáculos, lo que supone más de 70.000 millones de euros anuales a nivel nacional [2]. En Sevilla, referente turístico y cultural del sur de Europa, el sector turístico genera más de 3.000 millones de euros al año y emplea a más de 60.000 personas en la provincia, según la Encuesta de Ocupación Hotelera del INE [3]. Tanto los grandes eventos como los pequeños contribuyen a esta actividad económica: los primeros atraen visitantes y generan gasto directo, mientras que los segundos dinamizan el comercio local, la hostelería de proximidad y la escena cultural emergente. Una plataforma que mejore la visibilidad de toda la oferta, no solo la de los eventos consolidados, tiene por tanto un impacto económico real en el tejido comercial y cultural de la ciudad.

1.1. Motivación

La motivación de este proyecto surge de esa experiencia cotidiana: enterarse tarde. Alguien sube una foto de un concierto increíble, preguntas dónde fue y ya terminó. Un cartel en la calle anuncia una exposición que lleva dos semanas abierta.

Un amigo te habla de una ruta por el Aljarafe que no encontrarías por tu cuenta en ninguna web. La información existe, pero no está donde la gente la busca ni cuando la necesita.

Sevillaneando nace para resolver este problema. Es una aplicación móvil multiplataforma que centraliza la oferta de ocio y cultura de Sevilla en una única plataforma participativa, donde la propia comunidad construye y enriquece el contenido. Cualquier usuario puede publicar un evento, seguir a otras personas con sus mismos intereses, chatear con los asistentes antes y durante el evento, y recibir recomendaciones personalizadas según lo que le gusta y dónde se encuentra.

El enfoque participativo es clave: frente a los directorios estáticos de eventos gestionados por instituciones, *Sevillaneando* apuesta por un modelo donde el contenido lo genera la ciudadanía, con un sistema de moderación que garantiza la calidad y veracidad de lo publicado. Esto permite dar visibilidad tanto a la programación oficial como a iniciativas emergentes que de otro modo no tendrían canal de difusión.

Desde el punto de vista técnico, el proyecto afronta retos relevantes y actuales: comunicación bidireccional en tiempo real, consultas espaciales sobre datos geográficos, extracción automática de datos de fuentes externas mediante scraping, un motor de recomendaciones personalizado y una arquitectura distribuida cliente-servidor con múltiples servicios integrados.

Los objetivos del proyecto y del sistema se desarrollan de forma explícita en el capítulo 2; los requisitos funcionales, no funcionales y de información se detallan en el capítulo 3; y las tecnologías empleadas se describen en los capítulos 4 y 5, donde se justifica la arquitectura y su materialización técnica.

1.2. Soluciones existentes

Existen diversas plataformas que abordan, de forma parcial, el problema de la difusión de eventos. Sin embargo, ninguna de ellas cubre de forma integrada las necesidades que motivaron este proyecto.

Eventbrite [4] es la plataforma de gestión de eventos más extendida a nivel global. Está orientada principalmente a eventos de pago con registro formal, lo que deja fuera la mayoría de actividades informales, gratuitas o de pequeña escala. No ofrece funcionalidades sociales ni de comunidad entre asistentes.

Facebook Events [5] permite crear y descubrir eventos dentro del ecosistema de Facebook. Su principal limitación es la dependencia de una red social generalista, lo que provoca que la información quede enterrada entre otro tipo de contenido. Además, su uso entre jóvenes ha decaído significativamente en los últimos años [6].

Meetup [7] está orientado a grupos de interés y reuniones recurrentes, especialmente en el ámbito tecnológico y profesional. No está pensado para eventos culturales puntuales ni para el ocio urbano en general.

Plan de ocio del Ayuntamiento de Sevilla [8] y webs institucionales similares ofrecen una agenda cultural oficial, pero son unidireccionales (sin interacción), no personalizadas, y no recogen la oferta privada o emergente.

Fever [9] es una aplicación móvil de reciente lanzamiento orientada al descubrimiento de planes y eventos locales. Permite crear y compartir eventos y cuenta con funcionalidades de personalización básicas. Sin embargo, no incorpora un sistema de gestión de rutas entre eventos ni moderación de contenido por parte de usuarios con rol específico, lo que la diferencia de *Sevillaneando* en cuanto a profundidad de las funcionalidades sociales y calidad garantizada del contenido.

La tabla 1.1 resume las principales diferencias entre estas soluciones y *Sevillaneando*:

Tabla 1.1: Comparativa de soluciones existentes

Característica	Eventbrite	Facebook Events	Meetup	Web Ayto.	Fever	Sevillaneando
Enfoque local (Sevilla)	X	X	X	✓	X	✓
Eventos emergentes/gratuitos	X	✓	X	X	✓	✓
Contenido generado por usuarios	X	✓	✓	X	✓	✓
Moderación de contenido	X	Parcial	X	✓	X	✓
Chat / comunidad en el evento	X	X	X	X	X	✓
Mensajería directa entre usuarios	X	✓	X	X	X	✓
Eventos privados con acceso restringido	X	✓	X	X	X	✓
Mapa interactivo de eventos cercanos	X	✓	X	X	✓	✓
Rutas turísticas personalizadas	X	X	X	X	X	✓
Recomendaciones personalizadas	X	Parcial	✓	X	Parcial	✓
Moderación automática de imágenes (IA)	X	✓	X	X	X	X

Como se observa, *Sevillaneando* es la única solución que combina el enfoque local, la participación ciudadana, las funcionalidades sociales en tiempo real, la moderación de contenido por parte de usuarios con rol específico y la personalización mediante recomendaciones en una misma plataforma orientada específicamente al ocio y la cultura en Sevilla. Aunque Fever ofrece funcionalidades de descubrimiento local y personalización básica, no dispone del sistema de gestión de rutas entre eventos ni de la moderación por roles que caracterizan a *Sevillaneando*.

1.3. Estructura del documento

El presente documento está organizado de la siguiente manera. El capítulo 2 describe el estudio previo, incluyendo los objetivos del proyecto, la metodología de desarrollo y la planificación temporal. El capítulo 3 recoge el análisis del problema y los requisitos del sistema. El capítulo 4 presenta el diseño de la solución. El capítulo 5 detalla la implementación de los módulos principales. El capítulo 6 describe las pruebas realizadas. El capítulo 7 recoge las conclusiones y líneas de trabajo futuro. A continuación se incluyen los anexos: el glosario de términos, el manual de desarrollo (B), el manual de despliegue (C), el manual de usuario (D), el reporte de horas, los informes de avance y cambios, la validación con usuarios piloto, las actas de reunión y la declaración de autoría.

2. Estudio previo

2.1. Introducción

En este capítulo se presenta el estudio previo realizado para el desarrollo del sistema **Sevillaneando**. Este análisis tiene como finalidad definir el contexto del proyecto, establecer los objetivos principales, seleccionar la metodología de trabajo más adecuada junto la planificación de las distintas fases del proyecto.

El estudio previo constituye una base fundamental para el resto del proyecto, ya que permite organizar el trabajo de manera estructurada, analizar las tecnologías más apropiadas y estimar los recursos necesarios. Asimismo, facilita la toma de decisiones durante el desarrollo y asegura la coherencia entre los requisitos iniciales y la solución final implementada.

2.2. Objetivos

2.2.1. Objetivos del proyecto

Los objetivos del proyecto son aquellos que persigue la estudiante como futura ingeniera de Software en el marco del Trabajo de Fin de Grado:

- **OP-01.** Aplicar en un contexto real los conocimientos adquiridos a lo largo del Grado en Ingeniería del Software, especialmente en las áreas de diseño de software, arquitecturas de sistemas, ingeniería de requisitos y gestión de proyectos.
- **OP-02.** Desarrollar la capacidad de planificación, seguimiento y autogestión en un proyecto de desarrollo unipersonal, adoptando un proceso iterativo e incremental basado en metodologías ágiles.
- **OP-03.** Adquirir experiencia práctica en tecnologías modernas de desarrollo web y móvil (React Native, NestJS, PostgreSQL/PostGIS, Firebase) que complementen la formación académica con competencias técnicas de industria.
- **OP-04.** Adentrarse por primera vez en el desarrollo de aplicaciones móviles, un ámbito no cubierto durante el Grado, afrontando sus particularidades: ciclo de vida de la aplicación, adaptación a múltiples plataformas (iOS y Android) y compilación y distribución mediante Expo.
- **OP-05.** Consolidar el uso de TypeScript en un proyecto complejo de extremo a extremo, profundizando en el tipado estático y en las buenas prácticas del

lenguaje más allá del contacto puntual previo, aplicándolo tanto en el frontend (React Native) como en el backend (NestJS).

- **OP-06.** Enfrentar y resolver retos técnicos no triviales: comunicación bidireccional en tiempo real, consultas geoespaciales, scraping automatizado y diseño de un motor de recomendaciones.
- **OP-07.** Producir documentación técnica de calidad (memoria, manuales de usuario, de desarrollo y de despliegue) como parte de las entregas propias de un proyecto profesional.

2.2.2. Objetivos del sistema

El objetivo principal del sistema es el desarrollo de una aplicación móvil orientada al descubrimiento de eventos y actividades en Sevilla, con especial énfasis en dar visibilidad a eventos emergentes o poco conocidos.

A partir de este objetivo general, se definen los siguientes objetivos específicos del sistema:

- **OBJ-01.** Diseñar e implementar un sistema que permita listar eventos con información detallada (descripción, ubicación, imágenes, precio, etc.) y filtrado de eventos.
- **OBJ-02.** Permitir a los usuarios crear nuevos eventos, diferenciando entre eventos públicos y privados, e incorporando un sistema de moderación para garantizar la fiabilidad de los eventos públicos.
- **OBJ-03.** Permitir a los usuarios editar los eventos creados por sí mismos.
- **OBJ-04.** Integrar funcionalidades de geolocalización para mostrar eventos cercanos y permitir su visualización en un mapa interactivo para un acceso más cómodo.
- **OBJ-05.** Promover la interacción social entre usuarios mediante chat en eventos, mensajería privada, módulo de amistad basado en seguimiento entre usuarios y opiniones en las valoraciones.
- **OBJ-06.** Implementar un sistema de recomendaciones personalizado basado en la actividad del usuario (guardar un evento, apuntarse a un evento, compartir un evento, ver un evento, valorar un evento, etc.), sus intereses y su ubicación.
- **OBJ-07.** Desarrollar un sistema de rutas recomendadas de eventos que permita agrupar actividades según gustos y criterios de proximidad y horario.
- **OBJ-08.** Permitir a los usuarios crear rutas de eventos que sean visibles para el resto y que puedan valorarlas.
- **OBJ-09.** Incluir notificaciones para informar al usuario sobre eventos cercanos, recordatorios, seguidores nuevos y aprobación o rechazo de los eventos.

2.3. Metodología

En este apartado se describe la metodología seguida durante el proyecto, organizada en tres ámbitos de actuación con distintos niveles de abstracción: la gestión general del proyecto, la gestión del proceso de desarrollo y la gestión del código fuente. Adicionalmente, se recoge la metodología de seguimiento del tiempo.

2.3.1. Metodología de gestión del proyecto

El desarrollo del proyecto se ha llevado a cabo mediante una adaptación de la metodología ágil Scrum [10], ajustada al contexto de un Trabajo de Fin de Grado.

Al tratarse de un equipo unipersonal, la desarrolladora ha asumido simultáneamente los roles de *Product Owner*, *Scrum Master* y desarrolladora. Esto ha permitido una toma de decisiones ágil, aunque también ha requerido una mayor disciplina en la autogestión del tiempo y las prioridades.

Dado que el proyecto ha sido desarrollado por un único miembro, no se han realizado reuniones diarias (*daily stand-ups*). En su lugar, la comunicación se ha mantenido de forma continua mediante tutorías periódicas con el tutor académico. Estas sesiones han combinado la revisión del trabajo realizado con la planificación del siguiente sprint, permitiendo mantener una evolución iterativa del sistema y adaptar el alcance en función del tiempo disponible y de las prioridades del proyecto.

Para la gestión del backlog y el seguimiento de tareas se han utilizado **GitHub Projects** como tablero Kanban, complementado con un documento Word donde se ha mantenido el backlog detallado junto con las actas de reunión de cada tutoría, en las que se recogen las tareas completadas en el sprint anterior y los objetivos definidos para el siguiente.

2.3.2. Metodología de gestión del desarrollo

La estimación de tareas se ha realizado mediante la técnica de tallas de camiseta (*T-shirt sizing*) [11], que asigna un tamaño relativo a cada historia: XS (extra pequeña), S (pequeña), M (mediana), L (grande) y XL (extra grande). La talla se define en función de la complejidad, el riesgo y la cantidad de trabajo, comparando cada historia con otras ya conocidas. La descripción de las funcionalidades como historias de usuario sigue el formato propuesto por Mike Cohn [12]. Esta estimación permite priorizar el backlog combinando valor (prioridad) y esfuerzo (talla), facilitando la planificación ágil y la selección de historias por sprint.

Adicionalmente, se ha configurado un pipeline de integración y entrega continua (CI/CD) mediante **GitHub Actions** [13], que automatiza los despliegues

y la ejecución del linter y las comprobaciones de formato en cada push, contribuyendo a mantener la calidad del código a lo largo del desarrollo.

2.3.3. Metodología de gestión del código fuente

Para la gestión del código fuente se ha utilizado un sistema de control de versiones Git, permitiendo mantener el historial de cambios y asegurando el orden y la claridad en el desarrollo.

Se ha seguido una política de ramas sencilla basada en *main*, para la versión estable; *develop*, para la integración de nuevas funcionalidades antes de fusionarlas en *main*; y ramas de funcionalidad (*feature/**), derivadas de *develop*.

Además, se ha aplicado el estándar *Conventional Commits* [14] para mantener un historial de cambios claro y estructurado, siguiendo la siguiente convención:

`<tipo>[alcance opcional]: <descripción corta>`

Los tipos utilizados han sido:

- feat: Nueva funcionalidad.
- fix: Corrección de errores.
- docs: Cambios en la documentación.
- style: Ajustes de formato sin afectar a la funcionalidad.
- refactor: Cambios en el código sin alterar la funcionalidad.
- test: Añadir o modificar pruebas.
- chore: Tareas de mantenimiento o configuración.

También se ha seguido un versionado semántico de tres niveles (X.Y.Z) para la política de versiones del software:

- X: Versión mayor, ante cambios incompatibles o rediseños importantes.
- Y: Versión menor, al añadir nuevas funcionalidades.
- Z: Versión de parche, para corregir errores o problemas menores.

2.3.4. Metodología de seguimiento del tiempo

El seguimiento del tiempo dedicado al proyecto se ha realizado con la herramienta **Clockify** [15], lo que ha permitido monitorizar el esfuerzo real invertido en las distintas actividades del trabajo y contrastarlo posteriormente con las fases de planificación. El informe detallado de horas se recoge en el Anexo E.

2.4. Planificación

La planificación del proyecto se ha estructurado en varias fases, siguiendo un enfoque incremental:

- **Fase 0 – Determinación del alcance:** definición del alcance funcional del sistema, identificación del problema a resolver, análisis del contexto y de las soluciones existentes, y establecimiento de las restricciones y criterios de éxito del proyecto.
- **Fase 1 – Análisis y planificación:** definición de requisitos y alcance, elaboración del backlog inicial, planificación temporal y de recursos (cronograma inicial) y diseño del modelo conceptual del sistema.
- **Fase 2 – Diseño:** diseño de la arquitectura técnica, de la interfaz de usuario, de la base de datos, de la API REST y planificación de la gestión de usuarios y permisos.
- **Fase 3 – Desarrollo:** implementación del sistema, incluyendo el frontend móvil (React Native + Expo) y el backend (NestJS), así como funcionalidades como eventos, chat, mapa, notificaciones y recomendaciones.
- **Fase 4 – Pruebas y validación:** ejecución de pruebas funcionales, de integración y de usabilidad, así como la corrección de errores.
- **Fase 5 – Ampliaciones y extras:** funcionalidades adicionales condicionadas al tiempo disponible, como la generación de carteles con IA, la gestión de calendarios y el filtrado de contenido inapropiado (NSFW).
- **Fase 6 – Despliegue y cierre del proyecto:** despliegue del sistema, elaboración de la memoria técnica y los manuales, creación de los materiales de defensa del proyecto y cierre formal del trabajo.

2.4.1. Gestión de riesgos

Durante la fase de planificación se identificaron los principales riesgos que podían afectar al desarrollo del proyecto. Para cada uno se estimó su probabilidad de ocurrencia e impacto, y se definió un plan de mitigación o contingencia.

La probabilidad se clasifica en: **Baja** (poco probable), **Media** (posible) o **Alta** (probable). El impacto en: **Bajo** (afecta a tareas menores), **Medio** (retraso en una fase) o **Alto** (compromete el alcance o el plazo).

Tabla 2.1: Tabla de riesgos identificados

Riesgo	Prob.	Impacto	Mitigación	Resultado real
Complejidad técnica superior a la estimada en módulos nuevos (scraping, recomendaciones, rutas)	Alta	Alto	Planificar un margen de holgura por fase; repriorizar el backlog si se supera el tiempo estimado.	Ocurrió en scraping (30 h reales frente a 10–15 h estimadas) y en rutas (18 h frente a 12–14 h).
Incompatibilidades entre plataformas móviles (iOS / Android)	Media	Medio	Probar en ambas plataformas desde las primeras iteraciones; usar componentes multiplataforma siempre que sea posible.	Ocurrió en la fase de pruebas: el mapa no se visualizaba en Android y aparecieron descuadres de layout, resueltos en el Sprint 6.
Cambios externos en herramientas o servicios de terceros	Baja	Medio	Documentar las versiones utilizadas; mantener alternativas identificadas para los servicios críticos.	Ocurrió con Expo Go (12/05/2026): restricción de carga de actualizaciones EAS a miembros del proyecto, resuelta adoptando -tunnel.
Incertidumbre en la integración del mapa interactivo	Media	Medio	Prototipar la visualización del mapa en fases tempranas; comprobar la carga de teselas, marcadores y coordenadas en dispositivos reales.	No se produjeron problemas relevantes en la integración del mapa ni en la visualización de eventos geolocalizados.
Dificultad para integrar servicios externos (APIs, autenticación, almacenamiento)	Media	Medio	Prototipar las integraciones clave en fases tempranas; consultar documentación oficial y ejemplos de uso.	No se produjeron problemas relevantes en la integración de Firebase, Cloudinary y Ticketmaster.

Riesgo	Prob.	Impacto	Mitigación	Resultado real
Falta de tiempo para implementar todas las funcionalidades planificadas	Alta	Medio	Mantener una priorización estricta del backlog; clasificar las funcionalidades en esenciales y opcionales desde el inicio.	La Fase 5 (ampliaciones) no pudo completarse íntegramente: la generación de carteles con IA y la moderación NSFW quedaron fuera del alcance.

2.4.2. Desviaciones respecto a la planificación inicial

A lo largo del proyecto se han producido algunas desviaciones respecto a la planificación inicial, motivadas principalmente por la aparición de nuevas necesidades técnicas y por el ajuste del alcance en función del tiempo disponible.

- **Incorporación de flujos de CI/CD y ajustes de calidad** (*Fase 6 – Sprint 6*): inicialmente no estaba prevista la configuración de pipelines de integración continua. Sin embargo, durante el Sprint 6 se consideró conveniente automatizar las comprobaciones de calidad del código mediante GitHub Actions. La configuración del pipeline, la guía de estilo, la integración del linter, las comprobaciones de formato y los ajustes derivados del despliegue supusieron una dedicación aproximada de 2 horas, no contempladas en la estimación original de cierre del proyecto.
- **Cambio externo en la distribución mediante Expo Go** (*Fase 6 – Sprint 6*): la estrategia inicial de demostración contemplaba compartir la aplicación móvil mediante un código QR generado a partir de una actualización publicada con EAS Update, lo que permitía probar la aplicación sin pasar por tiendas oficiales. Sin embargo, el 12 de mayo de 2026 Expo modificó el comportamiento de Expo Go, restringiendo la carga de actualizaciones EAS a usuarios propietarios del proyecto o miembros de la organización correspondiente [16]. Esta circunstancia, externa al proyecto y no prevista en la planificación inicial, obligó a cambiar la estrategia de distribución para la defensa y las pruebas abiertas, adoptando `npx expo start -tunnel` como alternativa sin coste para iOS y Android. La desviación no afectó al alcance funcional del sistema, pero sí supuso un desfase aproximado de 2 horas, dedicado a revisar el manual de despliegue, actualizar la descripción del entorno de pruebas y ajustar la documentación de usuarios piloto.
- **No implementación de funcionalidades opcionales** (*Fase 5*): la Fase 5 del proyecto (ampliaciones y extras) estaba condicionada a la disponibilidad de tiempo una vez completadas las fases principales. Finalmente, la generación de carteles con IA y la moderación automática de imágenes mediante NSFWJS no pudieron implementarse, ya que el desarrollo de las funcionalidades core del sistema (especialmente rutas de eventos, sistema de

recomendaciones y scraping) requirió más esfuerzo del estimado inicialmente.

- **Mayor complejidad en el scraping** (*Fase 3 – Sprint 4*): la integración del motor de scraping implicó una complejidad técnica superior a la prevista. Se estimaban entre 10 y 15 horas de desarrollo, pero el trabajo real se situó finalmente en torno a 30 horas. La desviación se debió principalmente a la gestión de errores 429 (límite de peticiones) y la variabilidad en la estructura de las fuentes externas, lo que requirió iteraciones adicionales de corrección y ajuste. A ello se sumaron tres dificultades no previstas: en primer lugar, la clasificación de los eventos extraídos en las categorías existentes del sistema resultó especialmente laboriosa, ya que los títulos y descripciones de las fuentes externas no siguen ninguna taxonomía estándar; fue necesario diseñar y refinar iterativamente una lista de palabras clave por categoría hasta lograr una asignación razonablemente precisa. En segundo lugar, la extracción de direcciones reales presentó dificultades por la falta de uniformidad en cómo cada fuente representa la ubicación, mezclando nombres de recintos, barrios, referencias parciales y formatos libres. En tercer lugar, el tratamiento de fechas supuso un reto significativo: muchas fuentes presentan sus calendarios de eventos en formatos no lineales (calendarios visuales, rangos de fechas, sesiones múltiples), lo que obligó a reajustar en varias ocasiones la lógica de extracción y normalización de fechas para cubrir los distintos casos encontrados.
- **Impacto de las fechas del scraping en el sistema de rutas** (*Fase 3 – Sprint 5*): los problemas de normalización de fechas descritos en el punto anterior tuvieron también un efecto directo sobre el sistema de rutas recomendadas, cuyo desarrollo se concentró en el Sprint 5. Las rutas agrupan eventos por fecha como criterio de organización, por lo que los eventos del scraping que presentaban un calendario de fechas múltiples o indeterminadas no podían tratarse del mismo modo que los eventos con fecha única. Como solución, se decidió que cuando un evento procedente del scraping viene asociado a un calendario de sesiones, la fecha deja de usarse como factor de puntuación en la construcción de la ruta; en su lugar, se indica que existen varias fechas disponibles y el criterio de ordenación pasa a ser principalmente la distancia entre eventos. Fue necesario adaptar la lógica del algoritmo para detectar este caso y aplicar la estrategia adecuada. Esta incidencia supuso una desviación de aproximadamente 4–5 horas adicionales sobre las 12–14 horas estimadas para el algoritmo de rutas.
- **Mayor dedicación a la integración de mapas** (*Fase 3 – Sprint 4*): la integración del mapa interactivo no generó incidencias graves, pero sí requirió más ajustes de los previstos para coordinar la visualización de marcadores, la carga de teselas, el tratamiento de coordenadas y la experiencia en dispositivos reales. El seguimiento de horas recoge una dedicación aproximada de 14 horas, frente a una previsión inicial de unas 10 horas.
- **Problemas de compatibilidad con Android** (*Fases 5 y 6 – Sprints 5 y 6*): durante la fase de pruebas con usuarios piloto, llevada a cabo en el Sprint 5,

se detectaron incidencias significativas en dispositivos Android que no habían aparecido durante el desarrollo, realizado íntegramente sobre iOS. Los principales problemas fueron la no visualización del mapa (OpenStreetMap) en Android y varios descuadres en la disposición de elementos de pantalla. La corrección de estas incidencias se extendió hasta el Sprint 6 y supuso una dedicación adicional de aproximadamente 2 horas respecto a lo previsto. Esta desviación pone de manifiesto la importancia de incorporar pruebas sobre ambas plataformas desde las primeras fases del desarrollo.

Resumen cuantitativo de desviaciones. La tabla 2.2 recoge las principales partidas donde se produjo una desviación significativa entre la estimación inicial y el esfuerzo real registrado en Clockify:

Tabla 2.2: Resumen cuantitativo de desviaciones por área

Área	Horas estimadas	Horas reales (aprox.)	Desviación	Causa principal
Motor de scraping	10–15 h	~30 h	+15–20 h	Variabilidad de fuentes, gestión 429, clasificación
Sistema de rutas y recomendaciones	12–14 h	~18–19 h	+5–7 h	Impacto de fechas múltiples del scraping
Integración de mapas	10 h	~14 h	+4 h	Ajustes de marcadores, teselas y coordenadas en dispositivos reales
Compatibilidad Android	2 h	~4 h	+2 h	Desarrollo solo sobre iOS hasta la fase de pruebas
CI/CD y calidad de código	0 h (no previsto)	~2 h	+2 h	Decisión durante el Sprint 6 de automatizar checks
Cambio en Expo Go	0 h (no previsto)	~2 h	+2 h	Cambio externo en la distribución de actualizaciones EAS
Funcionalidades opcionales (Fase 5)	15 h	0 h	–15 h	Reasignadas al desarrollo core

En conjunto, el tiempo adicional dedicado a resolver incidencias y ajustes no previstos fue mayor que el tiempo ahorrado en las funcionalidades que quedaron fuera del alcance. Por ello, el esfuerzo real del proyecto se situó en torno a 320 horas, por encima de las 300 horas presupuestadas. Esta diferencia no se considera un fracaso de planificación, sino un resultado habitual en proyectos de desarrollo de software donde el alcance técnico real solo se revela durante la implementación.

Lecciones extraídas de las desviaciones. Las desviaciones anteriores permiten identificar mejoras concretas aplicables a futuros proyectos similares:

- **Sobreestimar el esfuerzo de integración con fuentes externas.** El scraping de datos no estructurados es inherentemente impredecible. En futuros proyectos, se reservará al menos el doble de la estimación inicial para cualquier tarea de ingesta de datos externos, dado que la variabilidad de formatos y los límites de tasa son difíciles de anticipar sin haberlos explorado previamente.
- **Probar en todas las plataformas objetivo desde el primer sprint.** Posponer las pruebas en Android hasta la fase final hizo que los problemas de compatibilidad fueran más costosos de corregir. Incorporar una comprobación básica en Android al final de cada sprint de desarrollo de interfaz habría evitado esa acumulación.
- **Analizar dependencias entre módulos antes de cerrar las estimaciones.** La relación entre el scraping y el sistema de rutas no estaba prevista; los problemas de normalización de fechas de uno repercutieron en la lógica del otro. Un análisis de dependencias en la fase de diseño habría permitido secuenciar el desarrollo en el orden adecuado.
- **Incluir una partida explícita para calidad e infraestructura técnica.** La configuración de CI/CD y los ajustes de linter forman parte integral de un proyecto profesional. Contemplarlos desde el inicio evita que aparezcan como sobrecarga imprevista al final del proyecto.

En términos generales, el proyecto ha completado el alcance funcional definido como prioritario, aunque para ello ha sido necesario dedicar más horas de las previstas inicialmente y ajustar algunas funcionalidades opcionales. El seguimiento realizado con Clockify se recoge en el Anexo [E](#) y permite justificar el esfuerzo real registrado, mientras que el presupuesto mantiene una estimación prevista y redondeada por fases, definida en la Sección de Presupuesto.

2.4.3. Estructura de Desglose del Trabajo inicial

La planificación inicial del proyecto se ha definido mediante una Estructura de Desglose del Trabajo (EDT), organizando el proyecto en fases, tareas y subtareas principales.

- **Nivel 1: Proyecto “Sevillaneando”**
 - **Fase 0 – Determinación del alcance**
 - 0.1 Análisis del problema y contexto.
 - 0.2 Estudio de soluciones existentes.
 - 0.3 Definición del alcance funcional y restricciones del proyecto.
 - **Fase 1 – Análisis y planificación**
 - 1.1 Revisión de requisitos y alcance.
 - 1.2 Elaboración del backlog inicial.
 - 1.3 Planificación temporal y de recursos (cronograma inicial).

- 1.4 Creación del modelo conceptual.
- **Fase 2 – Diseño**
 - 2.1 Diseño de arquitectura técnica.
 - 2.2 Diseño de la interfaz (navegación, usabilidad).
 - 2.3 Diseño de la base de datos (PostgreSQL + PostGIS).
 - 2.4 Diseño de la API REST (endpoints principales).
 - 2.5 Planificación de la gestión de usuarios y permisos.
- **Fase 3 – Desarrollo paralelo (Frontend y Backend)**
 - **Fase 3A – Desarrollo del frontend (React Native)**
 - 3A.1 Configuración del entorno móvil (dependencias, entornos, etc.).
 - 3A.2 Implementación del listado y detalle de eventos.
 - 3A.3 Creación y moderación de eventos, categorías y usuarios.
 - 3A.4 Integración del mapa (OpenStreetMap).
 - 3A.5 Integración de búsqueda.
 - 3A.6 Sistema de notificaciones.
 - 3A.7 Integración de chat.
 - 3A.8 Módulo de subida de fotos.
 - 3A.9 Implementación de reseñas y valoraciones.
 - 3A.10 Personalización de recomendaciones.
 - **Fase 3B – Desarrollo del backend (Node.js + NestJS)**
 - 3B.1 Configuración del servidor, base de datos e integración de autenticación.
 - 3B.2 Implementación de endpoints de eventos, usuarios y categorías (CRUD, lógica de eventos privados y sistema de moderación).
 - 3B.3 Sistema de notificaciones.
 - 3B.4 Infraestructura de chat: WebSockets para chat.
 - 3B.5 Integración de imágenes mediante Cloudinary.
 - 3B.6 Motor de scraping: implementación de scrapers para captura periódica de eventos externos.
 - 3B.7 Algoritmo de recomendaciones: lógica de filtrado colaborativo o basado en etiquetas para rutas e intereses.
- **Fase 4 – Pruebas y validación**
 - 4.1 Pruebas unitarias (de backend).

- 4.2 Pruebas de integración (API + app).
- 4.3 Pruebas de usabilidad con usuarios reales.
- 4.4 Corrección de errores y mejoras.

■ Fase 5 – Ampliaciones y extras

Esta fase se ejecutará únicamente si el cronograma de las fases anteriores lo permite.

- 5.1 Generación de carteles con IA.
- 5.2 Gestión de calendarios. *(Implementado.)*
- 5.3 Filtrado de contenido inapropiado (NSFW) en imágenes subidas por los usuarios.

■ Fase 6 – Despliegue y cierre del proyecto

- 6.1 Despliegue de backend y frontend.
- 6.2 Publicación de la app. *(No realizado: la publicación en las tiendas oficiales (Google Play Store y App Store) requiere el pago de una tarifa de desarrollador, lo que queda fuera del alcance de este proyecto.)*
- 6.3 Elaboración de la memoria técnica y manuales de usuario, de despliegue y de desarrollo.
- 6.4 Preparación de los materiales de defensa del proyecto (presentación y vídeo demostrativo).

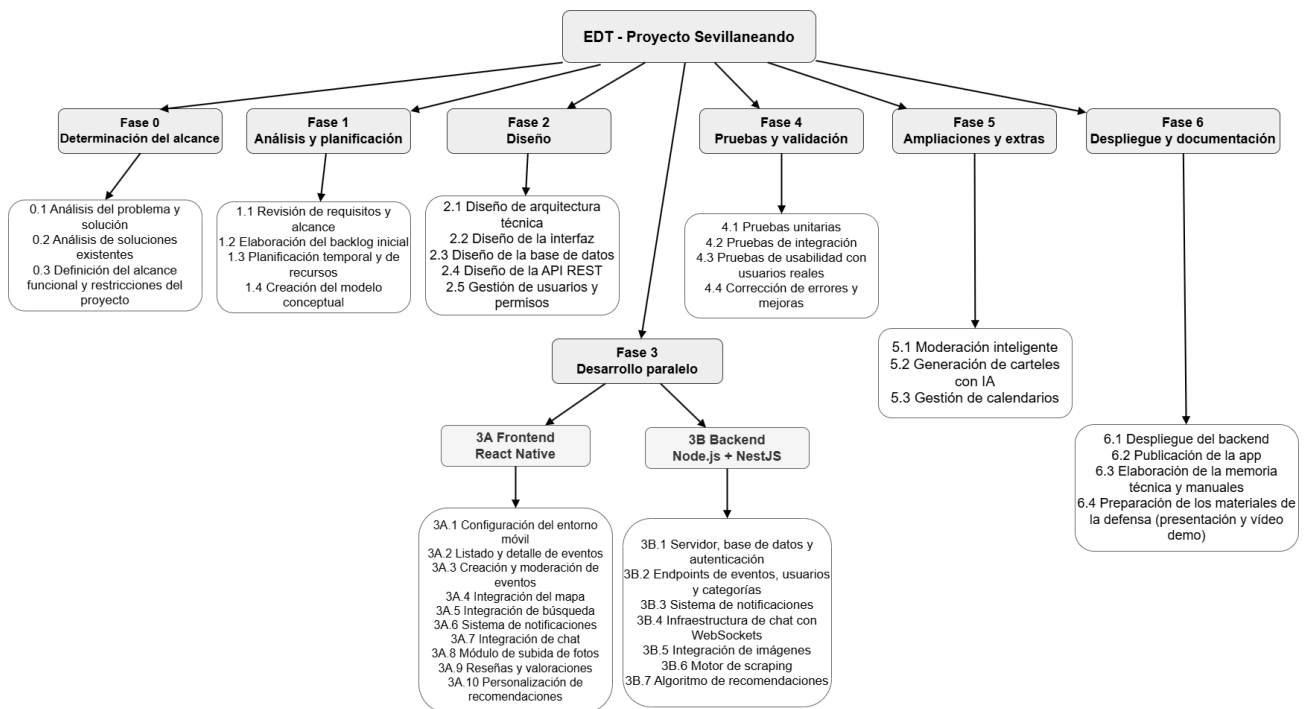


Figura 2.1: Estructura de Desglose del Trabajo del proyecto

2.4.4. Planificación por sprints

El desarrollo del proyecto se ha organizado en sprints de duración variable, adaptados a las necesidades del proyecto y a la disponibilidad temporal. En general, los sprints han tenido una duración aproximada de 4 semanas, ajustándose a 3 semanas en la fase final del proyecto.

Sprint	Duración	Objetivo	Funcionalidades principales
Sprint 1	4 semanas	MVP inicial	Sistema de autenticación, modelo conceptual, visualización básica de mapa
Sprint 2	4 semanas	Gestión de eventos	Creación y moderación de eventos, categorías, visualización en mapa, notificaciones
Sprint 3	4 semanas	Funcionalidades sociales	Chat en tiempo real (WebSockets), subida de imágenes, mejoras en búsqueda y filtrado geográfico
Sprint 4	4 semanas	Ampliación funcional	Scraping de eventos externos, gestión avanzada de imágenes, sistema de precios, optimización de chat
Sprint 5	3 semanas	Funcionalidad principal	Implementación del sistema de rutas de eventos y mejoras finales
Sprint 6	3 semanas	Cierre del proyecto	Refactorización, testing, validación, documentación y elaboración de la memoria

La duración de los sprints no implica una dedicación homogénea en horas. El seguimiento posterior mostró que algunas actividades concentraron una carga de trabajo superior a la prevista, especialmente la redacción de la memoria, el scraping, el sistema de recomendaciones y rutas, la integración de mapas y los ajustes finales de despliegue y calidad.

2.4.5. Diagrama de planificación

A continuación, se presenta un diagrama de Gantt simplificado que refleja la planificación temporal del proyecto por fases, ajustada a la duración real de los sprints:

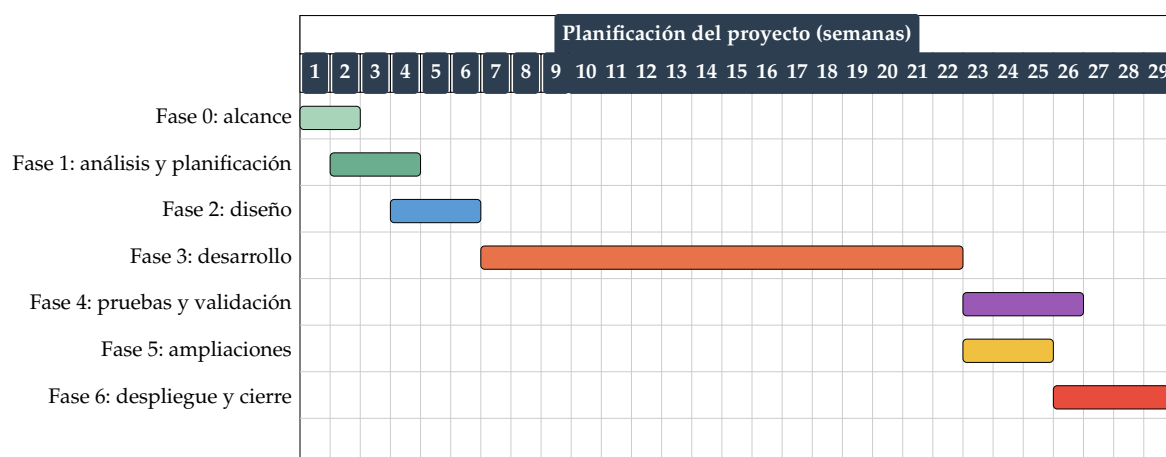


Figura 2.2: Diagrama de Gantt – planificación temporal por fases

2.5. Presupuesto

El proyecto ha sido desarrollado en un contexto académico, por lo que no ha requerido una inversión económica directa significativa. No obstante, a efectos del TFG se elabora una **estimación hipotética** del coste que tendría el proyecto si hubiera sido encargado a una empresa de desarrollo de software, siguiendo la estructura estándar de presupuestación de proyectos informáticos. Los valores que se presentan a continuación son, por tanto, proyecciones orientativas y no reflejan un gasto real incurrido.

2.5.1. Recursos humanos

El desarrollo ha sido llevado a cabo por una única persona asumiendo todos los roles del equipo. El coste unitario se estima a partir del informe de la Junta de Andalucía sobre precios/hora de perfiles profesionales del ámbito informático, tomando como referencia la columna *Media acotada todos*, que es la indicada en el propio informe para determinar importes de licitación [17]. Por la naturaleza del proyecto, se considera el perfil de **Analista programador J2EE / Web (Junior)**:

Tabla 2.4: Estimación del coste base de personal

Concepto	Valor
Coste hora base	28,37 €/h
Horas previstas presupuestadas	300 h
Coste base de personal	$300 \text{ h} \times 28,37 \text{ €/h} = 8.511 \text{ €}$

Sobre el coste base de personal se aplican los **costes sociales** a cargo de la empresa (Seguridad Social, formación, contingencias), estimados en un 30% del coste base:

Tabla 2.5: Costes sociales y coste total de personal

Concepto	Importe
Costes sociales	$8.511 \text{ €} \times 30 \% = 2.553 \text{ €}$
Coste total de personal (base + sociales)	11.064 €

2.5.2. Distribución del esfuerzo

El esfuerzo total previsto para el presupuesto se ha estimado en aproximadamente 300 horas de trabajo. Esta cifra se plantea como una valoración profesional redondeada por fases, no como una transcripción exacta del informe de seguimiento temporal. El registro de Clockify se ha utilizado como referencia para agrupar el esfuerzo por bloques de trabajo y ajustar el peso relativo de cada fase.

La distribución se ha alineado con la planificación y la EDT definidas anteriormente, evitando repetir aquí el detalle de tareas ya descrito. Únicamente se separa la Fase 3 entre frontend y backend por ser el bloque de mayor peso técnico. Además, se incrementa el peso de la Fase 6 porque el informe refleja una dedicación notable a memoria, manuales, anexos, revisión final y preparación de la defensa.

Tabla 2.6: Distribución estimada del esfuerzo y coste base por fase

Fase	Horas estimadas	Coste base
Fase 0 – Determinación del alcance	5 h	142 €
Fase 1 – Análisis y planificación	15 h	426 €
Fase 2 – Diseño del sistema	25 h	709 €
Fase 3A – Desarrollo frontend	75 h	2.128 €
Fase 3B – Desarrollo backend	75 h	2.128 €
Fase 4 – Pruebas y validación	15 h	426 €
Fase 5 – Ampliaciones y extras	15 h	426 €
Fase 6 – Despliegue y cierre	75 h	2.128 €
Total	300 h	8.511 €

Los importes de la tabla anterior se calculan aplicando el coste horario base de 28,37 €/h y redondeando al euro más cercano. La Fase 2 incluye la elaboración de mockups, modelo conceptual y diagramas técnicos; la Fase 5 se mantiene como partida acotada para las ampliaciones previstas inicialmente, aunque finalmente parte de ese tiempo se reasignó a funcionalidades principales del sistema.

2.5.3. Infraestructura y costes indirectos

Para el desarrollo del proyecto han sido necesarios los siguientes recursos materiales e infraestructura, cuyos costes se imputan mediante amortización durante la vida útil estimada:

Tabla 2.7: Infraestructura y costes indirectos

Recurso	Coste total	Vida útil	Duración proyecto	Coste imputable
Ordenador portátil	900 €	4 años	7 meses	131 €
Conexión a internet	40 €/mes	—	7 meses	280 €
Licencias y software	0 €	—	—	0 €
Herramientas en la nube (plan gratuito: Render, Supabase, Cloudinary, Firebase, GitHub)	0 €	—	—	0 €
Total costes indirectos				411 €

2.5.4. Coste total del proyecto

Tabla 2.8: Coste total del proyecto

Concepto	Importe
Coste de personal (base)	8.511 €
Costes sociales (30 %)	2.553 €
Costes indirectos (infraestructura)	411 €
Coste total del proyecto	11.475 €

2.5.5. Presupuesto estimado

El presupuesto hipotético representa la cantidad que se cobraría al cliente si el proyecto hubiera sido encargado externamente. A los costes totales estimados se suman dos partidas adicionales:

Tabla 2.9: Partidas adicionales del presupuesto

Partida	Descripción	Cálculo
Margen de contingencia (12 %)	Cubre desviaciones razonables respecto a la estimación inicial.	$11.475 \text{ €} \times 12 \% = 1.377 \text{ €}$
Beneficio industrial (20 %)	Margen de beneficio habitual en el sector para proyectos de desarrollo de software.	$(11.475 \text{ €} + 1.377 \text{ €}) \times 20 \% = 2.570 \text{ €}$

Tabla 2.10: Presupuesto final del proyecto

Concepto	Importe
Coste total del proyecto	11.475 €
Margen de contingencia (12 %)	1.377 €
Beneficio industrial (20 %)	2.570 €
Presupuesto total (sin IVA)	15.422 €
IVA (21 %)	3.239 €
Presupuesto total (con IVA)	18.661 €

Esta estimación representa el valor de mercado hipotético del trabajo realizado, calculado conforme a tarifas profesionales de referencia del sector. En el contexto del TFG no ha supuesto ningún gasto económico real: las herramientas y servicios utilizados están disponibles en planes gratuitos y el trabajo ha sido realizado por la propia estudiante.

2.5.6. Coste real del proyecto

Aplicando la misma tarifa (28,37 €/h) a las **319,61 horas** reales registradas en Clockify, y añadiendo los mismos costes sociales (30 %) e indirectos (411 €), el coste real del proyecto asciende a **12.198 €**, frente a los 11.475 € estimados. La diferencia de **723 €** (19,6 h adicionales) se explica por las desviaciones recogidas en la Sección 2.4.2: mayor dedicación al scraping, a las pruebas en Android y a la documentación final.

Tabla 2.11: Comparativa estimación hipotética vs. coste real

Concepto	Estimado	Real
Horas de trabajo	300 h	319,61 h
Coste base de personal	8.511 €	9.067 €
Costes sociales (30 %)	2.553 €	2.720 €
Costes indirectos	411 €	411 €
Coste total	11.475 €	12.198 €

3. Análisis del problema

3.1. Introducción

En este capítulo se analiza el dominio del problema y se especifican los requisitos del sistema **Sevillaneando**: actores involucrados, funcionalidad esperada, restricciones de calidad, reglas de negocio y normativa aplicable.

3.2. Actores del sistema

Los actores del sistema son las personas que interactúan con la aplicación. Cada actor representa un perfil de usuario real con motivaciones, necesidades y capacidades de uso diferenciadas. La identificación de actores es el punto de partida del análisis: determina qué historias de usuario existen, qué requisitos funcionales aplican a cada perfil y qué casos de prueba deben cubrirse para validar cada uno.

3.2.1. Visitante

Persona que llega a la aplicación sin cuenta o que aún no ha iniciado sesión. Su interacción con el sistema es mínima: puede registrarse, iniciar sesión o recuperar su contraseña.

Casos de prueba relacionados: [CP-01](#) (registro), [CP-02](#) (login incorrecto), [CP-04](#) (recuperación de contraseña).

3.2.2. Ciudadano

Persona registrada que utiliza la aplicación para descubrir y participar en la vida cultural y de ocio de Sevilla. Es el actor principal y el más frecuente. Sus motivaciones son encontrar planes que se adapten a sus intereses, compartir eventos con otras personas, conocer a gente con gustos similares y mantenerse informado de lo que sucede en la ciudad.

Sus actividades principales incluyen: consultar el listado y el mapa de eventos, filtrar por sus preferencias, apuntarse a actividades, compartir eventos, chatear con otros asistentes, buscar usuarios, consultar perfiles, gestionar el módulo de amistad mediante seguimiento, enviar mensajes privados, guardar eventos para más tarde, valorar los eventos a los que ha asistido, consultar las recomendaciones personalizadas, explorar las rutas sugeridas y las creadas por otros usuarios, y valorarlas.

Casos de prueba relacionados: [CP-03](#) (perfil), [CP-05](#) a [CP-18](#) (consulta, búsqueda, mapa, participación, chat, recomendaciones y rutas), [CP-29](#) y [CP-30](#) (notificaciones).

3.2.3. Organizador

Todo ciudadano registrado puede crear y gestionar sus propios eventos, por lo que el rol de organizador no es un perfil técnico diferenciado sino un patrón de uso del mismo actor. Sin embargo, dentro de este perfil conviven motivaciones muy distintas: desde una persona particular que organiza una quedada o un taller esporádico, hasta un negocio local, una asociación cultural o un creador de contenido cuya actividad principal es precisamente la publicación de eventos. Este último grupo, aunque técnicamente igual al ciudadano estándar, hace un uso intensivo de las funcionalidades de creación, edición y gestión de visibilidad.

Sus necesidades específicas incluyen publicar eventos públicos o privados, editarlos, gestionar los accesos, crear rutas y recibir información sobre el resultado de la moderación.

Casos de prueba relacionados: [CP-08](#) y [CP-09](#) (apuntarse a evento), [CP-19](#) a [CP-23](#) (creación, acceso y edición de eventos privados y cambio de visibilidad), [CP-29](#) (notificación de moderación).

3.2.4. Moderador de contenido

Ciudadano con responsabilidad de revisión del contenido publicado por otros usuarios. Su función es garantizar la calidad y fiabilidad de los eventos que aparecen en el listado público. Puede revisar los eventos pendientes, editarlos directamente para corregir errores menores, aprobarlos o rechazarlos, y editar los eventos públicos ya aprobados. Este rol es asignado por el administrador.

Casos de prueba relacionados: [CP-26](#) (panel de moderación), [CP-27](#) (evento no visible durante moderación), [CP-28](#) (acceso denegado a usuario sin rol).

3.2.5. Administrador de la plataforma

Responsable de la configuración estructural del sistema. Gestiona el catálogo de categorías de eventos y los roles de los usuarios (puede promover a moderador o administrador, o eliminar cuentas), consulta estadísticas globales de uso y puede restablecer los eventos importados por scraping. No interviene directamente en la revisión de contenido, separando así las responsabilidades de configuración y moderación.

Casos de prueba relacionados: [CP-24](#) (panel de administración – usuarios), [CP-25](#) (gestión de categorías), [CP-28](#) (verificación de acceso por rol).

La figura 3.1 muestra las relaciones de extensión entre actores y su vinculación con las principales áreas funcionales del sistema.

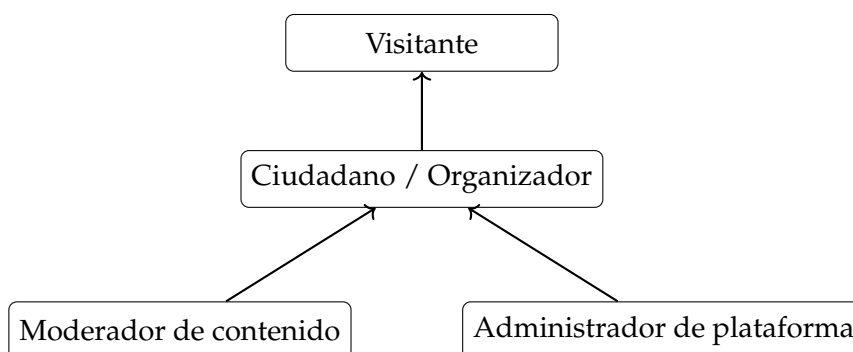


Figura 3.1: Diagrama de actores del sistema Sevillaneando

3.3. Diagrama de casos de uso

El diagrama de casos de uso muestra las principales funcionalidades del sistema y los actores que las utilizan. Se presentan agrupados en tres bloques funcionales para facilitar la lectura.

Dado el número de funcionalidades del sistema, el diagrama de casos de uso se divide en dos partes: la primera cubre el acceso, el perfil y la gestión de eventos; la segunda cubre las funcionalidades sociales, la moderación y la administración.

*Nota: el moderador y el administrador heredan todas las funcionalidades del ciudadano, tal como refleja el diagrama de actores (figura 3.1), a excepción de **crear evento** y **editar evento propio**. Por claridad visual, estas relaciones no se representan explícitamente en los diagramas de casos de uso.*

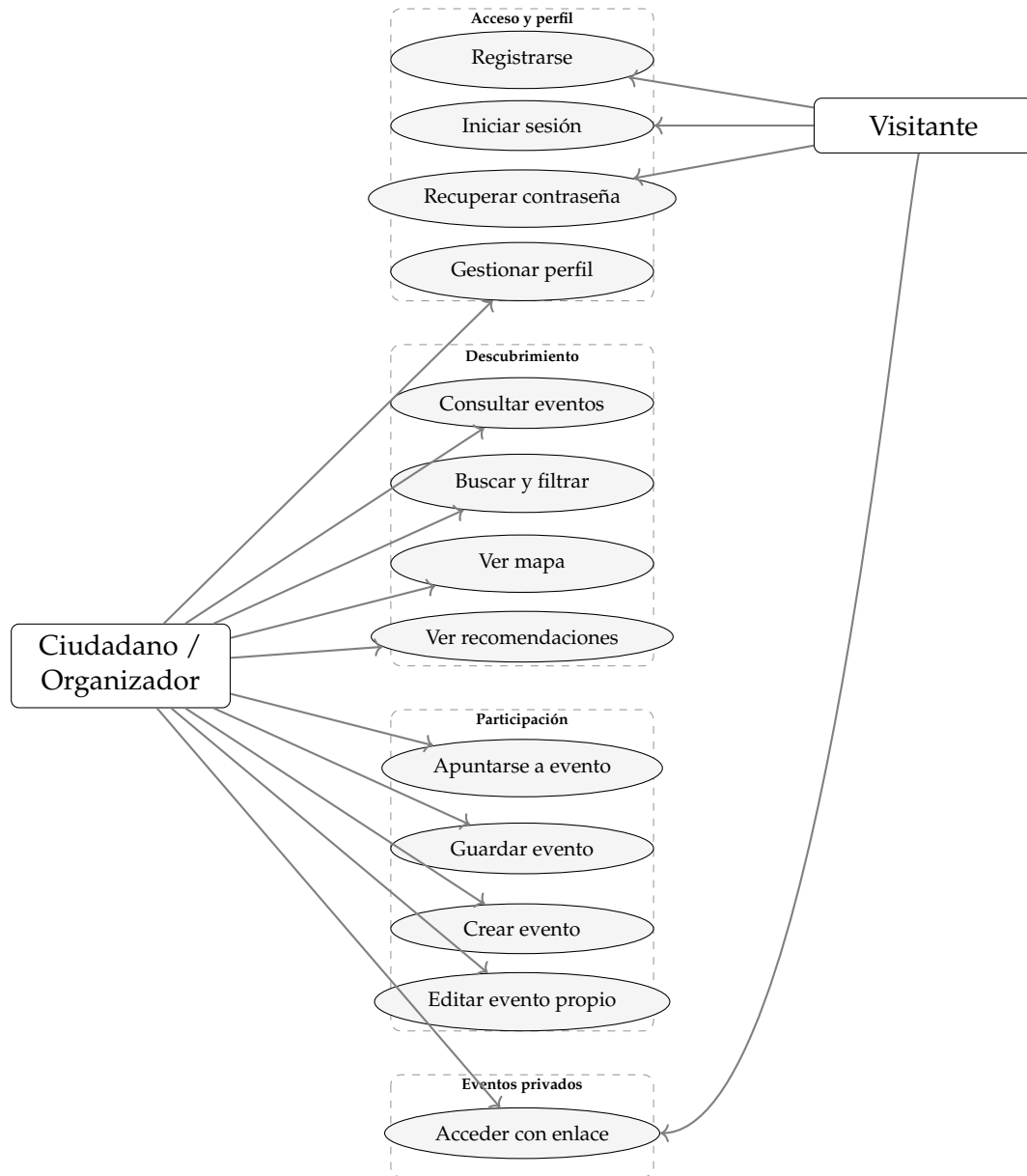


Figura 3.2: Diagrama de casos de uso (parte 1): acceso, perfil, descubrimiento y gestión de eventos

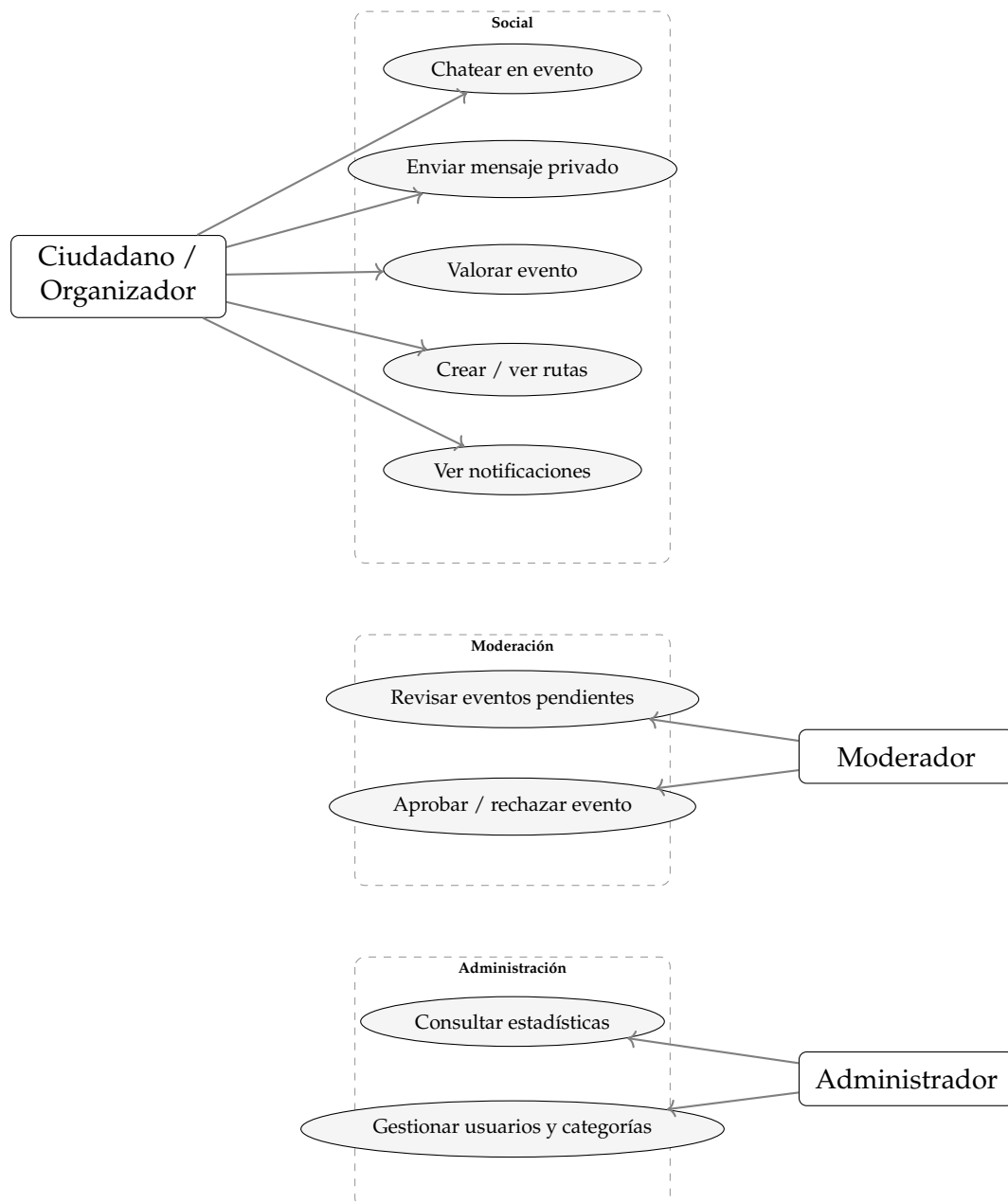


Figura 3.3: Diagrama de casos de uso (parte 2): funcionalidades sociales, moderación y administración

3.4. Requisitos de información

Los requisitos de información describen los datos que el sistema debe gestionar para poder ofrecer sus funcionalidades.

En el sistema **Sevillaneando**, los principales elementos de información son los recogidos en la tabla 3.1.

Tabla 3.1: Requisitos de información del sistema

Elemento de información	Descripción
Usuarios	Información básica del usuario: identidad, credenciales, foto de perfil, ubicación, intereses y rol en el sistema (usuario, moderador o administrador). También se almacenan sus preferencias de navegación, la aceptación de la política de privacidad y las relaciones sociales con otros usuarios, así como los eventos con los que ha interactuado: asistidos, guardados, compartidos y visitados. La preferencia de tema visual se gestiona en el cliente y no se persiste en base de datos.
Roles y permisos	Información asociada al tipo de usuario y a las acciones que puede realizar dentro del sistema, como crear eventos, moderarlos, gestionar usuarios o administrar categorías.
Categorías	Clasificación de los eventos según su temática. Son gestionadas por el administrador y utilizadas por los usuarios para filtrar eventos y definir intereses para las recomendaciones.
Eventos	Cada evento contiene su identificación, descripción, categoría, ubicación geográfica, fechas de inicio y fin, posibilidad de recurrencia, precio fijo o variable según tramo, galería de imágenes, estado de moderación, creador y visibilidad. Los eventos privados incluyen además un enlace de acceso por invitación.
Solicitudes de edición de eventos	Cuando el creador edita un evento público ya aprobado, se registra una solicitud con los cambios propuestos que debe pasar de nuevo por moderación. Se almacena el estado de la solicitud y, en caso de rechazo, el motivo indicado por el moderador.
Moderación de eventos	Datos necesarios para controlar el estado de revisión de los eventos públicos propuestos por usuarios, incluyendo aprobación, rechazo con motivo y posibles modificaciones realizadas por el moderador antes de publicar.
Asistencia a eventos	Relación entre usuarios y eventos que permite conocer qué usuarios están apuntados a cada evento y obtener listados posteriores.
Eventos guardados	Relación entre usuarios y eventos almacenados como guardados para su consulta posterior.

Elemento de información	Descripción
Eventos compartidos y visitados	Registro de los eventos que cada usuario ha compartido con terceros o ha consultado, utilizado como señal de comportamiento en el motor de recomendaciones.
Calendario	Información derivada de los eventos con fecha a los que el usuario está apuntado, para su visualización agrupada por día.
Mensajes de chat de evento	Datos asociados a la comunicación grupal dentro de un evento: contenido, imagen opcional, fecha de envío y usuario emisor.
Mensajes privados	Datos asociados a las conversaciones directas entre dos usuarios: contenido, imagen opcional, fecha de envío, estado de lectura, emisor y receptor.
Imágenes	Fotografías asociadas a eventos, chats o perfiles de usuario, incluyendo imágenes subidas en tiempo real durante los eventos.
Reseñas de eventos	Puntuaciones y comentarios que los usuarios realizan sobre eventos a los que han asistido, con su fecha y autor.
Calificaciones de rutas	Valoraciones numéricas que los usuarios asignan a las rutas. Cada usuario puede calificar una ruta como máximo una vez.
Notificaciones	Avisos enviados al usuario sobre eventos relevantes del sistema: recordatorios de asistencia, eventos cercanos según ubicación, resultado de moderación y nuevos seguidores. Se registra el tipo de notificación, su fecha y si ha sido leída.
Recomendaciones	Información utilizada para sugerir eventos en función de los intereses, ubicación y actividad previa del usuario. Se almacenan los criterios empleados en el cálculo y si la recomendación ya ha sido visualizada.
Rutas	Agrupaciones de eventos con un itinerario definido, creadas por el sistema o por usuarios. Incluyen nombre, descripción, secuencia de eventos, trayecto geográfico, duración estimada, fecha de creación y puntuación media acumulada.
Filtros de búsqueda	Información necesaria para buscar eventos por estado, categoría, título, descripción, precio, cercanía y radio de distancia.
Datos geográficos	Coordenadas y ubicación de usuarios y eventos, utilizadas para mostrar eventos en el mapa, calcular cercanía, calcular recomendaciones y generar rutas recomendadas.

Estos datos se almacenan en una base de datos relacional, concretamente **PostgreSQL con PostGIS**. Esta permite gestionar de forma eficiente las relaciones entre entidades, así como realizar consultas tanto convencionales como geoespaciales. En particular, las capacidades de PostGIS se utilizan para operaciones como el cálculo de distancias, filtrado por radio, búsqueda de eventos cercanos, visualización en mapas y generación de rutas, entre otras funcionalidades basadas en localización.

La figura 3.4 muestra el diagrama del modelo conceptual del sistema, representando las entidades del dominio y sus relaciones.

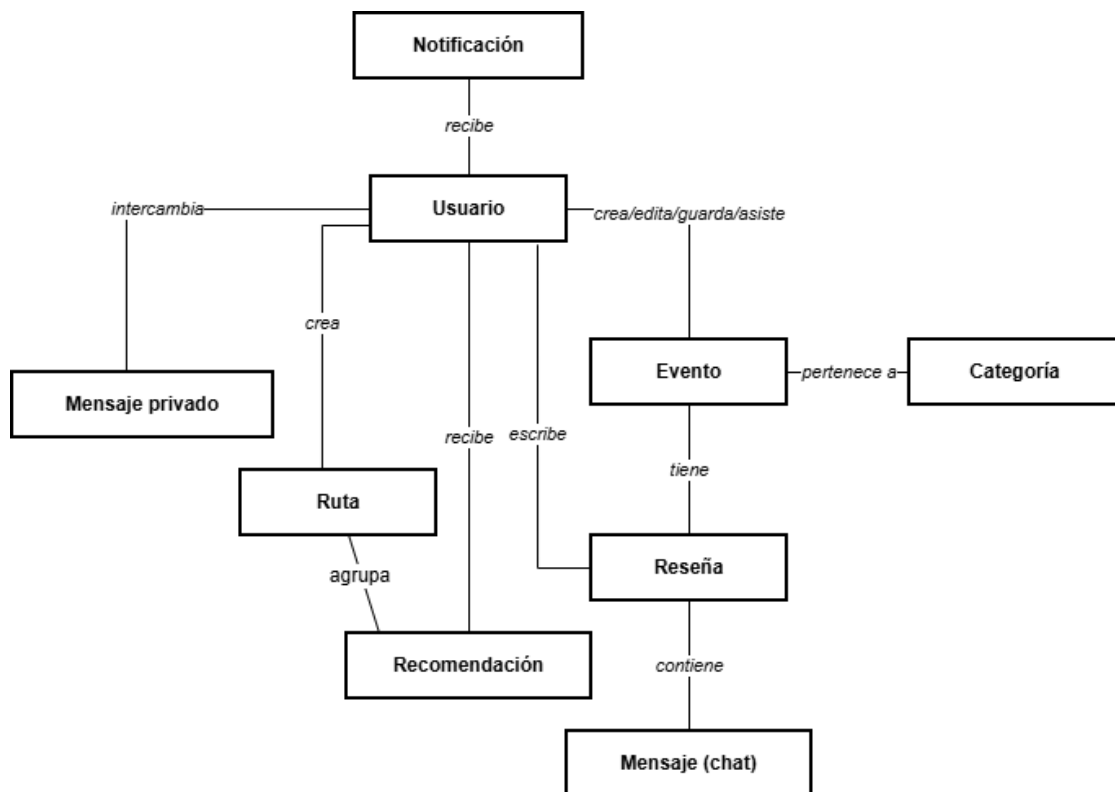


Figura 3.4: Diagrama del modelo conceptual del sistema

3.5. Requisitos funcionales

Los requisitos funcionales describen las funcionalidades que el sistema debe ofrecer a los usuarios. Se expresan siguiendo el formato estándar de historia de usuario: *Como [actor], quiero [acción], para [objetivo]*, lo que permite relacionar cada requisito con el actor que lo origina y con el objetivo del sistema que satisface.

La tabla 3.2 recoge los requisitos funcionales identificados, junto con su historia de usuario y su criterio de aceptación.

Tabla 3.2: Requisitos funcionales del sistema

Código	Funcionalidad	Detalle
RF-01	Registro, inicio de sesión y recuperación de contraseña	Historia de usuario: <i>Como usuario no autenticado, quiero registrarme con correo y contraseña, iniciar sesión y recuperar mi contraseña en caso de olvidarla, para acceder a las funcionalidades de la aplicación.</i> Criterio de aceptación: un usuario puede crear una cuenta e iniciar sesión correctamente. Un intento con credenciales incorrectas muestra un mensaje de error. Un usuario puede solicitar el restablecimiento de contraseña mediante su correo electrónico.
RF-02	Gestión de perfil	Historia de usuario: <i>Como usuario autenticado, quiero editar mi información personal, intereses, ubicación y contraseña, para mantener mi perfil actualizado y recibir recomendaciones ajustadas a mis preferencias.</i> Criterio de aceptación: los cambios en el perfil se reflejan inmediatamente en la aplicación. El usuario puede eliminar su cuenta y todos sus datos asociados son eliminados del sistema.
RF-03	Consulta de eventos	Historia de usuario: <i>Como usuario autenticado, quiero consultar un listado de eventos con información básica y acceder al detalle de cada uno, para descubrir actividades culturales y de ocio en Sevilla.</i> Criterio de aceptación: el listado muestra título, categoría, fecha y ubicación. Al pulsar sobre un evento se accede a su pantalla de detalle con toda la información disponible.
RF-04	Búsqueda y filtrado	Historia de usuario: <i>Como usuario autenticado, quiero filtrar eventos por categoría, precio, distancia, fecha, título o descripción, para encontrar rápidamente los eventos que se ajustan a mis intereses.</i> Criterio de aceptación: al aplicar uno o varios filtros, el listado se actualiza mostrando únicamente los eventos que cumplen los criterios seleccionados.
RF-05	Mapa de eventos	Historia de usuario: <i>Como usuario autenticado, quiero visualizar los eventos cercanos en un mapa interactivo, para descubrir actividades en mi entorno de forma visual e intuitiva.</i> Criterio de aceptación: los eventos aparecen como marcadores en el mapa. Al pulsar sobre un marcador se muestra información básica del evento y se puede navegar a su detalle.

Código	Funcionalidad	Detalle
RF-06	Creación de eventos	Historia de usuario: <i>Como usuario autenticado, quiero crear eventos públicos o privados, para dar visibilidad a actividades que organizo o que quiero compartir con otras personas.</i> Criterio de aceptación: el formulario de creación permite seleccionar visibilidad. Un evento público queda en estado pendiente tras su creación; un evento privado es accesible inmediatamente por su creador.
RF-07	Moderación de eventos	Historia de usuario: <i>Como moderador, quiero revisar los eventos públicos pendientes, editarlos si es necesario, y aprobarlos o rechazarlos; y además poder editar o eliminar cualquier evento público ya aprobado, para garantizar la calidad y fiabilidad del contenido publicado en todo momento.</i> Criterio de aceptación: el moderador ve en su panel los eventos pendientes, puede aprobarlos, rechazarlos o modificarlos. Tras la resolución, el creador recibe una notificación con el resultado. Adicionalmente, el moderador dispone de un listado independiente con todos los eventos públicos (aprobados) desde el que puede editarlos o eliminarlos sin afectar su estado de moderación.
RF-08	Participación en eventos	Historia de usuario: <i>Como usuario autenticado, quiero apuntarme a eventos públicos o acceder a eventos privados mediante enlace, para gestionar mi asistencia y organizar mis planes.</i> Criterio de aceptación: un usuario puede apuntarse a un evento activo y aparece en la lista de asistentes. Un evento privado solo es accesible si el usuario dispone del enlace de invitación.
RF-09	Chat y mensajería	Historia de usuario: <i>Como usuario autenticado, quiero comunicarme con otros usuarios mediante el chat de un evento o con mensajes privados directos, para coordinarme y conocer a personas con intereses similares.</i> Criterio de aceptación: los mensajes en el chat de un evento se reciben en tiempo real por todos los participantes. Los mensajes privados entre dos usuarios se entregan de forma inmediata.

Código	Funcionalidad	Detalle
RF-10	Gestión de imágenes	Historia de usuario: Como usuario autenticado, quiero subir y visualizar imágenes en eventos, chats y perfiles, para enriquecer el contenido y compartir momentos de los eventos. Criterio de aceptación: las imágenes se suben correctamente y se muestran en el contexto correspondiente. Se rechaza la subida de archivos que superen 10 MB o que no sean de tipo permitido.
RF-11	Valoraciones	Historia de usuario: Como usuario autenticado, quiero valorar eventos mediante puntuaciones y comentarios, para compartir mi experiencia y ayudar a otros usuarios a decidir si asistir. Criterio de aceptación: un usuario puede dejar una puntuación de 1 a 5 estrellas y un comentario opcional. La valoración media del evento se actualiza y es visible en el detalle del evento.
RF-12	Recomendaciones	Historia de usuario: Como usuario autenticado, quiero recibir recomendaciones personalizadas de eventos, para descubrir actividades que se ajusten a mis intereses sin tener que buscarlas activamente. Criterio de aceptación: la pantalla de recomendaciones muestra eventos distintos a los ya vistos, ordenados según la puntuación calculada por el algoritmo en base a intereses, ubicación y actividad previa.
RF-13	Rutas de eventos	Historia de usuario: Como usuario autenticado, quiero consultar rutas de eventos recomendadas automáticamente o creadas por otros usuarios, y poder crear mis propias rutas públicas, para planificar itinerarios completos de ocio. Criterio de aceptación: el sistema genera rutas recomendadas al usuario. Un usuario puede crear una ruta pública que es visible y valorable por otros usuarios.
RF-14	Notificaciones	Historia de usuario: Como usuario autenticado, quiero recibir notificaciones sobre eventos cercanos, recordatorios, seguidores nuevos y resultados de moderación, para estar informado sin tener que consultar manualmente cada sección de la aplicación. Criterio de aceptación: el usuario recibe notificaciones dentro de la aplicación cuando hay eventos cercanos, se acerca la fecha de un evento al que está apuntado, un evento suyo es aprobado o rechazado, o un usuario comienza a seguirlo.

Código	Funcionalidad	Detalle
RF-15	Gestión de usuarios y roles	Historia de usuario: <i>Como administrador, quiero consultar estadísticas, gestionar el rol de los usuarios y eliminar cuentas, para mantener la integridad del sistema y gestionar el equipo de moderación.</i> Criterio de aceptación: un usuario sin permisos no puede acceder a las pantallas de administración ni moderación. El administrador puede consultar totales de usuarios y eventos, cambiar el rol de cualquier usuario desde el panel de administración y eliminar cuentas. Un usuario eliminado no puede volver a iniciar sesión con esa cuenta.
RF-16	Gestión de categorías	Historia de usuario: <i>Como administrador, quiero crear, editar y eliminar categorías de eventos, para mantener una taxonomía coherente y representativa de la oferta cultural de la ciudad.</i> Criterio de aceptación: las categorías creadas por el administrador aparecen disponibles inmediatamente en el formulario de creación de eventos y en los filtros de búsqueda.
RF-17	Organización de eventos	Historia de usuario: <i>Como usuario autenticado, quiero guardar eventos, apuntarme a ellos y consultarlos en mi calendario, para organizar mis planes y no perder de vista las actividades que me interesan.</i> Criterio de aceptación: los eventos guardados y los eventos a los que el usuario se ha apuntado aparecen en la sección correspondiente. Los eventos con fecha a los que el usuario está apuntado aparecen en el calendario en su fecha correcta.
RF-18	Actualización automática de eventos	Historia de usuario: <i>Como sistema, quiero obtener y actualizar eventos de fuentes externas de forma periódica, para ampliar automáticamente el catálogo con eventos de plataformas institucionales y comerciales.</i> Criterio de aceptación: los eventos importados aparecen en el listado como eventos aprobados. Si un evento ya existe, sus datos se actualizan en lugar de crear un duplicado. Los eventos sin ubicación válida no se almacenan.

Código	Funcionalidad	Detalle
RF-19	Restablecimiento del scraping	Historia de usuario: <i>Como administrador, quiero poder restablecer los eventos importados por scraping, para limpiar el catálogo de eventos externos y forzar una reimportación limpia desde las fuentes.</i> Criterio de aceptación: el administrador puede eliminar todos los eventos importados por scraping desde el panel de administración. Tras el restablecimiento, el catálogo queda sin eventos de origen externo hasta la siguiente ejecución del proceso de scraping.
RF-20	Edición de eventos	Historia de usuario: <i>Como usuario autenticado, quiero editar los eventos que he creado, para corregir errores o actualizar la información antes o después de la moderación.</i> Criterio de aceptación: el creador de un evento puede modificar sus datos. Si el evento es público, la edición vuelve a pasar por moderación antes de publicarse.
RF-21	Acceso a eventos privados	Historia de usuario: <i>Como usuario autenticado, quiero acceder a eventos privados mediante el enlace que me comparte el creador, para participar en eventos restringidos a invitados.</i> Criterio de aceptación: un usuario que accede mediante el enlace correcto puede ver el detalle del evento privado. Un usuario sin el enlace no puede ver ni listar el evento.

3.5.1. Matriz de trazabilidad de requisitos a objetivos

La tabla 3.3 muestra la relación entre los requisitos funcionales y los objetivos del sistema definidos en la Sección 2.2.2 del capítulo 2.

Tabla 3.3: Matriz de trazabilidad: requisitos funcionales y objetivos del sistema

Requisito funcional	Objetivo(s) del sistema relacionado(s)
RF-01: Registro e inicio de sesión	Base habilitadora para todos los objetivos (§ 2.2.2)
RF-02: Gestión de perfil	Recomendaciones personalizadas (OBJ-06)
RF-03: Consulta de eventos	Listado y detalle de eventos (OBJ-01)
RF-04: Búsqueda y filtrado	Listado y filtrado de eventos (OBJ-01)
RF-05: Mapa de eventos	Geolocalización e interacción en mapa (OBJ-04)
RF-06: Creación de eventos	Creación de eventos públicos y privados (OBJ-02)
RF-07: Moderación de eventos	Sistema de moderación (OBJ-02)
RF-08: Participación en eventos	Interacción social y asistencia (OBJ-05)
RF-09: Chat y mensajería	Interacción social en tiempo real (OBJ-05)
RF-10: Gestión de imágenes	Interacción social y contenido enriquecido (OBJ-05)
RF-11: Valoraciones	Interacción social y contenido enriquecido (OBJ-05)
RF-12: Recomendaciones	Sistema de recomendaciones personalizadas (OBJ-06)
RF-13: Rutas de eventos	Sistema de rutas recomendadas y de usuario (OBJ-07, OBJ-08)
RF-14: Notificaciones	Sistema de notificaciones (OBJ-09)
RF-15: Gestión de usuarios y roles	Moderación y administración (OBJ-02)
RF-16: Gestión de categorías	Moderación y administración (OBJ-02); filtrado de eventos por temática (OBJ-01)
RF-17: Organización de eventos	Asistencia y organización personal (OBJ-01, OBJ-05)
RF-18: Actualización automática	Listado con eventos de fuentes externas (OBJ-01)
RF-19: Restablecimiento del scraping	Gestión del catálogo de eventos externos (OBJ-01)
RF-20: Edición de eventos	Gestión del ciclo de vida de eventos (OBJ-03)
RF-21: Acceso a eventos privados	Eventos privados con acceso restringido (OBJ-02)

3.6. Diseño de mockups

Durante la fase de análisis se elaboraron mockups de las pantallas principales de la aplicación con el objetivo de validar el flujo de navegación y el diseño de la interfaz antes de iniciar la implementación. Los mockups sirvieron como punto de partida para definir la estructura visual de cada pantalla y para identificar los elementos de interacción más importantes.

Las siguientes imágenes recogen una vista general de los mockups realizados para las principales áreas funcionales de la aplicación.

3.6.1. Vista general de mockups

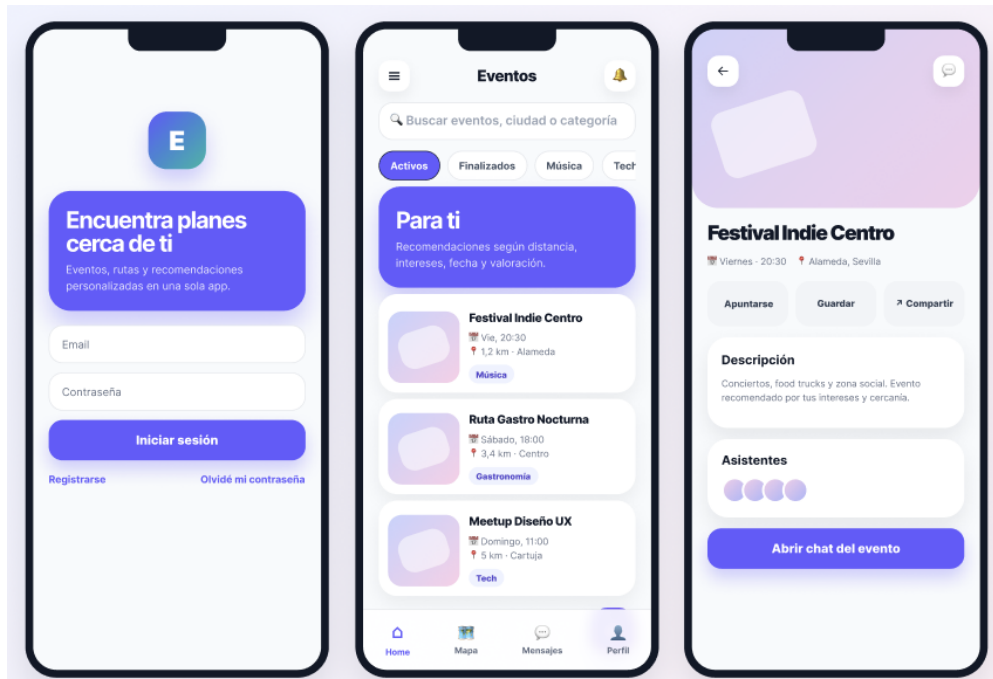


Figura 3.5: Acceso, listado de eventos y detalle de evento

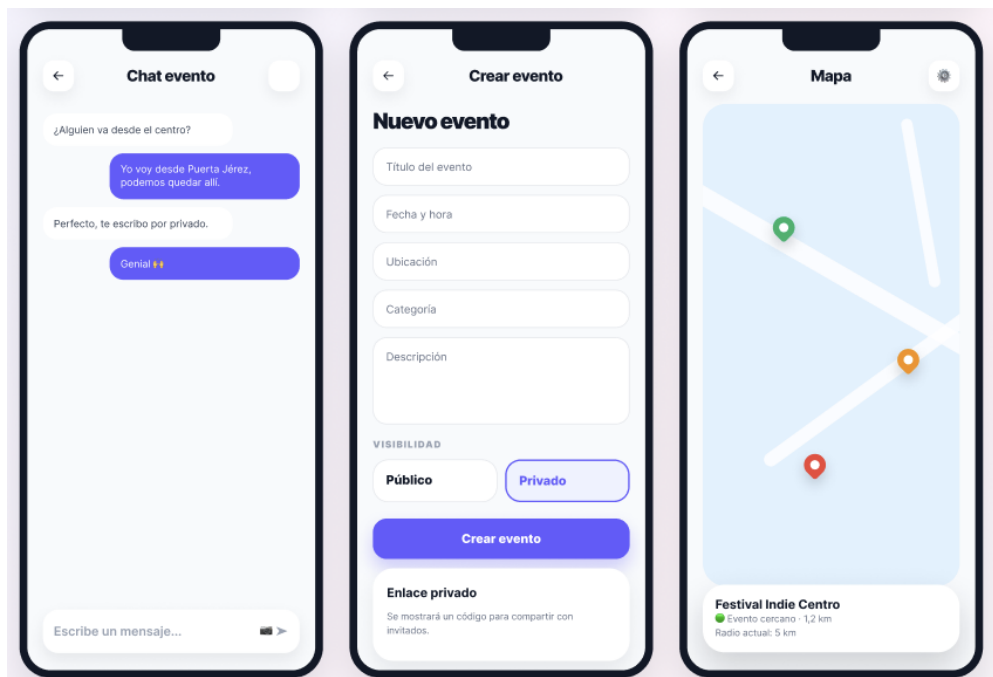


Figura 3.6: Chat de evento, creación de eventos y mapa

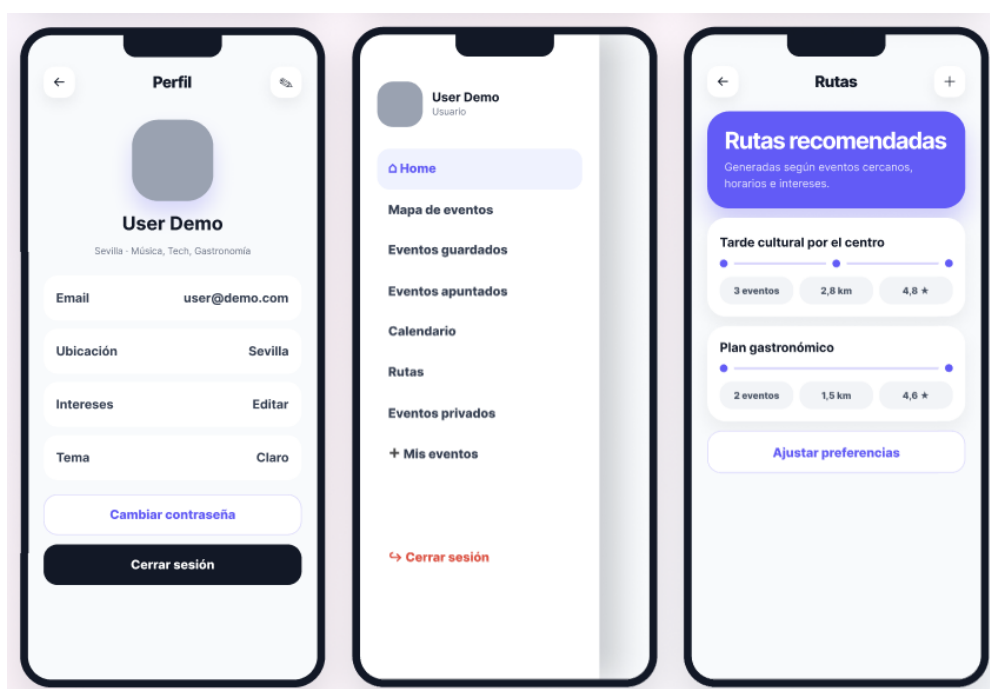


Figura 3.7: Perfil, menú lateral y rutas recomendadas

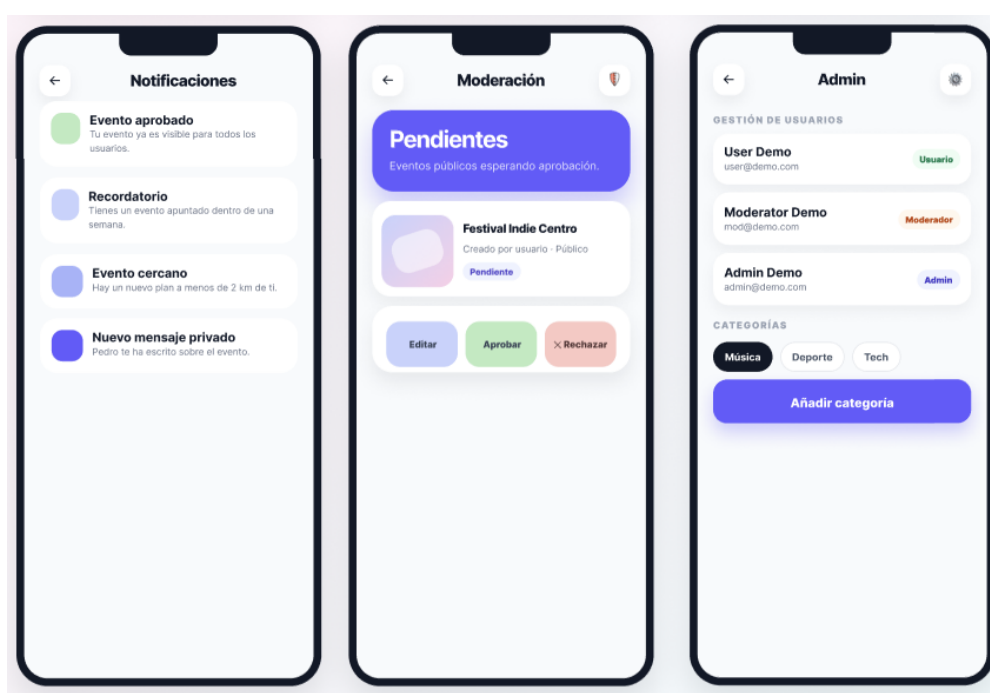


Figura 3.8: Notificaciones, moderación y administración

3.7. Requisitos no funcionales

Los requisitos no funcionales describen las características de calidad que debe cumplir el sistema.

La tabla 3.4 resume los requisitos no funcionales definidos para el sistema.

Tabla 3.4: Requisitos no funcionales del sistema

Código	Nombre	Descripción
RNF-01	Usabilidad	La aplicación debe ofrecer una interfaz intuitiva y fácil de usar, adaptada a dispositivos móviles.
RNF-02	Rendimiento	El sistema debe responder de forma rápida a las acciones del usuario, especialmente en la carga de eventos y mapas.
RNF-03	Escalabilidad	La arquitectura debe permitir el crecimiento del sistema en número de usuarios y eventos.
RNF-04	Seguridad	La autenticación debe ser segura, utilizando servicios como Firebase Auth, y protegiendo los datos del usuario.
RNF-05	Disponibilidad	El sistema debe estar disponible de forma continua para garantizar el acceso a los usuarios.
RNF-06	Mantenibilidad	El código debe estar organizado y documentado para facilitar futuras modificaciones.
RNF-07	Compatibilidad	La aplicación debe funcionar tanto en dispositivos Android como iOS.
RNF-08	Integración	El sistema debe integrarse correctamente con servicios externos como mapas, almacenamiento de imágenes y notificaciones.

3.8. Reglas de negocio

Las reglas de negocio definen las restricciones y condiciones del sistema, asegurando la coherencia de los datos y el correcto funcionamiento de las funcionalidades. Estas reglas de negocio se han definido evitando duplicar funcionalidades ya descritas en los requisitos funcionales, centrándose únicamente en restricciones, validaciones y condiciones del sistema.

3.8.1. Gestión de usuarios y roles

- **RN-01:** Todo usuario debe estar autenticado para acceder a las funcionalidades principales del sistema.
- **RN-02:** El rol por defecto al registrarse es el de usuario.
- **RN-03:** Solo los administradores pueden modificar roles o eliminar usuarios.

3.8.2. Gestión de eventos

- **RN-04:** Todo evento público debe pasar por un proceso de moderación antes de ser visible.
- **RN-05:** No se pueden aprobar o rechazar eventos que ya tengan ese mismo estado.
- **RN-06:** Un evento público solo será visible para los usuarios si su estado es aprobado.
- **RN-07:** Los eventos finalizados no permiten nuevas inscripciones.
- **RN-08:** Cuando un evento pasa de privado a público, debe someterse a moderación antes de ser visible.

3.8.3. Eventos privados

- **RN-09:** Los eventos privados solo son accesibles mediante un enlace de invitación.
- **RN-10:** Solo el creador del evento privado puede compartir el enlace de acceso.
- **RN-11:** Los eventos privados no son visibles en el listado general de eventos.
- **RN-12:** Un evento privado solo puede ser consultado por su creador o usuarios invitados.
- **RN-13:** Los eventos privados no requieren moderación.

3.8.4. Participación en eventos

- **RN-14:** Un usuario solo puede apuntarse a eventos activos.
- **RN-15:** Un usuario no puede apuntarse más de una vez al mismo evento.
- **RN-16:** Un usuario no puede guardar más de una vez el mismo evento.
- **RN-17:** Solo los eventos con fecha definida deben aparecer en el calendario del usuario.

3.8.5. Comunicación y contenido

- **RN-18:** Las imágenes deben ser de contenido adecuado; el cumplimiento de esta política queda bajo la responsabilidad del usuario.
- **RN-19:** Las imágenes deben ser de tipo permitido (JPEG, PNG, WEBP, HEIC, HEIF).
- **RN-20:** Las imágenes no deben superar un tamaño máximo establecido de 10 MB.

3.8.6. Valoraciones y reseñas

- **RN-21:** Las valoraciones deben estar comprendidas entre 1 y 5 estrellas.

3.8.7. Recomendaciones y geolocalización

- **RN-22:** La ubicación del usuario debe estar definida para ofrecer funcionalidades basadas en proximidad.

3.8.8. Scraping

- **RN-23:** Los eventos sin ubicación válida no deben almacenarse.
- **RN-24:** Los eventos importados mediante scraping se consideran públicos y se aprueban automáticamente.

3.8.9. Restricciones de integridad de datos

- **RN-25:** Todo evento debe tener título, descripción y categoría.
- **RN-26:** Todo evento debe tener una ubicación geográfica válida.
- **RN-27:** La latitud y longitud deben ser valores numéricos y en el rango (latitud entre -90 y 90, longitud entre -180 y 180).
- **RN-28:** La fecha de inicio no puede ser posterior a la fecha de finalización.
- **RN-29:** El precio de un evento no puede ser negativo.
- **RN-30:** El radio de búsqueda debe ser un valor positivo.
- **RN-31:** Un evento no puede tener simultáneamente un precio fijo y un rango de precios mínimo y máximo.
- **RN-32:** La dirección de un evento debe ser coherente con sus coordenadas geográficas.
- **RN-33:** En un rango de precios, el precio mínimo no puede ser superior al precio máximo.

3.8.10. Notificaciones

- **RN-34:** Las notificaciones de eventos cercanos solo pueden generarse si el usuario tiene una ubicación definida.
- **RN-35:** Las notificaciones de recordatorio solo pueden generarse para eventos a los que el usuario esté apuntado.

- **RN-36:** Las notificaciones de moderación solo deben enviarse al creador del evento aprobado o rechazado.

3.9. Cumplimiento del Reglamento General de Protección de Datos (RGPD)

El sistema **Sevillaneando** trata datos de carácter personal de sus usuarios, por lo que está sujeto al Reglamento (UE) 2016/679, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales (RGPD) [18], así como a la Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales (LOPDGDD) [19].

3.9.1. Datos personales tratados

La aplicación recoge y trata los siguientes datos personales:

- **Identificación:** nombre completo y dirección de correo electrónico, recogidos durante el registro.
- **Fotografía de perfil:** imagen opcional que el usuario sube voluntariamente.
- **Ubicación geográfica:** coordenadas del usuario, recogidas opcionalmente para mostrar eventos cercanos.
- **Intereses culturales:** categorías seleccionadas por el usuario para personalizar las recomendaciones.
- **Contenido generado:** eventos, rutas, reseñas, valoraciones y mensajes de chat creados por el usuario.

3.9.2. Finalidad y base legal del tratamiento

El tratamiento de los datos se realiza con las siguientes finalidades:

- Gestión de la cuenta del usuario y autenticación.
- Personalización de la experiencia: recomendación de eventos según intereses, historial y ubicación.
- Funcionalidades sociales: chat, mensajería privada y sistema de seguimiento entre usuarios.
- Moderación de contenido para garantizar la calidad de los eventos publicados.

La base legal del tratamiento es el **consentimiento expreso del interesado** (art. 6.1.a RGPD), otorgado mediante la aceptación de la política de privacidad en el

momento del registro. Este consentimiento puede retirarse en cualquier momento mediante la eliminación de la cuenta.

3.9.3. Terceros encargados del tratamiento

El sistema integra servicios externos que actúan como encargados del tratamiento:

- **Firebase (Google LLC):** autenticación de usuarios y almacenamiento de imágenes. Google LLC cuenta con certificaciones de cumplimiento RGPD y cláusulas contractuales tipo para transferencias internacionales.
- **Cloudinary:** alojamiento de imágenes de eventos y perfiles.
- **Ticketmaster:** fuente de datos de eventos culturales públicos; no se transfieren datos de usuarios.
- **Google Gemini:** enriquecimiento de información de eventos; no se transfieren datos de usuarios.
- **OpenStreetMap / Nominatim:** mapas y geocodificación; se usan únicamente coordenadas de búsqueda, sin enviar datos identificativos.

3.9.4. Derechos de los usuarios

De acuerdo con el RGPD, los usuarios tienen los siguientes derechos sobre sus datos personales:

- **Derecho de acceso** (art. 15): el usuario puede consultar todos sus datos desde su perfil.
- **Derecho de rectificación** (art. 16): el usuario puede modificar su nombre, correo, fotografía e intereses desde la pantalla de edición de perfil.
- **Derecho de supresión o “derecho al olvido”** (art. 17): el usuario puede eliminar su cuenta desde la aplicación, lo que provoca la anonimización o eliminación de todos sus datos personales.
- **Derecho a la portabilidad** (art. 20): aunque la aplicación móvil no expone un mecanismo directo de exportación, los datos se almacenan en formatos estándar (PostgreSQL, JSON en la API REST), lo que facilita su extracción por parte del responsable del tratamiento a petición del usuario.
- **Derecho a retirar el consentimiento** (art. 7.3): el usuario puede retirar su consentimiento eliminando su cuenta en cualquier momento.

3.9.5. Medidas técnicas y organizativas

Para garantizar la seguridad de los datos se han implementado las siguientes medidas (art. 32 RGPD):

- Comunicaciones cifradas mediante HTTPS/TLS en todas las peticiones entre cliente y servidor.
- Autenticación delegada a Firebase, que implementa mecanismos de seguridad reconocidos industrialmente.
- Validación y saneamiento de todas las entradas de usuario en el servidor (NestJS + class-validator [20]).
- Limitación de peticiones (*rate limiting*) mediante @nestjs/throttler [21] para prevenir ataques de fuerza bruta, en línea con las recomendaciones del OWASP Top 10 [22].
- Cabeceras de seguridad HTTP configuradas mediante Helmet.js [23].
- Control de acceso basado en roles (RBAC): usuario, moderador y administrador.

3.9.6. Consentimiento explícito en el registro

La aplicación incorpora un mecanismo de consentimiento explícito en la pantalla de registro: el usuario debe marcar una casilla aceptando la política de privacidad antes de poder crear su cuenta. Esta pantalla es accesible desde el formulario de registro y detalla los datos recogidos, la finalidad del tratamiento, los terceros implicados, los derechos del usuario y las medidas de seguridad aplicadas.

3.10. Conclusiones

En este capítulo se han identificado los actores del sistema y sus relaciones, elaborado los diagramas de casos de uso, especificado los requisitos de información, definido 21 requisitos funcionales, 8 requisitos no funcionales y 36 reglas de negocio, diseñado mockups de las pantallas principales y analizado el cumplimiento del RGPD. El conjunto resultante sirve de contrato entre el problema identificado y la solución diseñada en el capítulo siguiente.

4. Diseño de la solución

4.1. Introducción

En este capítulo se describe el diseño de la solución adoptada para el sistema **Sevillaneando**. El diseño parte de los requisitos identificados en el capítulo anterior y define la arquitectura general del sistema, el modelo de datos, la interfaz de usuario y el flujo de navegación de la aplicación.

El objetivo de este capítulo es explicar las decisiones estructurales tomadas antes y durante la implementación, justificando por qué se han elegido determinados patrones y organizaciones frente a otras alternativas posibles.

4.2. Arquitectura del sistema

El sistema **Sevillaneando** sigue una arquitectura cliente-servidor distribuida en dos capas principales: una aplicación móvil que actúa como cliente y un servidor backend que expone una API REST. Además, el sistema se apoya en servicios externos para cubrir necesidades específicas como autenticación, almacenamiento de imágenes y obtención de datos de fuentes externas.

La figura [4.1](#) muestra la vista lógica del sistema con los componentes principales y sus relaciones.

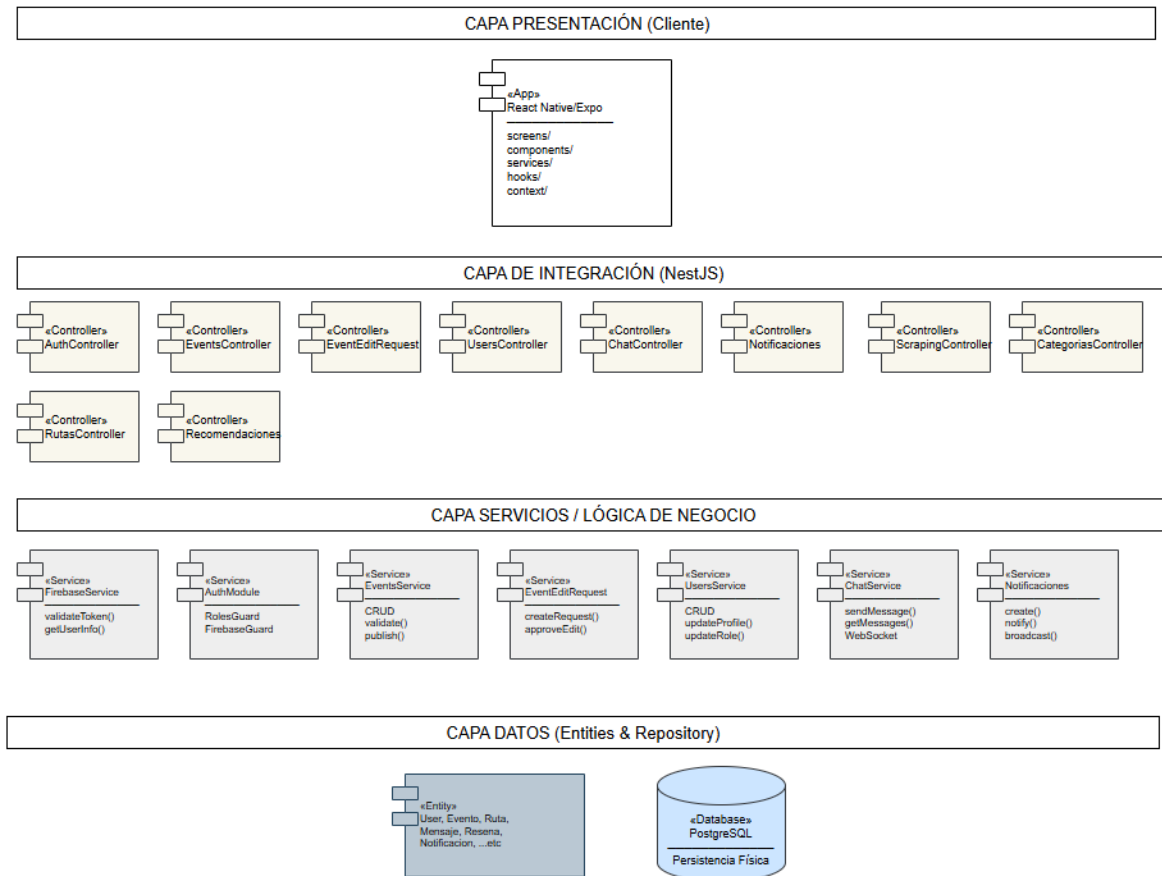


Figura 4.1: Vista lógica del sistema Sevillaneando

4.2.1. Aplicación móvil (frontend)

La aplicación móvil actúa como único punto de interacción con el usuario. Ha sido desarrollada con **React Native** [24, 25] sobre **React** [26] y empaquetada con **Expo** [27], lo que permite generar una única base de código compatible con Android e iOS. La versión final del cliente contiene 83 ficheros TypeScript/TSX en src, con 18.992 líneas de código distribuidas en pantallas, componentes, navegación, servicios, contextos, hooks, tipos y utilidades. La arquitectura interna del frontend se estructura en los siguientes niveles:

- **Pantallas:** 31 componentes que representan vistas completas navegables.
- **Componentes:** 11 elementos reutilizables que componen la interfaz, como botones temáticos, cabeceras de perfil, selectores, esqueletos de carga, modales y atribuciones de OpenStreetMap.
- **Servicios:** capa de comunicación con el backend mediante **Axios** [28]. Todos los servicios comparten un cliente Axios centralizado que inyecta automáticamente el token de Firebase [29] en la cabecera Authorization de cada petición, gestiona un *timeout* global de 60 segundos y aplica un interceptor de reintento ante errores de red.

- **Contextos:** gestión del estado global compartido entre pantallas (AuthContext para la sesión, SocketContext para la conexión WebSocket, ThemeContext para el tema visual y NotificacionesContext para los avisos pendientes).

La navegación entre pantallas se implementa con **React Navigation** [30] mediante una navegación en pila (*stack navigator*), separando las rutas autenticadas de las públicas. El AuthContext determina en tiempo de montaje qué pila de navegación se activa según el estado de la sesión.

La figura 4.2 muestra cómo interactúan estos niveles desde la acción del usuario hasta la respuesta del servidor.

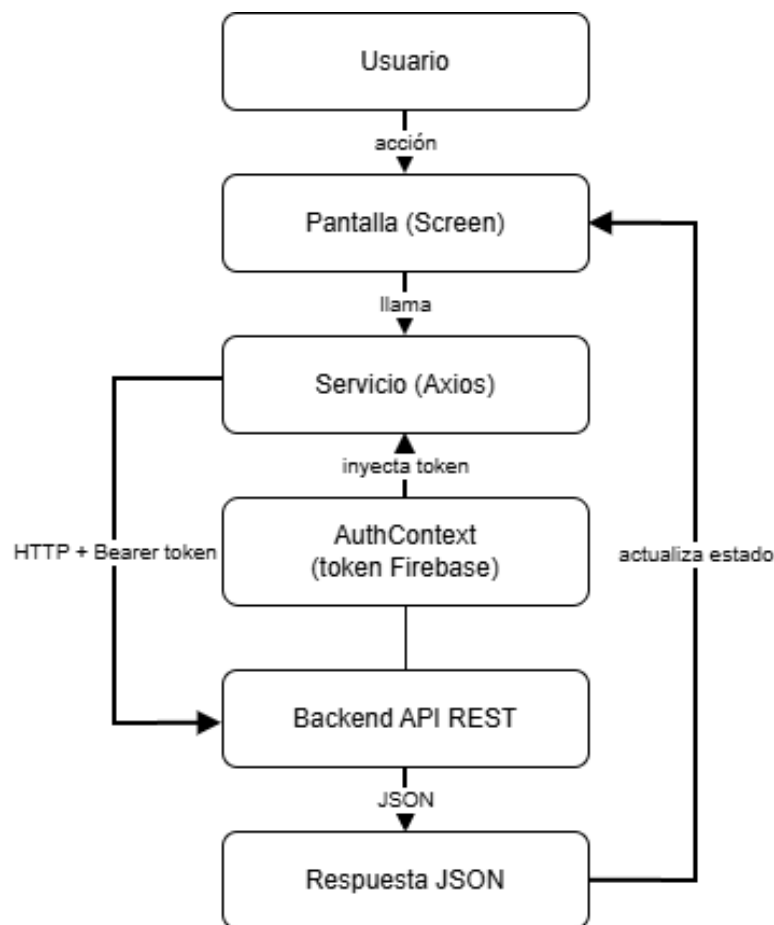


Figura 4.2: Flujo de una petición en el frontend desde la acción del usuario hasta la actualización de la pantalla

4.2.2. Servidor backend

El backend es una API REST desarrollada con **NestJS** [31, 32] sobre **Node.js** [33] y **TypeScript** [34, 35]. NestJS organiza el servidor en módulos independientes, cada uno con sus controladores, servicios y entidades. Esta separación facilita el mantenimiento y permite extender el sistema sin afectar al resto de componentes.

Cada petición HTTP entrante pasa por la siguiente cadena de responsabilidades:

1. **Guards:** FirebaseAuthGuard intercepta la petición, extrae el token Bearer de la cabecera Authorization y lo verifica con el SDK de administración de Firebase. Si el token es inválido o ha expirado, la petición es rechazada con 401 Unauthorized antes de llegar al controlador.
2. **Controlador:** recibe la petición validada, extrae los parámetros de ruta, *query* y cuerpo, y delega la lógica de negocio en el servicio correspondiente.
3. **Servicio:** ejecuta la lógica de negocio, interactúa con TypeORM para consultar o modificar la base de datos, y si es necesario llama a servicios externos (por ejemplo, Cloudinary para imágenes).
4. **TypeORM:** traduce las operaciones del servicio en consultas SQL optimizadas y gestiona la conexión con PostgreSQL.

La figura 4.3 ilustra esta cadena para una petición autenticada típica.

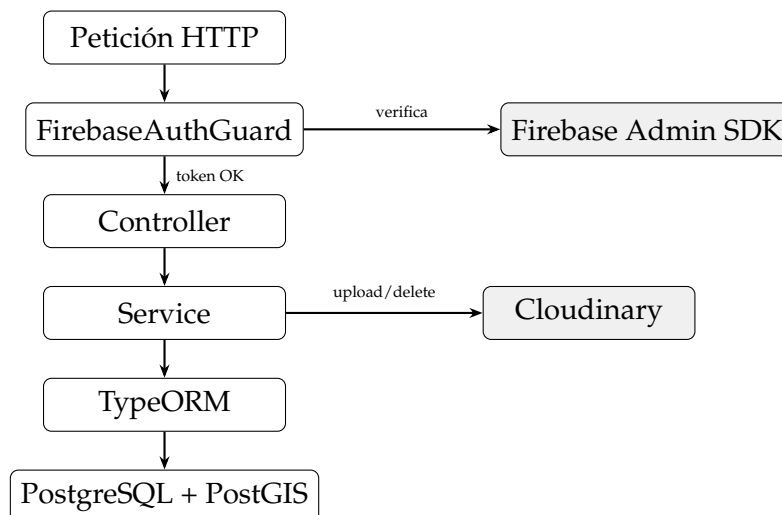


Figura 4.3: Cadena de responsabilidades en el backend para una petición autenticada

La figura 4.4 muestra en detalle el flujo completo de autenticación, desde el inicio de sesión del usuario hasta la verificación del token en el backend.

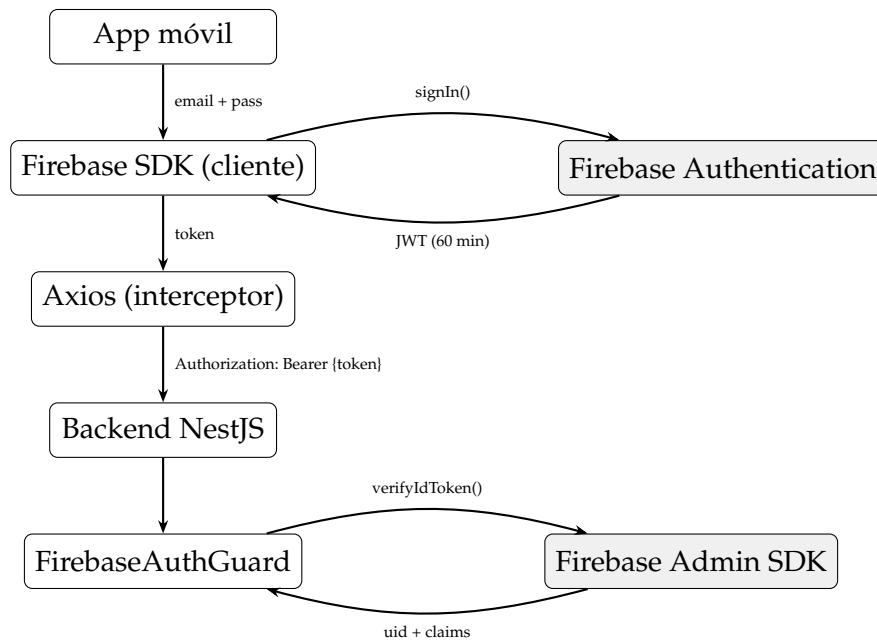


Figura 4.4: Flujo de autenticación: desde el inicio de sesión hasta la verificación del token en el backend

Además de la API REST, el backend integra un servidor **Socket.IO** [36], construido sobre el protocolo WebSocket [37], para la comunicación en tiempo real del chat. La conexión WebSocket se autentica en el *handshake* inicial mediante el mismo mecanismo que la API REST: el cliente envía el token de Firebase como parámetro de la URL de conexión, y el middleware de Socket.IO lo verifica antes de aceptar la conexión. Una vez autenticada, la conexión se mantiene abierta y el servidor puede emitir eventos al cliente en cualquier momento sin necesidad de una nueva petición.

4.2.3. Base de datos

La persistencia de datos se gestiona con **PostgreSQL 15** [38] junto con la extensión **PostGIS** [39, 40]. PostgreSQL cubre el modelo relacional general del sistema, mientras que PostGIS añade soporte para datos geográficos, permitiendo almacenar coordenadas como puntos geography en SRID 4326 [41] y ejecutar consultas espaciales eficientes como el filtrado por radio (`ST_DWithin`) o el cálculo de distancias en metros (`ST_Distance`). La comunicación entre el backend y la base de datos se realiza a través de **TypeORM** [42], que mapea las entidades TypeScript a tablas SQL y genera las consultas correspondientes.

4.2.4. Servicios externos

El sistema delega en servicios externos las siguientes responsabilidades:

- **Firebase Authentication** [43]: gestión del ciclo de vida de la sesión del usuario (registro, inicio de sesión y tokens JWT). El cliente móvil usa el SDK de Firebase para obtener y renovar tokens; el backend usa el SDK de administración para verificarlos sin almacenar contraseñas ni gestionar sesiones propias.
- **Cloudinary** [44]: almacenamiento y servicio de imágenes asociadas a eventos, perfiles y chats. El backend sube las imágenes directamente a Cloudinary mediante su SDK y almacena únicamente la URL resultante en PostgreSQL. Cloudinary sirve las imágenes desde su CDN (Content Delivery Network) con transformaciones automáticas de formato y calidad.
- **OpenStreetMap** [45]: visualización de mapas interactivos en el frontend mediante react-native-maps [46]. Los *tiles* cartográficos se obtienen directamente del servidor de OpenStreetMap sin pasar por el backend.
- **Eventbrite** [4] / **Ticketmaster** [47] / **Visita Sevilla** / **webs institucionales y de ocio de Sevilla**: fuentes externas de eventos consultadas automáticamente mediante scraping. Eventbrite se procesa extrayendo tarjetas de eventos y bloques JSON-LD del HTML; la API oficial de Ticketmaster se consume directamente; el portal visitasevilla.es dispone de un scraper dedicado que interpreta su formato de fechas propio; el resto de webs (portal municipal, guías de ocio, teatros y fundaciones) se procesan mediante el GeminiScraperService, que envía el texto limpio de cada página al modelo **Gemini 2.5 Flash** (Google Generative AI) y obtiene los eventos en formato JSON estructurado. El backend ejecuta diariamente un *cron job* que invoca cada scraper, transforma los datos al modelo interno y los persiste en PostgreSQL asignándolos al usuario técnico *scraper-bot*.
- **Nominatim (OpenStreetMap)** [48]: API de geocodificación utilizada por el GeminiScraperService [49] para resolver las coordenadas geográficas de los eventos cuando Gemini no puede deducirlas directamente del texto. Se realizan hasta tres variantes de búsqueda por dirección para maximizar la cobertura.

La figura 4.5 resume cómo se conectan todas las tecnologías del sistema en tiempo de ejecución, mostrando el camino que recorre una petición desde que el usuario interactúa con la aplicación hasta que los datos son persistidos o devueltos.

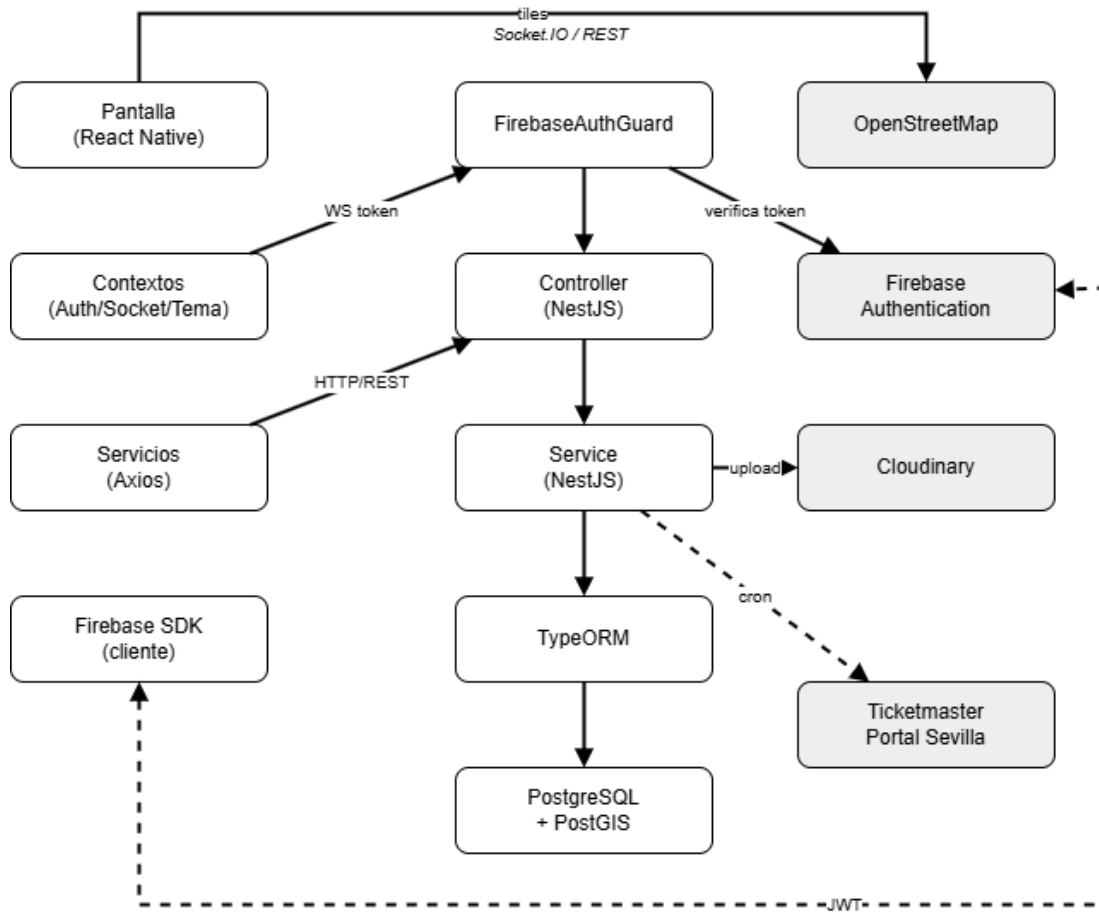


Figura 4.5: Conexión entre tecnologías del sistema en tiempo de ejecución

Una vez descritos los componentes internos y los servicios externos, la figura 4.6 sitúa esos elementos sobre la infraestructura de ejecución utilizada por el sistema.

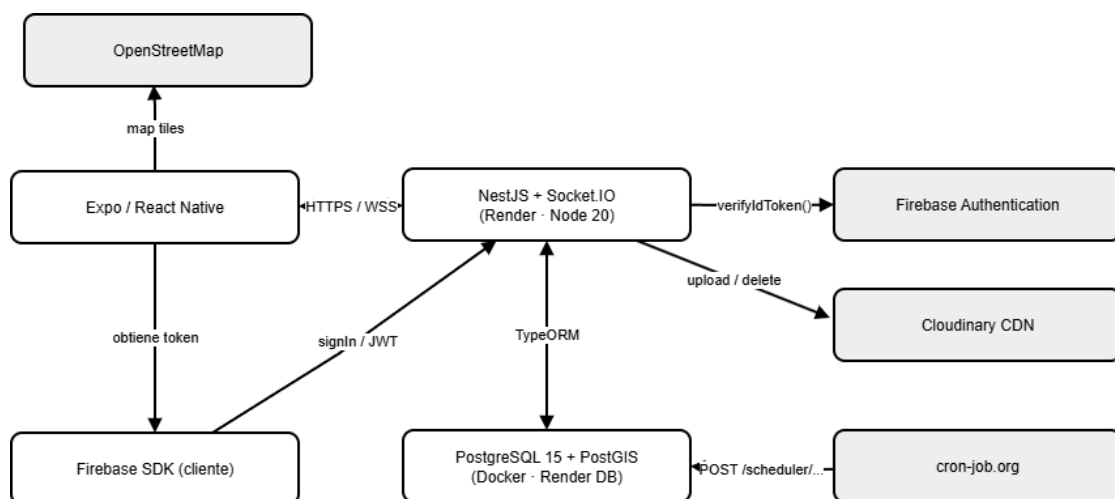


Figura 4.6: Vista de despliegue del sistema Sevillaneando

4.3. Modelo de datos

El modelo de datos del sistema refleja las entidades identificadas en el análisis y las relaciones entre ellas. La tabla 4.1 describe las entidades principales y sus atributos más relevantes.

Tabla 4.1: Entidades del modelo de datos

Entidad	Descripción y atributos principales
User	Representa a cada persona registrada. Atributos: UUID, UID de Firebase, correo, nombre, foto de perfil (fotoPerfilPublicId en Cloudinary), ubicación geográfica (punto PostGIS SRID 4326), rol (user, moderator o admin), intereses por categoría, fecha de aceptación de privacidad (privacyAcceptedAt), orden de categorías personalizado (categoryOrder) y radios de búsqueda configurados (radiusOptions).
Event	Representa un evento cultural o de ocio. Atributos: título, descripción, dirección, ubicación geográfica (punto PostGIS SRID 4326), fechas de inicio y fin, precio fijo o en rango, imagen principal, hasta cinco imágenes adicionales, estado de moderación (Pendiente, Aprobado o Rechazado), visibilidad, enlace de acceso para eventos privados (linkAcceso), recurrencia (diario, semanal, quincenal o mensual), fecha final de recurrencia e indicador hasMultipleDatesAvailable para eventos con varias sesiones.
Categoria	Clasifica los eventos por temática. Contiene nombre y descripción. Solo los administradores pueden crearlas, editarlas o eliminarlas.
Resena	Valoración de un evento por parte de un usuario. Incluye puntuación de 1 a 5 estrellas, comentario opcional, referencia al usuario autor y al evento valorado.
Mensaje	Mensaje del chat grupal de un evento. Incluye contenido de texto, URL de imagen opcional, usuario emisor y referencia al evento.
MensajePrivado	Mensaje directo entre dos usuarios. Contiene contenido, imagen opcional, referencias a emisor y receptor, e indicador de lectura (leído).

Entidad	Descripción y atributos principales
Notificacion	Aviso enviado a un usuario. Incluye tipo (aprobación, rechazo, evento cercano, evento próximo, mensaje privado, nuevo seguidor o recomendación), mensaje, estado de lectura, y referencias auxiliares opcionales al evento relacionado (targetEventId) o al usuario origen (sourceUserId) para navegar directamente a la pantalla correspondiente.
Ruta	Itinerario formado por una secuencia ordenada de eventos. Almacena título, descripción, secuencia de eventos (M:N con Event), trayecto geográfico (array de puntos GeoJSON), duración estimada en minutos, creador, puntuación promedio y número de calificaciones recibidas.
CalificacionRuta	Valoración de un usuario sobre una ruta. Almacena puntuación numérica y fechas de creación y actualización. Restricción de unicidad compuesta (usuario, ruta): cada usuario solo puede calificar una ruta una vez. Se borra en cascada al eliminar el usuario o la ruta.
Recomendacion	Sugerencia personalizada generada por el motor de recomendaciones. Almacena los eventos recomendados y los criterios aplicados
EventEditRequest	Solicitud de edición de un evento público creada por su autor. Almacena únicamente los campos que se quieren modificar, todos opcionales: título, descripción, dirección, ubicación, fechas, precio, visibilidad, enlace de acceso e imágenes. El estado se crea siempre como pendiente y solo el moderador puede cambiarlo a aprobada o rechazada. Si se rechaza, puede almacenarse un motivo opcional. Permite separar el contenido publicado de los cambios pendientes de moderación.

El moderador puede además editar directamente los eventos públicos pendientes antes de aprobarlos o rechazarlos, y supervisar cualquier evento público aprobado desde un listado específico. Esta operación no aplica a eventos privados ajenos, que quedan bajo control exclusivo de su creador.

4.3.1. Diagrama entidad-relación

La figura 4.7 muestra el diagrama entidad-relación del sistema con las relaciones principales entre entidades.

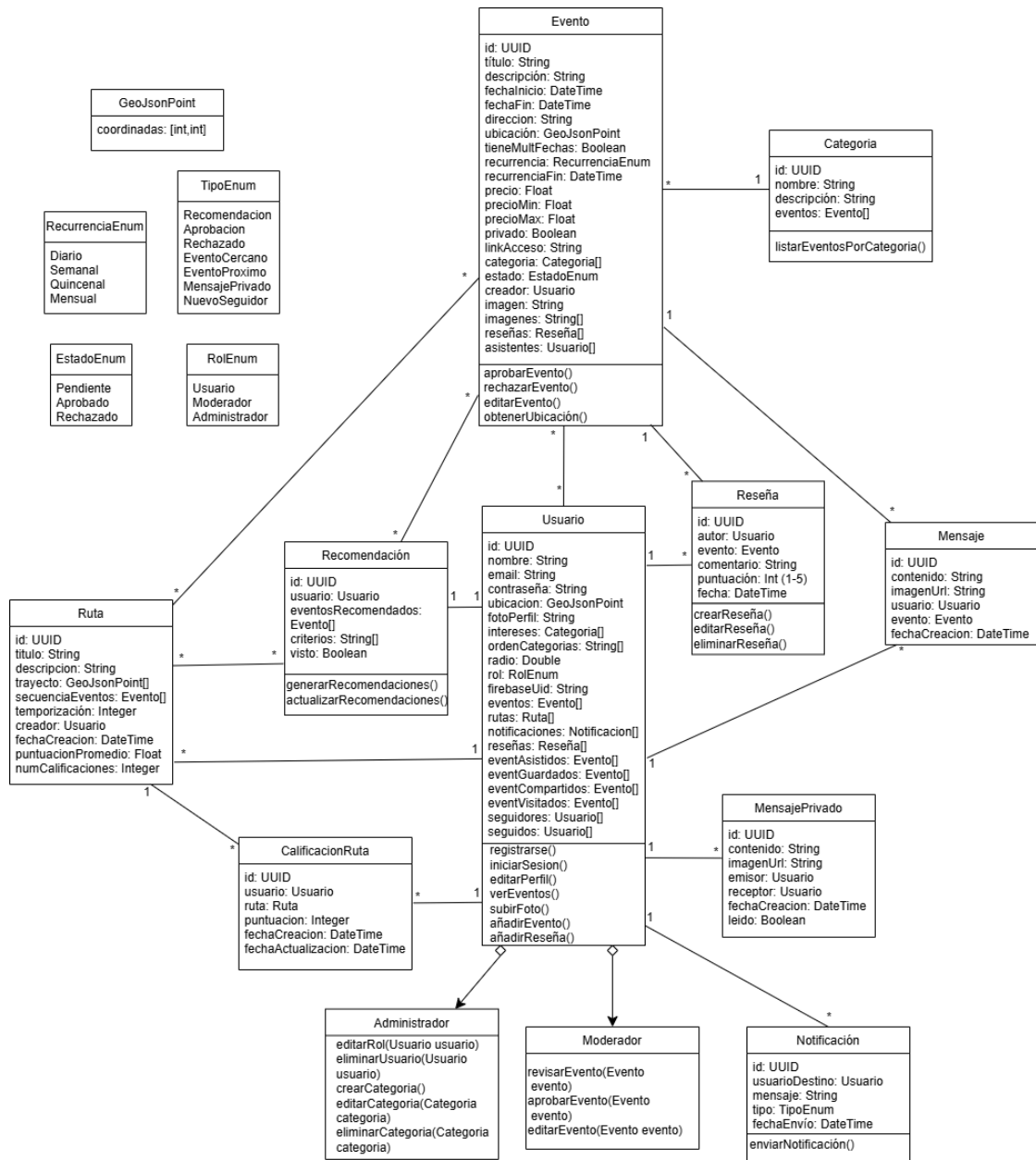


Figura 4.7: Diagrama entidad-relación del sistema

Las relaciones principales entre entidades son las siguientes:

- Un usuario puede crear múltiples eventos (1:N) y múltiples rutas (1:N).
- Un usuario mantiene cuatro relaciones M:N independientes con los eventos: asistencia (event_asistentes), guardado (user_saved_events), compartición (user_shared_events) y visita (user_visited_events).
- Un usuario puede seguir a otros usuarios mediante una relación reflexiva M:N (user_seguidores).
- Un evento pertenece a una categoría (N:1).

- Un evento puede tener múltiples reseñas (1:N) y múltiples mensajes de chat grupal (1:N).
- Dos usuarios cualesquiera pueden intercambiar mensajes privados: cada MensajePrivado referencia a un emisor y a un receptor (N:1 hacia User en ambos extremos).
- Una notificación pertenece a un único usuario (N:1).
- Una ruta contiene una secuencia ordenada de eventos (M:N mediante ruta_eventos) y puede recibir múltiples calificaciones de usuarios (1:N a CalificacionRuta); cada calificación referencia también al usuario autor (N:1).
- Una recomendación pertenece a un usuario (N:1) y agrupa múltiples eventos recomendados para ese usuario (M:N).

4.4. Diseño de la API REST

El backend expone una API REST [50] cuya organización sigue la estructura modular de NestJS [51]. Cada módulo define un conjunto de endpoints agrupados por recurso. La documentación interactiva completa se genera automáticamente mediante @nestjs/swagger siguiendo la especificación OpenAPI [52] y queda disponible en /api/docs cuando el servidor está en ejecución.

4.4.1. Versionado

La API no aplica versionado explícito en la URL (p.ej. /v1/) en esta primera versión, dado que cliente y servidor se desarrollan y despliegan de forma coordinada en el mismo proyecto. Si en el futuro el sistema expusiera la API a clientes externos independientes, se adoptaría versionado mediante prefijo de ruta (/v1/, /v2/) o cabecera Accept-Version, siguiendo la convención habitual de NestJS.

4.4.2. Autenticación y control de acceso

Todos los endpoints protegidos reciben el token de Firebase en la cabecera Authorization: Bearer <token>. Los endpoints que exigen un rol específico aplican un guard adicional de roles. En la tabla 4.2 la columna **Auth** indica el acceso funcional expuesto por la aplicación: — (público), **U** (usuario), **T** (cualquier rol autenticado: usuario, moderador o administrador), **C** (creador o propietario del recurso), **C/M** (creador o moderador sobre evento público), **M** (solo moderador) o **A** (solo administrador).

4.4.3. Endpoints principales

La tabla 4.2 recoge los endpoints de cada módulo extraídos directamente del contrato OpenAPI exportado desde el backend. El contenido fue generado con asistencia de Claude (Anthropic) a partir del YAML del Swagger y revisado por la autora.

Tabla 4.2: Endpoints de la API REST (fuente: OpenAPI 3.0)

Módulo	Método / Ruta	Auth	Parámetros / Cuerpo	Respuestas
Eventos	POST /events	U	Body: CreateEventDto	201, 400, 401
	GET /events	—	Query: userId?, lat?, lng?, radiusKm?, limit?, offset?	200
	GET /events/{id\}	—	Path: id (UUID)	200, 404
	PUT /events/{id\}	C/M	Path: id; Body: UpdateEventDto	200, 400, 401, 404
	DELETE /events/{id\}	C/M	Path: id	200, 401, 403, 404
	PATCH /events/{id\}/aprobar	M	Path: id	200, 401, 403, 404
	PATCH /events/{id\}/rechazar	M	Path: id	200, 401, 403, 404
	GET /events/moderacion/list	M	—	200, 401
	GET /events/moderacion/publicos	M	—	200, 401
	GET /events/acceso/{linkAcceso\}	—	Path: linkAcceso	200, 404
	GET /events/{id\}/private-share-link	C	Path: id	200, 401, 403, 404
	POST /events/upload-image	—	Form: file (binary)	201, 400, 401
	GET /events/{id\}/attendees	—	Path: id	200, 404
	GET /events/{id\}/attendees/me	T	Path: id	200, 401
	POST /events/{id\}/attendees	T	Path: id	201, 401, 404
	DELETE /events/{id\}/attendees	T	Path: id	200, 401, 404
	GET /events/{id\}/reviews	—	Path: id	200
	GET /events/user/{userId\}	—	Path: userId	200
	GET /events/attending/{userId\}	—	Path: userId	200
Usuarios	GET /users/me	T	—	200, 401
	PATCH /users/me/firebase	T	Body: UpdateProfileDto	200, 400, 401
	DELETE /users/me/firebase	T	—	200, 401
	GET /users/search	T	Query: q (mín. 2 car.)	200, 401
	GET /users/admin/stats	A	—	200, 401, 403
	GET /users	M	—	200, 401, 403
	GET /users/{id\}	T	Path: id	200, 401
	DELETE /users/{id\}	A	Path: id	200, 401, 403, 404
	PATCH /users/{id\}/role	A	Path: id; Body: UpdateRoleDto	200, 401, 403, 404
	POST /users/{id\}/seguir	T	Path: id	201, 401
	DELETE /users/{id\}/seguir	T	Path: id	200, 401
	GET /users/{id\}/siguiendo	T	Path: id	200, 401
	GET /users/{id\}/seguidos	T	Path: id	200, 401

Módulo	Método / Ruta	Auth	Parámetros / Cuerpo	Respuestas
	GET /users/{id}/seguidores	T	Path: id	200, 401
	POST /users/upload-profile-image/firebase	T	Form: file (binary)	201, 400, 401
Categorías	GET /categorias	—	—	200
	POST /categorias	A	Body: CreateCategoriaDTO	201, 400, 401, 403
	PUT /categorias/{id}	A	Path: id; Body: CreateCategoriaDTO	200, 400, 401, 403, 404
	DELETE /categorias/{id}	A	Path: id	200, 401, 403, 404
Rutas	GET /rutas	—	Query: userId?	200
	POST /rutas	T	Body: CreateRutaDto	201, 400, 401
	GET /rutas/search	—	Query: q	200
	GET /rutas/{id}	—	Path: id	200, 404
	GET /rutas/{id}/mi-calificacion	T	Path: id	200, 401
	PATCH /rutas/{id}	C	Path: id; Body: UpdateRutaDto	200, 401, 403, 404
	DELETE /rutas/{id}	C	Path: id	200, 401, 403, 404
Recomend.	POST /rutas/{id}/calificar	T	Path: id	201, 400, 401, 404
	GET /recomendaciones/me/events	T	Query: lat?, lng?, radiusKm?, from?, to?, limit?	200, 401
	GET /recomendaciones/me/rutas	T	Query: lat?, lng?, strategy?, limit?	200, 401
	GET /recomendaciones/me/guardados	T	—	200, 401
	POST /recomendaciones/events/{id}/guardar	T	Path: eventId	201, 401, 404
	DELETE /recomendaciones/events/{id}/guardar	T	Path: eventId	200, 401
	POST /recomendaciones/events/{id}/compartir	T	Path: eventId	201, 401
	POST /recomendaciones/events/{id}/visitar	T	Path: eventId	201, 401
	POST /recomendaciones/events/{id}/valorar	T	Path: eventId; Body: RateEventDto	201, 400, 401
Notific.	GET /recomendaciones/events/{id}/valorar/me	T	Path: eventId	200, 401
	GET /notificaciones/usuario/{id}	T	Path: usuarioId	200, 401
	PATCH /notificaciones/{id}/leida	T	Path: id	200, 401, 404
Chat	DELETE /notificaciones/{id}	T	Path: id	200, 401, 404
	POST /chat/upload	T	Form: file (binary)	201, 400, 401
Solic. edición	POST /event-edit-requests/{eventId}/{userId}	—	Path: eventId, userId; Body: EventEditRequestDto	201, 400
	PATCH /event-edit-requests/{requestId}/approve	—	Path: requestId	200, 404
	PATCH /event-edit-requests/{requestId}/reject	—	Path: requestId; Body: motivoRechazo?	200, 404
	GET /event-edit-requests/pending/{eventId}	—	Path: eventId	200
	GET /event-edit-requests	—	—	200
Scraping	POST /scraping/run-all	A	—	200, 401
	POST /scraping/run/{name}	A	Path: scraperName	200, 401
	GET /scraping/scrapers	A	—	200, 401

Módulo	Método / Ruta	Auth	Parámetros / Cuerpo	Respuestas
	POST /scraping/reset	A	—	200, 401, 500
	POST /scraping/delete-all-events	T	— (solo fuera de producción)	200, 403
Scheduler	GET /scheduler/health	—	—	200
	POST /scheduler/run-scraping	—	—	200
	POST /scheduler/run-notifications	—	—	200

La API aplica limitación de velocidad (*throttling*) mediante @nestjs/throttler con tres niveles configurables:

- **Estricto** (5 req/min): operaciones costosas reservadas a administradores, como los endpoints de scraping (POST /scraping/run-all, POST /scraping/reset, POST /scraping/run/:name).
- **Moderado** (20 req/min): escrituras habituales, como la creación de eventos (POST /events).
- **Subida** (10 req/min): carga de imágenes (POST /events/upload-image, POST /chat/upload, POST /users/upload-profile-image/firebase).

4.5. Flujo de interacción del usuario

El usuario ordinario puede registrarse, iniciar sesión, explorar eventos, crear eventos propios, gestionar su asistencia, interactuar con el chat, escribir reseñas y gestionar su perfil. La figura 4.8 detalla estas interacciones principales.

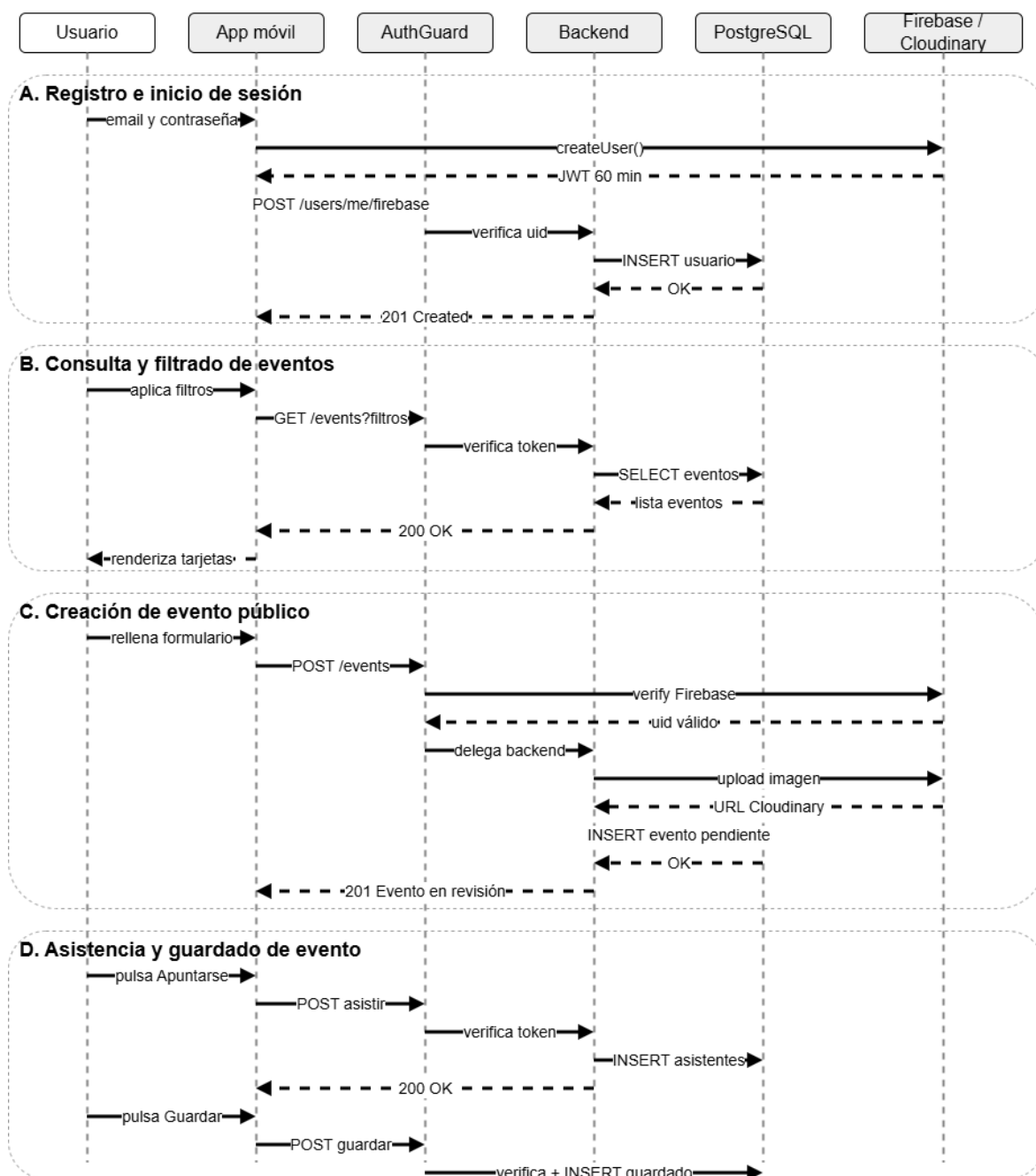


Figura 4.8: Diagrama de secuencias del flujo del usuario (registro, consulta, creación y asistencia)

4.6. Diseño del sistema de moderación

La moderación de eventos públicos es una de las piezas centrales del sistema. El moderador accede al panel de moderación, revisa los eventos en estado **Pendiente**, puede editarlos directamente antes de tomar una decisión, y los aprueba o rechaza. Además, dispone de un listado independiente con los eventos públicos, que puede editar o eliminar si detecta errores, información

desactualizada o contenido inadecuado. La notificación correspondiente se envía al creador cuando el evento pendiente se aprueba o rechaza. La figura 4.9 muestra este flujo.

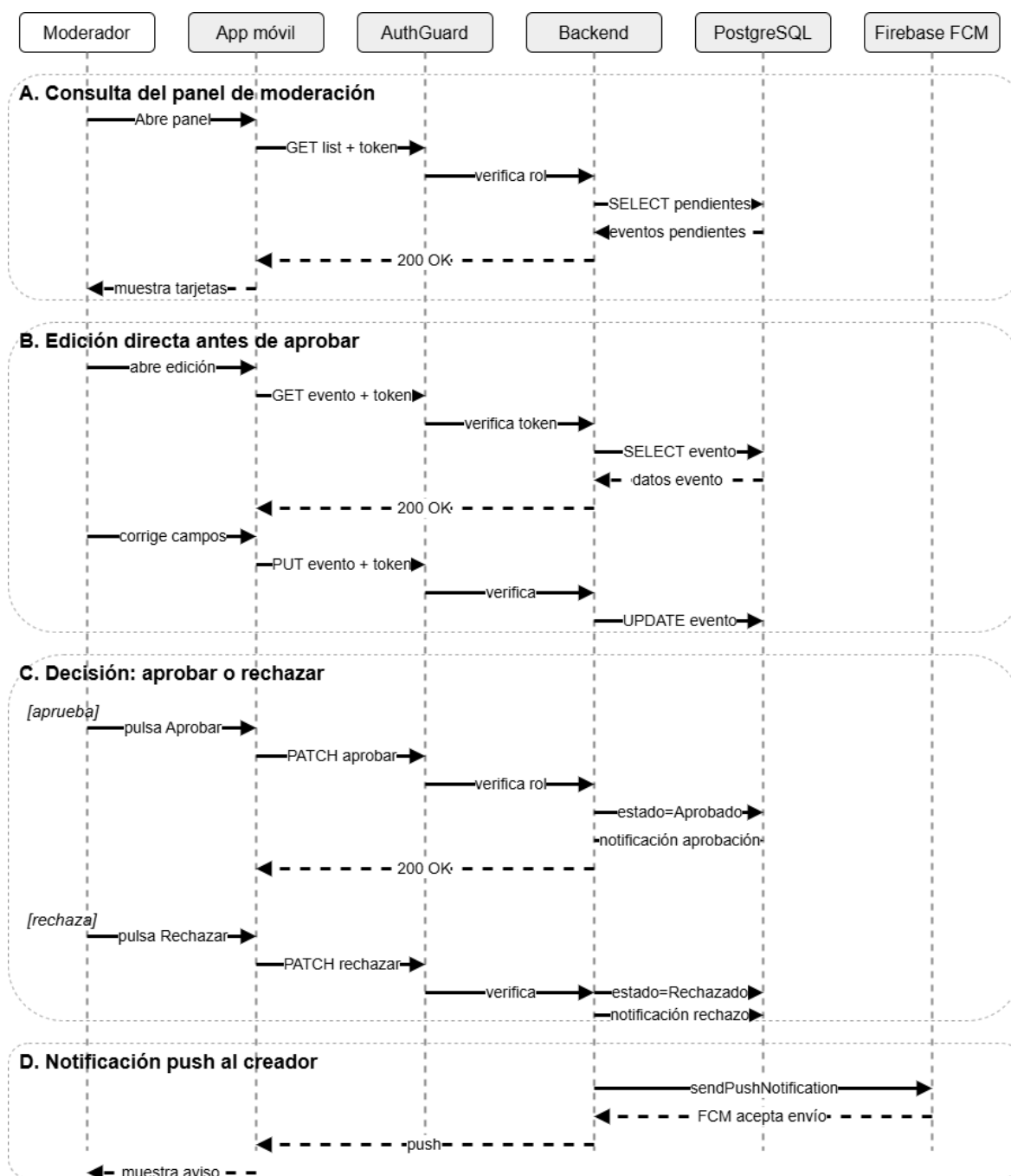


Figura 4.9: Diagrama de secuencias del flujo del moderador

El ciclo de vida de un evento depende principalmente de su visibilidad:

- **Eventos públicos:** al crearse quedan en estado **Pendiente**. No se muestran en el listado general ni en el listado de edición del usuario mientras esperan revisión. El moderador puede corregirlos si detecta errores menores y después

aprobarlos o rechazarlos. Si se aprueban, pasan a estado **Aprobado** y aparecen en la aplicación; si se rechazan, dejan de estar disponibles para los usuarios.

- **Eventos privados:** no pasan por moderación. Se crean directamente como **Aprobados** y solo son accesibles mediante su enlace de invitación. El botón de compartir se sustituye por la acción *invitar*, disponible únicamente para el creador del evento.
- **Ediciones de eventos públicos:** cuando el creador modifica un evento público ya aprobado, el evento vuelve a estado **Pendiente** y deja de mostrarse hasta que el moderador revise los cambios. Si la modificación la realiza un moderador desde su panel, el evento conserva el estado **Aprobado**.
- **Cambios de visibilidad:** si un evento privado pasa a público, entra en moderación como **Pendiente**. Si un evento público pasa a privado, se aprueba automáticamente y se genera un enlace de acceso.

El panel de moderación separa dos tareas: por un lado, la revisión de eventos pendientes; por otro, la supervisión de eventos públicos ya aprobados. Esto permite que el moderador apruebe o rechace nuevo contenido, pero también corrija o elimine eventos públicos que hayan quedado desactualizados o contengan información incorrecta.

4.7. Flujo de interacción del administrador

El administrador gestiona la estructura del sistema: consulta el listado de usuarios, cambia sus roles y administra las categorías de eventos. No interviene en el flujo de moderación de contenido. La figura 4.10 muestra estas interacciones.

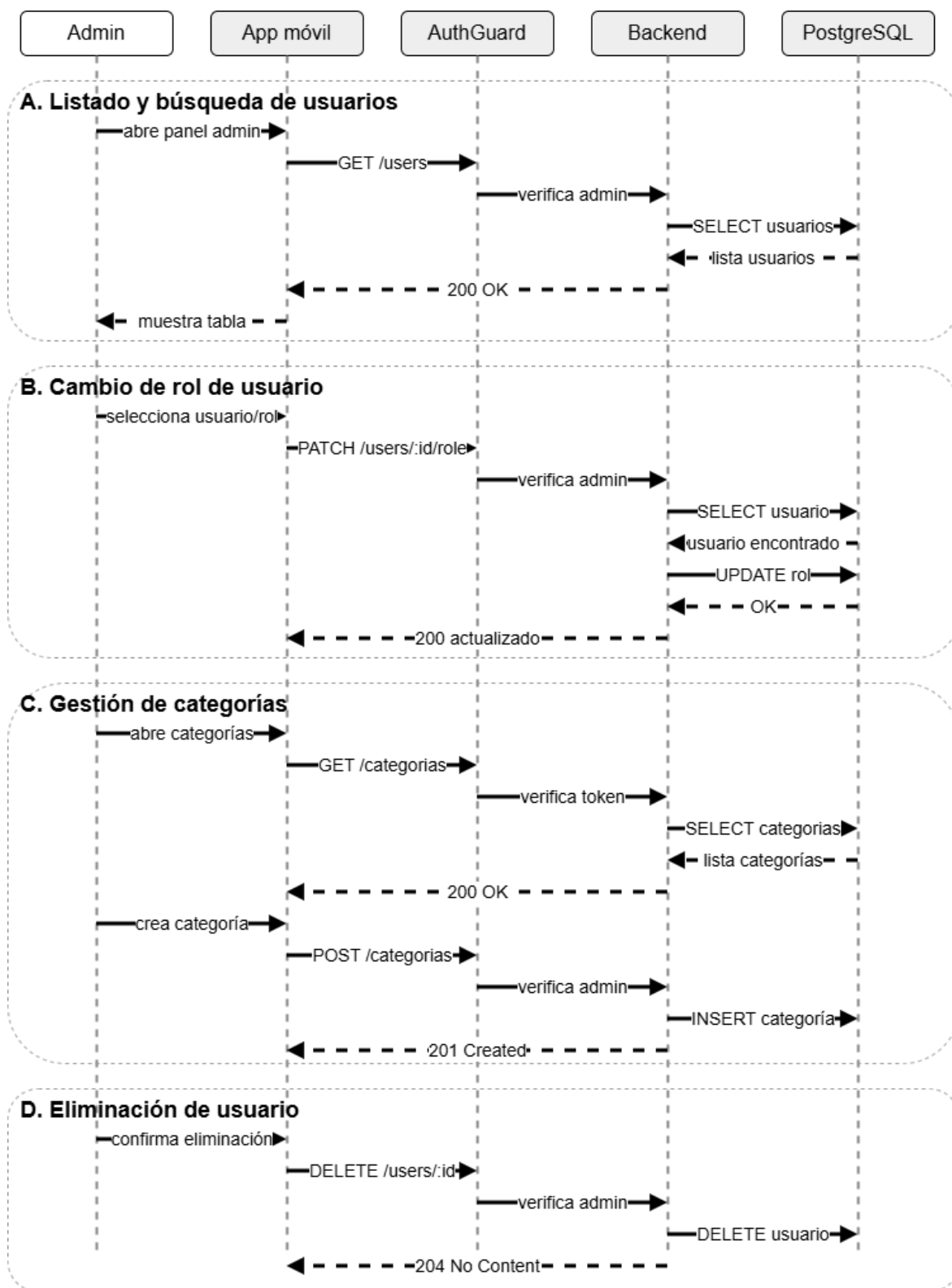


Figura 4.10: Diagrama de secuencias del flujo del administrador

4.8. Diseño de la interfaz de usuario

4.8.1. Principios de diseño

La interfaz de usuario ha sido diseñada siguiendo los principios de usabilidad para aplicaciones móviles:

- **Consistencia visual:** se utiliza un sistema de componentes temáticos (ThemedView, ThemedCard, ThemedButton, ThemedText) que garantizan una apariencia coherente en todas las pantallas, tanto en modo claro como en modo oscuro.
- **Adaptabilidad:** la aplicación soporta modo claro y modo oscuro, adaptándose automáticamente a la preferencia del sistema o permitiendo el cambio manual desde el perfil.
- **Jerarquía de información:** cada pantalla prioriza la información más relevante para el usuario, ocultando las opciones avanzadas o de administración según el rol.
- **Accesibilidad de acciones:** las acciones principales (apuntarse, guardar, compartir) están siempre visibles en el detalle de evento sin necesidad de desplazarse.

4.8.2. Grafo de navegación

La aplicación se divide en dos grupos de pantallas según el estado de autenticación del usuario. La figura [4.11](#) muestra el grafo de navegación completo.

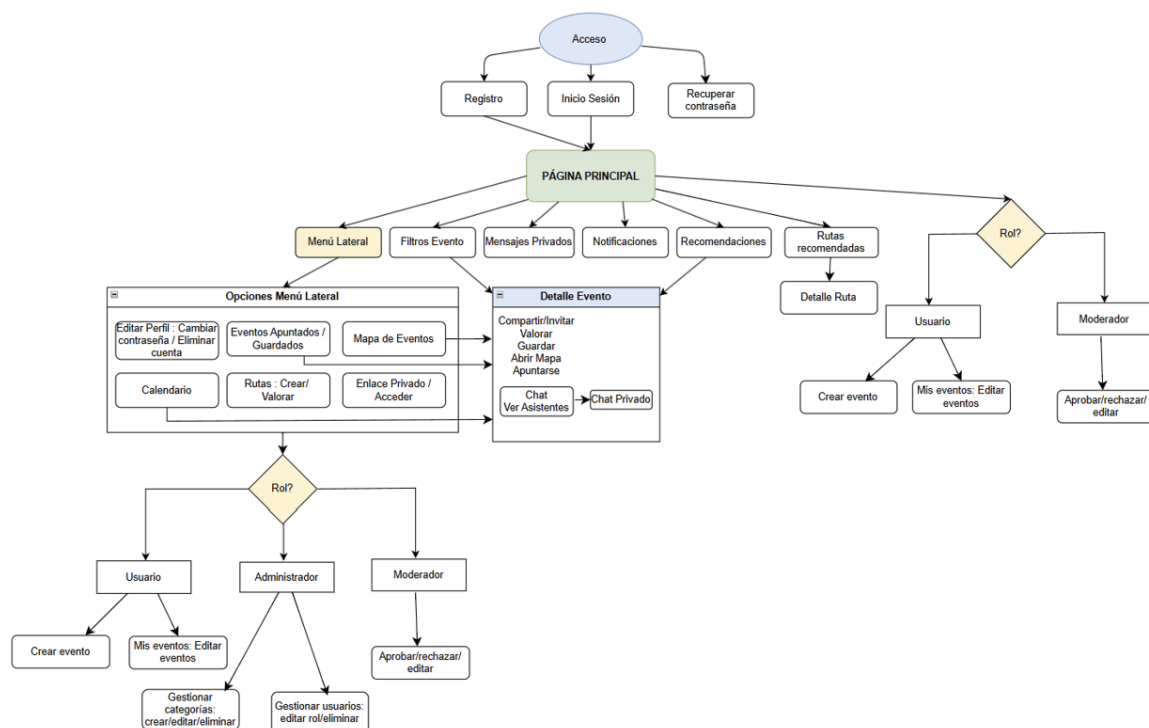


Figura 4.11: Grafo de navegación de la aplicación organizado por rol de acceso

El grupo de pantallas no autenticadas comprende las pantallas de inicio de sesión, registro y política de privacidad. El grupo de pantallas autenticadas incluye todas las funcionalidades de la aplicación, organizadas alrededor de la pantalla principal.

Las pantallas exclusivas de moderador y administrador solo son accesibles si el usuario dispone del rol correspondiente; de lo contrario, no aparecen en el menú lateral ni en ningún punto de entrada de la navegación.

4.8.3. Pantallas del sistema

La tabla 4.3 describe las pantallas del sistema y su accesibilidad según el rol del usuario.

Tabla 4.3: Pantallas del sistema y roles que pueden acceder

Pantalla	Descripción	Acceso
Inicio de sesión	Autenticación con correo y contraseña	No autenticado
Registro	Creación de cuenta nueva	No autenticado
Pantalla principal	Listado de eventos con filtros y recomendaciones	Todos los roles

Pantalla	Descripción	Acceso
Detalle de evento	Información completa, chat, reseñas y asistencia	Todos los roles
Mapa de eventos	Vista cartográfica de los eventos cercanos	Todos los roles
Calendario	Eventos con fecha a los que está apuntado el usuario	Todos los roles
Acceso a evento privado	Acceso a evento privado mediante enlace de invitación	Todos los roles
Guardados y privados	Eventos guardados y eventos privados del usuario	Todos los roles
Crear evento	Formulario de creación de evento público o privado	Usuario
Editar evento	Modificación de un evento propio	Usuario creador
Mis eventos	Listado de eventos creados por el usuario	Usuario
Rutas	Listado de rutas públicas	Todos los roles
Detalle de ruta	Información de la ruta y secuencia de eventos	Todos los roles
Rutas recomendadas	Itinerarios generados automáticamente para el usuario	Todos los roles
Vista previa de ruta	Previsualización de una ruta recomendada	Todos los roles
Crear ruta	Formulario para crear una ruta propia	Todos los roles
Notificaciones	Listado de notificaciones del usuario	Todos los roles
Mensajes	Lista de conversaciones privadas	Todos los roles
Mensaje directo	Chat privado entre dos usuarios	Todos los roles
Perfil de usuario	Vista del perfil de otro usuario, con acciones sociales y eventos a los que asiste	Todos los roles
Amigos	Módulo social con búsqueda de usuarios, seguidores, seguidos y amistades mutuas	Todos los roles
Conexiones	Listado de seguidores y seguidos de un usuario concreto	Todos los roles
Editar perfil	Modificación de datos personales e intereses	Todos los roles
Cambiar contraseña	Modificación de la contraseña de la cuenta	Todos los roles
Panel de administración	Estadísticas globales, gestión de usuarios, roles y scraping	Administrador
Gestión de categorías	Creación, edición y eliminación de categorías	Administrador

Pantalla	Descripción	Acceso
Moderación de eventos	Revisión y aprobación de eventos pendientes	Moderador
Edición moderada	Edición directa de un evento pendiente antes de aprobarlo	Moderador
Ayuda	Preguntas frecuentes y guía de uso de la aplicación	Todos los roles
Política de privacidad	Política de tratamiento de datos personales	Todos
Atribuciones legales	Licencias y atribuciones de recursos de terceros	Todos los roles

4.9. Diseño del sistema de comunicación en tiempo real

El chat de la aplicación está implementado sobre **Socket.IO** [36], integrado en el mismo servidor NestJS que la API REST. Se definen dos canales de comunicación independientes:

- **Chat de evento:** sala identificada por `event:{eventId}` en la que todos los asistentes pueden leer y enviar mensajes.
- **Mensajes privados:** sala personal identificada por `user:{userId}` que enruta los mensajes directos entre dos usuarios.

La conexión WebSocket se autentica con el mismo token de Firebase que los endpoints REST, verificado en el middleware del socket antes de aceptar la conexión. Para prevenir el abuso, el servidor aplica un límite de 10 mensajes por ventana de 10 segundos por conexión.

4.9.1. Ciclo de vida de la conexión y los listeners

La conexión WebSocket se centraliza en el `SocketProvider`. Cuando la aplicación se monta con una sesión activa, el proveedor abre una única conexión Socket.IO autenticada con el token de Firebase y la comparte con el resto de pantallas mediante el contexto de React. La conexión se cierra al desmontarse el proveedor o si pasan 24 horas sin que el cliente envíe mensajes a través de `sendMessage`; cada envío reinicia ese temporizador.

Los *listeners* no se registran de forma global, sino en las pantallas que los necesitan. En el detalle de un evento se emite `join_room` para entrar en la sala del evento y se registran listeners como `chat_history`, `chat_message`, `delete_event_message_success` y `chat_error`. En el chat privado se solicita el historial con `dm_history` y se escuchan `dm_history`, `dm_message` y `delete_dm_success`. Las pantallas que muestran conversaciones emiten

`get_conversations` y escuchan eventos como `conversations`, `refresh_conversations` y `dm_message`.

Al salir de estas pantallas, el `cleanup` de sus `useEffect` elimina los listeners con `socket.off()`. Así se reutiliza la misma conexión durante la navegación, pero se evita acumular manejadores duplicados al entrar y salir de chats o conversaciones.

La figura 4.12 muestra el flujo completo desde el establecimiento de la conexión hasta la recepción del mensaje en otro cliente.

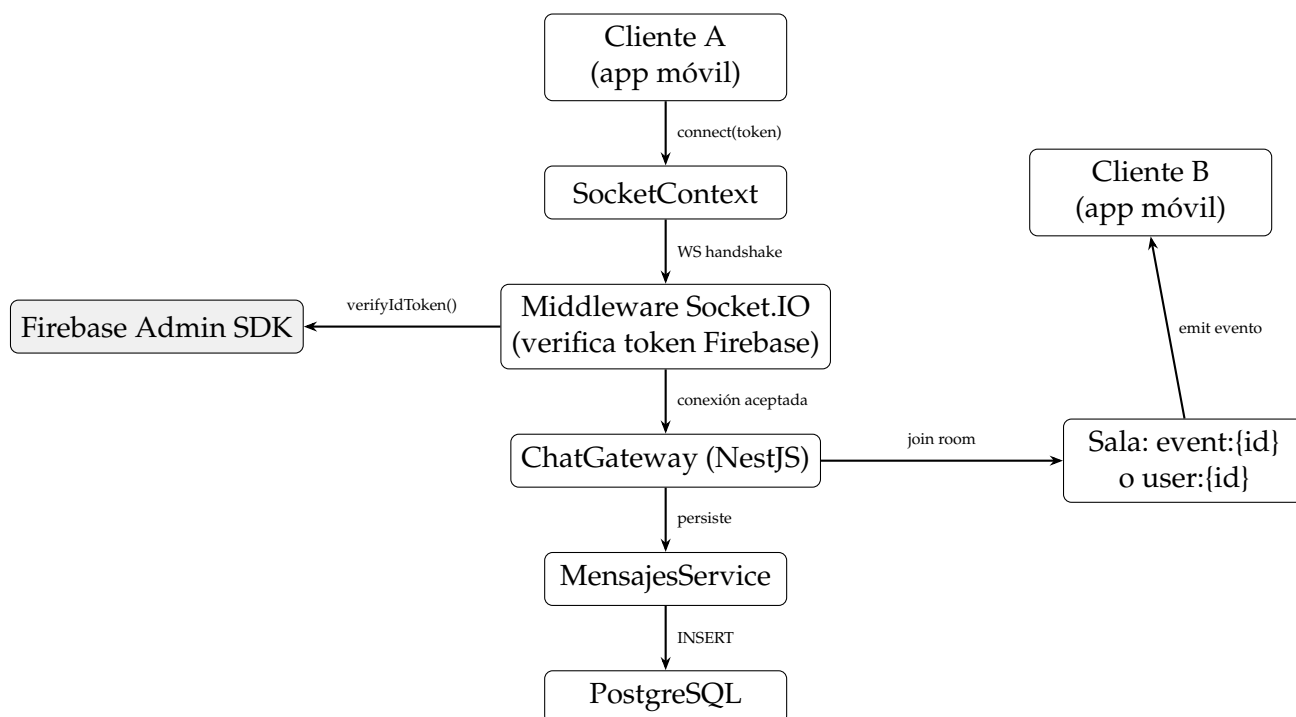


Figura 4.12: Flujo de comunicación en tiempo real mediante Socket.IO: conexión, autenticación y entrega de mensajes

4.10. Diseño del sistema de recomendaciones

El motor de recomendaciones es un componente del backend que genera listas de eventos personalizadas para cada usuario. Su diseño se basa en un algoritmo de puntuación multifactorial [53]: cada evento candidato recibe una puntuación calculada como suma ponderada de varios factores relacionados con el comportamiento del usuario, sus intereses declarados, la proximidad temporal y la proximidad geográfica.

Los factores con mayor peso en el diseño son la afinidad por categoría (hasta 12 puntos), el haber guardado previamente el evento (+10) y la proximidad temporal al evento (hasta 8 puntos). Los eventos se ordenan de mayor a menor puntuación y se devuelven los más relevantes.

Cabe destacar que este algoritmo de *scoring* multifactorial no es un patrón de

diseño de software en el sentido estructural o de comportamiento (GoF), sino un algoritmo de dominio propio del problema de recomendación. Su característica clave es que cada factor es independiente y ponderado: modificar el peso de un criterio o incorporar uno nuevo no afecta al cálculo del resto, lo que facilita su ajuste y extensión.

La figura 4.13 muestra las entradas que consulta el motor y el proceso de puntuación hasta producir la lista final.

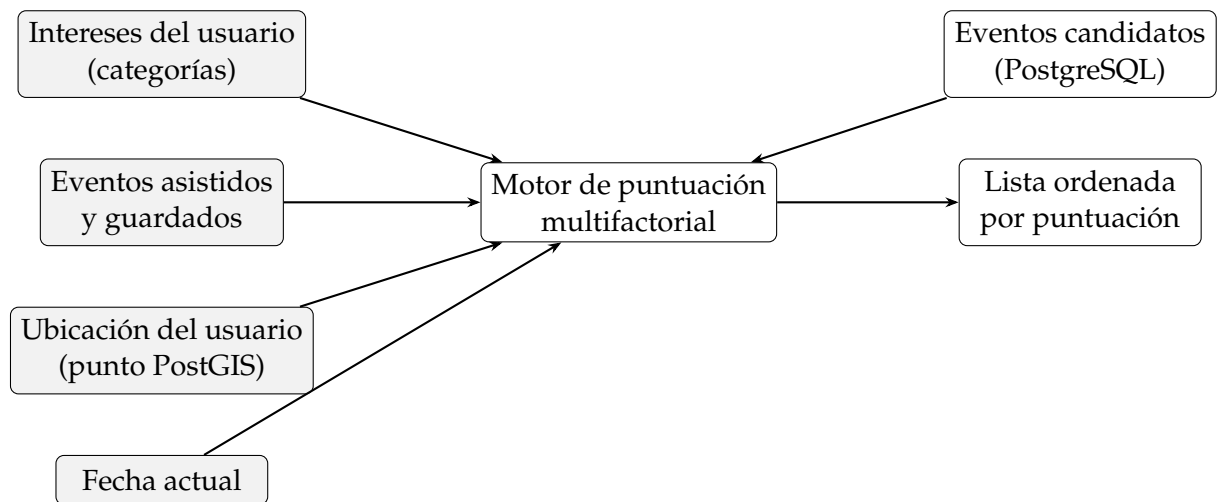


Figura 4.13: Entradas y proceso del motor de recomendaciones

El mismo motor puede generar rutas recomendadas aplicando tres estrategias seleccionables:

- **balanced**: equilibra puntuación y distancia recorrida entre eventos.
- **walkable**: minimiza la distancia total del itinerario.
- **score**: maximiza la puntuación agregada de los eventos incluidos.

4.11. Patrones de diseño aplicados

A lo largo del desarrollo de **Sevillaneando** se han aplicado varios patrones de diseño reconocidos [54], algunos inherentes a los frameworks elegidos y otros aplicados de forma deliberada. La tabla 4.4 resume los más relevantes.

Tabla 4.4: Patrones de diseño aplicados en el sistema

Patrón	Dónde se aplica	Por qué
Módulo	Backend NestJS	Cada dominio (<i>EventosModule</i> , <i>UsuariosModule</i> , etc.) encapsula controlador, servicio y entidades. Facilita el mantenimiento y la extensión sin afectar a otros módulos.
Repositorio [55]	TypeORM	Los servicios acceden a la base de datos a través de repositorios tipados, desacoplando la lógica de negocio del SQL y permitiendo sustituirlos por dobles en pruebas unitarias.
Guard	FirebaseAuthGuard, RolesGuard	Centraliza la autenticación y la autorización interceptando cada petición antes del controlador, evitando duplicar esa lógica en cada endpoint.
Interceptor	Cliente Axios (frontend)	Injecta automáticamente el token Firebase en cada petición y aplica reintentos ante errores de red, sin repetir esa lógica en cada llamada al servicio.
Contexto / Proveedor	AuthContext, SocketContext, ThemeContext, Notificaciones Context	Gestiona el estado global del frontend evitando <i>prop drilling</i> . La separación en cuatro contextos independientes evita rerenders innecesarios entre dominios.
Estrategia	IScraper	Cada fuente externa implementa la misma interfaz; el orquestador trabaja con la abstracción. Permite añadir nuevas fuentes sin modificar la lógica de orquestación.
Observador	Socket.IO (chat)	El servidor emite eventos a las salas y los clientes suscritos los reciben automáticamente, desacoplando al emisor del receptor.
Estado	Flujo de moderación de eventos	El ciclo de vida del evento (Pendiente → Aprobado / Rechazado) tiene transiciones controladas; no todas son válidas, lo que garantiza la coherencia del flujo.

4.12. Decisiones de diseño relevantes

A lo largo del proceso de diseño se tomaron una serie de decisiones que merecen una justificación explícita, ya que no son evidentes a partir de los

requisitos y condicionan la arquitectura y el comportamiento de la aplicación.

4.12.1. Roles, permisos y responsabilidades

Este bloque agrupa las decisiones relacionadas con el reparto de responsabilidades entre usuario, moderador y administrador, así como con las operaciones que pueden afectar al contenido creado por otros usuarios.

Por qué tres roles y no más ni menos

El sistema define tres roles: **usuario**, **moderador** y **administrador**. Esta jerarquía mínima responde a la necesidad de separar tres tipos de responsabilidad claramente diferenciados:

- El **usuario** es el actor principal del sistema: crea, consulta y consume contenido. No tiene autoridad sobre el contenido de otros ni sobre la configuración del sistema.
- El **moderador** actúa como filtro de calidad: revisa y valida eventos antes de que sean visibles públicamente, y también puede editar eventos públicos cuando sea necesario corregir o completar su información. Requiere más permisos que un usuario ordinario, pero no debe tener acceso a la gestión del sistema, lo que delimita claramente su responsabilidad.
- El **administrador** gestiona la estructura del sistema (roles y categorías) pero no interviene en el contenido concreto. Esta separación evita que una única persona acumule tanto la capacidad de gestionar usuarios como la de aprobar contenido, reduciendo el riesgo de abusos y errores.

Añadir más roles (por ejemplo, un *superadmin* o un *editor*) habría incrementado la complejidad de la gestión de permisos sin aportar valor real al caso de uso concreto de la aplicación. Un único rol con todos los permisos habría eliminado la separación de responsabilidades que protege la integridad del contenido y del sistema.

Por qué la eliminación de eventos es explícita y restringida

La eliminación de eventos se realiza mediante el endpoint `DELETE /events/{id}`, protegido con autenticación Firebase. Aunque el endpoint no declara un rol concreto en el controlador, el servicio vuelve a comprobar los permisos antes de borrar: el creador puede eliminar sus propios eventos, y los usuarios con rol de moderador o administrador solo pueden eliminar eventos que no sean privados. Por tanto, no es una acción automática ni accesible para cualquier usuario autenticado.

- **Control de autoría:** el creador conserva la capacidad de retirar un evento propio que ya no desea mantener publicado o accesible.

- **Control de calidad:** el moderador puede eliminar eventos no privados cuando detecta contenido inadecuado, duplicado o incorrecto.
- **Reducción de riesgo:** los eventos privados ajenos quedan fuera del alcance de moderadores y administradores para preservar su naturaleza restringida.

En la interfaz, la eliminación aparece en la pantalla de edición del creador y en la pantalla de edición de moderación, siempre con confirmación previa. Al ejecutarse, el backend elimina primero relaciones asociadas al evento, como asistentes, mensajes y reseñas, y después borra el propio evento.

4.12.2. Moderación y ciclo de vida de eventos

Las siguientes decisiones explican cómo se conserva el historial de eventos, cómo intervienen moderadores y administradores y qué reglas evitan que el contenido publicado dependa de acciones automáticas poco transparentes.

Por qué los eventos finalizados no se borran

Un evento finalizado sigue teniendo valor informativo y social dentro del sistema:

- Los usuarios que asistieron pueden seguir leyendo el chat del evento y las reseñas dejadas por otros asistentes.
- Las reseñas escritas sobre el evento permanecen accesibles y contribuyen a la reputación del creador del evento.
- Las rutas que incluyen ese evento mantienen su coherencia: una ruta cultural puede referenciar un evento ya celebrado como parte del itinerario.
- El historial de eventos pasados permite al motor de recomendaciones analizar patrones de asistencia y mejorar las sugerencias futuras.

Borrar eventos finalizados de forma automática habría reducido la utilidad histórica y social del sistema sin beneficio claro para el usuario.

Por qué el moderador puede editar un evento directamente antes de aprobarlo

El rol del moderador no se limita a aprobar o rechazar: puede detectar que un evento tiene un error menor (fecha incorrecta, descripción incompleta, categoría equivocada) que no justifica el rechazo completo. Rechazar ese evento obliga al creador a corregirlo y volver a someterlo a moderación, lo que genera fricción innecesaria.

La pantalla de edición moderada permite al moderador corregir directamente los campos necesarios y publicar el evento en un único paso. Esto:

- Reduce el número de ciclos de moderación para eventos con errores menores.

- Agiliza la publicación de contenido de calidad sin depender de la disponibilidad del creador para aplicar los cambios.
- Concentra en el moderador la responsabilidad de garantizar que el contenido publicado cumple los estándares del sistema.

Por qué el administrador gestiona la configuración y la operación del sistema

El administrador tiene un alcance orientado a la configuración y operación general de la plataforma. Desde la aplicación puede consultar estadísticas globales, gestionar usuarios y roles, administrar las categorías de eventos y restablecer los eventos procedentes del sistema de scraping. En cambio, no participa en la revisión diaria del contenido desde el flujo principal de moderación. Esta separación es deliberada:

- **Separación de poderes:** el administrador configura quién puede hacer qué y mantiene los datos estructurales del sistema, pero la revisión habitual de eventos queda delegada en el moderador.
- **Supervisión operativa:** el panel de estadísticas permite conocer el estado global de la plataforma, incluyendo usuarios registrados, eventos por estado y eventos obtenidos mediante scraping.
- **Mantenimiento del scraping:** el restablecimiento del scraping permite eliminar y regenerar eventos importados automáticamente cuando las fuentes externas cambian o se necesita refrescar el catálogo.
- **Principio de mínimo privilegio:** aunque el administrador dispone de permisos técnicos amplios, la interfaz separa sus tareas de gestión y mantenimiento de las tareas editoriales propias del moderador.

4.12.3. Comunicación, tiempo real y notificaciones

Este bloque reúne las decisiones relacionadas con el chat, la conexión WebSocket y las notificaciones generadas automáticamente.

Por qué el chat incluye persistencia y no es solo en tiempo real

El chat de evento está implementado con **Socket.IO** para la transmisión en tiempo real, pero todos los mensajes se persisten en base de datos (entidades `Mensaje` y `MensajePrivado`). Esta decisión responde a dos necesidades diferenciadas: el valor social del chat antes y durante el evento, y su utilidad después de que este haya terminado.

Antes y durante el evento, el chat es el espacio donde los asistentes pueden coordinarse: quedar para ir juntos, resolver dudas sobre el lugar o el horario, compartir fotos o simplemente conocer a otras personas que van al mismo plan. Para que esto sea posible, el historial debe estar disponible para cualquiera que se

una al chat más tarde; sin persistencia, quien llegara después solo vería el estado actual de la conversación y perdería todo el contexto previo. Lo mismo aplica a los mensajes privados: si el receptor no está conectado en el momento del envío, el mensaje se perdería sin persistencia.

Después del evento, el chat conserva el mismo valor que el resto del historial del evento: otros usuarios pueden consultar la conversación para saber cómo fue, qué opinaron los asistentes o si merece la pena acudir a futuras ediciones. Esto conecta directamente con las razones por las que los eventos tampoco se eliminan del sistema.

Por qué las notificaciones programadas se separan en tres métodos

El sistema no define tres clases scheduler independientes, sino una clase `NotificacionesScheduler` con tres métodos diferenciados. Cada método cubre un caso de notificación distinto: eventos cercanos en el espacio (radio de 1 km), eventos próximos en el tiempo (7 días) y eventos con menos de 24 horas. El endpoint `POST /scheduler/run-notifications` ejecuta los tres métodos de forma conjunta, pero la lógica permanece separada para que cada criterio pueda revisarse, probarse y modificarse de forma independiente.

Por qué la conexión WebSocket se autentica en el *handshake* y no por mensaje

La autenticación del socket se realiza en el middleware de conexión de `Socket.IO`, antes de que se establezca la sala y se procese cualquier mensaje. Si la autenticación fallara por mensaje (es decir, si se aceptara la conexión y se verificara el token en el primer evento recibido), existiría una ventana de tiempo en la que un cliente no autenticado tendría una conexión activa y podría enviar eventos al servidor. Verificar en el *handshake* garantiza que ninguna conexión sin token válido llega a establecerse, simplificando además la lógica de cada handler de eventos, que puede asumir que el socket ya está autenticado.

Por qué el historial de chat se limita a los últimos 50 mensajes

Al incorporarse a la sala de chat de un evento, el servidor envía al cliente los últimos 50 mensajes almacenados. Un límite superior habría aumentado el tamaño del payload inicial y el tiempo de carga de la pantalla de detalle de evento, especialmente en eventos con chats muy activos. Un límite inferior habría privado al usuario de contexto suficiente para entender la conversación reciente. El valor de 50 mensajes representa un equilibrio razonable entre utilidad y rendimiento para el caso de uso típico de la aplicación.

4.12.4. Visibilidad, recurrencia y datos geográficos

Este bloque recoge las decisiones que afectan a la forma en que se publican, comparten y consultan los eventos dentro del modelo de datos.

Por qué los eventos privados se aprueban automáticamente

Cuando un usuario crea un evento privado, el sistema le asigna directamente el estado **Aprobado** sin pasar por el flujo de moderación. La razón es que los eventos privados no son visibles en el listado público: solo acceden a ellos las personas que disponen del enlace de invitación. El objetivo de la moderación es garantizar la calidad del contenido que el público en general puede encontrar; un evento cuyo alcance está restringido a las personas que el creador decide invitar no supone un riesgo para la comunidad y no requiere revisión externa.

Someter los eventos privados a moderación habría generado una experiencia confusa: el creador crearía un evento para compartir con amigos y tendría que esperar aprobación antes de poder enviar el enlace. Esta fricción no se corresponde con el modelo de uso esperado.

Por qué los eventos privados usan un enlace UUID en lugar de una lista de control de acceso

El acceso a eventos privados se implementa mediante un enlace único generado automáticamente (UUID v4) en lugar de una lista explícita de usuarios autorizados. Esta decisión responde a varias consideraciones:

- **Simplicidad de uso:** el creador puede compartir el enlace por cualquier canal externo (mensaje, correo, red social) sin necesidad de que los invitados tengan cuenta en la aplicación o deban ser añadidos manualmente.
- **Flexibilidad:** el creador decide el canal y el momento del reparto sin intervención del sistema.
- **Sin fricción para el invitado:** no se requiere autenticación para acceder al detalle del evento por enlace, lo que facilita que alguien consulte los datos antes de registrarse.

La contrapartida es que quien obtiene el enlace puede reenviarlo. Este riesgo es aceptable en el contexto de la aplicación, ya que los eventos privados son de naturaleza social e informal.

Por qué los eventos recurrentes generan instancias independientes en base de datos

Cuando se crea un evento con recurrencia (diaria, semanal, quincenal o mensual), el sistema genera un registro independiente en base de datos por cada

ocurrencia hasta la fecha de fin de recurrencia. La alternativa habría sido almacenar una regla de recurrencia única y calcular las ocurrencias en tiempo de consulta.

Se eligió la generación de instancias por las siguientes razones:

- Cada instancia puede tener su propio estado de moderación, chat, reseñas y lista de asistentes de forma independiente. Con una regla única esto sería mucho más complejo de gestionar.
- Las consultas espaciales y temporales de PostGIS operan sobre registros concretos. Calcular ocurrencias en tiempo de consulta habría requerido lógica adicional dentro de las queries geoespaciales, complicando el motor de filtrado.
- La coherencia con el resto del modelo de datos es total: una instancia recurrente es un evento como cualquier otro para el resto del sistema (recomendaciones, rutas, notificaciones).

La desventaja es que la base de datos crece proporcionalmente al número de ocurrencias generadas, pero dado el alcance de la aplicación este coste es despreciable.

Por qué se usa PostGIS con tipo *geography* y SRID 4326

Las ubicaciones de usuarios y eventos se almacenan como puntos *geography* de PostGIS con sistema de referencia SRID 4326 (WGS84), que es el mismo sistema que utilizan el GPS y la mayoría de servicios cartográficos. Se eligió el tipo *geography* frente al tipo *geometry* porque *geography* trabaja directamente en coordenadas esféricas: las funciones *ST_DWithin* y *ST_Distance* devuelven distancias en metros sin necesidad de reproyectar, lo que simplifica las consultas de filtrado por radio y elimina el error de distorsión que aparece con *geometry* en coordenadas geográficas.

La alternativa de almacenar latitud y longitud como dos columnas numéricas habría impedido aprovechar los índices espaciales de PostGIS (GIST), lo que habría degradado significativamente el rendimiento de las consultas de filtrado por proximidad a medida que crece el número de eventos.

4.12.5. Automatización y fuentes externas

Las siguientes decisiones justifican cómo se obtienen eventos desde fuentes externas y cómo se planifican las tareas periódicas fuera de la interacción directa del usuario.

Por qué el scraping se ejecuta a las 3:00 AM

La tarea de obtención de eventos externos (Eventbrite, Ticketmaster y webs institucionales y de ocio procesadas con Gemini AI) se programa a las 3:00 AM

mediante un *cron job* diario. Esta hora se eligió porque es el período de menor actividad de usuarios en la aplicación, lo que reduce la competencia por recursos del servidor y de la base de datos durante el proceso de scraping. Además, ejecutarlo una vez al día es suficiente para mantener el catálogo actualizado: los eventos de ocio y cultura no cambian con frecuencia mayor a la diaria.

Por qué el cron job es externo y no interno al backend

La planificación de las tareas periódicas (scraping diario y envío de notificaciones) se gestiona mediante un servicio de *cron* externo (cron-job.org) [56], basado en el formato cron estándar [57], que invoca los endpoints POST /scheduler/run-scraping y POST /scheduler/run-notifications, en lugar de usar el planificador interno de NestJS (@nestjs/schedule).

La razón principal es la infraestructura de despliegue: el backend se aloja en Render con el plan gratuito, que suspende el proceso tras 15 minutos de inactividad. Cuando el proceso está suspendido, el planificador interno de NestJS no corre; y cuando Render lo reactiva ante una nueva petición, el planificador se reinicia desde cero. Esto hace que los @Cron internos sean poco fiables: la tarea diaria de las 3:00 AM o la de notificaciones horarias podrían no ejecutarse si el proceso lleva inactivo más de 15 minutos antes de la hora programada.

Un *cron* externo es independiente del ciclo de vida del proceso del servidor: aunque el backend esté suspendido, la petición del *cron* externo lo reactiva y ejecuta la tarea. Además, el servicio de *keep-alive* (que llama periódicamente a GET /scheduler/health) se gestiona también desde el mismo sistema externo, lo que centraliza en un único panel el control de todas las tareas programadas y su historial de ejecuciones.

El *cron* interno de NestJS sería la opción preferida si el backend corriera de forma continua en un servidor dedicado o en un plan de pago con instancias siempre activas: en ese caso eliminaría la dependencia de un servicio externo y la latencia de la petición HTTP entrante. Para el contexto actual de la aplicación, con infraestructura gratuita, el *cron* externo es la solución más fiable.

Por qué los scrapers siguen el patrón estrategia con interfaz IScraper

Cada fuente de eventos externos (Eventbrite, Ticketmaster, Visita Sevilla y webs institucionales y de ocio de Sevilla procesadas con Gemini AI) está implementada como un scraper independiente que implementa la interfaz IScraper. Esto permite añadir nuevas fuentes de datos sin modificar la lógica de orquestación del servicio de scraping. La alternativa de implementar el scraping como código monolítico habría acoplado la lógica de cada fuente con el proceso de guardado en base de datos, dificultando el mantenimiento cuando alguna de las fuentes externas cambia su estructura.

Además, todos los eventos scraped se asignan a un único usuario técnico del sistema (*scraper-bot*), centralizado con un UID y correo fijos. Esto simplifica la

trazabilidad: es inmediato identificar qué eventos provienen de fuentes externas y quién es su “creador” a efectos del modelo de datos.

Por qué se ha implementado scraping en lugar de usar únicamente APIs oficiales

El contenido de eventos de la aplicación proviene de dos grandes bloques: eventos creados por usuarios y eventos obtenidos automáticamente de fuentes externas. Para estas últimas se utilizan cuatro estrategias complementarias:

- **Scraping HTML con JSON-LD (Eventbrite):** el `SevillaScrapperService` descarga la página de eventos de Sevilla en Eventbrite mediante `axios` y parsea el HTML con `cheerio`, extrayendo tarjetas de eventos y bloques JSON-LD estructurados embebidos en la página.
- **API oficial de Ticketmaster:** el `TicketmasterScrapperService` consume la API Discovery de Ticketmaster paginando resultados en bloques de hasta 100 eventos, filtrando por ciudad y fecha.
- **Scraping dedicado de Visita Sevilla:** el `VisitaSevillaScrapperService` descarga la agenda de `visitasevilla.es` y parsea su HTML con `cheerio`. Se ha implementado un scraper específico porque la web utiliza un formato de fecha propio (DD.MM.YY y rangos DD.MM.YY-DD.MM.YY) que no puede interpretarse correctamente mediante extracción genérica con IA. El scraper descarta los placeholders de eventos recurrentes sin fecha concreta y marca como *consultar fechas* los rangos que superan los catorce días, evitando así persistir duraciones ficticias.
- **Extracción asistida por IA (Gemini):** el `GeminiScrapperService` accede a varias webs de ocio e institucionales de Sevilla (portal municipal, Teatro de la Maestranza, Fundación Cajasol, guías de ocio locales, entre otras), limpia el HTML con `cheerio`, y envía el texto resultante al modelo **Gemini 2.5 Flash** de Google, que extrae y estructura los eventos en formato JSON. Las coordenadas no deducidas por el modelo se resuelven mediante geocodificación con Nominatim. Esto permite integrar fuentes con estructuras HTML variables o sin patrón fijo, que serían inviables de parsear con selectores estáticos.

La razón de esta combinación de estrategias es la cobertura: Ticketmaster y Eventbrite recogen principalmente grandes eventos comerciales, pero la oferta cultural de Sevilla incluye multitud de actividades locales (ferias de barrio, exposiciones, talleres, mercadillos, programación de teatros) que solo están publicadas en webs institucionales sin API pública ni estructura HTML uniforme. Las webs con formato HTML predecible y propio, como Visita Sevilla, disponen de scrapers dedicados más precisos; el resto se integran mediante Gemini AI, lo que hace el sistema robusto ante cambios de estructura. El patrón `IScraper` garantiza que añadir nuevas fuentes es una tarea acotada sin tocar la lógica central.

4.12.6. Estado del frontend y experiencia de usuario

Este bloque agrupa decisiones de interfaz y cliente móvil: gestión de estado, preferencias locales, experiencia visual y navegación.

Por qué se usan contextos de React en lugar de una librería de estado global

La gestión de estado del frontend se implementa con los contextos de React (AuthContext, ThemeContext, SocketContext, NotificacionesContext) en lugar de librerías especializadas como Redux o Zustand. Los motivos son:

- El estado global de la aplicación es reducido y bien delimitado: autenticación, tema visual, conexión de socket y notificaciones. Estos estados no tienen interdependencias complejas ni requieren middleware de efectos secundarios.
- Los contextos de React son suficientes para este volumen de estado compartido sin introducir dependencias externas ni curvas de aprendizaje asociadas.
- La separación en cuatro contextos independientes evita rerenders innecesarios: un cambio en el tema no provoca la actualización de los componentes suscritos al contexto de notificaciones.

Por qué la preferencia de tema se guarda en AsyncStorage y no en el servidor

La preferencia de modo claro u oscuro se persiste en AsyncStorage (almacenamiento local del dispositivo) y no en la base de datos del servidor. Esta elección obedece a que la preferencia de tema es una configuración estrictamente local del dispositivo: no tiene sentido sincronizarla entre dispositivos diferentes del mismo usuario (quien puede preferir modo oscuro en el móvil y modo claro en otro entorno) y no requiere estar disponible en el servidor para ningún proceso de negocio. Guardarla localmente elimina una llamada de red en cada inicio de sesión y no genera dependencia del servidor para que la interfaz cargue con el tema correcto.

Por qué las imágenes del evento usan un campo principal y un array adicional

La entidad Event tiene un campo imagen (imagen principal, cadena de texto con URL) y un campo imagenes (array serializado en JSON con hasta 5 imágenes adicionales). La imagen principal es la que se muestra en los listados y tarjetas de eventos, donde el rendimiento es crítico y no tiene sentido deserializar un array para obtener solo una URL. Las imágenes adicionales solo se cargan en la pantalla de detalle. Esta separación evita una deserialización innecesaria en la mayoría de las consultas y simplifica el código de los componentes de listado.

4.12.7. Seguridad, administración y reglas de acceso

Este bloque reúne decisiones de seguridad, autenticación y administración que condicionan quién puede ejecutar cada operación sensible.

Por qué el token de Firebase se refresca cada 50 minutos en el frontend

Los tokens de Firebase tienen una validez de 60 minutos. El frontend programa un refresco automático cada 50 minutos, dejando un margen de 10 minutos antes de la expiración. Refrescar bajo demanda (cuando la API devuelve 401) habría generado una mala experiencia de usuario: la primera petición tras la expiración fallaría, el cliente debería detectar el error, obtener un nuevo token y reintentar la llamada, con la consiguiente latencia percibida y la posibilidad de que alguna operación de escritura se perdiera en el proceso. El refresco proactivo garantiza que el token siempre sea válido sin interrumpir el flujo de uso.

Por qué solo el administrador puede crear y gestionar categorías

Las categorías son la taxonomía global del sistema: todos los eventos se clasifican en ellas, los usuarios declaran sus intereses en términos de categorías, y el motor de recomendaciones las usa como señal de afinidad. Por eso, su creación y modificación está restringida al rol de administrador.

Permitir que cualquier usuario o moderador crease categorías generaría un catálogo caótico con duplicados, variantes ortográficas y categorías de un solo uso. El valor de las categorías reside precisamente en su estabilidad y en que el conjunto sea pequeño, coherente y representativo de la oferta cultural de la ciudad. El administrador actúa como curador de esta estructura, garantizando que el catálogo sea mantenible a largo plazo.

Por qué cualquier usuario autenticado puede crear eventos

Cualquier usuario que haya iniciado sesión puede publicar un evento. Esta decisión es intencionada: el modelo de negocio de la aplicación es comunitario y participativo. Reservar la creación de eventos a moderadores o administradores habría convertido la plataforma en un directorio cerrado gestionado por un equipo reducido, en lugar de una comunidad donde cualquier persona puede dar visibilidad a un plan cultural.

El riesgo de que se publique contenido de baja calidad o inapropiado se controla mediante el flujo de moderación posterior: los eventos públicos no son visibles hasta ser aprobados. La creación está abierta; la visibilidad pública está controlada.

Por qué los roles de administrador y moderador no se han unificado

A medida que la aplicación crece, garantizar la fiabilidad del contenido requiere contar con un número elevado de moderadores: cuantos más eventos se publiquen, más revisores se necesitan para mantener tiempos de respuesta razonables. Sin embargo, no es deseable que ese grupo numeroso tenga acceso a la configuración estructural del sistema. Separar ambos roles permite escalar el equipo de moderación sin ampliar el conjunto de personas con privilegios administrativos, que debe mantenerse lo más reducido posible.

- El **administrador** gestiona quién es moderador y qué categorías existen. Son acciones infrecuentes, de alto impacto y reservadas a un grupo muy reducido de responsables del sistema.
- El **moderador** revisa y valida contenido de forma continua. Es un rol operativo que puede asignarse a muchas personas sin riesgo, porque sus acciones están acotadas al flujo de moderación de eventos.

Unificar ambos roles implicaría que cualquiera del amplio grupo de moderadores podría cambiar roles de usuario o modificar las categorías globales del sistema, lo que ampliaría innecesariamente la superficie de error y el riesgo asociado a cada cuenta.

4.12.8. Elección de tecnologías e infraestructura

Las siguientes decisiones explican por qué se eligieron los principales servicios, frameworks y herramientas que sostienen la aplicación.

Por qué se usa Cloudinary para el almacenamiento de imágenes

Las imágenes de eventos, perfiles y mensajes de chat se almacenan en Cloudinary en lugar de en el propio servidor o en un servicio como Amazon S3. La elección se basa en tres ventajas concretas:

- **Transformaciones automáticas:** Cloudinary puede entregar las imágenes con el formato, tamaño y calidad óptimos para cada contexto (miniatura en listado, imagen completa en detalle) sin que el backend ni el frontend necesiten código de procesamiento de imagen. El parámetro `quality: 'auto:good'` reduce el peso de las imágenes de forma automática.
- **CDN incluido:** las imágenes se sirven desde el nodo de red más cercano al usuario, lo que reduce la latencia de carga respecto a servir las desde el servidor de aplicación.
- **Tier gratuito:** el plan gratuito de Cloudinary es suficiente para el volumen de un proyecto en fase de desarrollo, sin coste hasta alcanzar límites de almacenamiento y transformaciones que un MVP difícilmente supera.

Almacenar imágenes en el propio servidor habría generado problemas de escalabilidad y persistencia (las imágenes se pierden si el servidor se redespliega). Amazon S3 habría ofrecido mayor capacidad pero con mayor complejidad de configuración (IAM, políticas de bucket, CORS) y sin las transformaciones automáticas.

Por qué la aplicación ofrece modo claro y modo oscuro

La interfaz soporta dos temas visuales que el usuario puede cambiar manualmente o que se sincronizan con la preferencia del sistema operativo. Esta decisión responde a razones tanto de usabilidad como de accesibilidad:

- El modo oscuro reduce la fatiga visual en entornos de poca luz, un contexto habitual cuando se consulta la agenda de ocio nocturno.
- En dispositivos con pantalla OLED (habituales en gamas medias y altas de Android), el fondo negro consume menos energía que el fondo blanco, lo que alarga la autonomía de la batería.
- Los usuarios con fotosensibilidad o con dificultades para leer sobre fondos blancos se benefician directamente del modo oscuro.
- El soporte de tema oscuro es una expectativa de usuario consolidada en aplicaciones móviles modernas: tanto iOS como Android lo ofrecen a nivel de sistema desde 2019.

La implementación mediante un ThemeContext con una paleta de colores intercambiable permite que todos los componentes de la interfaz respondan al cambio de tema con una única fuente de verdad, sin necesidad de estilos duplicados.

Por qué se usa Firebase Authentication en lugar de autenticación propia

La gestión de identidad (registro, inicio de sesión, recuperación de contraseña y emisión de tokens) se delega completamente en Firebase Authentication. El backend únicamente verifica los tokens emitidos por Firebase mediante el SDK de administración.

Implementar un sistema de autenticación propio requeriría gestionar correctamente el almacenamiento de contraseñas (hash con bcrypt o Argon2), la generación y rotación de tokens JWT, la invalidación de sesiones, la recuperación de contraseña por correo y la protección contra ataques de fuerza bruta. Cada uno de estos aspectos es una fuente potencial de vulnerabilidades de seguridad si no se implementa con rigor. Firebase Authentication es un servicio auditado, probado a escala global y que asume toda esta responsabilidad.

Además, el tier gratuito de Firebase cubre el volumen de usuarios de cualquier MVP sin coste, y la integración con el cliente React Native es directa gracias al SDK oficial.

Por qué se usa React Native con Expo y no otra tecnología móvil

La aplicación móvil se desarrolla con React Native y Expo, lo que genera una única base de código compatible con Android e iOS. Las alternativas considerables habrían sido desarrollo nativo separado (Swift para iOS, Kotlin para Android), Flutter o una aplicación web progresiva.

El desarrollo nativo habría producido la mejor experiencia de usuario posible en cada plataforma, pero habría requerido mantener dos bases de código distintas, duplicando el esfuerzo de desarrollo para un proyecto unipersonal. Flutter comparte el modelo de una única base de código pero utiliza Dart, un lenguaje con menor ecosistema y curva de aprendizaje adicional. Una PWA no tiene acceso a las APIs nativas necesarias para la geolocalización continua y las notificaciones del sistema.

React Native permite reutilizar el conocimiento de JavaScript y React del frontend web, tiene un ecosistema de librerías amplio (navegación, mapas, calendarios, animaciones) y Expo elimina la necesidad de configurar entornos de compilación nativos durante el desarrollo.

Por qué se usa NestJS como framework de backend

El backend está construido sobre NestJS, un framework de Node.js orientado a la construcción de aplicaciones empresariales con TypeScript. La alternativa más directa habría sido Express, que es más ligero y flexible pero no impone ninguna convención de organización.

NestJS proporciona una arquitectura modular por defecto (cada funcionalidad en su propio módulo con controlador, servicio y entidades) que obliga a mantener el código organizado a medida que el proyecto crece. Los decoradores (@UseGuards, @Roles, @Throttle, @Cron) reducen el código repetitivo y hacen explícita la intención de cada endpoint. La inyección de dependencias nativa facilita las pruebas unitarias. Y la integración oficial con TypeORM, Socket.IO y el planificador de tareas (@nestjs/schedule) evita tener que ensamblar manualmente estas piezas.

Con Express habría sido necesario tomar todas estas decisiones de arquitectura de forma manual, con el riesgo de inconsistencias a medida que el proyecto crece.

Por qué se usa PostgreSQL en lugar de una base de datos NoSQL

La elección de PostgreSQL frente a opciones como MongoDB o Firebase Firestore responde principalmente a dos necesidades del sistema:

- **Datos relacionales con integridad referencial:** el modelo de datos tiene múltiples entidades fuertemente relacionadas (usuarios, eventos, categorías, reseñas, rutas, notificaciones) con dependencias en cascada. Una base de datos relacional garantiza la consistencia de estas relaciones mediante claves

foráneas y transacciones ACID, lo que evita estados inconsistentes si una operación falla a mitad de ejecución.

- **Consultas geoespaciales con PostGIS:** la extensión PostGIS convierte PostgreSQL en una base de datos espacial completa, con soporte nativo para almacenar puntos geográficos e índices GiST que aceleran las consultas de filtrado por radio. Replicar esta funcionalidad en MongoDB o Firestore habría requerido soluciones más limitadas o el uso de servicios externos adicionales.

Por qué se usa TypeORM en lugar de Prisma u otro ORM

TypeORM se integra de forma nativa con NestJS mediante el módulo `@nestjs/typeorm` y ofrece soporte directo para los tipos espaciales de PostGIS (`geography`, `geometry`) mediante el decorador `@Column({ type: 'geography', spatialFeatureType: 'Point', srid: 4326 })`. Prisma, que sería la alternativa más moderna, no tiene soporte nativo para tipos PostGIS y requeriría recurrir a SQL crudo para las consultas geoespaciales, eliminando la ventaja principal del ORM. Dado que la geolocalización es una funcionalidad central del sistema, TypeORM resultó la elección más coherente.

Por qué se usa OpenStreetMap en lugar de Google Maps

Los mapas de la aplicación se renderizan con tiles de OpenStreetMap a través del componente `UrlTile` de `react-native-maps`. Google Maps habría ofrecido una experiencia visual más pulida, pero su uso en producción requiere una API key de pago a partir de un determinado volumen de peticiones de tiles. OpenStreetMap es completamente gratuito y de uso libre incluso en aplicaciones comerciales, bajo licencia ODbL.

Para el caso de uso de la aplicación (mostrar eventos geolocalizados en un mapa de Sevilla) la cartografía de OpenStreetMap es suficientemente detallada y precisa. La calidad del mapa en ciudades españolas es alta, ya que la comunidad de contribuidores locales mantiene el detalle urbano actualizado.

Por qué se usa Axios en el frontend en lugar de fetch

Las llamadas HTTP del frontend se realizan con Axios en lugar de la API `fetch` nativa del navegador. Axios permite configurar un cliente centralizado con la URL base del servidor, un `timeout` global de 60 segundos y un interceptor de respuesta que reintenta automáticamente las peticiones que fallan por `timeout` o pérdida de conexión. Este último punto es especialmente relevante en una aplicación móvil, donde las interrupciones de red son habituales.

Con `fetch` habría sido necesario implementar manualmente el `timeout` (mediante `AbortController`), la lógica de reintento y el manejo centralizado de errores (mapeo de códigos HTTP a mensajes de usuario). Axios proporciona todo esto con configuración mínima.

4.12.9. Funcionalidades sociales y reglas de dominio

Este bloque agrupa decisiones propias del dominio de la aplicación: rutas, relaciones sociales, precios, enlaces externos e integridad al eliminar datos.

Por qué existen tanto rutas creadas por usuarios como rutas recomendadas automáticamente

El sistema de rutas tiene dos orígenes bien diferenciados que responden a necesidades distintas y se complementan para reforzar el carácter social de la aplicación:

- **Rutas creadas por usuarios:** un usuario que publica una ruta ya la ha recorrido y ha comprobado que funciona. Ese itinerario lleva implícita una validación humana que un algoritmo no puede aportar: la experiencia real de alguien que estuvo allí. Además, compartir y valorar rutas de otros usuarios fomenta la interacción social dentro de la plataforma, ya que los usuarios pueden descubrir planes a través de personas en las que confían más que en una recomendación automática.
- **Rutas recomendadas:** son itinerarios generados automáticamente según los intereses del usuario, su ubicación y los eventos disponibles en ese momento. No han sido recorridas por nadie, pero tienen la ventaja de estar personalizadas y de no requerir ningún esfuerzo por parte del usuario. Son especialmente útiles para quienes no conocen la ciudad o no saben por dónde empezar.

Ofrecer solo rutas automáticas habría eliminado el componente social y la fiabilidad que aporta la experiencia de otro usuario. Ofrecer solo rutas de usuarios habría dejado sin servicio a quienes no tienen referencias previas. La combinación de ambas cubre el espectro completo: desde el usuario que confía en las recomendaciones del sistema hasta quien prefiere fiarse de alguien que ya lo ha hecho.

Por qué existe un módulo de amistad basado en seguimiento

El sistema implementa un módulo de amistad apoyado en un grafo de seguimiento unidireccional entre usuarios: una persona puede seguir a otra sin que esa otra deba aceptar ni corresponder. En la interfaz, esta funcionalidad se presenta como un módulo de *Amigos*, aunque internamente diferencia entre usuarios a los que se sigue y usuarios que siguen al usuario actual.

La pantalla muestra una pestaña de *Seguidos* con las personas que el usuario sigue, una pestaña de *Seguidores* con todas las personas que siguen al usuario actual y una pestaña de *Amigos* con las relaciones mutuas, es decir, usuarios que aparecen simultáneamente como seguidores y seguidos. Esta separación evita perder información: una persona seguida de vuelta sigue apareciendo como seguidor, pero además queda reflejada como amistad mutua.

Esta funcionalidad refuerza el carácter social de la aplicación al permitir descubrir planes a través de otras personas:

- El usuario puede buscar a otras personas y acceder a su perfil público.
- Desde el perfil de otro usuario se pueden consultar los eventos a los que va a asistir, lo que facilita coordinarse o descubrir planes que de otro modo no habrían aparecido en el listado general.
- El perfil permite seguir o dejar de seguir a esa persona según el estado actual de la relación.
- El perfil de otro usuario es también el punto de entrada para iniciar una conversación privada, lo que permite ponerse en contacto con alguien para quedar o resolver dudas sobre un evento concreto.

Por qué el precio de un evento puede ser fijo o expresarse como rango

La entidad Event permite tres variantes de precio: un valor fijo (precio), un rango mínimo-máximo (precioMin y precioMax), o ninguno (evento gratuito o precio no especificado). Esta flexibilidad responde a la diversidad de la oferta real:

- Algunos eventos tienen una entrada de precio único (un concierto con entrada a 15€).
- Otros tienen precios diferenciados por tipo de entrada o categoría de asiento (de 10€ a 40€), por lo que expresar solo el mínimo o el máximo sería engañoso.
- Muchos eventos (talleres, ferias, exposiciones) son gratuitos o no tienen precio definido en el momento de publicarse.

La validación en la entidad impide que se proporcionen simultáneamente precio fijo y rango, evitando estados ambiguos.

Por qué se implementa deep linking

La aplicación gestiona URLs externas que abren directamente una pantalla concreta dentro de la app. Esto es necesario para dos flujos específicos:

- **Compartir eventos:** cuando un usuario comparte un evento, el sistema genera una URL que, al abrirse en el dispositivo, lleva directamente a la pantalla de detalle de ese evento sin pasar por la pantalla principal. Sin deep linking, el destinatario tendría que abrir la app y buscar el evento manualmente.
- **Acceso a eventos privados:** el enlace de invitación a un evento privado (UUID) debe resolverse directamente en la pantalla de acceso, tanto desde un navegador externo como desde otra app. Sin deep linking este flujo no sería posible.

Qué ocurre al eliminar un usuario

Cuando se elimina una cuenta de usuario, el sistema aplica las siguientes reglas de integridad referencial:

- **Eventos creados:** se eliminan en cascada junto con el usuario. Al ser el creador el responsable del evento, se considera que el evento pierde su razón de existir si la cuenta que lo publicó desaparece.
- **Rutas creadas:** se eliminan en cascada por el mismo motivo.
- **Mensajes en chats de eventos:** se eliminan en cascada, ya que están ligados directamente a la identidad del emisor.
- **Mensajes privados:** el hilo completo de la conversación se elimina en cascada junto con el usuario eliminado, incluidos todos los mensajes intercambiados. Dado que la conversación privada es una relación bilateral, no tiene sentido conservar un hilo sin uno de sus dos participantes.
- **Resto de relaciones** (notificaciones, reseñas, recomendaciones, calificaciones de rutas): se eliminan en cascada.
- **Tablas de unión M:N** (`user_seguidores`, `user_saved_events`, `user_shared_events`, `user_visited_events`, `event_asistentes`): se limpian manualmente antes del DELETE en el servicio de usuarios, ya que TypeORM no propaga onDelete automáticamente sobre relaciones @JoinTable.

Qué ocurre al eliminar una categoría

Las categorías son el eje de clasificación de todos los eventos del sistema. Si se eliminara una categoría sin reasignar los eventos que la usan, estos quedarían sin categoría, lo que violaría la integridad del modelo y los excluiría de los filtros y del motor de recomendaciones.

Para evitarlo, el sistema obliga a que exista siempre una categoría llamada **Otros**. Cuando el administrador elimina una categoría, todos los eventos que la tenían asignada se reasignan automáticamente a *Otros* antes de ejecutar el borrado. Si la categoría *Otros* no existe en ese momento, la operación se rechaza con un error que indica al administrador que debe crearla previamente.

Se descartó el uso de SET NULL en la clave foránea porque la categoría es un campo obligatorio en los eventos: un evento sin categoría no puede participar en la navegación por categorías, en los filtros de búsqueda ni en el cálculo de afinidades del motor de recomendaciones.

4.13. Conclusiones

En este capítulo se ha presentado el diseño de la solución adoptada para el sistema **Sevillaneando**. Se ha descrito la arquitectura cliente-servidor, el modelo de

datos con sus entidades y relaciones, el diseño de la API REST, el flujo de moderación de eventos, el diseño de la interfaz de usuario con el grafo de navegación, los sistemas de comunicación en tiempo real y de recomendaciones, los patrones de diseño aplicados y las decisiones de diseño más relevantes.

Las decisiones de diseño adoptadas responden directamente a los requisitos funcionales y no funcionales identificados en el análisis: la separación en cliente y servidor facilita la escalabilidad, el uso de PostGIS permite cubrir los requisitos de geolocalización, el sistema de roles garantiza el control de acceso, y la arquitectura modular de NestJS simplifica el mantenimiento y la extensión del sistema. Los patrones de diseño identificados ponen de manifiesto que la solución no solo responde a los requisitos funcionales, sino que también incorpora buenas prácticas de ingeniería del software consolidadas. Las decisiones específicas sobre roles, persistencia de eventos y diseño del chat han sido justificadas en la sección anterior, poniendo de manifiesto que cada elección responde a una necesidad concreta del dominio y no a una preferencia arbitraria.

5. Implementación

5.1. Introducción

En este capítulo se describe la implementación del sistema **Sevillaneando**, detallando las herramientas utilizadas, la organización del proyecto y la forma en la que se han desarrollado las principales funcionalidades de la aplicación.

La implementación se ha organizado siguiendo una arquitectura cliente-servidor. Por un lado, la aplicación móvil actúa como cliente y permite al usuario interactuar con los eventos, mapas, recomendaciones, chats y demás funcionalidades. Por otro lado, el backend expone una API REST encargada de procesar las peticiones, aplicar la lógica de negocio y acceder a la base de datos.

Además, el sistema integra servicios externos para resolver necesidades concretas como la autenticación, el almacenamiento de imágenes, la geolocalización y la gestión de la infraestructura local.

5.2. Estructura del proyecto

El proyecto se organiza como un monorepositorio, separando el código del frontend y del backend dentro de una misma estructura.

- **apps/frontend**: contiene la aplicación móvil desarrollada con React Native y Expo.
- **apps/backend**: contiene la API REST desarrollada con NestJS.
- **docker-compose.yml**: define los servicios necesarios para el entorno local, especialmente la base de datos PostgreSQL con PostGIS.

Esta estructura permite mantener en un único repositorio todos los componentes principales del sistema, facilitando la instalación, el desarrollo y la coordinación entre frontend y backend.

En la revisión final del repositorio, la aplicación contiene 165 ficheros TypeScript/TSX de producción dentro de src: 82 en el backend, con 10.289 líneas, y 83 en el frontend, con 18.992 líneas. A esta base se añaden 3 ficheros de pruebas unitarias del backend, con 1.269 líneas adicionales, por lo que el total auditado asciende a 168 ficheros y 30.550 líneas. La estructura funcional se resume en 13 módulos NestJS, 10 controladores, 11 entidades TypeORM, 12 DTOs principales, 31 pantallas móviles, 11 componentes reutilizables y 5 servicios de comunicación con la API.

5.3. Implementación del backend

5.3.1. Organización modular

El backend se organiza en módulos de NestJS. Cada módulo agrupa controladores, servicios y entidades relacionadas con una parte concreta del dominio.

La estructura básica sigue el siguiente patrón:

- **Controladores:** reciben las peticiones HTTP y definen los endpoints.
- **Servicios:** contienen la lógica de negocio.
- **Entidades:** representan las tablas de la base de datos.
- **DTOs:** definen y validan los datos recibidos desde el frontend.

Esta separación permite que la lógica de cada funcionalidad esté localizada y sea más sencilla de mantener.

5.3.2. Persistencia geográfica con TypeORM

La ubicación de los eventos se almacena en PostgreSQL [58] como un punto geográfico compatible con PostGIS [39] mediante TypeORM [42], utilizando el sistema de referencia SRID 4326. El siguiente fragmento muestra la definición del campo en la entidad Event:

```
1 @Entity({ name: 'events' })
2 export class Event {
3   @PrimaryGeneratedColumn('uuid')
4   id!: string;
5
6   @Column({ length: 100, nullable: true })
7   title!: string | null;
8
9   @Column({ type: 'varchar', length: 255, nullable: false })
10  address!: string;
11
12  @Column({ type: 'geography', spatialFeatureType: 'Point', srid: 4326 })
13  location!: GeoJsonPoint;
14
15  @Column('float', { nullable: true })
16  precio?: number | null; // precio fijo (0 = gratuito)
17
18  @Column('float', { nullable: true })
19  precioMin?: number | null; // precio minimo del rango
20
21  @Column('float', { nullable: true })
```

```

22 precioMax?: number | null;           // precio maximo del rango
23
24 @Column({ type: 'enum', enum: RecurrenciaEnum, nullable: true })
25 recurrencia?: RecurrenciaEnum | null;
26
27 @Column({ type: 'enum', enum: EstadoEnum, default: EstadoEnum.Pendiente
28         })
29 estado!: EstadoEnum;

```

Extracto de código 5.1: Definición del campo geográfico y de precios en la entidad Event con TypeORM

5.3.3. Scraping de eventos externos

El sistema incluye una funcionalidad de scraping para incorporar eventos procedentes de fuentes externas. Se han integrado cuatro fuentes concretas:

- **Eventbrite Sevilla:** el SevillaScrapperService descarga la página de eventos de Sevilla en Eventbrite mediante axios [28] y parsea el HTML con cheerio [59], extrayendo tarjetas de eventos y bloques JSON-LD embebidos en la página.
- **Ticketmaster API:** el TicketmasterScrapperService consume la API oficial de Ticketmaster Discovery paginando los resultados en bloques de hasta 100 eventos, filtrando por ciudad y fecha de inicio.
- **Visita Sevilla:** el VisitaSevillaScrapperService descarga directamente la agenda de visitasevilla.es y parsea el HTML con cheerio. Interpreta el formato de fechas propio del portal (DD.MM.YY y rangos DD.MM.YY-DD.MM.YY), descarta placeholders de eventos sin fecha concreta y marca como *consultar fechas* aquellos con rangos superiores a catorce días, evitando así duraciones ficticias.
- **Scraper multi-web con Gemini AI:** el GeminiScrapperService procesa varias webs institucionales y de ocio de Sevilla (portal municipal, Teatro de la Maestranza, Fundación Cajasol, guías de ocio locales, entre otras). Para cada URL, descarga el HTML, lo limpia con cheerio [59] eliminando scripts, estilos y elementos de navegación, y envía el texto resultante al modelo **Gemini 2.5 Flash** de Google [49], que devuelve los eventos en formato JSON estructurado. Las coordenadas geográficas que Gemini no puede deducir se resuelven mediante geocodificación con la API de **Nominatim** [48] (OpenStreetMap), con hasta tres variantes de búsqueda por dirección.

La figura 5.1 muestra cómo se orquesta la obtención de eventos externos, desde el *cron job* hasta la persistencia en base de datos.

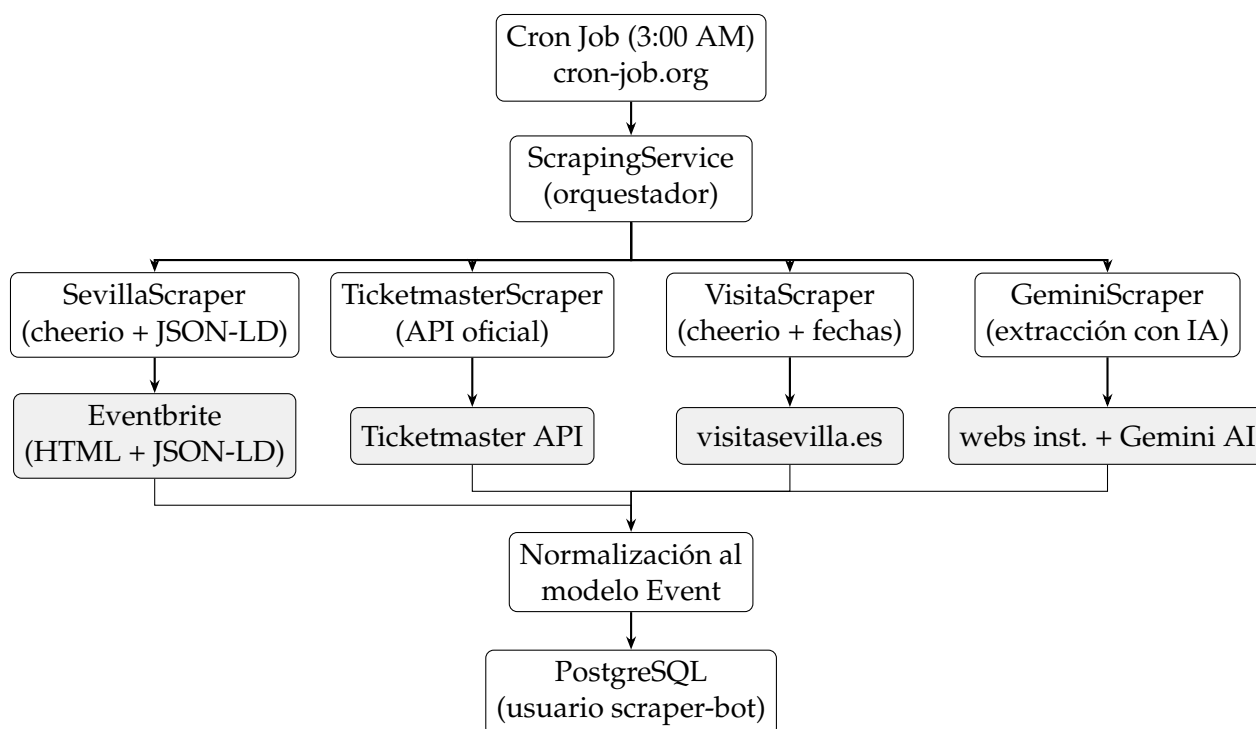


Figura 5.1: Flujo del sistema de scraping: orquestación, fuentes externas y persistencia

Antes de almacenar los eventos obtenidos, el sistema comprueba que no existan duplicados y que la información de ubicación sea válida, descartando aquellos sin coordenadas o con datos incompletos.

5.3.4. Tareas programadas

El backend incluye un SchedulerModule que agrupa la lógica de scraping y notificaciones automáticas. En despliegue, estas tareas se lanzan mediante un cron externo que invoca endpoints HTTP del backend, lo que evita depender de que el proceso permanezca siempre activo.

- **Scraping diario:** el endpoint POST `/scheduler/run-scraping` invoca el `ScrapingScheduler`, que ejecuta `ScrapingService.scrapeAll()` para actualizar el catálogo de eventos externos.
- **Notificaciones automáticas:** el cron externo invoca cada hora el endpoint POST `/scheduler/run-notifications`, que ejecuta los tres métodos de `NotificacionesScheduler`: eventos cercanos a la ubicación del usuario, eventos a los que el usuario está apuntado y que ocurren en los próximos siete días, y eventos que ocurren en menos de 24 horas. En cada caso, el sistema evita crear notificaciones duplicadas.

La eliminación de mensajes privados al borrar un usuario se gestiona directamente mediante la política CASCADE en la base de datos: al eliminar

cualquiera de los dos participantes (emisor o receptor), sus mensajes se borran automáticamente sin necesidad de lógica adicional.

El mismo controlador expone GET /scheduler/health, utilizado como comprobación de disponibilidad del backend y como apoyo al sistema externo de activación periódica.

5.3.5. API de eventos

La API de eventos permite crear, consultar y editar eventos. Las operaciones de escritura están protegidas mediante autenticación, mientras que la consulta de eventos puede utilizarse para mostrar el catálogo en la aplicación móvil.

Durante la creación de un evento, el backend recibe campos como título, descripción, dirección, latitud y longitud. A partir de las coordenadas recibidas, el servicio construye un punto geográfico en formato compatible con PostGIS.

```
1 SRID=4326;POINT(longitude latitude)
```

Extracto de código 5.2: Construcción conceptual de la ubicación geográfica de un evento

De esta forma, aunque el frontend trabaja con latitud y longitud por separado, la base de datos almacena la ubicación como un único campo geográfico.

5.3.6. Validación de datos

La validación de datos se realiza mediante DTOs apoyados en la librería class-validator [20], integrada en el ValidationPipe global de NestJS. Por ejemplo, al crear un evento se exige que determinados campos estén presentes y tengan el tipo adecuado.

Esto permite evitar que lleguen a la base de datos eventos incompletos o con coordenadas inválidas. Además, centralizar estas validaciones en el backend garantiza que las reglas se apliquen aunque la petición no provenga directamente de la interfaz móvil.

5.3.7. Actualización de eventos

En la edición de eventos se permite modificar solo los campos enviados por el usuario. Si se proporcionan nuevas coordenadas, el backend recalcula la ubicación geográfica. Si no se proporcionan, se mantiene la ubicación anterior.

Este comportamiento evita sobrescribir información existente de forma accidental y permite implementar formularios de edición parciales.

5.3.8. Control de roles

El sistema distingue tres roles de usuario mediante el enum `RoleEnum`: `user` (usuario estándar), `moderator` y `admin`. El rol se almacena en la entidad `User` y se comprueba en los endpoints que requieren privilegios elevados mediante un guard de roles adicional al de autenticación.

Los administradores pueden cambiar el rol de cualquier usuario. Los moderadores acceden a funcionalidades de revisión de eventos que no están disponibles para usuarios estándar.

5.3.9. Limitación de velocidad

Para proteger la API frente a abusos, el backend aplica throttling HTTP mediante `@nestjs/throttler` [21], una medida recomendada por OWASP frente a ataques de fuerza bruta y abuso de recursos [22]. Se definen tres niveles configurables mediante decoradores personalizados:

- `ThrottleStrict`: 5 peticiones por minuto. Se aplica a los endpoints de administración del scraping (`POST /scraping/run-all`, `POST /scraping/reset`, `POST /scraping/run/:name`), que son operaciones costosas reservadas a administradores.
- `ThrottleModerate`: 20 peticiones por minuto. Se aplica a la creación de eventos (`POST /events`), operación de escritura habitual pero que podría usarse de forma abusiva para inundar el sistema de contenido pendiente de moderación.
- `ThrottleUpload`: 10 peticiones por minuto. Se aplica a todos los endpoints de subida de archivos: imágenes de eventos (`POST /events/upload-image`), imágenes en el chat (`POST /chat/upload`) y foto de perfil (`POST /users/upload-profile-image/firebase`).

Los endpoints de solo lectura pueden omitir el throttling con el decorador `SkipThrottle`.

5.3.10. Autenticación en endpoints protegidos

Los endpoints que requieren autenticación utilizan un guard basado en Firebase. Este guard extrae el token de la cabecera `Authorization`, comprueba que tenga el formato correcto y lo valida mediante `Firebase Admin`.

Si el token es válido, la petición continúa y se añade la información del usuario autenticado al contexto de la petición. Si el token no existe o no es válido, el backend rechaza la operación.

```
1 @Injectable()
2 export class FirebaseAuthGuard implements CanActivate {
3   async canActivate(context: ExecutionContext): Promise<boolean> {
```

```

4     const request = context.switchToHttp().getRequest();
5     const header = request.headers['authorization'];
6     if (!header || !header.startsWith('Bearer '))
7         throw new UnauthorizedException('Token requerido');
8     const token = header.replace('Bearer ', '');
9     const decoded = await this.firebaseService.verifyToken(token);
10    if (!decoded) throw new UnauthorizedException('Token invalido');
11    request.user = decoded;
12    return true;
13  }
14 }

```

Extracto de código 5.3: Guard de autenticación Firebase en NestJS

5.3.11. Eventos recurrentes

La entidad Event contempla eventos con fechas repetidas mediante el campo `recurrencia`, que puede tomar los valores diario, semanal, quincenal o mensual, junto con un campo `recurrenciaFin` que delimita hasta qué fecha se repite el evento.

Adicionalmente, el campo booleano `hasMultipleDatesAvailable` permite indicar que un evento dispone de varias fechas posibles sin seguir un patrón de recurrencia fijo. Esta distinción es relevante en el sistema de recomendaciones, donde los eventos con fechas flexibles reciben un tratamiento diferente en el cálculo del factor de proximidad temporal.

5.4. Implementación del frontend

5.4.1. Estructura de pantallas

La aplicación móvil se organiza en 31 pantallas conectadas mediante React Navigation. La navegación se divide en dos pilas: una pila pública con inicio de sesión, registro y política de privacidad, y una pila autenticada que contiene el resto de la aplicación. Algunas pantallas solo se registran cuando el usuario tiene el rol adecuado, como el panel de administración o las pantallas de moderación.

Las pantallas principales se agrupan funcionalmente de la siguiente forma:

- **Acceso público:** `LoginScreen`, `SignUpScreen` y `PrivacyPolicyScreen`. Cubren autenticación, registro y consulta de la política de privacidad antes de iniciar sesión.
- **Descubrimiento:** `HomeScreen`, `EventDetailScreen`, `EventDetailLinkScreen`, `AccessPrivateEventScreen` y `EventsMapScreen`. La pantalla principal concentra el listado, filtros, eventos recomendados, rutas sugeridas, menú lateral y acceso a eventos privados.

- **Gestión de eventos:** `CreateEventScreen`, `UserEditEventScreen`, `UserEventsScreen`, `CalendarEventsScreen`, `SavedAndPrivateEventsScreen`, `ModeratorEventsScreen` y `ModeratorEditEventScreen`. Estas pantallas cubren creación, edición, eventos propios, calendario, guardados, privados y moderación.
- **Rutas:** `RoutesListScreen`, `RouteDetailScreen`, `RoutePreviewScreen` y `CreateRouteScreen`. Permiten consultar rutas de usuarios, crear nuevas rutas, valorar itinerarios y previsualizar rutas recomendadas por el sistema.
- **Social:** `FriendsScreen`, `UserProfileScreen`, `ProfileConnectionsScreen`, `MessagesScreen`, `DirectMessageScreen` y `NotificacionesScreen`. Implementan búsqueda de usuarios, seguimiento, perfiles públicos, mensajería privada y notificaciones.
- **Cuenta:** `EditProfileScreen` y `EditPasswordScreen`. Permiten modificar datos personales, preferencias, foto de perfil y contraseña.
- **Administración e información:** `AdminScreen`, `CategoriesScreen`, `HelpScreen` y `LegalAttributionsScreen`.

Esta organización permite separar la interfaz según los casos de uso principales de la aplicación, manteniendo tipadas las rutas mediante `RootStackParamList` y `AuthStackParamList`. El sistema de *deep linking* conecta URLs externas con pantallas internas, especialmente para abrir detalles de eventos compartidos y resolver enlaces de acceso a eventos privados.

5.4.2. Estado global y proveedores

El frontend utiliza cuatro contextos principales:

- **AuthContext:** observa el estado de Firebase Authentication, obtiene el token JWT, consulta `/users/me`, almacena el usuario actual y su rol, e inyecta el token en el cliente Axios. Además, fuerza la renovación del token cada 50 minutos para evitar que expire durante el uso de la aplicación.
- **SocketContext:** mantiene una única conexión `Socket.IO` autenticada con el token de Firebase. La conexión se reutiliza entre pantallas de chat y mensajería privada, con reconexión automática y desconexión tras un periodo prolongado de inactividad.
- **ThemeContext:** gestiona modo claro y oscuro, con una paleta compartida por todos los componentes temáticos. La preferencia se persiste localmente mediante `AsyncStorage`.
- **NotificacionesContext:** consulta periódicamente las notificaciones del usuario y mantiene el contador de no leídas, que se muestra en la interfaz sin tener que recargar manualmente la pantalla de notificaciones.

La raíz de la aplicación envuelve el navegador en estos proveedores mediante `AppProviders`, de forma que autenticación, tema, socket y notificaciones quedan

disponibles para todas las pantallas sin propagar propiedades manualmente.

5.4.3. Comunicación con la API

El frontend utiliza una capa de servicios para comunicarse con el backend. Esta capa centraliza la configuración de Axios, la URL base de la API y la inclusión del token de autenticación en las peticiones protegidas. El cliente Axios define un *timeout* de 60 segundos y reintenta automáticamente una vez las peticiones que fallan por timeout o por error de red, comportamiento especialmente útil cuando el backend desplegado en Render se encuentra suspendido y necesita unos segundos para reactivarse.

La capa de servicios se divide por dominio:

- **events.ts**: consulta de eventos, detalle, asistentes, asistencia del usuario, reseñas y acceso por enlace privado. También normaliza la respuesta del backend para añadir coordenadas latitude/longitude cuando la ubicación llega como GeoJSON o WKT.
- **recommendations.ts**: eventos recomendados, rutas recomendadas, guardado, compartición, visitas y valoraciones.
- **routes.ts**: creación, consulta, búsqueda, edición, eliminación y valoración de rutas creadas por usuarios.
- **users.ts**: perfil público, búsqueda de usuarios, seguimiento, seguidores y seguidos.
- **api.ts**: configuración base de Axios, gestión del token y traducción de errores HTTP a mensajes de usuario.

Además, esta capa transforma los datos geográficos recibidos desde el backend. Como la ubicación puede llegar en formato WKT (POINT(lon lat)) o GeoJSON, el frontend extrae las coordenadas y las convierte en valores numéricos de latitud y longitud para poder mostrarlas en el mapa.

```
1 export const parseLocationPoint = (location?: string | any)
2   : { latitude: number; longitude: number } | null => {
3   if (!location) return null;
4
5   if (typeof location === 'object' && location.coordinates) {
6     const [lon, lat] = location.coordinates;
7     if (Number.isFinite(lat) && Number.isFinite(lon))
8       return { latitude: lat, longitude: lon };
9   }
10
11  if (typeof location === 'string') {
12    const match = location.match(/POINT\(((\[-\d.]+\s+(\[-\d.]+\s+)\s+)\)/);
13    if (!match) return null;
14    const lon = parseFloat(match[1]);
```

```

15     const lat = parseFloat(match[2]);
16     if (Number.isFinite(lat) && Number.isFinite(lon))
17         return { latitude: lat, longitude: lon };
18     }
19
20     return null;
21 };

```

Extracto de código 5.4: Función de parseo de coordenadas geográficas en el frontend

5.4.4. Gestión de mapas

Los eventos se representan en el mapa mediante marcadores. Cada marcador se posiciona utilizando las coordenadas del evento y permite al usuario acceder a información resumida o navegar hacia el detalle del evento.

La ubicación del usuario también se utiliza para calcular cercanía y aplicar filtros por radio, siempre que el usuario haya definido una ubicación en su perfil. En el lado cliente se mantiene una utilidad de cálculo de distancia Haversine para ordenar, colorear o comparar eventos cuando es necesario trabajar sin una consulta espacial directa al backend. La constante `SEVILLE_COORDINATES` proporciona un centro inicial razonable para el mapa, y `OSM_TILE_URL_TEMPLATE` define los tiles de OpenStreetMap usados por react-native-maps.

5.4.5. Gestión de imágenes

La aplicación permite trabajar con imágenes asociadas a perfiles, eventos y chats. Estas imágenes no se almacenan directamente en la base de datos, sino que se suben a un servicio externo de almacenamiento.

En la base de datos se almacena la referencia o URL de acceso a la imagen, lo que reduce la carga del backend y permite gestionar los archivos multimedia de forma más eficiente. En el frontend, `imageUrl.ts` normaliza URLs absolutas, relativas, locales y procedentes de Cloudinary. También añade transformaciones de Cloudinary como `f_auto` y `q_auto` para optimizar formato y calidad, y genera candidatos alternativos de URL para aumentar la robustez ante imágenes importadas por scraping con formatos inconsistentes.

5.4.6. Utilidades de presentación y normalización

El frontend contiene varias utilidades específicas para adaptar los datos del backend al uso móvil:

- **Precios:** `price.ts` formatea precios fijos y rangos. Si el evento procede del usuario técnico de scraping y tiene precio cero, se muestra como “Consultar

precio” para evitar etiquetar como gratuitos eventos cuyo precio no venía especificado en la fuente original.

- **Fechas:** `sevillaTime.ts` formatea fechas en la zona Europe/Madrid, contemplando horario de verano. `dateTime.ts` convierte fechas introducidas en hora local de Sevilla a ISO con offset explícito para que el backend guarde el instante correcto.
- **Mapas:** `map.ts` parsea puntos GeoJSON o WKT, calcula distancias mediante la fórmula de Haversine [60] y centraliza las coordenadas de referencia de Sevilla.
- **Errores:** `firebaseErrors.ts` y `getErrorMessage` traducen errores técnicos de Firebase, Axios y HTTP a mensajes comprensibles para el usuario.

5.5. Implementación de funcionalidades principales

5.5.1. Eventos, administración y moderación

Este bloque recoge las funcionalidades que giran alrededor del ciclo de vida de los eventos: creación, edición, revisión, privacidad, asistencia y herramientas administrativas.

Creación y edición de eventos

En la aplicación, la creación de eventos está expuesta al rol user. El formulario permite introducir datos como título, descripción, categoría, ubicación, fechas, precio e imágenes. El sistema distingue entre eventos públicos y privados.

Los eventos públicos quedan pendientes de moderación antes de ser visibles para el resto de usuarios. En cambio, los eventos privados se gestionan mediante enlace de invitación y no aparecen en el listado general.

En la creación y edición se incluye un selector de recurrencia con las opciones *Sin recurrencia*, *Diario*, *Semanal*, *Quincenal* y *Mensual*. Cuando el usuario selecciona una recurrencia, las pantallas `CreateEventScreen`, `UserEditEventScreen` y `ModeratorEditEventScreen` muestran además el campo `recurrenciaFin`; ambos valores se envían al backend como `recurrencia` y `recurrenciaFin`. El backend genera registros independientes para las ocurrencias incluidas en el rango indicado. En paralelo, los eventos importados con varias fechas posibles pero sin regla fija se marcan con `hasMultipleDatesAvailable`, lo que se muestra en el detalle del evento, en los listados y en la vista previa de rutas.

En la edición de eventos se mantienen las reglas de negocio definidas en el análisis, como la coherencia de fechas, la coherencia de precios y la validez de la ubicación geográfica.

Panel de administración

El panel de administración se implementa en AdminScreen mediante tres pestañas: *Estadísticas*, *Usuarios* y *Scraping*. La primera consume el endpoint GET /users/admin/stats y muestra indicadores globales de la plataforma: número total de usuarios, total de eventos, eventos aprobados, pendientes y rechazados, eventos importados por scraping y eventos creados por usuarios.

La pestaña *Usuarios* permite consultar las cuentas registradas, cambiar su rol entre usuario, moderador y administrador, y eliminar cuentas cuando sea necesario. La pestaña *Scraping* expone la acción administrativa de restablecer los eventos importados automáticamente, invocando POST /scraping/reset y limpiando la caché local de eventos para que el cliente vuelva a solicitar datos actualizados.

Moderación de eventos

Los moderadores pueden revisar eventos públicos pendientes. Durante este proceso pueden aprobarlos, rechazarlos o realizar modificaciones antes de su publicación. Esta funcionalidad no forma parte del flujo de administración: el administrador gestiona usuarios, roles, categorías, estadísticas y scraping, pero no interviene en la aprobación o rechazo de eventos desde la aplicación.

Esta funcionalidad permite controlar la calidad de la información publicada y evita que eventos incompletos o incorrectos aparezcan directamente en el sistema.

Eventos privados

Los eventos privados se implementan mediante un mecanismo de acceso restringido por enlace. El creador del evento puede compartir un enlace de invitación, y cualquier persona que disponga de ese enlace puede resolver el detalle del evento mediante el endpoint público de acceso privado.

Estos eventos no aparecen en el listado general y se mantienen separados de los eventos públicos.

Participación y organización de eventos

El sistema permite que los usuarios se apunten a eventos, los guarden y los consulten posteriormente. Además, los eventos con fecha definida pueden aparecer en el calendario del usuario.

Estas acciones también sirven como información de entrada para el sistema de recomendaciones, ya que reflejan los intereses reales del usuario.

5.5.2. Recomendaciones y rutas

Estas funcionalidades generan sugerencias personalizadas y permiten construir itinerarios a partir de eventos disponibles.

Recomendaciones

Las recomendaciones se generan mediante un algoritmo de puntuación que evalúa cada evento candidato en función de múltiples factores ponderados. La puntuación final de cada evento se calcula sumando las contribuciones de cada factor, y los eventos se ordenan de mayor a menor puntuación para devolver los más relevantes al usuario.

Los factores considerados y sus pesos por defecto son los siguientes:

- **Afinidad por categoría** (máx. 12 puntos): se acumula en función de si el usuario ha marcado interés en esa categoría, se ha apuntado o guardado eventos de ella, la ha visitado o la ha valorado positiva o negativamente.
- **Evento guardado previamente** (+10): el usuario ya guardó ese evento.
- **Evento compartido** (+8): el usuario lo compartió anteriormente.
- **Evento al que está apuntado** (+6): el usuario ya asistirá.
- **Evento visitado** (+4): el usuario accedió al detalle del evento.
- **Valoración propia** (± 2 por punto sobre 3): la puntuación que el propio usuario dio al evento.
- **Valoración media de la comunidad** (+hasta 1,5 por punto sobre 3): media de reseñas del resto de usuarios.
- **Popularidad** (máx. +3): calculada como $\min(\text{asistentes} / 5, 3)$, donde 5 es el factor de escala y 3 el tope máximo (`maxAttendeesBoost`).
- **Proximidad temporal** (máx. +8, decae linealmente): mayor puntuación cuanto más próxima es la fecha de inicio. Los eventos con fechas flexibles reciben un boost reducido al 20 %.
- **Proximidad geográfica** (máx. +6, decae linealmente): mayor puntuación cuanto más cerca esté el evento de la ubicación del usuario. Para eventos de fecha flexible el boost se amplifica un 35 %.

Todos los pesos son configurables mediante variables de entorno (por ejemplo, `RECO_WEIGHT_SAVED_EVENT`), lo que permite ajustar el comportamiento del sistema sin modificar el código.

```
1 private readonly weights = {  
2     categoryInterest: 5,    categoryAttend: 4,  
3     categorySave: 3,       categoryShare: 3,  
4     categoryVisit: 3,      categoryRatedHigh: 4,  
5     categoryRatedNeutral: 1, categoryRatedLow: -2,
```

```

6  maxCategoryContribution: 12,
7  savedEvent: 10,          sharedEvent: 8,
8  attendingEvent: 6,       visitedEvent: 4,
9  myRatingFactor: 2,       globalRatingFactor: 1.5,
10 attendeesFactor: 5,      maxAttendeesBoost: 3,
11 maxDateBoost: 8,         daysDecayFactor: 3,
12 maxDistanceBoost: 6,    distanceDecayFactor: 2,
13 };

```

Extracto de código 5.5: Pesos del sistema de recomendaciones (valores por defecto)

Rutas

El sistema permite trabajar con rutas de eventos. Estas rutas pueden ser generadas automáticamente a partir del motor de recomendaciones o creadas manualmente por los usuarios para compartir recorridos de interés.

Cada ruta almacena una secuencia ordenada de eventos, un trayecto geográfico, una temporización estimada en minutos y una puntuación promedio calculada a partir de las calificaciones de otros usuarios. Los usuarios pueden calificar las rutas, lo que retroalimenta el ranking de rutas disponibles.

La generación automática admite tres estrategias seleccionables:

- **balanced**: equilibra puntuación y distancia recorrida.
- **walkable**: minimiza la distancia total entre eventos consecutivos.
- **score**: maximiza la puntuación agregada de los eventos de la ruta.

Para construir rutas coherentes se comprueban además restricciones de solapamiento temporal entre eventos y separaciones máximas entre ellos.

5.5.3. Usuarios y funciones sociales

Este bloque agrupa la implementación del perfil, las relaciones entre usuarios y los mecanismos de interacción social fuera de un evento concreto.

Gestión de perfiles de usuario

Al autenticarse por primera vez, el backend crea automáticamente el registro de usuario en la base de datos a partir de los datos del token de Firebase (UID, email y nombre). Si el usuario ya existe por email pero aún no tiene `firebaseUid` asignado, el sistema lo vincula automáticamente.

El perfil almacena, además de los datos básicos, la ubicación geográfica del usuario (utilizada por las notificaciones de cercanía y las recomendaciones), una foto de perfil, un array de categorías de interés (intereses), un orden de categorías

preferido (categoryOrder) y los radios de búsqueda configurados por el usuario (radiusOptions).

Módulo de amistad y seguimiento

El sistema implementa un módulo de amistad basado en seguimiento entre usuarios. Un usuario puede buscar a otras personas, acceder a su perfil público, seguir las o dejar de seguir las, iniciar una conversación privada y consultar los eventos a los que asisten. La relación se almacena en la tabla intermedia `user_seguidores` como una relación muchos a muchos reflexiva sobre la entidad `User`, distinguiendo entre seguidores (quienes siguen al usuario) y seguidos (a quienes sigue el usuario).

En la interfaz se muestran tres pestañas sociales: *Seguidos*, con los usuarios a los que sigue la cuenta actual; *Seguidores*, con todos los usuarios que siguen a la cuenta actual; y *Amigos*, calculada como la intersección entre seguidores y seguidos. Cuando un usuario empieza a seguir de vuelta a un seguidor, este no desaparece de *Seguidores*, sino que pasa a aparecer también en *Amigos*.

Cuando un usuario empieza a seguir a otro, el sistema genera automáticamente una notificación de tipo **NuevoSeguidor** para el destinatario.

5.5.4. Comunicación y notificaciones

Las funcionalidades de comunicación combinan mensajes persistentes, eventos WebSocket y notificaciones consultables desde la aplicación.

Chat y mensajería

La aplicación incorpora dos tipos de comunicación distintos entre usuarios:

- **Chat de evento:** canal de mensajes asociado a un evento concreto. Cualquier usuario con acceso al evento puede leer y enviar mensajes en ese hilo. Los mensajes se almacenan en la tabla `mensajes` con referencia al evento y al usuario emisor, y pueden incluir imagen adjunta.
- **Mensajes privados:** conversaciones directas entre dos usuarios. Se almacenan en la tabla `mensajes_privados` con los campos `emisor`, `receptor` y un indicador de lectura (`leído`), lo que permite calcular conversaciones con mensajes no leídos.

Esta distinción permite separar la comunicación pública ligada a un evento de la comunicación personal entre usuarios, reforzando el componente social del sistema.

El chat se implementa en tiempo real mediante **Socket.IO**, integrado directamente en el servidor NestJS. La comunicación por WebSocket se autentica con el mismo token de Firebase que los endpoints REST: el middleware del socket

verifica el token antes de permitir la conexión. Los eventos principales del protocolo son los siguientes:

- **join_room**: el cliente se une a la sala del evento y recibe el historial de los últimos 50 mensajes.
- **chat_message**: envío de mensaje al chat de un evento; se retransmite a todos los usuarios de la sala.
- **dm_history** / **dm_message**: carga de historial y envío de mensajes privados entre dos usuarios.
- **mark_as_read**: marca como leídos los mensajes privados de un remitente concreto.
- **delete_dm** / **delete_event_message**: borrado de mensajes, restringido al propio emisor.
- **get_conversations**: devuelve la lista de conversaciones privadas con el último mensaje y el contador de no leídos.

Para prevenir el abuso, el servidor aplica un límite de velocidad por socket: máximo 10 mensajes por ventana de 10 segundos. Si se supera, el servidor rechaza el mensaje con un error `rate_limit` sin desconectar al usuario.

Notificaciones

El sistema implementa notificaciones persistentes almacenadas en base de datos. Cada notificación pertenece a uno de los siguientes tipos, definidos mediante un enumerado:

- **Aprobacion** / **Rechazado**: informan al creador de un evento del resultado de la moderación.
- **EventoProximo**: recuerda a los usuarios apuntados que el evento ocurre dentro de los próximos siete días o en menos de 24 horas.
- **EventoCercano**: avisa al usuario cuando un evento aprobado se encuentra a menos de 1 km de su ubicación registrada.
- **MensajePrivado**: tipo reservado para avisos asociados a conversaciones privadas.
- **NuevoSeguidor**: informa cuando otro usuario comienza a seguir al destinatario.
- **Recomendacion**: reservado para notificaciones generadas por el motor de recomendaciones.

Las notificaciones de tipo **EventoProximo** y **EventoCercano** se generan automáticamente desde los métodos de `NotificacionesScheduler`, invocados cada hora por el cron externo a través del endpoint de tareas programadas (véase la sección 5.3.4). Las notificaciones de moderación se generan en el momento en que se aprueba o rechaza el evento.

Cada notificación tiene un campo leída que permite al usuario marcarla como leída y filtrar las pendientes.

5.6. Carga inicial y mantenimiento de datos

La base de datos almacena las entidades principales del sistema: usuarios, eventos, categorías, asistencia, eventos guardados, valoraciones, mensajes, rutas, notificaciones y datos geográficos.

Durante el arranque del backend se puede ejecutar un proceso de carga inicial de datos, evitando duplicados si ya existen registros. Esta carga facilita disponer de datos de prueba durante el desarrollo y validación de la aplicación.

5.7. Despliegue y entorno de desarrollo

El procedimiento detallado para configurar el entorno local y desplegar el sistema se recoge en el Anexo [B](#) (Manual de desarrollo) y el Anexo [C](#) (Manual de despliegue).

5.8. Conclusiones

En este capítulo se ha descrito la implementación del sistema **Sevillaneando**, mostrando cómo las tecnologías seleccionadas permiten materializar los requisitos definidos en el análisis.

La combinación de React Native y Expo en el frontend, NestJS en el backend y PostgreSQL con PostGIS en la capa de persistencia permite construir una aplicación móvil con soporte para geolocalización, mapas, eventos personalizados y funcionalidades sociales.

Además, el uso de Docker facilita la reproducción del entorno de desarrollo, mientras que la integración con servicios externos como Firebase y el almacenamiento multimedia reduce la complejidad de la implementación y mejora la escalabilidad del sistema.

6. Pruebas

6.1. Introducción

En este capítulo se describe el proceso de validación del sistema **Sevillaneando**. Su objetivo es comprobar que las funcionalidades implementadas cumplen con los requisitos definidos durante el análisis y que el comportamiento observado en la aplicación coincide con el esperado.

El plan de pruebas combina tres tipos complementarios: pruebas unitarias automatizadas sobre los servicios críticos del backend, pruebas de integración que verifican la comunicación entre frontend, API y base de datos, y pruebas de aceptación manuales ejecutadas sobre la aplicación real en dispositivos físicos y emuladores. Estas últimas, que constituyen el grueso del capítulo, se organizan por bloques funcionales siguiendo la estructura de las historias de usuario del backlog.

6.2. Metodología y tipos de pruebas

6.2.1. Pruebas unitarias del backend

Se han implementado pruebas unitarias automatizadas sobre los tres servicios de mayor complejidad lógica del backend, utilizando **Jest** [61] como framework de pruebas. Las pruebas utilizan mocks de los repositorios de TypeORM para aislar la lógica de negocio de la base de datos, permitiendo verificar el comportamiento de cada método de forma independiente.

Los archivos de pruebas se encuentran en `sevillaneando/apps/backend/tests/` y se ejecutan con el comando `npm.cmd test - --runInBand` desde el directorio del backend en Windows.

EventsService

Se han implementado **20 pruebas unitarias** sobre `events.service.ts`, cubriendo la gestión de asistentes, la lógica de enlaces de acceso a eventos privados, la consulta de eventos y las reglas de edición según creador y moderador:

N.º	Descripción del test	Método	Comportamiento verificado
1	Genera enlace de acceso para evento privado sin enlace previo	<code>getPrivateShareLink</code>	Asigna un UUID nuevo y persiste el evento

2	Devuelve el enlace existente sin volver a guardar	getPrivateShareLink	No llama a save si el enlace ya existe
3	Lanza NotFoundException si el evento no existe	getPrivateShareLink	Excepción correcta ante evento inexistente
4	Lanza ForbiddenException si el evento no es privado	getPrivateShareLink	Rechaza la operación en eventos públicos
5	Lanza ForbiddenException si el solicitante no es el creador	getPrivateShareLink	Rechaza acceso de usuarios no propietarios
6	Añade asistente cuando no estaba apuntado	addAttendee	El usuario aparece en la lista de asistentes
7	No duplica asistente si ya estaba apuntado	addAttendee	No llama a save si el usuario ya asiste
8	Lanza NotFoundException si el evento no existe al añadir asistente	addAttendee	Excepción correcta ante evento inexistente
9	Lanza NotFoundException si el usuario no existe al apuntarse	addAttendee	Excepción correcta ante usuario inexistente
10	Elimina asistente correctamente	removeAttendee	El usuario desaparece de la lista de asistentes
11	Lanza NotFoundException al eliminar asistente de evento inexistente	removeAttendee	Excepción correcta ante evento inexistente
12	Devuelve evento por ID cuando existe	findOne	El resultado contiene el ID buscado
13	Lanza NotFoundException cuando el evento no existe	findOne	Excepción correcta ante búsqueda fallida
14	Devuelve la lista de asistentes de un evento existente	getAttendees	La lista contiene los usuarios correctos
15	Lanza NotFoundException al consultar asistentes de evento inexistente	getAttendees	Excepción correcta ante evento inexistente
16	Devuelve true si el usuario asiste al evento	isAttending	Detecta correctamente la asistencia existente
17	Devuelve false si el usuario no asiste al evento	isAttending	Detecta correctamente la ausencia de asistencia
18	Lanza ForbiddenException si un usuario que no es creador intenta editar un evento	update	Rechaza modificaciones no autorizadas
19	Lanza ForbiddenException si un moderador intenta editar un evento privado ajeno	update	Limita la edición moderada a eventos no privados

20	Permite al moderador editar un evento público sin devolverlo a pendiente	update	Mantiene el estado aprobado cuando la edición la realiza el moderador
----	--	--------	---

RecomendacionesService

Se han implementado **13 pruebas unitarias** sobre `recomendaciones.service.ts`, cubriendo la gestión de eventos guardados, el sistema de valoraciones y el algoritmo de scoring de recomendaciones:

N.º	Descripción del test	Método	Comportamiento verificado
1	Guarda un evento para el usuario	saveEvent	El evento se añade a <code>eventosGuardados</code> y se persiste
2	Elimina un evento guardado	unsaveEvent	El evento desaparece de <code>eventosGuardados</code>
3	Lanza <code>NotFoundException</code> al desguardar si el usuario no existe	unsaveEvent	Excepción correcta ante usuario inexistente
4	Crea una reseña nueva si el usuario no había valorado el evento	rateEvent	Se llama a <code>save</code> con la reseña creada
5	Devuelve valoración vacía si el usuario no tiene reseña	getMyEventRating	Respuesta con <code>hasRating: false</code> y campos nulos
6	Puntúa más alto el evento cuya categoría coincide con los intereses del usuario	recommendEvents	El score del evento de categoría preferida supera al de categoría no preferida
7	Incluye razón de intereses en el evento cuya categoría coincide	recommendEvents	El campo <code>reasons</code> contiene una entrada relativa a intereses
8	No añade razón de intereses si el usuario no tiene intereses configurados	recommendEvents	El campo <code>reasons</code> no menciona intereses cuando el perfil está vacío
9	Puntúa más alto el evento guardado frente a un evento sin historial	recommendEvents	El <code>bonus</code> de <code>eventosGuardados</code> eleva el score del evento ya guardado

10	Filtra eventos fuera del radio cuando se pasa ubicación del usuario	recommendEvents	El evento en Madrid no aparece en resultados con radio de 10 km centrado en Sevilla
11	Respetar el límite de resultados indicado	recommendEvents	La respuesta contiene como máximo el número de eventos solicitado
12	Normaliza categorías con acento — “Música” coincide con interés “musica”	recommendEvents	La normalización NFD equipara categoría acentuada con interés sin acento
13	Devuelve todos los criterios esperados en la respuesta	recommendEvents	El campo criterios incluye intereses-perfil, distancia, fecha y demás dimensiones

UsersService

Se han implementado **14 pruebas unitarias** sobre `users.service.ts`, cubriendo la creación y actualización de usuarios, el módulo de amistad basado en seguimiento social y la gestión de roles:

N.º	Descripción del test	Método	Comportamiento verificado
1	Crea usuario desde Firebase si no existe en BD	ensureFromFirebase	El usuario se crea con el UID y rol USER
2	Actualiza datos del usuario si han cambiado	ensureFromFirebase	Email y nombre quedan actualizados
3	No realiza ninguna acción si el seguidor y seguido son el mismo	seguir	No consulta la BD cuando los IDs son iguales
4	Añade seguimiento cuando no existe previamente	seguir	El seguido aparece en seguidos
5	No duplica el seguimiento si ya existía	seguir	No llama a save si ya seguía al usuario
6	Elimina el seguimiento correctamente	dejarDeSeguir	El usuario desaparece de seguidos
7	Devuelve la lista de seguidos de un usuario	getSeguidos	La lista contiene los IDs correctos
8	Devuelve lista vacía si el usuario no existe en getSeguidos	getSeguidos	Respuesta vacía sin lanzar excepción
9	Devuelve true si el usuario sigue a otro	isSiguiendo	Resultado correcto para relación existente

10	Devuelve false si el usuario no sigue a otro	isSiguiendo	Resultado correcto para relación inexistente
11	Actualiza el rol de un usuario existente	updateRole	El rol queda guardado con el nuevo valor
12	Actualiza campos de perfil correctamente	updateProfile	Nombre y email se actualizan y se persisten
13	Omite campos de perfil no proporcionados	updateProfile	Los campos no enviados conservan su valor anterior
14	No crea usuario cuando el UID de Firebase ya existe y el correo coincide	ensureFromFirebase	Devuelve el usuario existente sin crear duplicados

6.2.2. Resumen de pruebas unitarias

En total se han implementado **47 pruebas unitarias automatizadas** sobre los tres servicios de mayor complejidad lógica del backend. La ejecución actual con `npm.cmd test -- --runInBand` produce el siguiente resultado:

```
PASS tests/users/users.service.spec.ts
PASS tests/recomendaciones/recomendaciones.service.spec.ts
PASS tests/events/events.service.spec.ts
```

```
Test Suites: 3 passed, 3 total
Tests:       47 passed, 47 total
Snapshots:   0 total
Time:        7.936 s
```

Servicio	Pruebas	Estado
EventsService	20	Todas superadas
RecomendacionesService	13	Todas superadas
UsersService	14	Todas superadas
Total	47	47/47 superadas

Tabla 6.4: Resumen de pruebas unitarias del backend

6.2.3. Cobertura de código

La cobertura de código se midió con el informe integrado de Jest (`npm.cmd test -- --coverage --runInBand`), ejecutado sobre el backend completo. Los valores de la tabla corresponden al informe generado tras confirmar que las 47 pruebas se superan correctamente.

Tabla 6.5: Métricas de cobertura de código del backend

Ámbito	Sentencias	Ramas	Funciones	Líneas
Global (todos los archivos)	12,98 %	8,80 %	9,41 %	13,01 %
EventsService	45,49 %	26,02 %	41,86 %	44,76 %
RecomendacionesService	28,50 %	28,40 %	23,95 %	30,32 %
UsersService	52,34 %	50,68 %	44,00 %	54,62 %

La cobertura global del proyecto es baja (12,98 % de sentencias) porque la herramienta analiza la totalidad del código fuente del backend, incluyendo controladores, guards, scrapers, módulos de configuración y entidades TypeORM, ninguno de los cuales dispone de pruebas unitarias. Estos componentes son, por diseño, difíciles de aislar sin infraestructura real (base de datos, Firebase, Cloudinary), por lo que su validación se delega en las pruebas de aceptación sobre el entorno desplegado.

En los tres servicios que sí disponen de pruebas unitarias, la cobertura de sentencias oscila entre el 28 % y el 52 %, reflejando que los métodos de mayor complejidad lógica quedan cubiertos mientras que los métodos de acceso directo a datos (pasarela hacia TypeORM) quedan fuera del alcance de los mocks utilizados.

La ampliación de la cobertura mediante pruebas de integración con base de datos real y la incorporación de pruebas para controladores y guards se identifican como trabajo futuro en el capítulo de conclusiones.

6.2.4. Pruebas de aceptación

Las pruebas de aceptación son el tipo principal de prueba documentado en este capítulo. Se basan en las historias de usuario del backlog y verifican que el sistema cumple los criterios de aceptación definidos en cada historia. Se han ejecutado manualmente sobre dispositivos reales y emuladores, tal como se describe en la siguiente sección.

6.3. Entorno de pruebas

Las pruebas de aceptación se han ejecutado sobre dos entornos:

- **Entorno local:** servidor NestJS ejecutado en local, conectado a una instancia de PostgreSQL con PostGIS en Docker. Frontend compilado con Expo y ejecutado en un emulador Android a través de Android Studio.
- **Entorno desplegado:** backend desplegado en Render con base de datos en Supabase. Frontend ejecutado mediante **Expo Go con túnel**, probado principalmente sobre un **iPhone 16 Pro** con iOS. Se ha verificado que todos los módulos de la aplicación se abren correctamente, incluyendo el mapa (OpenStreetMap), el chat en tiempo real y las notificaciones.

En ambos entornos se comprobó el correcto funcionamiento de la interfaz, la navegación entre pantallas, la carga de datos desde la API y el comportamiento ante acciones del usuario.

6.3.1. Incidencias de compatibilidad con Android

Durante la fase de pruebas con usuarios piloto (véase el Anexo H), se detectaron problemas de compatibilidad en dispositivos Android reales que no habían aparecido en el emulador ni en las pruebas sobre iOS. Los principales problemas identificados fueron:

- **Mapa no visible:** el componente de mapa (OpenStreetMap mediante react-native-maps) no se renderizaba correctamente en algunos dispositivos Android, mostrando una pantalla en blanco. El problema estaba relacionado con la configuración del proveedor de tiles en la compilación de Expo Go para Android.
- **Descuadres de pantalla:** varios elementos de la interfaz (modales, cabeceras, botones flotantes) presentaban desajustes de posicionamiento en Android debido a diferencias en el comportamiento del *safe area* y el teclado virtual respecto a iOS.

Estos problemas fueron corregidos en una iteración adicional de la fase de cierre del proyecto, no prevista en la planificación original. La detección se produjo al distribuir la aplicación a usuarios piloto con dispositivos Android, ya que el desarrollo previo se había realizado íntegramente sobre iOS. Esta incidencia se recoge como desviación en el capítulo de gestión del proyecto y como lección aprendida en el capítulo de conclusiones.

6.4. Plan de pruebas de aceptación

6.4.1. Pruebas de autenticación y perfil

Este bloque valida el acceso al sistema y la correcta gestión de la información personal del usuario. Corresponden a HU-10, RF-01 y RF-02.

ID	HU / RF	Entrada	Acción	Validación esperada
CP-01	HU-10 / RF-01	Correo válido, contraseña válida y datos mínimos de registro	Crear una nueva cuenta e iniciar sesión con las credenciales registradas	El sistema autentica al usuario y muestra la pantalla principal sin errores. La sesión queda iniciada correctamente.

CP-02	HU-10 / RF-01	Correo o contraseña incorrectos	Intentar iniciar sesión con credenciales inválidas	El sistema deniega el acceso y muestra un mensaje de error, sin permitir entrar.
CP-03	HU-10 / RF-02	Nombre, intereses, ubicación e imagen de perfil modificados	Editar los datos del perfil desde el menú lateral	Los cambios se reflejan tras guardar y persisten al volver a abrir el perfil.
CP-04	HU-10 / RF-01	Correo electrónico asociado a la cuenta	Solicitar recuperación de contraseña	El sistema inicia el flujo de recuperación y acepta el correo indicado.

6.4.2. Pruebas de consulta, búsqueda y detalle de eventos

Verifican el núcleo principal de la aplicación: listado, filtrado y detalle de eventos. Corresponden a HU-1, HU-2, HU-14, HU-15, RF-03, RF-04 y RF-08.

ID	HU / RF	Entrada	Acción	Validación esperada
CP-05	HU-1 / RF-03	Aplicación autenticada con eventos en BD	Acceder a la página principal	Se muestra el listado con imagen, nombre, categoría y fecha, sin errores de carga.
CP-06	HU-2 / RF-03	Evento existente con datos completos	Seleccionar un evento del listado	Se abre la ficha detallada con descripción, ubicación, precio, asistentes, valoraciones y acciones disponibles.
CP-07	HU-14 / RF-04	Categoría, texto de búsqueda, radio y estado	Aplicar filtros desde el buscador avanzado	El listado muestra únicamente los eventos que cumplen los filtros seleccionados.
CP-08	HU-15 / RF-08	Evento público activo	Pulsar la acción de apuntarse en el detalle	El usuario queda como asistente y el cambio se refleja en el detalle y en el listado de apuntados.
CP-09	HU-15 / RF-08	Evento finalizado	Acceder al detalle desde el listado de finalizados	La información del evento es visible pero la acción de apuntarse aparece deshabilitada.

6.4.3. Pruebas de geolocalización y mapa

Validan el uso de la ubicación del usuario y el mapa de eventos. Corresponden a HU-6 y RF-05.

ID	HU / RF	Entrada	Acción	Validación esperada
CP-10	HU-6 / RF-05	Perfil con ubicación y eventos geolocalizados	Acceder al mapa de eventos	El mapa se carga correctamente con los eventos posicionados según su ubicación.
CP-11	HU-6 / RF-05	Distintos radios configurables	Cambiar el radio de búsqueda en el mapa	Los eventos visibles se ajustan al radio seleccionado.
CP-12	HU-6 / RF-05	Evento con coordenadas válidas	Seleccionar un marcador en el mapa	Se muestra el resumen del evento y al pulsarlo se accede a su ficha completa.

6.4.4. Pruebas de interacción social: chat, mensajería y valoraciones

Verifican el chat de evento, los mensajes privados y el sistema de reseñas. Corresponden a HU-8, HU-9, RF-09 y RF-11.

ID	HU / RF	Entrada	Acción	Validación esperada
CP-13	HU-8 / RF-09	Texto de mensaje e imagen opcional	Enviar mensaje en el chat de un evento	El mensaje aparece en el chat y queda visible para los participantes.
CP-14	HU-8 / RF-09	Dos cuentas autenticadas en el mismo evento	Iniciar conversación privada desde la lista de asistentes	El chat privado se abre correctamente y los mensajes son visibles para ambas cuentas.
CP-15	HU-9 / RF-11	Puntuación entre 1 y 5 y comentario opcional	Valorar un evento desde su ficha de detalle	La valoración se registra y se refleja en la puntuación media del evento.

6.4.5. Pruebas de recomendaciones y rutas

Validan el sistema de personalización, las recomendaciones y la generación de rutas. Corresponden a HU-11, RF-12 y RF-13.

ID	HU / RF	Entrada	Acción	Validación esperada
----	---------	---------	--------	---------------------

CP-16	HU-11 / RF-12	Usuario con intereses configurados y actividad previa	Abrir la sección "Para ti"	El sistema muestra recomendaciones coherentes con los intereses y la actividad del usuario.
CP-17	HU-11 / RF-13	Preferencias del usuario y eventos con fecha	Abrir la sección de rutas recomendadas	Se genera al menos una ruta con una agrupación coherente de eventos.
CP-18	HU-11 / RF-13	Datos mínimos de una ruta nueva	Crear una ruta manual y consultarla	La ruta queda almacenada y es visible en el listado con su vista de detalle.

6.4.6. Pruebas de eventos privados y gestión de eventos creados

Validan el flujo de eventos privados con enlace de acceso restringido. Corresponden a HU-3, HU-15, RF-06, RF-08, RF-20 y RF-21.

ID	HU / RF	Entrada	Acción	Validación esperada
CP-19	HU-3 / RF-06	Datos válidos para evento privado	Crear un evento privado	El sistema genera el evento y muestra un enlace de acceso compartible. El evento no aparece en el listado público.
CP-20	HU-15 / RF-21	Enlace privado válido	Acceder con otra cuenta usando el enlace	El usuario abre el detalle del evento mediante el enlace y dicho enlace queda guardado para poder consultarlo después desde la sección de eventos privados.
CP-21	HU-15 / RF-21	Sin enlace válido	Intentar acceder al evento sin enlace	El sistema deniega el acceso y el evento no aparece en ningún listado.

CP-22	HU-3 / RF-20	Evento privado ya creado o enlace privado aceptado	Acceder a "Guardados y privados" y "Mis eventos"	El evento privado aparece en la sección de privados cuando el usuario es creador o dispone del enlace. Solo el creador puede editarlo desde "Mis eventos".
CP-23	HU-3 / RF-20	Evento privado editado a público	Editar y cambiar visibilidad a pública	El evento pasa a estado pendiente y entra en cola de moderación, sin hacerse visible de inmediato.

6.4.7. Pruebas de administración y moderación

Validan que cada rol accede únicamente a sus opciones y puede realizar sus operaciones. Corresponden a HU-4, HU-5, HU-12, RF-07, RF-15, RF-16 y RF-17.

ID	HU / RF	Entrada	Acción	Validación esperada
CP-24	HU-12 / RF-17	Cuenta con rol administrador	Acceder al panel de administración	Se muestra la gestión de usuarios y se pueden modificar roles o eliminar cuentas. El panel no ofrece acciones de aprobación o rechazo de eventos.
CP-25	HU-12 / RF-16	Cuenta con rol administrador	Crear, editar o eliminar una categoría	Los cambios se reflejan en el sistema y la categoría queda disponible en filtros y formularios.
CP-26	HU-4 / RF-07	Cuenta moderador y evento pendiente	Acceder al panel de moderación	Se muestra la lista de eventos pendientes y se puede aprobar, rechazar o editar cada uno.
CP-27	HU-3 / RF-07	Evento del CP-23 en moderación	Verificar visibilidad del evento durante moderación	El evento no aparece en el listado público hasta que el moderador lo aprueba.

CP-28	HU-12 / RF-15	Cuenta con rol usuario estándar	Intentar acceder al panel de administración o moderación	El sistema deniega el acceso y no muestra esas opciones en la interfaz.
-------	---------------	---------------------------------	--	---

6.4.8. Pruebas de notificaciones

Validan las notificaciones de moderación y de eventos cercanos. Corresponden a HU-5, HU-7 y RF-14.

ID	HU / RF	Entrada	Acción	Validación esperada
CP-29	HU-5 / RF-14	Evento moderado por un moderador	Abrir la pantalla de notificaciones tras la resolución	El sistema muestra una notificación con el resultado de la moderación (aprobado o rechazado).
CP-30	HU-7 / RF-14	Perfil con ubicación y eventos cercanos	Revisar las notificaciones con la app activa	El sistema muestra avisos de eventos próximos a la ubicación del usuario.

6.4.9. Pruebas complementarias incorporadas en la revisión final

Durante la revisión del capítulo se han añadido casos que no estaban reflejados en la versión anterior del plan, pero que sí forman parte del código actual: guardados, seguimiento social, acciones nuevas sobre rutas, administración de scraping y notificaciones temporales invocadas por el cron externo.

ID	HU / RF	Entrada	Acción	Validación esperada
CP-31	HU-11 / RF-12	Evento público disponible	Guardar y desguardar un evento desde su detalle	El evento aparece en "Guardados y privados" tras guardarlo y desaparece al desguardarlo.
CP-32	HU-13 / RF-18	Dos usuarios registrados	Seguir y dejar de seguir a otro usuario desde su perfil	Las listas de seguidos, seguidores y amigos mutuos se actualizan correctamente.
CP-33	HU-13 / RF-14	Usuario seguido por otra cuenta	Abrir las notificaciones después de recibir un nuevo seguidor	Se muestra una notificación informando del nuevo seguimiento.

CP-34	HU-8 / RF-09	Mensaje propio en chat de evento o mensaje directo	Eliminar el mensaje enviado por el usuario autenticado	El mensaje desaparece de la conversación y no se permite eliminar mensajes de otros usuarios.
CP-35	HU-11 / RF-13	Ruta existente con detalle visible	Valorar la ruta desde su pantalla de detalle	La valoración queda registrada y se actualiza la puntuación media de la ruta.
CP-36	HU-11 / RF-13	Ruta creada por el usuario autenticado	Eliminar la ruta desde su detalle	La ruta se elimina del listado del creador y no queda accesible desde su ficha.
CP-37	HU-12 / RF-17	Cuenta con rol administrador	Consultar la pestaña de estadísticas del panel de administración	El sistema muestra indicadores globales de usuarios, eventos y actividad sin permitir acciones de moderación.
CP-38	HU-12 / RF-17	Cuenta con rol administrador	Ejecutar el restablecimiento del scraping desde el panel	El sistema limpia el estado asociado al scraping y permite relanzar la carga externa controlada desde administración.
CP-39	HU-4 / RF-07	Cuenta con rol administrador y evento pendiente	Intentar acceder al flujo de aprobación o rechazo de eventos	El sistema no muestra el panel de moderación al administrador y el backend restringe aprobar/rechazar al rol moderador.
CP-40	HU-7 / RF-14	Usuario apuntado a eventos con fecha próxima	Ejecutar el scheduler mediante el endpoint invocado por el cron externo	Se generan notificaciones de eventos próximos en el tiempo, incluyendo avisos de siete días y de menos de 24 horas cuando procede.

6.5. Resultados de las pruebas de aceptación

Los 40 casos de prueba de aceptación se han ejecutado sobre los dos entornos descritos: entorno local con emulador Android y entorno móvil real sobre iPhone

16 Pro mediante Expo Go con túnel, conectando en ambos casos con el backend desplegado en Render. La siguiente tabla recoge los resultados obtenidos:

ID	Funcionalidad	Resultado	Observaciones
CP-01	Registro e inicio de sesión	Superada	
CP-02	Login con credenciales incorrectas	Superada	
CP-03	Edición de perfil	Superada	
CP-04	Recuperación de contraseña	Superada	
CP-05	Listado de eventos	Superada	
CP-06	Detalle de evento	Superada	
CP-07	Filtros de búsqueda	Superada	
CP-08	Apuntarse a evento público	Superada	
CP-09	Evento finalizado sin acción	Superada	
CP-10	Mapa de eventos	Superada	El mapa OpenStreetMap se carga correctamente en Android e iOS. Este caso de prueba verificó además la resolución del problema de visualización en Android detectado durante el Sprint 5 (véase sección 2).
CP-11	Radio de búsqueda en mapa	Superada	
CP-12	Detalle desde marcador de mapa	Superada	
CP-13	Chat de evento	Superada	
CP-14	Mensajes privados	Superada	
CP-15	Valoración de evento	Superada	
CP-16	Recomendaciones personalizadas	Superada	
CP-17	Rutas recomendadas	Superada	
CP-18	Creación de ruta manual	Superada	
CP-19	Creación de evento privado	Superada	
CP-20	Acceso con enlace privado	Superada	
CP-21	Acceso denegado sin enlace	Superada	
CP-22	Gestión de mis eventos privados	Superada	

CP-23	Cambio de privado a público	Superada	El evento entra en moderación correctamente.
CP-24	Panel de administrador – usuarios	Superada	
CP-25	Panel de administrador – categorías	Superada	
CP-26	Panel de moderación	Superada	
CP-27	Evento en moderación no visible	Superada	
CP-28	Acceso denegado a admin/moderación	Superada	
CP-29	Notificación de moderación	Superada	
CP-30	Notificación de evento cercano	Superada	
CP-31	Guardar y desguardar evento	Superada	
CP-32	Seguimiento social	Superada	
CP-33	Notificación de nuevo seguidor	Superada	
CP-34	Eliminación de mensaje propio	Superada	
CP-35	Valoración de ruta	Superada	
CP-36	Eliminación de ruta propia	Superada	
CP-37	Panel de administrador – estadísticas	Superada	
CP-38	Panel de administrador – scraping	Superada	
CP-39	Administrador sin moderación	Superada	
CP-40	Notificaciones temporales por cron externo	Superada	El scheduler se valida mediante el endpoint invocado por el cron externo, no mediante un @Cron interno del backend.

Todos los casos de prueba han sido superados satisfactoriamente. El sistema se comporta de acuerdo con los requisitos funcionales definidos en el análisis tanto en el entorno de desarrollo local como en el entorno móvil conectado al backend de producción.

6.6. Conclusiones

El plan de pruebas del sistema **Sevillaneando** ha combinado pruebas unitarias automatizadas del backend y pruebas de aceptación manuales ejecutadas sobre dispositivos reales, cubriendo los principales requisitos funcionales del sistema.

Las **47 pruebas unitarias** implementadas sobre los servicios de mayor complejidad lógica (EventsService, RecomendacionesService y UsersService) han superado correctamente todos los casos, incluyendo tanto flujos normales como situaciones de error. En particular, las pruebas sobre el algoritmo de scoring de RecomendacionesService verifican aspectos críticos como la influencia de los intereses del usuario en la puntuación, el filtrado por radio geográfico, el bonus por historial de guardado y la normalización de acentos en categorías. Los **40 casos de prueba de aceptación** ejecutados sobre la aplicación real han confirmado que el sistema funciona correctamente en los dos entornos probados, tanto en Android (emulador local) como en iOS (iPhone 16 Pro con Expo Go mediante túnel y backend desplegado en Render).

El resultado global es que todos los casos de prueba han sido superados, lo que valida que el sistema cumple con los requisitos funcionales definidos y que los principales flujos de uso de la aplicación funcionan de forma correcta y coherente.

7. Conclusiones

7.1. Introducción

Este capítulo recoge las conclusiones obtenidas tras el desarrollo del sistema **Sevillaneando**. Se revisan los objetivos planteados al inicio del proyecto y el grado en que han sido alcanzados, se analizan las lecciones técnicas aprendidas durante el desarrollo, se cuantifican y justifican las desviaciones respecto a la planificación original y se proponen líneas de trabajo futuro que permitirían ampliar y mejorar el sistema.

7.2. Objetivos alcanzados

El capítulo de estudio previo definió dos categorías de objetivos: los objetivos del proyecto, orientados al aprendizaje y la formación de la estudiante, y los objetivos del sistema, que describen las funcionalidades de la aplicación. A continuación se revisa el grado de cumplimiento de ambos.

7.2.1. Objetivos del proyecto

Los siete objetivos del proyecto definidos en el capítulo 2 ([OP-01–OP-07](#)) han sido alcanzados en su totalidad:

- **OP-01 — Aplicación de conocimientos del Grado:** alcanzado. El proyecto ha integrado de forma transversal contenidos de diseño de software, arquitecturas de sistemas, ingeniería de requisitos y gestión de proyectos, aplicándolos en un contexto real y complejo.
- **OP-02 — Planificación y autogestión:** alcanzado. La gestión unipersonal del proyecto mediante metodología ágil adaptada ha requerido priorizar el backlog, revisar el alcance en función del tiempo disponible y tomar decisiones autónomas en cada sprint.
- **OP-03 — Experiencia en tecnologías modernas:** alcanzado. Se ha trabajado con React Native, NestJS, PostgreSQL/PostGIS y Firebase en un proyecto de extremo a extremo con despliegue real en producción.
- **OP-04 — Primer contacto con desarrollo móvil:** alcanzado. Se ha afrontado el ciclo de vida de una aplicación móvil, la adaptación a iOS y Android, y la compilación y distribución mediante Expo.
- **OP-05 — Consolidación de TypeScript:** alcanzado. TypeScript se ha utilizado de forma rigurosa tanto en el frontend como en el backend, profundizando en

el tipado estático y las buenas prácticas del lenguaje.

- **OP-06 — Retos técnicos no triviales:** alcanzado. El sistema integra comunicación bidireccional en tiempo real, consultas geoespaciales con PostGIS, scraping automatizado de fuentes externas y un motor de recomendaciones multifactorial.
- **OP-07 — Documentación técnica:** alcanzado. El proyecto incluye memoria, manual de usuario, manual de desarrollo y manual de despliegue como parte de las entregas.

7.2.2. Objetivos del sistema

El objetivo principal del sistema era desarrollar una aplicación móvil multiplataforma que centralizase la oferta de ocio y cultura de Sevilla, con especial énfasis en dar visibilidad a eventos emergentes. A continuación se revisa el grado de cumplimiento de cada uno de los objetivos específicos definidos en la Sección 2.2.2 del capítulo 2:

- **OBJ-01 — Listado y filtrado de eventos:** implementado completamente. La pantalla principal ofrece filtrado por categoría, precio, fecha y radio de distancia, con soporte para geolocalización en tiempo real.
- **OBJ-02 — Creación de eventos con moderación:** implementado completamente. Los eventos públicos siguen un flujo de moderación con estados *Pendiente*, *Aprobado* y *Rechazado*. Los moderadores pueden revisar eventos pendientes en su listado propio y editar o eliminar todos los eventos públicos desde un listado específico, mientras que los eventos privados se aprueban automáticamente y son accesibles mediante enlace de invitación.
- **OBJ-03 — Edición de eventos propios:** implementado completamente, incluyendo la lógica de vuelta a estado *Pendiente* al editar un evento público aprobado y la posibilidad de eliminar eventos desde la pantalla de edición.
- **OBJ-04 — Geolocalización y mapa interactivo:** implementado completamente mediante OpenStreetMap y react-native-maps, con filtrado por radio y visualización de marcadores en mapa.
- **OBJ-05 — Interacción social:** implementado completamente. El sistema incluye chat de evento (sala grupal por evento), mensajería privada entre usuarios, valoraciones con puntuación y comentario, y módulo social basado en seguimiento, con listados diferenciados de seguidores, seguidos y amistades mutuas.
- **OBJ-06 — Recomendaciones personalizadas:** implementado completamente. El motor aplica un algoritmo multifactorial ponderado que tiene en cuenta categorías de interés, historial de interacciones (guardar, asistir, compartir, ver, valorar), proximidad geográfica y temporal.
- **OBJ-07 — Rutas recomendadas:** implementado completamente. El motor genera itinerarios automáticos con tres estrategias: *balanced*, *walkable* y *score*.

- **OBJ-08 — Rutas de usuario:** implementado completamente. Los usuarios pueden crear rutas visibles para el resto de la comunidad, que pueden ser valoradas por otros usuarios.
- **OBJ-09 — Notificaciones:** implementado completamente. El sistema envía notificaciones por aprobación o rechazo de eventos, eventos cercanos, eventos próximos, nuevos mensajes privados, nuevos seguidores y recomendaciones generadas. Además, las notificaciones relacionadas con perfiles o eventos incluyen acciones para navegar directamente al perfil o al evento correspondiente.

Adicionalmente, se implementó la documentación automática de la API con Swagger: el backend expone un contrato OpenAPI completo mediante @nestjs/swagger, accesible en /api/docs. En cuanto a los objetivos no alcanzados, la generación automática de carteles con IA y la moderación de imágenes mediante NSFWJS (detección de contenido inapropiado) quedaron fuera del alcance final del proyecto, tal como se detalló en el análisis de desviaciones del capítulo de estudio previo.

7.3. Resultados obtenidos

El resultado principal del proyecto es una aplicación móvil multiplataforma funcional para el descubrimiento, creación y gestión de eventos en Sevilla. La solución final integra una aplicación cliente desarrollada con React Native y Expo, un backend basado en NestJS, una base de datos PostgreSQL con extensión PostGIS y distintos servicios externos para autenticación, almacenamiento de imágenes, mapas, geocodificación y obtención de eventos desde fuentes externas.

Desde el punto de vista funcional, el sistema permite consultar eventos, filtrarlos por criterios relevantes, visualizarlos en un mapa, crear, editar y eliminar eventos públicos y privados, moderar contenido, participar en chats, enviar mensajes privados, buscar usuarios, consultar sus perfiles, seguir o dejar de seguir a otras personas, revisar seguidores, seguidos y amistades mutuas, ver los eventos a los que asisten, valorar eventos, recibir notificaciones accionables y obtener recomendaciones personalizadas. Además, se han incorporado rutas de eventos recomendadas y rutas creadas por los propios usuarios, ampliando el alcance de la aplicación más allá de un listado convencional de actividades.

Desde el punto de vista documental, el proyecto ha generado una memoria completa, un manual de usuario, un manual de desarrollo, un manual de despliegue, actas de reunión, informes de avance y cambio, el reporte de horas de Clockify y la declaración de autoría correspondiente. Todo ello permite presentar el trabajo no solo como una aplicación terminada, sino como un proyecto de ingeniería documentado, planificado, probado y preparado para su mantenimiento o evolución futura.

7.4. Lecciones aprendidas

Los objetivos del proyecto (OP-01–OP-07), definidos en la Sección 2.2.2 del capítulo 2, establecían compromisos de aprendizaje concretos. Esta sección reflexiona sobre qué ha aportado realmente cada uno de ellos.

- **OP-01 — Aplicación de conocimientos del Grado.** El proyecto ha demostrado que las asignaturas de diseño de software, arquitecturas, ingeniería de requisitos y gestión de proyectos no son compartimentos estancos. Aplicarlas simultáneamente en un contexto con consecuencias reales —una mala decisión de diseño en el modelo de datos repercutía directamente en las semanas siguientes— ha consolidado esos conocimientos de una forma que ningún ejercicio académico aislado puede reproducir.
- **OP-02 — Planificación y autogestión.** Gestionar el proyecto sin equipo ha evidenciado que la priorización del backlog es una habilidad no trivial. La combinación de estimación por tallas de camiseta y seguimiento real con Clockify ha sido fundamental para detectar desviaciones a tiempo y replantear el alcance de forma razonada, en lugar de acumular deuda y tomar decisiones precipitadas al final.
- **OP-03 — Experiencia en tecnologías modernas.** Trabajar con React Native, NestJS, PostgreSQL/PostGIS y Firebase en un proyecto de extremo a extremo ha confirmado que la curva de aprendizaje real de estas tecnologías está en su integración conjunta, no en cada una por separado. Las fricciones aparecen en los bordes: entre el frontend y el backend, entre la autenticación y los guards, entre el ORM y las funciones espaciales.
- **OP-04 — Primer contacto con desarrollo móvil.** El ciclo de vida de una aplicación móvil, las diferencias de comportamiento entre iOS y Android, el sistema de permisos y el proceso de compilación con Expo son conceptos sin equivalente directo en el desarrollo web. La lección más relevante ha sido aprender a leer documentación de alto nivel rápidamente y distinguir qué hay que entender a fondo de qué puede tratarse inicialmente como caja negra.
- **OP-05 — Consolidación de TypeScript.** Usar TypeScript de forma rigurosa en frontend y backend a la vez ha sido cualitativamente distinto a su uso en ejercicios académicos. Los DTOs con validación, los guards tipados de NestJS y los tipos de retorno de los hooks de React han hecho que muchos errores de integración fueran detectables en tiempo de compilación. El valor real de TypeScript no está en los tipos básicos, sino en modelar contratos entre módulos y hacerlos verificables automáticamente.
- **OP-06 — Retos técnicos no triviales.** Cada módulo complejo —scraping, recomendaciones, rutas, chat en tiempo real, consultas geoespaciales— ha requerido una fase de investigación previa no contemplada en la estimación inicial. La incertidumbre técnica es un riesgo de primer orden en ingeniería del software: prototipar y validar un enfoque antes de comprometerse con él en el backlog es una práctica que este proyecto ha confirmado como

imprescindible.

- **OP-07 — Documentación técnica.** Redactar la memoria, los manuales y los anexos ha supuesto una parte significativa del esfuerzo total. Esta experiencia ha cambiado la percepción de la documentación: no es un trámite que se hace al final, sino una actividad que revela inconsistencias en el diseño y obliga a articular decisiones que de otro modo quedarían implícitas en el código.

7.5. Desviaciones respecto a la planificación

Las desviaciones producidas durante el proyecto —incorporación de CI/CD, cambio en Expo Go, mayor complejidad del scraping, ajustes en rutas y recomendaciones, integración de mapas, no implementación de NSFWJS y generación de carteles con IA, y corrección de compatibilidad con Android— se describen en detalle, con su impacto cuantitativo y justificación, en la sección [2.4.2](#) del capítulo 2.

En conjunto, el proyecto se completó dentro del plazo establecido y con el alcance funcional prioritario cubierto al 100 %.

Con la perspectiva que da el cierre del proyecto, estas desviaciones revelan tres lecciones de fondo sobre la naturaleza de los proyectos de software:

- **La incertidumbre técnica es inherente, no un fallo de planificación.** Las desviaciones más relevantes —el scraping, el sistema de rutas y recomendaciones y la integración de mapas— no eran completamente predecibles con la información disponible al inicio. En ingeniería del software, una parte del esfuerzo real solo se descubre durante la implementación. Aceptar esto como una característica del trabajo, y no como un error de estimación, es una forma de madurez profesional que este proyecto ha ayudado a interiorizar.
- **Las dependencias ocultas entre módulos son más peligrosas que la complejidad individual.** El impacto del scraping sobre el sistema de rutas no era visible en el diseño inicial porque cada módulo parecía independiente. La lección no es solo técnica —analizar dependencias antes de estimar— sino de gestión: cuando dos módulos comparten datos o supuestos, un retraso en uno se convierte automáticamente en un retraso en el otro, y eso hay que anticiparlo al planificar el orden de desarrollo.
- **Entregar algo que funciona vale más que cumplir el plan original.** La decisión de no implementar las funcionalidades de la Fase 5 no fue un fracaso: fue una elección consciente de priorizar la calidad del núcleo sobre la cantidad de características. Esta capacidad de ajustar el alcance sin perder el objetivo principal es, probablemente, la lección de gestión más transferible de todo el proyecto.

7.6. Consideraciones de seguridad y privacidad

Durante el desarrollo se han aplicado medidas de seguridad a distintos niveles del sistema:

- **Autenticación:** gestionada íntegramente por Firebase Authentication, que garantiza el almacenamiento seguro de credenciales y la emisión de tokens JWT de corta duración.
- **Autorización:** todos los endpoints protegidos verifican el token en cada petición mediante un guard de Firebase. Los endpoints de moderación y administración aplican un guard adicional de roles.
- **Limitación de velocidad:** se aplica *throttling* en tres niveles (estricto, moderado y subida) mediante @nestjs/throttler para prevenir abusos y ataques de fuerza bruta.
- **Validación de entrada:** los DTOs del backend utilizan class-validator para validar y sanear todos los datos recibidos del cliente antes de procesarlos.
- **Privacidad de datos:** la aplicación no almacena contraseñas (delegadas en Firebase) ni datos de pago. Los datos de ubicación del usuario solo se utilizan para consultas espaciales y no se registran con seguimiento histórico.
- **Eventos privados:** el acceso a eventos privados requiere un enlace de invitación único generado en el backend, que no es predecible ni enumerable.
- **Row-Level Security en Supabase:** tras recibir un aviso de seguridad crítico de Supabase, se activó la política de *Row-Level Security* (RLS) en las 18 tablas del esquema público de la base de datos. Sin esta medida, cualquier persona con la URL del proyecto y la clave anónima podía leer, modificar y eliminar datos directamente a través de la API REST de Supabase, sin pasar por el backend. El backend NestJS no se ve afectado, ya que se conecta como *service role*, que omite las políticas RLS por diseño.

7.6.1. Limitaciones respecto al RGPD

La aplicación recoge y almacena datos personales sensibles sujetos al Reglamento General de Protección de Datos (RGPD, Reglamento UE 2016/679): ubicación geográfica en tiempo real, fotografías de perfil, historial de interacciones con eventos y mensajes privados entre usuarios. En el contexto de este TFG, la aplicación no está publicada en producción para el público general, por lo que estas obligaciones legales no son exigibles en esta fase. No obstante, se identifican a continuación las medidas implementadas y las limitaciones conocidas que deberían abordarse antes de cualquier despliegue real.

Medidas implementadas. La pantalla de registro incluye un checkbox de aceptación de los términos de uso y la política de privacidad, basando el tratamiento de datos en el consentimiento del usuario (RGPD art. 6.1.a). La fecha y hora de dicha aceptación se registran en la base de datos (privacyAcceptedAt)

desde el primer inicio de sesión, lo que permite acreditar el consentimiento si fuera necesario. El endpoint de eliminación de cuenta (DELETE /users/me/firebase) propaga la baja a Firebase Authentication y también elimina la fotografía de perfil almacenada en Cloudinary mediante el publicId persistido en la entidad de usuario. Los mensajes privados se eliminan en cascada al borrar cualquiera de los dos participantes (emisor o receptor), gracias a la política CASCADE definida en la entidad MensajePrivado, garantizando así que ningún mensaje quede huérfano en la base de datos. Además, la pantalla de edición de perfil incluye un aviso informativo específico sobre el uso de la ubicación: se explica que se utiliza para mostrar eventos cercanos, ordenar resultados por distancia y mejorar las recomendaciones, y que no se comparte con terceros ni se registra en tiempo real, cumpliendo así el requisito RGPD de consentimiento informado y específico para este dato.

Limitaciones conocidas:

- **Consentimiento explícito para la geolocalización:** implementado. La aplicación no solicita permiso de GPS del sistema operativo — la ubicación se establece manualmente en la pantalla de edición de perfil. Dicha pantalla incluye un aviso informativo que explica al usuario para qué se utiliza su localización y que no se comparte con terceros ni se registra en tiempo real, cumpliendo el requisito RGPD de consentimiento libre, específico, informado e inequívoco.
- **Procedimiento formal de ejercicio del derecho al olvido:** aunque la eliminación técnica de la cuenta y los datos principales está implementada, no existe un canal documentado para que el usuario pueda ejercer formalmente este derecho frente a la responsable del tratamiento, ni un registro de solicitudes recibidas y atendidas, tal como exige el RGPD.

Estas limitaciones se documentan como trabajo futuro en la sección siguiente y constituyen una de las tareas prioritarias antes de un lanzamiento real de la aplicación.

7.7. Trabajo futuro

A lo largo del desarrollo han surgido ideas de mejora y nuevas funcionalidades que, por restricciones de tiempo o alcance, no han podido implementarse en esta versión del sistema. Se organizan a continuación en dos bloques: mejoras sobre funcionalidades existentes y nuevas funcionalidades.

7.7.1. Mejoras sobre funcionalidades existentes

- **Cumplimiento RGPD:** las medidas de consentimiento (checkbox en el registro con privacyAcceptedAt), derecho al olvido para imágenes de perfil (eliminación en Cloudinary al borrar la cuenta) y la eliminación en cascada de mensajes privados al borrar cualquiera de los participantes ya están

implementadas. La única tarea pendiente antes de un despliegue real es establecer un procedimiento formal y documentado para el ejercicio del derecho al olvido. Estas limitaciones se identifican en la sección de consideraciones de seguridad y privacidad.

- **Moderación de contenido inapropiado (NSFW):** Completar esta funcionalidad evitaría la subida de imágenes inapropiadas en eventos y chats, mejorando la seguridad del contenido sin requerir revisión manual para cada imagen.
- **Verificación de correos electrónicos:** actualmente, el registro de usuarios no incluye un paso de validación del correo mediante enlace de confirmación. Implementar esta funcionalidad mediante Firebase Auth o un servicio de correo transaccional (SendGrid, Mailgun) aumentaría la confiabilidad de las direcciones de correo registradas y permitiría re-enviar invitaciones o notificaciones críticas con mayor seguridad. Además, facilitaría la recuperación de cuentas y reduciría registros fraudulentos o con errores tipográficos.
- **Cobertura de pruebas:** el plan de pruebas actual cubre los flujos principales pero deja fuera casos límite en el motor de recomendaciones, el scraping y la lógica de moderación. Ampliar la suite con pruebas de carga y pruebas de regresión automatizadas aumentaría la fiabilidad del sistema en producción.

7.7.2. Nuevas funcionalidades

- **Generación de carteles con IA:** identificada desde el inicio como objetivo de la Fase 5, esta funcionalidad permitiría a los usuarios generar automáticamente un cartel visual para su evento a partir del título, la descripción y las imágenes proporcionadas, integrando un modelo generativo de imagen a través de una API externa.
- **Modo sin conexión básico:** actualmente no existe detección de red ni caché local. Mediante AsyncStorage o React Query se podrían almacenar localmente los últimos eventos cargados y mostrarlos sin conexión, mejorando la experiencia de usuario en entornos con conectividad intermitente.
- **Achievements y gamificación:** dado que el sistema ya registra visitas, asistencias, guardados y compartidos, sería natural introducir un sistema de insignias: *Explorador* (10 eventos distintos visitados), *Foodie* (5 eventos de gastronomía asistidos), *Organizador* (3 eventos propios aprobados), entre otras. Este mecanismo aumentaría la retención y la participación activa de los usuarios.
- **Módulo avanzado de amigos:** actualmente el sistema dispone de un módulo social basado en seguimiento, con búsqueda de usuarios, perfiles públicos, opción de seguir o dejar de seguir, distinción entre seguidores, seguidos y amistades mutuas, mensajería privada y consulta de eventos a los que asisten otros usuarios. Como ampliación futura, se podría incorporar una gestión

más explícita de contactos cercanos, invitaciones directas a eventos y recomendaciones sociales basadas en intereses compartidos.

- **Integración con el calendario nativo del dispositivo:** la pantalla CalendarEventsScreen ya existe pero no hay integración con el calendario nativo del teléfono. Mediante expo-calendar sería posible añadir directamente los eventos a los que el usuario se apunta en Google Calendar o Apple Calendar, sin salir de la aplicación.
- **Reseñas con fotos:** las reseñas (Resena) actualmente contienen puntuación y comentario de texto. Permitir adjuntar fotografías en las valoraciones aumentaría la credibilidad de las reseñas y la confianza de los usuarios a la hora de apuntarse a un evento.
- **Asistencia mediante código QR:** integrar un sistema de códigos QR que el organizador pueda mostrar en el evento permitiría registrar la asistencia de forma sencilla. Combinado con un enlace de inscripción externo (Eventbrite, formularios propios), facilitaría la interoperabilidad con otras plataformas de gestión de eventos.
- **Cuentas de empresa y promoción de eventos:** actualmente todos los usuarios tienen el mismo tipo de cuenta. Introducir un perfil de empresa o entidad permitiría a organizaciones y negocios registrarse con un perfil verificado y destacado, con la posibilidad de promocionar sus eventos dentro de la plataforma mediante un modelo de visibilidad preferente.
- **Soporte de eventos online:** el modelo de evento actual está orientado a actividades presenciales, con dirección física y ubicación geográfica como atributos centrales. Ampliar el modelo para contemplar eventos online (conciertos en streaming, talleres por videoconferencia, presentaciones virtuales) requeriría añadir un campo de tipo de celebración (*presencial*, *online* o *híbrido*), sustituir la ubicación por un enlace de acceso y adaptar el motor de filtrado para que los eventos online sean visibles independientemente del radio de búsqueda del usuario. Esta extensión ampliaría significativamente el catálogo disponible y abriría la plataforma a organizadores que operan sin espacio físico fijo.
- **Pasarela de pago integrada:** actualmente la aplicación muestra el precio de un evento como información orientativa pero no gestiona la compra de entradas. Integrar una pasarela de pago (por ejemplo, Stripe) permitiría al usuario adquirir su entrada directamente desde la pantalla de detalle del evento, sin necesidad de redirigirle a una plataforma externa. Esta funcionalidad requeriría ampliar el modelo de datos con entidades de pedido y ticket, implementar los webhooks de confirmación de pago, y gestionar la emisión de entradas en formato QR escaneables en el acceso al evento. Desde el punto de vista del organizador, ofrecería un canal de venta propio con control sobre la lista de asistentes.
- **Soporte multiidioma (i18n):** la aplicación incorpora actualmente un soporte parcial de internacionalización mediante react-i18next, con español como idioma principal e inglés como segundo idioma seleccionable desde la

edición del perfil. No obstante, esta traducción cubre únicamente los literales de la interfaz (menús, botones, mensajes del sistema), mientras que el contenido dinámico generado por los usuarios o importado por los scrapers (títulos, descripciones y direcciones de los eventos, comentarios de las valoraciones, mensajes del chat, nombres de las categorías, etc.) sigue mostrándose en su idioma original. Completar el soporte multidioma requeriría integrar un servicio de traducción automática (DeepL, Google Translate, o un modelo propio) que traduzca el contenido bajo demanda según el idioma seleccionado por el usuario, así como añadir nuevos idiomas relevantes como francés, alemán o italiano, especialmente útiles dado el carácter turístico de Sevilla y la diversidad de visitantes internacionales que recibe.

7.8. Uso de herramientas de inteligencia artificial

Durante el desarrollo de este proyecto se han utilizado herramientas de inteligencia artificial como apoyo en determinadas tareas. A continuación se detalla el uso realizado, las herramientas empleadas y el proceso de verificación seguido en cada caso, en cumplimiento con los principios de transparencia exigidos por la Escuela Técnica Superior de Ingeniería Informática de la Universidad de Sevilla.

7.8.1. Herramientas utilizadas

- **GitHub Copilot:** asistente de autocompletado de código integrado en el entorno de desarrollo. Se ha utilizado para sugerir fragmentos de código repetitivo (DTOs, validadores, controladores NestJS con estructura similar) y para acelerar la escritura de código boilerplate en el frontend (componentes con estructura análoga a otros ya existentes). Además, se ha empleado puntualmente para proponer mejoras visuales en componentes de interfaz, como ajustes de estilos y disposición de elementos.
- **Claude (Anthropic):** modelo de lenguaje utilizado para resolver dudas técnicas concretas (p.ej., uso de funciones espaciales de PostGIS, configuración de guardias de autenticación en NestJS), proponer mejoras visuales en la interfaz y como guía en procesos técnicos más elaborados, como el despliegue en producción y la revisión de vulnerabilidades de seguridad. Asimismo, la redacción de la memoria ha sido revisada con ayuda de esta herramienta para mejorar la expresión y coherencia del texto. De forma específica, se utilizó para generar el contenido inicial de la tabla de endpoints de la API REST (Tabla 4.2) a partir de la estructura de controladores y DTOs del backend, dado el carácter mecánico y repetitivo de esta tarea; el resultado fue revisado, corregido y validado íntegramente por la autora.

7.8.2. Alcance del uso

El uso de estas herramientas ha sido complementario y supervisado en todo momento. No se ha delegado en ellas la toma de decisiones de diseño, arquitectónicas ni funcionales; éstas han sido adoptadas íntegramente por la autora. El código generado automáticamente ha sido siempre revisado, comprendido y adaptado antes de integrarse en el proyecto, nunca copiado directamente sin verificación. Las guías proporcionadas para procesos como el despliegue o la revisión de seguridad han servido como referencia técnica, pero las decisiones finales y su implementación han sido responsabilidad de la autora.

La redacción de la memoria ha sido elaborada por la autora. Las herramientas de IA se han utilizado como apoyo para mejorar la expresión y coherencia del texto, pero el contenido, la estructura y las conclusiones son propios.

7.8.3. Verificación

Todo el código sugerido por herramientas de IA ha pasado por el mismo proceso de revisión que el código escrito manualmente: revisión de lógica, ejecución de pruebas unitarias y de integración, y comprobación del comportamiento en la aplicación. No se ha aceptado ningún fragmento cuyo funcionamiento no haya podido ser explicado y verificado por la autora.

7.9. Conclusión final

Sevillaneando es el resultado de un proceso de ingeniería completo: desde la identificación de un problema real —la fragmentación de la información sobre eventos en Sevilla— hasta la entrega de un sistema funcional, desplegado y documentado. El proyecto ha cubierto todos los objetivos prioritarios definidos al inicio, integrando tecnologías actuales de desarrollo móvil, backend, geolocalización, comunicación en tiempo real y recomendaciones personalizadas.

Más allá del resultado técnico, el desarrollo ha supuesto un aprendizaje profundo en la gestión de proyectos software en solitario: tomar decisiones de alcance bajo incertidumbre, priorizar con criterio y entregar algo que funciona sobre algo que es perfecto en papel. Las funcionalidades no implementadas no son fracasos sino decisiones conscientes de gestión del alcance, y quedan documentadas como trabajo futuro para una versión posterior del sistema.

Bibliografía

- [1] Ministerio de Cultura y Deporte. Encuesta de hábitos y prácticas culturales en España 2022–2023, 2023. Último acceso: mayo 2025.
- [2] Instituto Nacional de Estadística. Encuesta de presupuestos familiares. año 2022, 2023. Último acceso: mayo 2025.
- [3] Instituto Nacional de Estadística. Encuesta de ocupación en alojamientos turísticos. encuesta de ocupación hotelera, 2023. Último acceso: mayo 2025.
- [4] Eventbrite, Inc. Eventbrite – discover events or create your own, 2024. Último acceso: mayo 2025.
- [5] Meta Platforms, Inc. Facebook events, 2024. Último acceso: mayo 2025.
- [6] Pew Research Center. Teens, social media and technology 2023, 2023. Último acceso: mayo 2025.
- [7] Meetup, LLC. Meetup – find local groups, 2024. Último acceso: mayo 2025.
- [8] Ayuntamiento de Sevilla. Agenda cultural – ayuntamiento de Sevilla, 2024. Último acceso: mayo 2025.
- [9] Fever Labs, Inc. Fever – discover the best experiences in your city, 2024. Último acceso: mayo 2025.
- [10] Ken Schwaber and Jeff Sutherland. The scrum guide: The definitive guide to scrum: The rules of the game, 2020. Último acceso: mayo 2025.
- [11] Mike Cohn. *Agile Estimating and Planning*. Prentice Hall, Upper Saddle River, NJ, 2005.
- [12] Mike Cohn. *User Stories Applied: For Agile Software Development*. Addison-Wesley, Boston, MA, 2004.
- [13] GitHub, Inc. Github actions – automate your workflow from idea to production, 2024. Último acceso: mayo 2025.
- [14] Conventional Commits. Conventional commits 1.0.0, 2024. Último acceso: mayo 2025.
- [15] Cake.com d.o.o. Clockify – free time tracker and timesheet app, 2024. Último acceso: mayo 2025.
- [16] Expo Technology, Inc. Changes to project loading behavior in expo go, 2026. Último acceso: mayo 2026.
- [17] Junta de Andalucía. Consejería de Agricultura, Pesca y Desarrollo Rural. Informe final sobre la consulta preliminar del mercado “perfiles profesionales ámbito informático”. Documento PDF, 2018. Informe local consultado: [references/informe_precios_perfiles_informaticos.pdf](#).

- [18] Parlamento Europeo y Consejo de la Unión Europea. Reglamento (ue) 2016/679 del parlamento europeo y del consejo, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos (rgpd), 2016. Diario Oficial de la Unión Europea L 119, 4.5.2016. Último acceso: mayo 2025.
- [19] Jefatura del Estado (España). Ley orgánica 3/2018, de 5 de diciembre, de protección de datos personales y garantía de los derechos digitales, 2018. Boletín Oficial del Estado (BOE) núm. 294, 6.12.2018. Último acceso: mayo 2025.
- [20] TypeStack. class-validator – decorator-based property validation for classes, 2024. Último acceso: mayo 2025.
- [21] NestJS. Nestjs throttler – a rate-limiting module for nestjs, 2024. Último acceso: mayo 2025.
- [22] OWASP Foundation. Owasp top 10:2021 – the ten most critical web application security risks, 2021. Último acceso: mayo 2025.
- [23] Helmet contributors. Helmet – help secure express apps with various http headers, 2024. Último acceso: mayo 2025.
- [24] Meta Open Source. React native – learn once, write anywhere, 2024. Último acceso: mayo 2025.
- [25] Bonnie Eisenman. *Learning React Native: Building Native Mobile Apps with JavaScript*. O'Reilly Media, Sebastopol, CA, 2 edition, 2018.
- [26] Meta Open Source. React – the library for web and native user interfaces, 2024. Último acceso: mayo 2025.
- [27] Expo Technology, Inc. Expo – build one javascript/typescript project that runs natively on all your users' devices, 2024. Último acceso: mayo 2025.
- [28] Axios contributors. Axios – promise based http client for the browser and node.js, 2024. Último acceso: mayo 2025.
- [29] Michael Jones, John Bradley, and Nat Sakimura. Rfc 7519: Json web token (jwt), 2015. Internet Engineering Task Force (IETF). Último acceso: mayo 2025.
- [30] React Navigation contributors. React navigation – routing and navigation for expo and react native apps, 2024. Último acceso: mayo 2025.
- [31] NestJS. Nestjs – a progressive node.js framework, 2024. Último acceso: mayo 2025.
- [32] NestJS. Nestjs documentation – modules, controllers, providers and guards, 2024. Último acceso: mayo 2025.
- [33] OpenJS Foundation. Node.js – javascript runtime built on chrome's v8 javascript engine, 2024. Último acceso: mayo 2025.
- [34] Microsoft Corporation. Typescript – javascript with syntax for types, 2024. Último acceso: mayo 2025.

- [35] Boris Cherny. *Programming TypeScript: Making Your JavaScript Applications Scale*. O'Reilly Media, Sebastopol, CA, 2019.
- [36] Socket.IO. Socket.io – bidirectional and low-latency communication for every platform, 2024. Último acceso: mayo 2025.
- [37] Ian Fette and Alexey Melnikov. Rfc 6455: The websocket protocol, 2011. Internet Engineering Task Force (IETF). Último acceso: mayo 2025.
- [38] The PostgreSQL Global Development Group. Postgresql 15 documentation, 2023. Último acceso: mayo 2025.
- [39] PostGIS Project Steering Committee. Postgis 3.3 manual, 2023. Último acceso: mayo 2025.
- [40] Regina O. Obe and Leo S. Hsu. *PostGIS in Action*. Manning Publications, Shelter Island, NY, 3 edition, 2020.
- [41] International Association of Oil & Gas Producers. Epsg:4326 – wgs 84: Geodetic coordinate system for world, 2024. Último acceso: mayo 2025.
- [42] TypeORM. Typeorm – orm for typescript and javascript, 2024. Último acceso: mayo 2025.
- [43] Google LLC. Firebase authentication, 2024. Último acceso: mayo 2025.
- [44] Cloudinary Ltd. Cloudinary – image and video api platform, 2024. Último acceso: mayo 2025.
- [45] OpenStreetMap contributors. Openstreetmap, 2024. Último acceso: mayo 2025.
- [46] React Native Community. react-native-maps – react native mapview component for ios + android, 2024. Último acceso: mayo 2025.
- [47] Ticketmaster Entertainment, Inc. Ticketmaster – tickets to concerts, sports, theatre and more, 2024. Último acceso: mayo 2025.
- [48] OpenStreetMap Foundation. Nominatim – open-source geocoding with openstreetmap data, 2024. Último acceso: mayo 2025.
- [49] Google LLC. Gemini api – build with google’s most capable ai models, 2024. Último acceso: mayo 2025.
- [50] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000. Último acceso: mayo 2025.
- [51] Sam Newman. *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, Sebastopol, CA, 2 edition, 2021.
- [52] OpenAPI Initiative. Openapi specification 3.0.3, 2020. Último acceso: mayo 2025.
- [53] Francesco Ricci, Lior Rokach, and Bracha Shapira. *Recommender Systems Handbook*. Springer, New York, NY, 2 edition, 2015.

- [54] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, Reading, MA, 1994.
- [55] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, Boston, MA, 2002.
- [56] cron-job.org. cron-job.org – free cronjobs from the cloud, 2024. Último acceso: mayo 2025.
- [57] The Open Group. crontab(5) – tables for driving cron, 2018. IEEE Std 1003.1-2017 (POSIX.1-2017). Último acceso: mayo 2025.
- [58] The PostgreSQL Global Development Group. PostgreSQL: The world’s most advanced open source relational database, 2024. Último acceso: mayo 2025.
- [59] Cheerio contributors. Cheerio – fast, flexible & lean implementation of core jquery designed specifically for the server, 2024. Último acceso: mayo 2025.
- [60] C. Carl Robusto. The cosine-haversine formula. *The American Mathematical Monthly*, 64(1):38–40, 1957.
- [61] OpenJS Foundation. Jest – delightful javascript testing, 2024. Último acceso: mayo 2025.
- [62] Google LLC. Firebase admin sdk – server-side libraries for firebase, 2024. Último acceso: mayo 2025.
- [63] Docker, Inc. Docker – accelerated container application development, 2024. Último acceso: mayo 2025.
- [64] Render Services, Inc. Render – cloud application hosting, 2024. Último acceso: mayo 2025.
- [65] Expo Technology, Inc. Eas build – expo application services, 2024. Último acceso: mayo 2025.
- [66] Supabase, Inc. Supabase documentation, 2024. Último acceso: mayo 2025.

Anexos

A. Glosario de términos

Este anexo recoge los principales términos técnicos utilizados a lo largo de la memoria, con el objetivo de facilitar la lectura del documento y unificar el significado de los conceptos más relevantes del proyecto.

Tabla A.1: Glosario de términos del proyecto

Término	Definición
API REST	Interfaz de comunicación basada en HTTP que permite al frontend consultar y modificar información gestionada por el backend.
Backend	Parte del sistema encargada de procesar la lógica de negocio, exponer la API, validar peticiones y comunicarse con la base de datos y servicios externos.
Cloudinary	Servicio externo utilizado para almacenar y servir imágenes asociadas a eventos, chats y perfiles de usuario.
DTO	Objeto de transferencia de datos utilizado en el backend para definir y validar la estructura de la información recibida desde el cliente.
Expo	Plataforma y conjunto de herramientas que facilitan el desarrollo, prueba y distribución de aplicaciones React Native.
Firebase Authentication	Servicio empleado para gestionar el registro, inicio de sesión y verificación de identidad de los usuarios.
Frontend	Parte cliente de la aplicación, desarrollada como aplicación móvil, con la que interactúa directamente el usuario.
Geocodificación	Proceso de convertir una dirección textual en coordenadas geográficas que puedan almacenarse y consultarse en el sistema.
GitHub Actions	Plataforma de integración continua utilizada para automatizar comprobaciones de calidad y flujos de despliegue.
Mockup	Representación visual preliminar de una pantalla o flujo de interfaz, utilizada durante la fase de análisis y diseño.
NestJS	Framework de backend para Node.js utilizado para construir la API REST del sistema.
OpenStreetMap	Proyecto cartográfico abierto empleado como base para mapas y servicios de geolocalización.
PostGIS	Extensión de PostgreSQL que permite almacenar y consultar datos geográficos de forma eficiente.

Continúa en la página siguiente

Término	Definición
React Native	Framework utilizado para desarrollar la aplicación móvil multiplataforma a partir de una base de código común.
Scraping	Técnica utilizada para extraer eventos desde páginas web o fuentes externas y transformarlos en datos estructurados.
Sprint	Iteración de trabajo dentro de la metodología ágil empleada para planificar, desarrollar y revisar funcionalidades.
TypeORM	Herramienta ORM utilizada para mapear entidades TypeScript con tablas de la base de datos PostgreSQL.
TypeScript	Lenguaje de programación empleado tanto en frontend como en backend, que añade tipado estático sobre JavaScript.
WebSocket	Tecnología de comunicación bidireccional utilizada para funcionalidades en tiempo real, como el chat.

B. Manual de Desarrollo

B.1. Objetivo

El presente documento describe el entorno de desarrollo del sistema **Sevillaneando**, proporcionando la información necesaria para instalar, ejecutar y mantener el proyecto en un entorno local. Se detallan los componentes del sistema, la arquitectura adoptada, la configuración del entorno, la estructura del proyecto y las convenciones seguidas durante el desarrollo. El objetivo es facilitar tanto la comprensión del sistema como su evolución futura por parte de cualquier desarrollador que se incorpore al proyecto.

B.2. Arquitectura del sistema

El sistema **Sevillaneando** sigue una arquitectura cliente-servidor distribuida, estructurada en tres capas principales: cliente móvil, servidor backend y sistema de persistencia de datos. Esta separación permite desacoplar responsabilidades, mejorar la mantenibilidad y facilitar la escalabilidad.

B.2.1. Visión general

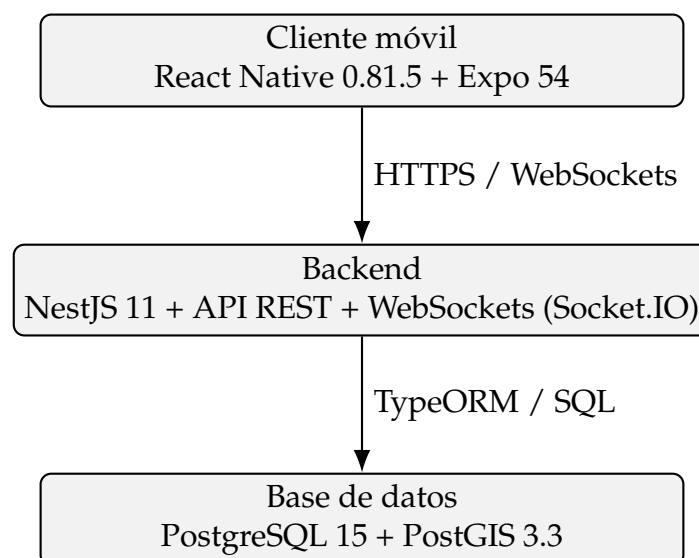


Figura B.1: Visión general de la arquitectura del sistema

El flujo principal de comunicación se realiza mediante peticiones HTTP a la API REST. Para funcionalidades en tiempo real (chat y notificaciones) se utilizan

WebSockets mediante Socket.IO.

B.2.2. Cliente móvil

El cliente ha sido desarrollado con **React Native 0.81.5** y **Expo SDK 54**, lo que permite crear una aplicación multiplataforma (iOS y Android) con una única base de código en TypeScript.

Sus responsabilidades principales son:

- Interacción con el usuario final.
- Visualización de eventos, mapas y rutas turísticas.
- Gestión del estado global mediante Context API.
- Comunicación con el backend mediante la librería axios.
- Conexión en tiempo real con el servidor mediante socket.io-client.

El cliente se organiza en pantallas (screens/), componentes reutilizables (components/), servicios de API (services/), contextos de estado global (context/), hooks personalizados (hooks/) y utilidades (utils/).

B.2.3. Servidor backend

El backend se ha implementado con **NestJS 11**, que proporciona una arquitectura modular basada en controladores, servicios y módulos. La comunicación se expone como API REST, con soporte adicional de WebSockets para la mensajería en tiempo real.

Sus responsabilidades son:

- Gestión de la lógica de negocio.
- Exposición de la API REST.
- Autenticación delegada en Firebase Authentication.
- Control de roles (usuario, moderador y administrador).
- Gestión de comunicación en tiempo real con Socket.IO.
- Web scraping de eventos externos (Eventbrite Sevilla, Ticketmaster, visitasevilla.es y webs procesadas con Gemini AI).
- Ejecución de tareas programadas (cron jobs) para scraping y notificaciones.

El backend se organiza en módulos funcionales e infraestructurales independientes: app, auth, events, users, rutas, chat, notificaciones, recomendaciones, categorías, scraping, scheduler, database y cloudinary. El endpoint de comprobación de disponibilidad se expone desde el controlador de scheduler.

B.2.4. Base de datos

La persistencia de datos se realiza mediante **PostgreSQL 15** con la extensión **PostGIS 3.3**, que añade soporte para operaciones geoespaciales. El acceso desde el backend se realiza mediante **TypeORM**.

Las funcionalidades geoespaciales permiten:

- Búsqueda de eventos cercanos a una ubicación.
- Filtrado por radio de distancia.
- Almacenamiento y consulta de coordenadas en formato GeoJSON.

B.2.5. Servicios externos

El sistema integra los siguientes servicios externos:

- **Firebase Authentication**: autenticación de usuarios (login, registro, gestión de sesión).
- **Cloudinary**: almacenamiento y transformación de imágenes de eventos, perfiles y chats.
- **OpenStreetMap + react-native-maps**: visualización de mapas interactivos.
- **Nominatim**: geocodificación de direcciones cuando las fuentes externas no proporcionan coordenadas.
- **Eventbrite**: importación de eventos de Sevilla mediante extracción HTML y JSON-LD.
- **visitasevilla.es**: importación de eventos locales mediante extracción HTML.
- **Google Gemini AI**: extracción inteligente de eventos desde webs externas mediante IA.
- **Ticketmaster API**: importación de eventos desde la plataforma Ticketmaster.

B.3. Tecnologías y versiones

Las tecnologías utilizadas y sus versiones exactas son las siguientes:

Componente	Versión	Portal de descarga
Node.js	20.x LTS	nodejs.org
npm	10.x	npmjs.com
TypeScript	5.3.x (backend) / 5.9.x (frontend)	typescriptlang.org
NestJS	11.x	nestjs.com
TypeORM	0.3.20	typeorm.io
Socket.IO	4.8.3	socket.io

Componente	Versión	Portal de descarga
Expo SDK	54.x	expo.dev
React Native	0.81.5	reactnative.dev
React	19.1.0	react.dev
PostgreSQL	15	postgresql.org/download
PostGIS	3.3	postgis.net/install
Docker	4.x	docker.com/products/docker-desktop
Firebase Admin SDK	12.7.0	firebase.google.com/docs/admin/setup
Cloudinary	2.9.0	cloudinary.com

B.4. Prerrequisitos

Para trabajar con el proyecto en local es necesario tener instalado:

- **Node.js 20 LTS** y **npm 10**: gestión de dependencias y ejecución del backend.
- **Docker**: para levantar la base de datos PostgreSQL con PostGIS en un contenedor.
- **Expo Go** (en dispositivo físico) o un emulador Android/iOS para ejecutar la aplicación móvil.
- Credenciales de los servicios externos: Firebase, Cloudinary, Ticketmaster API y Gemini API.

B.5. Obtención del código fuente

El repositorio del proyecto está alojado en GitHub. Para obtener el código fuente:

```

1 git clone https://github.com/cyncasjua/sevillaneando.git
2 cd sevillaneando

```

B.6. Instalación de dependencias

El proyecto está organizado como un monorepo con dos aplicaciones. Para instalar las dependencias de cada una:

```

1 # Backend
2 cd apps/backend
3 npm install
4

```

```
5 # Frontend
6 cd apps/frontend
7 npm install
```

B.7. Variables de entorno

El sistema utiliza ficheros `.env` separados por entorno. Nunca deben incluirse en el repositorio (están en `.gitignore`). La estructura es la siguiente:

```
apps/backend/
.env.development  <- valores locales (Docker, localhost, SSL desactivado)
.env.production   <- plantilla para produccion (rellenar con secretos reales)

apps/frontend/
.env.development  <- apunta al backend local
.env.production   <- apunta al backend en produccion
.env              <- fichero activo (se genera copiando uno de los anteriores)
```

El backend carga automáticamente `.env.${NODE_ENV}` al arrancar gracias a `ConfigModule`. El frontend requiere un `.env` activo en su raíz; los scripts `npm run start:dev` y `npm run start:prod` copian el fichero correcto antes de lanzar Expo.

B.7.1. Backend (apps/backend/.env.development)

```
1 NODE_ENV=development
2 PORT=3000
3
4 # Base de datos (Docker local, puerto 5433)
5 DATABASE_HOST=localhost
6 DATABASE_PORT=5433
7 DATABASE_USER=postgres
8 DATABASE_PASSWORD=postgres
9 DATABASE_NAME=sevillaneando
10 DATABASE_SSL=false
11
12 # CORS (origenes permitidos en local)
13 ALLOWED_ORIGINS=http://localhost:8081,http://localhost:19006
14
15 # Firebase Authentication (Admin SDK)
16 FIREBASE_PROJECT_ID=...
17 FIREBASE_CLIENT_EMAIL=...
18 # Incluir los \n literales: "-----BEGIN PRIVATE KEY-----\n...\n-----END
    PRIVATE KEY-----\n"
19 FIREBASE_PRIVATE_KEY=...
20
```

```
21 # Clouidary (imagenes de eventos)
22 CLOUDINARY_CLOUD_NAME=...
23 CLOUDINARY_API_KEY=...
24 CLOUDINARY_API_SECRET=...
25
26 # APIs externas
27 TICKETMASTER_API_KEY=...
28 GEMINI_API_KEY=...
29
30 # Usuario de sistema para scraping
31 SCRAPER_SYSTEM_UID=system-scraper-uid
32 SCRAPER_SYSTEM_EMAIL=scraper.bot@sevillaneando.local
33 SCRAPER_SYSTEM_NAME=Sevillaneando Bot
34 LEGACY_SCRAPER_EMAIL=mod@demo.com
```

B.7.2. Frontend (apps/frontend/.env.development)

```
1 # Para emulador Android usar: http://10.0.2.2:3000
2 # Para dispositivo fisico usar la IP local: http://192.168.1.X:3000
3 EXPO_PUBLIC_API_URL=http://localhost:3000
4 EXPO_PUBLIC_SHARE_BASE_URL=http://localhost:3000/events
5
6 # Firebase (cliente web)
7 EXPO_PUBLIC_FIREBASE_API_KEY=...
8 EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN=...
9 EXPO_PUBLIC_FIREBASE_PROJECT_ID=...
10 EXPO_PUBLIC_FIREBASE_STORAGE_BUCKET=...
11 # Android e iOS tienen APP_ID distintos:
12 #   iOS:      1:NNNNN:ios:XXXX
13 #   Android:  1:NNNNN:android:XXXX
14 EXPO_PUBLIC_FIREBASE_APP_ID=...
```

B.8. Ejecución local

Para ejecutar el sistema completo en local es necesario arrancar los componentes en el orden indicado.

B.8.1. 1. Base de datos

La base de datos se levanta mediante Docker Compose. El archivo `docker-compose.yml` en la raíz del proyecto define un contenedor PostgreSQL 15 con PostGIS 3.3:

```
1 docker compose up -d db
```

El contenedor queda disponible en `localhost:5433`. Si es la primera vez que se levanta, TypeORM creará automáticamente las tablas al arrancar el backend gracias a la opción `synchronize: true`, activa únicamente cuando `NODE_ENV` es distinto de `production`. En producción esta opción se desactiva para evitar que TypeORM altere el esquema automáticamente; el esquema debe estar ya creado en la base de datos antes del primer arranque.

B.8.2. 2. Backend

```
1 cd apps/backend
2 npm run start:dev
```

El servidor arranca en `http://localhost:3000`. El modo `start:dev` activa el hot-reload mediante `ts-node-dev`.

B.8.3. 3. Frontend

```
1 cd apps/frontend
2 npm run start:dev
```

Este script copia `.env.development` a `.env` y lanza el servidor Expo. La aplicación puede ejecutarse en:

- Dispositivo físico: escaneando el QR con la aplicación **Expo Go**. La variable `EXPO_PUBLIC_API_URL` debe apuntar a la IP local de la máquina de desarrollo (no a `localhost`), por ejemplo `http://192.168.1.42:3000`.
- Emulador Android: pulsando `a` en la terminal. Usar `http://10.0.2.2:3000` como URL del backend, ya que `localhost` dentro del emulador apunta al propio dispositivo virtual.
- Simulador iOS (solo macOS): pulsando `i`.

B.9. Estructura del proyecto

El proyecto sigue una arquitectura monorepo con dos aplicaciones principales:

```
1 sevellaneando/
2 |-- apps/
3 |   |-- backend/
4 |   |   |-- src/
5 |   |   |   |-- app.module.ts
6 |   |   |   |-- main.ts
```

```

7 | | | |-- auth/          (Firebase Auth + guards + roles)
8 | | | |-- events/      (CRUD eventos + reseñas + moderación)
9 | | | |-- users/       (gestión de usuarios y perfiles)
10 | | | |-- rutas/      (rutas turísticas y calificaciones)
11 | | | |-- chat/       (mensajería pública y privada)
12 | | | |-- notificaciones/ (sistema de notificaciones)
13 | | | |-- recomendaciones/ (motor de recomendaciones)
14 | | | |-- categorias/  (categorías de eventos)
15 | | | |-- scraping/    (Eventbrite, Ticketmaster, Gemini, VisitaSevilla)
    | | | )
16 | | | |-- scheduler/   (cron jobs scraping/notificaciones y health)
17 | | | |-- database/    (configuración y seed de datos)
18 | | | |-- common/      (utilidades, filtros, clouinary)
19 | | |-- Dockerfile
20 | | '-- package.json
21 | '-- frontend/
22 |     |-- src/
23 |         |-- screens/  (31 pantallas de la aplicación)
24 |         |-- components/ (componentes reutilizables)
25 |         |-- navigation/ (navegación + deep linking)
26 |         |-- context/   (Auth, Socket, Theme, Notificaciones)
27 |         |-- hooks/     (useAuth, useTheme, useEvents)
28 |         |-- services/   (llamadas a la API REST)
29 |         |-- types/      (tipos TypeScript compartidos)
30 |         |-- utils/      (utilidades de fecha, mapa, imágenes)
31 |         '-- firebase/   (configuración Firebase cliente)
32 |     '-- package.json
33 |-- docker-compose.yml
34 '-- package.json

```

B.9.1. Módulos del backend

El backend sigue la convención de módulos de NestJS. Cada módulo contiene su controlador, servicio, entidades y DTOs:

- **auth:** integración con Firebase Admin SDK para validar tokens JWT. Incluye FirebaseAuthGuard y RolesGuard, aplicados en los controladores y endpoints protegidos.
- **events:** módulo principal. Gestiona el ciclo de vida de eventos (creación, aprobación por moderadores, edición, eliminación) y reseñas. La autorización distingue entre creadores y moderadores: los creadores pueden modificar y eliminar sus propios eventos, mientras que los moderadores pueden actuar sobre eventos públicos, incluidos los ya aprobados, pero no sobre eventos privados ajenos.
- **users:** gestión de perfiles, cambio de contraseña, asignación de roles y módulo social basado en seguimiento entre usuarios, con seguidores, seguidos y amistades mutuas.

- **rutas:** rutas turísticas con puntos de interés y sistema de calificación.
- **chat:** mensajería en tiempo real implementada con Socket.IO. Incluye mensajes públicos en eventos y mensajes privados entre usuarios.
- **notificaciones:** generación y entrega de notificaciones en tiempo real mediante WebSockets, incluyendo referencias contextuales a usuarios o eventos cuando la notificación permite navegar al contenido relacionado.
- **recomendaciones:** motor de recomendaciones basado en preferencias de categoría y valoraciones previas.
- **scraping:** scrapers para importar eventos de Eventbrite Sevilla, Ticketmaster, visitasevilla.es y webs procesadas mediante Gemini AI. Implementa el patrón estrategia con la interfaz ScraperInterface.
- **scheduler:** tareas programadas con @nestjs/schedule. Ejecuta scrapers periódicamente y envía recordatorios de eventos próximos.

B.9.2. Pantallas del frontend

El frontend organiza las vistas en grupos funcionales:

- **Autenticación:** LoginScreen, SignUpScreen, EditPasswordScreen.
- **Principal:** HomeScreen, CategoriesScreen.
- **Eventos:** EventsMapScreen, EventDetailScreen, EventDetailLinkScreen, CreateEventScreen, UserEditEventScreen, UserEventsScreen, ModeratorEventsScreen, ModeratorEditEventScreen, CalendarEventsScreen, SavedAndPrivateEventsScreen, AccessPrivateEventScreen.
- **Rutas:** RoutesListScreen, RouteDetailScreen, RoutePreviewScreen, CreateRouteScreen.
- **Social:** FriendsScreen, MessagesScreen, DirectMessageScreen, NotificacionesScreen, ProfileConnectionsScreen.
- **Perfil:** UserProfileScreen, EditProfileScreen, AdminScreen.
- **Información:** HelpScreen, PrivacyPolicyScreen, LegalAttributionsScreen.

B.10. Ecosistema de desarrollo

B.10.1. Herramientas de implementación

- **Visual Studio Code:** editor principal de código.
- **NestJS CLI:** scaffolding y gestión del backend (nest generate module/controller/service).

- **Expo CLI / EAS CLI**: desarrollo y construcción de la aplicación móvil.
- **Docker Desktop**: contenedores para la base de datos en local.
- **Postman**: pruebas manuales de la API REST.
- **Swagger / OpenAPI**: documentación interactiva de la API REST, accesible en /api/docs mediante @nestjs/swagger.
- **TablePlus / pgAdmin**: administración visual de la base de datos.
- **React Native Debugger**: depuración del cliente móvil.

B.10.2. Herramientas de documentación

- **Overleaf / LaTeX**: redacción de la memoria del TFG.
- **draw.io**: diagramas de arquitectura y flujos.

B.10.3. Herramientas de gestión del proyecto

- **GitHub**: alojamiento del repositorio y revisión de código.
- **GitHub Projects**: seguimiento del backlog y planificación de sprints.
- **GitHub Actions**: integración continua (linting y formateo automático).

B.11. Política de gestión del código fuente

El control de versiones se gestiona con **Git** y el repositorio está alojado en **GitHub**.

B.11.1. Estrategia de ramas

- **main**: rama de producción. Solo recibe merges desde develop cuando el sistema está estable.
- **develop**: rama de integración. Las features se integran aquí antes de pasar a producción.
- **feature/nombre**: ramas de desarrollo de funcionalidades individuales.

B.11.2. Convención de commits

El proyecto sigue el estándar **Conventional Commits**, lo que permite identificar fácilmente el tipo de cambio en cada commit:

```
1 feat: nueva funcionalidad
2 fix:  correccion de error
3 style: cambios de formato o estilo
4 refactor: refactorizacion sin cambio de funcionalidad
5 docs: cambios en documentacion
6 test: adicion o modificacion de pruebas
7 chore: tareas de mantenimiento (dependencias, configuracion)
```

B.12. Integración continua

El proyecto utiliza **GitHub Actions** para automatizar tareas de calidad en cada push o pull request:

- **Linting:** análisis estático del código con ESLint (configurado para TypeScript).
- **Formateo:** verificación del estilo de código con Prettier.

El workflow se ejecuta automáticamente sobre las ramas main y develop, garantizando que el código integrado cumple los estándares de calidad definidos.

B.13. Resolución de problemas comunes

B.13.1. Base de datos

- **Error de conexión:** verificar que el contenedor Docker está activo con docker ps. El puerto utilizado es el 5433 (no el estándar 5432) para evitar conflictos con instalaciones locales de PostgreSQL.
- **Extensión PostGIS no encontrada:** conectarse al contenedor y ejecutar CREATE EXTENSION postgis; en la base de datos se villaneando.
- **Tablas no creadas:** asegurarse de que NODE_ENV=development está definido en el .env del backend, ya que en ese modo TypeORM sincroniza el esquema automáticamente.

B.13.2. Backend

- **Error de Firebase:** verificar que FIREBASE_PRIVATE_KEY está correctamente entrecomillado en el .env (debe incluir los saltos de línea \n).
- **Rate limiting (429 Too Many Requests):** el backend aplica un límite de 60 peticiones por minuto por IP. En desarrollo, este límite puede aumentarse modificando la configuración de ThrottlerModule en app.module.ts.

B.13.3. Frontend

- **No conecta con el backend:** verificar que `EXPO_PUBLIC_API_URL` apunta a la IP de la máquina de desarrollo (no localhost) cuando se ejecuta en dispositivo físico. En emulador Android usar `http://10.0.2.2:3000`.
- **Mapas no visibles:** comprobar que el paquete `react-native-maps` está correctamente enlazado. En Expo managed workflow esto se gestiona automáticamente.
- **Error de Metro bundler:** limpiar la caché con `npx expo start -clear`.

B.14. Referencias

Las referencias técnicas empleadas en este manual corresponden a la documentación oficial de cada tecnología y se recogen en la bibliografía del documento principal: NestJS [31, 32], Node.js [33], TypeScript [34], React [26], React Native [24], Expo [27], TypeORM [42], PostgreSQL [38], PostGIS [39], Firebase Admin [62], Docker [63], Axios [28], Socket.IO [36], react-native-maps [46] y Conventional Commits [14].

C. Manual de despliegue

C.1. Objetivo

El presente documento describe el proceso de despliegue del sistema **Sevillaneando** en un entorno de producción. Su finalidad es servir como guía para poner en funcionamiento la aplicación fuera del entorno local de desarrollo, asegurando que todos los componentes del sistema queden correctamente configurados y comunicados entre sí.

Se detalla la arquitectura de despliegue, la configuración de la base de datos, la publicación del backend y la distribución de la aplicación móvil. Asimismo, se incluyen instrucciones para validar el correcto funcionamiento del sistema una vez desplegado, un checklist de verificación y la estrategia de rollback ante incidencias.

C.2. Arquitectura de despliegue

El sistema **Sevillaneando** se despliega mediante una arquitectura distribuida compuesta por tres elementos principales: la aplicación móvil, el servidor backend y la base de datos. Cada componente se despliega de forma independiente en su plataforma correspondiente.

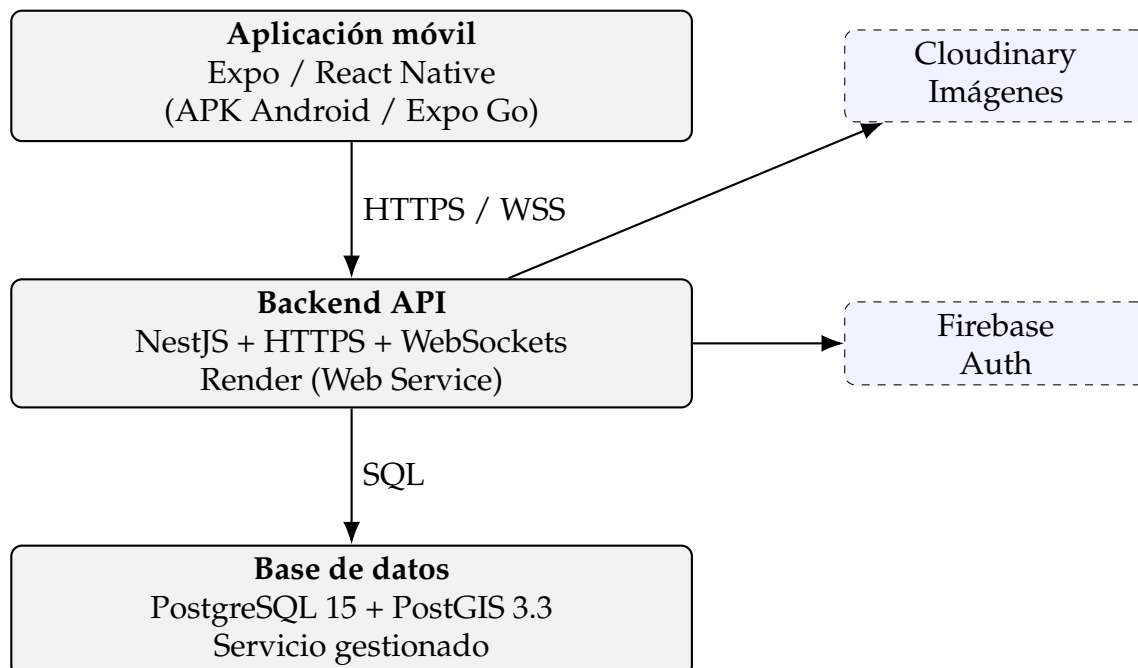


Figura C.1: Arquitectura general de despliegue del sistema

C.2.1. Componentes principales

- **Aplicación móvil:** se distribuye como cliente final mediante APK de Android (instalación directa) o mediante Expo Go. Se conecta al backend mediante HTTPS para consumir la API REST y mediante WebSockets para la mensajería en tiempo real.
- **Backend:** se despliega como servicio web en **Render**, plataforma de hosting en la nube que permite publicar servicios Node.js directamente desde un repositorio Git. Centraliza la lógica de negocio, la autenticación, la gestión de eventos y la comunicación con servicios externos.
- **Base de datos:** se despliega como instancia independiente de PostgreSQL 15 con la extensión PostGIS, preferiblemente como servicio gestionado para garantizar persistencia, disponibilidad y backups automáticos.
- **Firebase:** servicio externo de Google que gestiona la autenticación de usuarios. No requiere despliegue propio.
- **Cloudinary:** servicio externo para el almacenamiento y transformación de imágenes de eventos, perfiles y chats. No requiere despliegue propio.

C.3. Versiones

Las versiones de los componentes del sistema deben mantenerse alineadas con el entorno de desarrollo para evitar incompatibilidades:

Componente	Versión
Node.js	20.x LTS
NestJS	11.x
TypeORM	0.3.20
Expo SDK	54.x
EAS CLI	>= 13
React Native	0.81.5
PostgreSQL	15
PostGIS	3.3
Firebase Admin SDK	12.7.0

C.4. Despliegue de la base de datos

La base de datos debe desplegarse en primer lugar, ya que el backend depende de ella para poder iniciarse correctamente.

C.4.1. Opción recomendada: servicio gestionado

Para producción se recomienda utilizar un servicio gestionado de PostgreSQL, que simplifica la administración y proporciona backups automáticos, alta disponibilidad y soporte para extensiones. Las opciones compatibles son:

- **Supabase** (incluye PostGIS nativo, plan gratuito disponible).
- **AWS RDS** para PostgreSQL.
- **DigitalOcean Managed Databases**.
- **Railway** (sencillo de configurar, compatible con monorepos).

Una vez creada la instancia, es imprescindible habilitar la extensión PostGIS, ya que el sistema utiliza funcionalidades geoespaciales para búsquedas por proximidad:

```
1 CREATE EXTENSION IF NOT EXISTS postgis;
```

Tras habilitar la extensión, se deben anotar los datos de conexión (host, puerto, usuario, contraseña y nombre de la base de datos) para configurarlos como variables de entorno en el backend.

C.4.2. Configuración de SSL

En producción, la conexión a la base de datos debe realizarse mediante SSL. En el `.env` del backend debe incluirse:

```
1 DATABASE_SSL=true
```

C.4.3. Creación del esquema de base de datos en producción

En producción, TypeORM tiene la opción `synchronize` desactivada para evitar que altere el esquema automáticamente. Por tanto, las tablas deben existir antes del primer arranque del backend.

La forma más sencilla para un entorno académico es lanzar el backend una única vez apuntando a la base de datos de producción pero con `NODE_ENV=development`, de modo que TypeORM cree todas las tablas. Para ello, configurar temporalmente las variables `DATABASE_*` en `.env.development` con los datos de la BD de producción y arrancar con `npm run start:dev`. Una vez creadas las tablas, revertir `.env.development` a los valores locales.

Alternativamente, si el proveedor permite conectarse directamente, ejecutar el backend en modo producción con `synchronize: true` temporalmente (modificando `app.module.ts` para la primera puesta en marcha y revirtiendo el cambio tras el primer arranque).

C.4.4. Activación de Row-Level Security (RLS)

Si se utiliza **Supabase** como proveedor de base de datos, es imprescindible activar las políticas de seguridad a nivel de fila (*Row-Level Security*) en todas las tablas del esquema público. Supabase notifica mediante un aviso de seguridad crítico cuando estas políticas no están configuradas, ya que sin ellas cualquier petición autenticada puede acceder a todos los registros.

Para activar RLS en todas las tablas del esquema public, ejecutar el siguiente bloque SQL desde el editor de Supabase o mediante un cliente PostgreSQL:

```
1 DO $$
2 DECLARE
3   t TEXT;
4 BEGIN
5   FOR t IN
6     SELECT tablename FROM pg_tables
7     WHERE schemaname = 'public'
8   LOOP
9     EXECUTE format('ALTER TABLE public.%I ENABLE ROW LEVEL SECURITY;', t)
10    ;
11  END LOOP;
12 END;
13 $$;
```

A continuación, definir las políticas de acceso apropiadas para cada tabla según el rol que deba tener acceso. El backend accede a la base de datos mediante el rol de servicio de Supabase (*service_role*), que omite RLS por diseño, por lo que esta configuración no afecta al funcionamiento del backend pero protege el acceso directo a los datos a través del cliente público de Supabase.

C.4.5. Alternativa: despliegue con Docker

Para entornos de demostración o pruebas, la base de datos puede desplegarse mediante Docker:

```
1 docker run -d \
2   --name sevilaneando-db \
3   -e POSTGRES_USER=postgres \
4   -e POSTGRES_PASSWORD=secure-password \
5   -e POSTGRES_DB=sevilaneando \
6   -p 5432:5432 \
7   postgres/postgis:15-3.3
```

Esta opción no se recomienda para producción real, ya que no proporciona persistencia garantizada ni mecanismos de backup automático.

C.5. Despliegue del backend

El backend constituye el núcleo operativo del sistema. Antes de desplegarlo en producción es recomendable verificar que compila correctamente en local.

C.5.1. Construcción local previa

```
1 cd apps/backend
2 npm install
3 npm run build
```

Si el proceso finaliza sin errores, el directorio `dist/` contendrá la versión compilada lista para producción.

C.5.2. Variables de entorno de producción

El backend requiere las siguientes variables de entorno en producción. Deben configurarse en el panel del proveedor de hosting elegido, no en un archivo `.env` público:

```
1 NODE_ENV=production
2 PORT=3000
3
4 # Base de datos (servicio gestionado, SSL obligatorio)
5 DATABASE_HOST=<host-del-servicio-gestionado>
6 DATABASE_PORT=5432
7 DATABASE_USER=postgres
8 DATABASE_PASSWORD=<password-segura>
9 DATABASE_NAME=sevillaneando
10 DATABASE_SSL=true
11
12 # CORS: dejar vacio si solo hay clientes moviles nativos.
13 # Si se accede a Swagger desde navegador, anadir el dominio del frontend
    web.
14 ALLOWED_ORIGINS=
15
16 # Firebase Authentication (Admin SDK)
17 FIREBASE_PROJECT_ID=<proyecto-firebase>
18 FIREBASE_CLIENT_EMAIL=<email-service-account>
19 # Incluir los \n literales: "-----BEGIN PRIVATE KEY-----\n...\n-----END
    PRIVATE KEY-----\n"
20 FIREBASE_PRIVATE_KEY=<clave-privada-firebase>
21
22 # Cloudinary
23 CLOUDINARY_CLOUD_NAME=<cloud-name>
```

```
24 CLOUDINARY_API_KEY=<api-key>
25 CLOUDINARY_API_SECRET=<api-secret>
26
27 # APIs externas
28 TICKETMASTER_API_KEY=<api-key>
29 GEMINI_API_KEY=<api-key>
30
31 # Usuario de sistema para scraping
32 SCRAPER_SYSTEM_UID=system-scraper-uid
33 SCRAPER_SYSTEM_EMAIL=scraper.bot@sevillaneando.local
34 SCRAPER_SYSTEM_NAME=Sevillaneando Bot
35 LEGACY_SCRAPER_EMAIL=mod@demo.com
```

Importante: la variable `FIREBASE_PRIVATE_KEY` debe incluir los saltos de línea como `\n` literales dentro de comillas dobles, exactamente como los proporciona Firebase al descargar el archivo de credenciales JSON.

C.5.3. Despliegue en Render

El backend está configurado para desplegarse en **Render** como Web Service. El proceso es el siguiente:

Creación del servicio

1. Crear una cuenta en <https://render.com> y conectarla con la cuenta de GitHub donde está alojado el repositorio.
2. Seleccionar *New > Web Service* y elegir el repositorio del proyecto.
3. Seleccionar la rama `main` como rama de despliegue.

Configuración del proceso de build

En la pantalla de configuración del servicio, establecer los siguientes valores:

-
- ```
1 Root Directory: apps/backend
2 Build Command: npm install && npm run build
3 Start Command: node dist/main.js
```
- 

Seleccionar **Node 20** como entorno de ejecución.

#### Configuración de variables de entorno

Añadir todas las variables listadas en la sección anterior desde el panel *Environment* del servicio en Render. Este paso es crítico: sin las variables correctas, el backend no podrá conectar con la base de datos ni con los servicios externos.

## Publicación

Una vez completada la configuración, Render realiza el primer despliegue automáticamente y genera una URL pública del tipo:

---

```
1 https://sevillaneando-backend.onrender.com
```

---

Esta URL debe usarse como valor de `EXPO_PUBLIC_API_URL` en el frontend.

**Nota:** el plan gratuito de Render suspende los servicios tras 15 minutos de inactividad. Para una disponibilidad continua se recomienda un plan de pago.

## C.6. Distribución de la aplicación móvil

Al tratarse de un proyecto académico, la distribución de la aplicación móvil no se realiza a través de tiendas de aplicaciones, ya que publicar en Google Play requiere una tarifa de registro única de 25 USD y publicar en App Store requiere una cuenta de desarrollador de Apple con un coste de 99 USD anuales. En su lugar, la aplicación se distribuye mediante dos mecanismos sin coste: instalación directa mediante APK en Android y acceso mediante Expo Go.

Durante el desarrollo se valoró también el uso de **EAS Update** para compartir un código QR asociado a la versión publicada en Expo. Sin embargo, desde el 12 de mayo de 2026 Expo modificó el comportamiento de carga de proyectos en Expo Go: las actualizaciones publicadas mediante EAS Update solo pueden abrirse en Expo Go si el usuario autenticado es propietario del proyecto o pertenece a la organización propietaria [16]. Por este motivo, este mecanismo deja de ser adecuado para una defensa o prueba abierta con evaluadores externos no registrados en el proyecto.

La alternativa adoptada para la demostración en iOS y Android es el uso de **Expo Go con túnel**. Este mecanismo no publica una versión permanente de la aplicación en Expo, sino que expone temporalmente el servidor de desarrollo local mediante un túnel accesible desde Internet. La aplicación móvil se carga desde el equipo de desarrollo, pero sus peticiones siguen apuntando al backend desplegado en Render mediante la variable `EXPO_PUBLIC_API_URL`; por tanto, los datos utilizados son los del entorno de producción y no una base de datos local.

### C.6.1. Variables de entorno del frontend

El frontend dispone de dos ficheros de entorno: `apps/frontend/.env.development` (para local) y `apps/frontend/.env.production` (para producción). Para arrancar apuntando al backend de producción sin generar un build completo, ejecutar:

---

```
1 cd apps/frontend
2 npm run start:prod # copia .env.production a .env y lanza Expo
```

---

El archivo `apps/frontend/.env.production` debe contener:

---

```
1 EXPO_PUBLIC_API_URL=https://sevillaneando-backend.onrender.com
2 EXPO_PUBLIC_SHARE_BASE_URL=https://sevillaneando-backend.onrender.com/
 events
3
4 EXPO_PUBLIC_FIREBASE_API_KEY=<api-key>
5 EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN=<proyecto>.firebaseapp.com
6 EXPO_PUBLIC_FIREBASE_PROJECT_ID=<proyecto>
7 EXPO_PUBLIC_FIREBASE_STORAGE_BUCKET=<proyecto>.appspot.com
8 # Android e iOS tienen APP_ID distintos:
9 # iOS: 1:NNNNN:ios:XXXX
10 # Android: 1:NNNNN:android:XXXX
11 EXPO_PUBLIC_FIREBASE_APP_ID=<app-id>
```

---

### C.6.2. Opción 1: APK para Android

La opción más directa para dispositivos Android consiste en generar un archivo `.apk` e instalarlo directamente en el dispositivo sin pasar por ninguna tienda. Para ello se utiliza **EAS CLI**:

---

```
1 npm install -g eas-cli
2 eas login
3 cd apps/frontend
4 eas build -p android --profile preview
```

---

El perfil `preview` genera un `.apk` instalable directamente, a diferencia del perfil `production` que genera un `.aab` pensado para Google Play. Una vez finalizada la construcción en los servidores de Expo, se descarga el `.apk` y se transfiere al dispositivo Android. Es necesario habilitar la opción *Instalar aplicaciones de fuentes desconocidas* en los ajustes del dispositivo.

**Nota:** esta opción solo es válida para Android. iOS no permite la instalación de aplicaciones fuera de la App Store sin una cuenta de desarrollador de Apple, por lo que en ese caso se utiliza Expo Go con túnel.

### C.6.3. Opción 2: Expo Go con túnel

**Expo Go** permite ejecutar la aplicación directamente desde el código fuente sin necesidad de generar ninguna build. Es compatible con iOS y Android y no tiene ningún coste. Para permitir el acceso desde dispositivos que no están en la misma red local que el equipo de desarrollo, se utiliza el modo túnel:

---

```
1 cd apps/frontend
2 npx expo start --tunnel
```

---



Expo muestra un código QR en la terminal. El evaluador o usuario que desee probar la aplicación solo necesita:

1. Instalar la aplicación **Expo Go** desde la App Store o Google Play (gratuita).
2. Escanear el código QR con la cámara del dispositivo (iOS) o desde dentro de Expo Go (Android).

La aplicación se carga directamente en el dispositivo conectándose al servidor de desarrollo a través del túnel. Para que funcione, el equipo de desarrollo debe permanecer encendido y el proceso de Expo debe seguir ejecutándose. Este método se utiliza únicamente como mecanismo de demostración y validación, no como despliegue permanente de la aplicación.

## C.7. Validación posterior al despliegue

Una vez desplegado el sistema, es necesario verificar el correcto funcionamiento de todos los componentes en producción.

### C.7.1. Verificación del backend

Comprobar que el endpoint de health check responde:

---

```
1 curl https://sevillaneando-backend.onrender.com/scheduler/health
```

---

La respuesta esperada es un JSON con el estado del sistema:

---

```
1 {"status": "ok"}
```

---

### C.7.2. Funcionalidades a validar

- **Autenticación:** registro de nuevo usuario, inicio de sesión y cierre de sesión.
- **Eventos:** consulta del listado de eventos, visualización en el mapa y acceso al detalle.
- **Creación y gestión de eventos:** flujo completo de creación, edición y eliminación con imágenes (verifica Cloudinary).
- **Moderación:** revisión de eventos pendientes, listado independiente con todos los eventos públicos editables y eliminación de eventos públicos por parte del moderador.
- **Chat:** envío y recepción de mensajes en tiempo real (verifica WebSockets).
- **Notificaciones:** recepción de notificaciones en la aplicación.
- **Carga de imágenes:** subida de foto de perfil (verifica Cloudinary).

- **Mapas:** visualización de eventos en el mapa interactivo (verifica OpenStreetMap).
- **Geolocalización:** búsqueda de eventos cercanos (verifica PostGIS).

Esta validación permite detectar errores de configuración que no se manifiestan en local, como problemas de CORS, certificados SSL, credenciales incorrectas o URLs mal apuntadas.

## C.8. Checklist de despliegue

Lista de verificación antes de dar por concluido el despliegue:

- ☐ La base de datos está activa y la extensión PostGIS está habilitada.
- ☐ El backend compila y arranca correctamente (`npm run build` sin errores).
- ☐ Todas las variables de entorno están configuradas en el proveedor de hosting.
- ☐ El endpoint `/scheduler/health` responde con estado ok.
- ☐ La conexión SSL con la base de datos está activa.
- ☐ Las credenciales de Firebase, Cloudinary, Ticketmaster y Gemini son válidas.
- ☐ El frontend apunta a la URL correcta del backend de producción.
- ☐ El APK de Android se genera sin errores o la aplicación carga correctamente mediante Expo Go con túnel.
- ☐ Las funcionalidades principales se han validado en el entorno de producción.
- ☐ El CORS está configurado para aceptar únicamente los orígenes esperados.

## C.9. Actualización del sistema

Para desplegar una nueva versión del backend en Render, simplemente se hace push a la rama `main` del repositorio. Render detecta el nuevo commit y lanza automáticamente un nuevo despliegue.

Para actualizar la aplicación móvil, se genera una nueva build con los cambios aplicados (APK para Android) o se reinicia el servidor de Expo Go con túnel apuntando al código actualizado.

## C.10. Rollback y recuperación

En caso de que una nueva versión desplegada introduzca errores graves, es necesario revertir a la versión anterior estable.

### C.10.1. Rollback del backend en Render

1. Acceder al panel de Render y seleccionar el servicio del backend.
2. Ir a la sección *Events* o *Deploys*.
3. Localizar el último despliegue estable en el historial.
4. Hacer clic en *Rollback to this deploy*.
5. Verificar que el sistema vuelve a responder correctamente mediante el endpoint `/scheduler/health`.

### C.10.2. Rollback de la aplicación móvil

Para el APK de Android, basta con regenerar la build a partir del commit estable anterior y redistribuir el nuevo `.apk`:

---

```
1 git checkout <commit-estable>
2 cd apps/frontend
3 eas build -p android --profile preview
```

---

Para Expo Go, al ejecutar `npx expo start -tunnel` siempre se sirve el código fuente actual, por lo que el rollback consiste simplemente en volver al commit anterior con `git checkout` y reiniciar el servidor.

### C.10.3. Respaldo de la base de datos

Si se utiliza un servicio gestionado (Supabase, AWS RDS, etc.), los backups automáticos están disponibles en el panel de administración del servicio. Para restaurar:

1. Identificar el punto de restauración más reciente anterior al incidente.
2. Seguir el proceso de restauración del proveedor (generalmente un botón *Restore to point in time*).
3. Verificar la integridad de los datos restaurados.
4. Reiniciar el backend para que reconecte con la base de datos restaurada.

## C.11. Referencias

Las referencias técnicas empleadas en este manual corresponden a la documentación oficial de cada plataforma y se recogen en la bibliografía del documento principal: Render [64], Expo EAS Build [65], cambios en Expo Go [16], NestJS [31], PostGIS [39] y Supabase [66].

# D. Manual de usuario

---

## D.1. Introducción

Sevillaneando es una aplicación móvil orientada al descubrimiento de eventos culturales y de ocio en Sevilla, con especial interés en dar visibilidad a propuestas emergentes o menos conocidas. Su finalidad no es únicamente mostrar un catálogo de actividades, sino ofrecer al usuario una experiencia completa de exploración, interacción y personalización a partir de sus intereses, su ubicación y su comportamiento dentro de la plataforma.

La aplicación permite consultar eventos, filtrarlos, localizarlos sobre el mapa, guardar aquellos que resulten de interés, apuntarse a ellos, compartirlos, valorarlos y participar en espacios de comunicación asociados. Además, incorpora funcionalidades avanzadas, como recomendaciones personalizadas y generación de rutas, que ayudan al usuario a descubrir nuevas actividades ajustadas a sus preferencias y a planificar mejor su asistencia.

Desde el punto de vista de uso, el sistema contempla distintos perfiles. El rol de usuario permite crear actividades. Por su parte, los roles de moderador y administrador cubren necesidades de supervisión y gestión interna de la aplicación.

El objetivo de este manual es guiar al usuario en el uso de la aplicación, describiendo sus principales entidades, requisitos de uso y flujos de interacción más relevantes. A lo largo del documento se explica cómo utilizar las distintas funcionalidades del sistema de manera progresiva, de forma que cualquier persona pueda comprender el funcionamiento general de la aplicación y aprovechar todas sus posibilidades.

## D.2. Definición de entidades y funcionalidades

### Evento

El evento es la entidad principal del sistema y representa cada actividad cultural o de ocio disponible en la aplicación. Cada evento dispone de información descriptiva y funcional suficiente para que el usuario pueda decidir si le interesa asistir, guardarlo para más adelante o interactuar con él.

#### Funcionalidades

- Creación de eventos públicos y privados.
- Edición de la información del evento, como fecha, ubicación, descripción o precio.
- Inscripción de usuarios.
- Apertura de la ubicación del evento en aplicaciones externas de mapas.
- Guardado del evento para consultas posteriores.
- Visualización de asistentes.
- Valoración del evento.
- Acceso al chat asociado al evento.
- Compartición del evento mediante enlace.
- Gestión de estados: activo, finalizado y en moderación.
- Inclusión en distintos listados personalizados.

### Mapa de eventos

El mapa de eventos permite representar visualmente la distribución geográfica de las actividades disponibles. Esta funcionalidad resulta especialmente útil para usuarios que desean localizar eventos cercanos o comparar rápidamente varias opciones en función de su ubicación.

#### Funcionalidades

- Visualización de eventos sobre mapa interactivo.
- Clasificación aproximada por distancia mediante colores (verde, amarillo y rojo).
- Configuración del radio de búsqueda.
- Acceso directo al detalle del evento.
- Uso de la ubicación del usuario como referencia.

## Recomendaciones

El sistema de recomendaciones tiene como propósito adaptar la experiencia de uso a cada usuario. Para ello, tiene en cuenta factores como los intereses del perfil, la cercanía, las interacciones previas y otra información relevante del comportamiento dentro de la aplicación.

### Funcionalidades

- Recomendación de eventos según intereses del perfil.
- Aprendizaje basado en la interacción del usuario.
- Influencia de proximidad, valoraciones (propias y media global), eventos guardados, eventos apuntados, eventos vistos y eventos compartidos.
- Consideración de fechas y disponibilidad.
- Generación de rutas recomendadas.

## Perfil de usuario

El perfil recoge la información personal y las preferencias del usuario. Esta entidad resulta fundamental, ya que condiciona parte del comportamiento personalizado del sistema, especialmente en lo relativo a recomendaciones y funciones basadas en ubicación.

### Funcionalidades

- Edición de datos personales.
- Configuración de ubicación.
- Gestión de intereses (categorías preferidas).
- Cambio de contraseña.
- Eliminación de cuenta.

## Rutas

Las rutas son agrupaciones de eventos organizadas para facilitar la planificación del usuario. Pueden responder tanto a una lógica automática de recomendación como a la creación manual por parte de los propios usuarios. Este es otro punto clave, ya que permite agrupar eventos relacionados de forma coherente y optimizada en función de factores como la ubicación, el horario y las preferencias del usuario. De este modo, no solo se facilita la planificación de la asistencia a múltiples eventos, sino que también se incrementa la visibilidad de eventos relevantes que podrían pasar desapercibidos de forma individual, mejorando así el descubrimiento de contenido y la eficiencia en la experiencia de uso.

### Funcionalidades

- Generación automática de rutas recomendadas.
- Creación manual de rutas por usuarios.
- Optimización por distancia y horarios.
- Consulta y valoración de rutas creadas por otros usuarios.

## Notificaciones

El sistema de notificaciones mantiene informado al usuario sobre sucesos relevantes ocurridos dentro de la aplicación. Gracias a este módulo, la plataforma puede comunicar cambios de estado, recordatorios, eventos cercanos y actividad social de forma proactiva.

### Funcionalidades

- Notificación de aprobación o rechazo de eventos.
- Recordatorios previos a eventos apuntados.
- Avisos sobre eventos cercanos según la ubicación.
- Notificación de nuevos seguidores.

## Mensajería

Sistema de comunicación entre usuarios. Este módulo resulta fundamental dentro de la aplicación, ya que facilita la interacción directa entre participantes de un mismo evento o usuarios con intereses comunes. Gracias a esta funcionalidad, se promueve la conexión social, permitiendo coordinar asistencia, resolver dudas y organizar encuentros previos o durante el evento. Además, contribuye a mejorar la experiencia del usuario al fomentar la colaboración, la planificación compartida y la creación de redes sociales dentro de la plataforma.

### Funcionalidades

- Chat asociado a eventos.
- Mensajes privados.
- Acceso desde asistentes o perfiles de usuario.
- Envío de texto e imágenes.

## Amistad

El módulo de amistad permite descubrir y mantener contacto con otros usuarios de la plataforma. Aunque se apoya técnicamente en relaciones de seguimiento, la interfaz lo presenta como una sección social desde la que se pueden buscar usuarios, consultar perfiles, seguir o dejar de seguir a otras personas, enviar mensajes privados y ver los eventos a los que asisten.

### Funcionalidades

- Búsqueda de usuarios.
- Listado completo de seguidores en la pestaña *Seguidores*.
- Listado de usuarios seguidos en la pestaña *Seguidos*.
- Listado de amistades mutuas en la pestaña *Amigos*, cuando dos usuarios se siguen entre sí.
- Acceso a listas independientes de seguidores y seguidos desde los contadores del perfil público.
- Acceso al perfil de otros usuarios, con opción de seguir, dejar de seguir, enviar mensaje y consultar eventos a los que asisten.

### Administración

La administración cubre las funciones de gestión global de la plataforma reservadas al rol de administrador.

#### Funcionalidades

- Consulta de estadísticas globales de usuarios y eventos.
- Gestión de usuarios y roles.
- Gestión de categorías.
- Restablecimiento administrativo de eventos importados por scraping.

### Moderación

La moderación se ocupa de supervisar los contenidos que deben ser validados antes de su publicación y de mantener la coherencia y fiabilidad de los eventos ya publicados.

#### Funcionalidades

- Revisión de eventos pendientes de aprobación.
- Edición previa a la publicación.
- Aprobación o rechazo de eventos.
- Edición y eliminación de cualquier evento público ya aprobado.

## D.3. Requisitos técnicos

Para utilizar correctamente la aplicación es necesario disponer de los siguientes elementos:

- Un dispositivo móvil compatible con iOS o Android.
- Conexión a Internet, ya sea mediante WiFi o datos móviles.
- Una cuenta de correo electrónico válida para el registro y recuperación de contraseña.
- Permisos recomendados de acceso a cámara y galería, necesarios para funcionalidades de imagen.
- Permisos de ubicación, recomendables para el uso del mapa y la recepción de notificaciones cercanas.

## D.4. Guía de uso de la aplicación

Una vez definidas las entidades principales del sistema, en esta sección se describe el uso práctico de la aplicación desde la perspectiva del usuario. El objetivo no es únicamente enumerar pantallas, sino explicar cómo puede utilizarse la plataforma para cubrir necesidades reales, como descubrir eventos, organizar la asistencia, comunicarse con otros usuarios o gestionar actividades propias.



### D.4.1. Registro e inicio de sesión (para todos los roles)

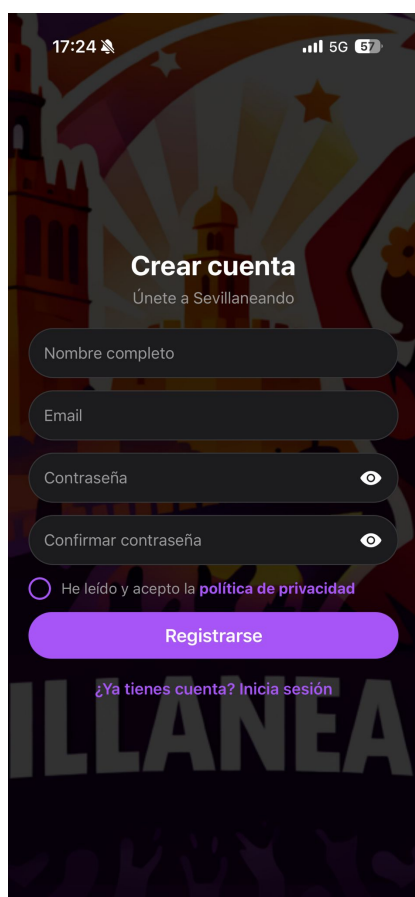
El primer paso para utilizar la aplicación consiste en crear una cuenta. Al abrir la aplicación por primera vez se muestra la pantalla de inicio de sesión, desde la que el usuario puede acceder al formulario de registro pulsando el enlace inferior que indica que aún no dispone de cuenta. En la pantalla de registro debe introducir su nombre completo, una dirección de correo electrónico válida y una contraseña. La contraseña debe escribirse dos veces, en el campo principal y en el campo de confirmación, para evitar errores; mediante el icono con forma de ojo situado a la derecha de cada campo puede mostrarse u ocultarse temporalmente lo escrito. La aplicación exige que la contraseña tenga al menos seis caracteres y que ambas coincidan.

Antes de poder completar el alta, el usuario debe marcar la casilla de aceptación de la política de privacidad situada bajo los campos de contraseña. El texto que la acompaña incluye un enlace remarcado que abre la política completa en una pantalla independiente, donde puede consultarse íntegramente antes de aceptar. Una vez marcada la casilla, basta con pulsar el botón principal para crear la cuenta. Si alguno de los datos es incorrecto, no se acepta la política o el correo ya está registrado, la pantalla muestra un mensaje de error explicativo bajo el formulario y permite corregir lo necesario sin perder lo escrito. Aunque la plataforma contempla distintos roles, el registro siempre crea una cuenta con rol de usuario; los roles de moderador y administrador son asignados manualmente por el administrador principal, al que puede contactarse a través del correo electrónico mostrado al final de la página de inicio.

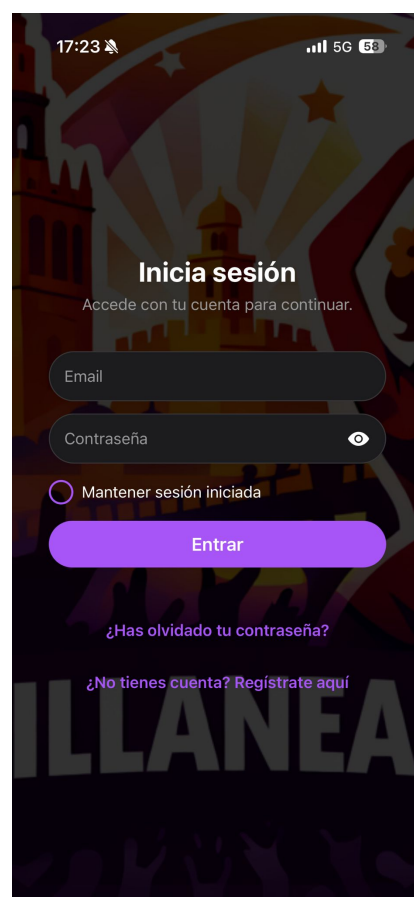
Una vez creada la cuenta, el acceso posterior se realiza desde la pantalla de inicio de sesión introduciendo el correo y la contraseña. Bajo los campos aparece la casilla *Mantener sesión iniciada*, que conviene marcar si se desea que el correo quede recordado en futuros accesos. Si el usuario no recuerda la contraseña, debe escribir su correo en el primer campo y pulsar el enlace *¿Olvidaste tu contraseña?* situado bajo el botón de inicio de sesión; la aplicación envía entonces un mensaje al correo indicado con las instrucciones para restablecerla. Tras completar el cambio desde el correo, puede volver a la pantalla de inicio de sesión e introducir la nueva contraseña con normalidad.

#### **Aspectos a tener en cuenta:**

- El correo electrónico debe ser válido.
- La contraseña debe recordarse o recuperarse mediante el correo asociado.
- La aceptación de la política de privacidad es obligatoria para crear la cuenta.
- El registro crea cuentas con rol de usuario.



(a) Crear cuenta



(b) Iniciar sesión

Figura D.1: Acceso a la aplicación

### D.4.2. Gestión del perfil (para todos los roles)

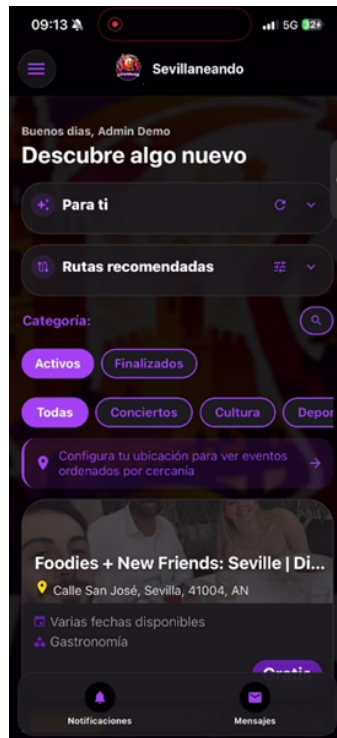
Tras iniciar sesión, el usuario puede abrir el menú lateral pulsando el icono de tres líneas situado en la esquina superior izquierda de la página principal. Este panel agrupa en su parte superior la información del usuario, los botones de selección del tema visual (*Claro* y *Oscuro*) y, justo debajo del avatar, un icono de lápiz que abre la pantalla de edición del perfil. El mismo menú incluye, en su parte inferior, el botón de cierre de sesión.

La pantalla de edición del perfil se compone de varios bloques que el usuario puede recorrer mediante desplazamiento vertical. En la parte superior aparece la fotografía de perfil: al pulsar sobre ella se abre la galería del dispositivo para seleccionar una nueva imagen, que se recorta en formato cuadrado y se sube automáticamente al guardarla; si ya existía una foto previa, junto a ella aparece un botón rojo *Quitar foto* para eliminarla. A continuación se editan el nombre y el correo electrónico mediante dos campos de texto. Más abajo, un recuadro informativo explica el uso de la ubicación: el usuario puede escribir una dirección o lugar en el campo de búsqueda y pulsar la lupa para localizarlo, o bien tocar directamente sobre el mapa para fijar el punto deseado, que queda marcado con un pin. Las coordenadas seleccionadas se muestran debajo del mapa y, si se desea descartar la ubicación, basta con pulsar el icono rojo de aspa situado junto a ellas.

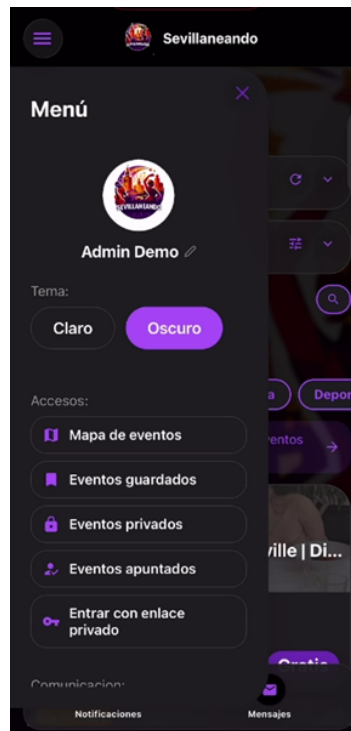
En el bloque de intereses se listan las categorías disponibles como etiquetas pulsables; al tocar una etiqueta esta se activa o desactiva, quedando resaltadas las seleccionadas. Estos intereses condicionan posteriormente las recomendaciones de eventos. Bajo este bloque, el botón principal *Guardar* confirma todos los cambios y vuelve a la pantalla anterior. El botón secundario *Cambiar contraseña* abre una pantalla independiente desde la que puede modificarse la contraseña actual. A continuación se muestra el selector de idioma, con dos opciones (*Español* e *English*) que cambian el idioma de la interfaz al pulsarlas. Por último, en la parte inferior aparece el botón rojo *Eliminar cuenta*: al pulsarlo se solicita confirmación mediante un cuadro de diálogo y, si se acepta, la cuenta se elimina de forma definitiva tanto en la aplicación como en el servicio de autenticación. Si el inicio de sesión es demasiado antiguo, la aplicación pide volver a autenticarse antes de completar el borrado.

#### **Validaciones y recomendaciones:**

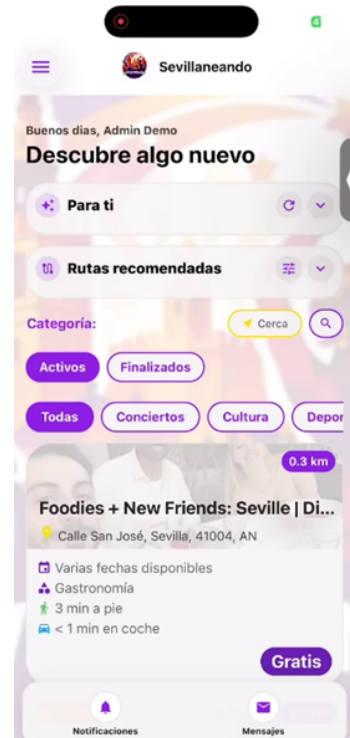
- Se recomienda indicar una ubicación para habilitar todas las funciones de proximidad.
- Los datos modificados deben guardarse correctamente antes de salir.
- El correo electrónico debe mantener un formato válido.



(a) Página principal

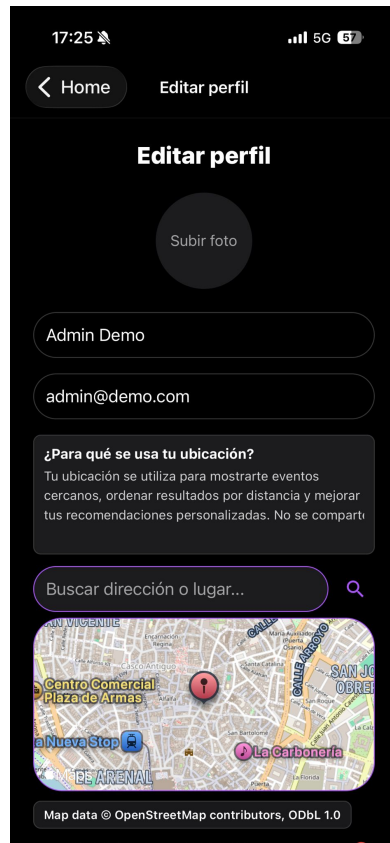


(b) Menú lateral

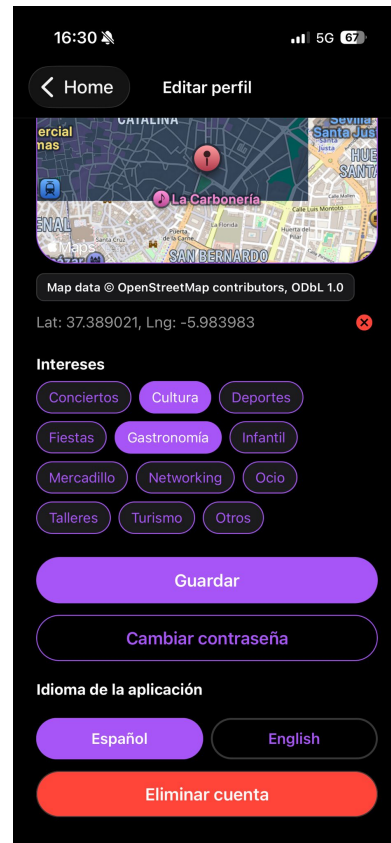


(c) Modo claro

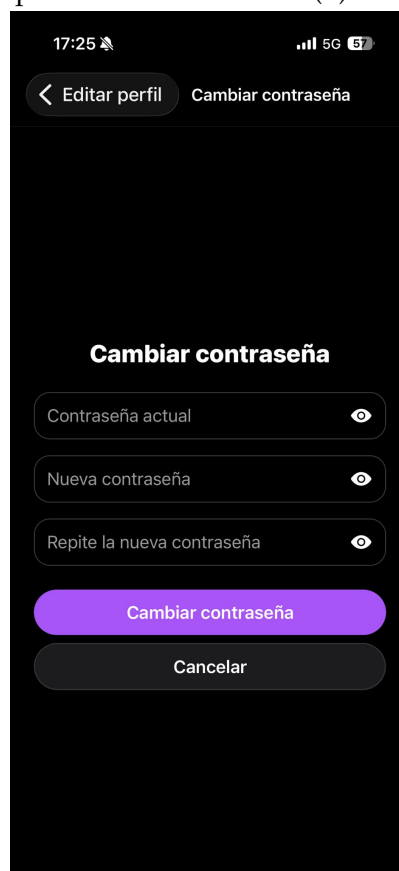
Figura D.2: Acceso y opciones del perfil



(a) Editar perfil 1



(b) Editar perfil 2



(c) Cambiar contraseña

Figura D.3: Gestión del perfil

### D.4.3. Mapa de eventos (para todos los roles)

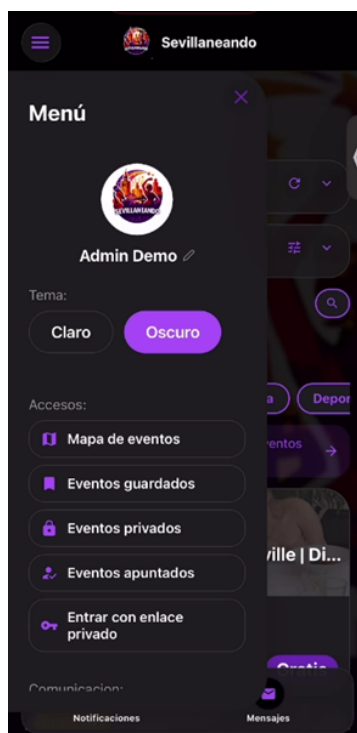
La aplicación permite consultar los eventos en un mapa interactivo, lo que resulta útil para usuarios que desean localizar actividades próximas a su ubicación. Para acceder a esta funcionalidad, el usuario debe abrir el menú lateral desde la esquina superior izquierda y pulsar la opción *Mapa de eventos*. La pantalla se carga centrada en la ubicación que el usuario tiene configurada en su perfil; si todavía no ha indicado ninguna, el mapa se centra en Sevilla y un aviso en la parte inferior recuerda que es necesario configurar la ubicación para que las funciones de proximidad tengan efecto.

Si el usuario tiene ubicación configurada, su posición se representa con un marcador morado distintivo y a su alrededor se dibuja un círculo translúcido que delimita el radio de búsqueda actual. Los eventos aparecen como chinchetas coloreadas en función de su distancia: verde cuando se encuentran a menos de un kilómetro, amarillo entre uno y tres, naranja entre tres y cinco, y rojo por encima de cinco; los eventos cuya distancia no puede calcularse aparecen en morado. En la parte inferior de la pantalla se muestra un recuadro con el radio actual en kilómetros y una barra deslizante que permite modificarlo entre doscientos metros y cinco kilómetros; el círculo del mapa se actualiza en tiempo real conforme se ajusta este valor.

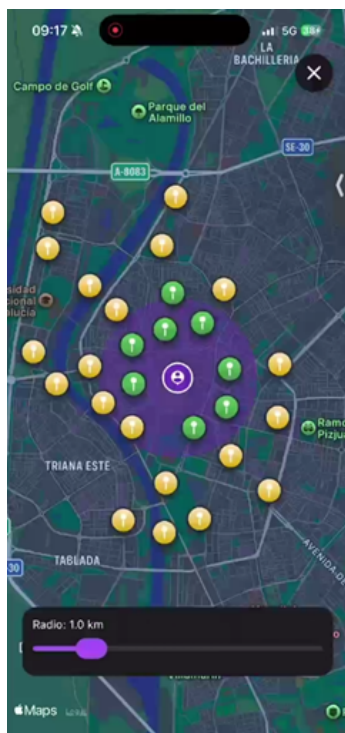
Al pulsar sobre la chincheta de un evento aparece una tarjeta resumen con el título, una breve descripción, la dirección y la distancia. En esta tarjeta, el botón *Ver detalles* (o un toque adicional sobre la propia tarjeta en iOS) abre la pantalla de detalle completo del evento. Para cerrar la tarjeta sin abandonar el mapa, basta con pulsar el icono de aspa que aparece en su esquina. Para salir de la pantalla del mapa se utiliza el botón circular con icono de cierre situado en la esquina superior derecha, que devuelve al usuario a la página principal.

#### **Aspectos a tener en cuenta:**

- Es necesario haber configurado una ubicación previamente en el perfil.



(a) Menú lateral



(b) Eventos en el mapa



(c) Etiqueta del evento

Figura D.4: Mapa de eventos

#### D.4.4. Página principal y filtros (para todos los roles)

La página principal es el espacio central de consulta de la aplicación y se muestra automáticamente tras iniciar sesión. En su parte superior aparece un saludo personalizado con el nombre del usuario que cambia según la hora del día, y debajo se sitúa una barra horizontal con las categorías disponibles, que el usuario puede arrastrar lateralmente para recorrerlas y pulsar para activar una categoría como filtro. Manteniendo pulsada una categoría es posible reordenarlas: al soltar, el nuevo orden queda guardado en el perfil para que las preferidas aparezcan primero en futuros accesos. La etiqueta *Todas* restablece la vista sin filtro de categoría.

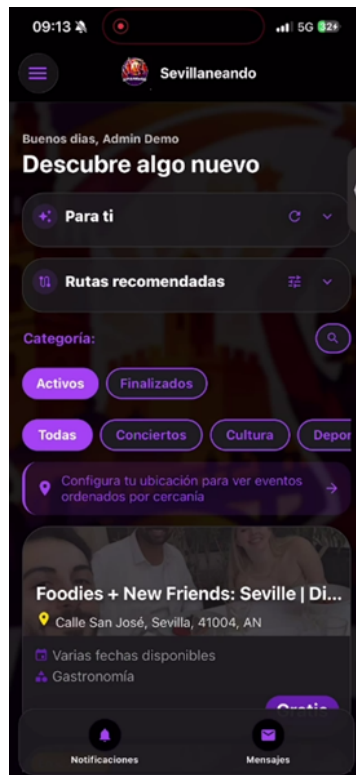
Bajo las categorías se encuentran dos botones de filtrado rápido. El primero, *Cerca*, se ilumina cuando se activa y muestra únicamente los eventos dentro del radio configurado; al hacerlo aparece sobre el listado una fila de valores de radio en kilómetros que el usuario puede pulsar para ajustar la distancia. Este filtro requiere tener ubicación configurada en el perfil; si no la hay, la aplicación muestra un mensaje que enlaza directamente con la pantalla de edición del perfil. El segundo botón abre el buscador avanzado en una ventana emergente que permite escribir un texto libre para buscar por título o descripción, definir un precio mínimo y máximo, y delimitar un intervalo de fechas mediante dos selectores de calendario. Dentro del buscador, el botón *Buscar* aplica los criterios y *Limpiar filtros* los descarta.

Entre los filtros y el listado aparecen dos pestañas, *Eventos activos* y *Eventos finalizados*, que separan los eventos vigentes de los ya finalizados. Cada evento se muestra como una tarjeta con su imagen de portada, título, dirección, fecha y categoría; al pulsar la tarjeta se accede a la pantalla de detalle. Por encima del listado, cuando hay datos suficientes, la página integra dos secciones de recomendaciones (eventos y rutas) que se describen más adelante. En la parte inferior de la pantalla se sitúa una barra fija con accesos directos a notificaciones, mensajes, creación de evento, moderación (si el rol lo permite) y amigos.

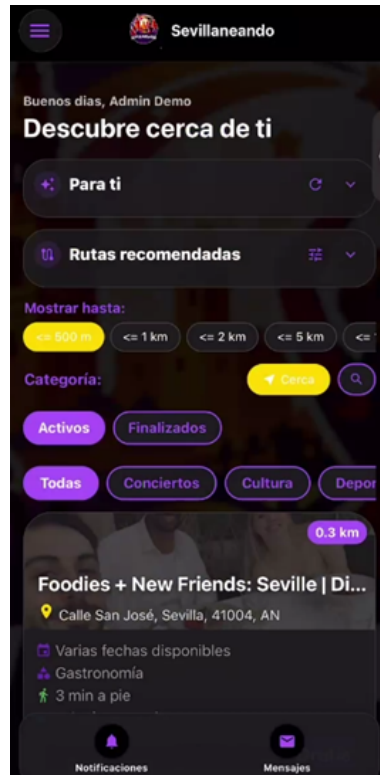
##### **Aspectos a tener en cuenta:**

- Los eventos finalizados siguen siendo visibles, pero no admiten inscripción.
- El filtrado por cercanía requiere ubicación.
- Las categorías pueden organizarse arrastrándolas horizontalmente para dejar primero las favoritas.

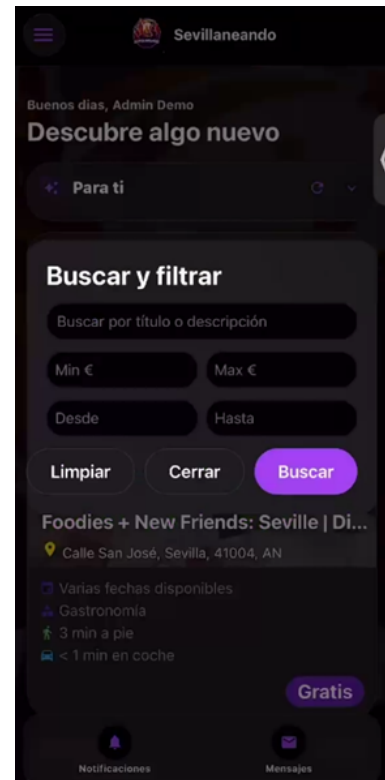




(a) Página principal



(b) Búsqueda por cercanía



(c) Buscador avanzado

Figura D.5: Filtros de búsqueda

#### D.4.5. Detalle de evento

La pantalla de detalle se abre al pulsar cualquier evento desde la página principal, el mapa, un listado personalizado o una notificación. En la parte superior aparece un carrusel de imágenes que puede desplazarse horizontalmente con el dedo, y bajo este se muestran el título, la dirección, la descripción, el rango de fechas, la duración total, el tiempo restante hasta el inicio o el final, la categoría y, si procede, la recurrencia y la valoración media en estrellas. El precio se muestra destacado en un recuadro morado en la parte inferior derecha del bloque informativo. Si la descripción contiene un enlace a la fuente original (común en eventos importados por scraping), este aparece resaltado y abre el navegador del dispositivo al pulsarlo.

Más abajo se incluye un pequeño mapa centrado en la ubicación del evento; por defecto está bloqueado para no interferir con el desplazamiento de la pantalla, pero pulsando el icono de candado de su esquina puede desbloquearse para hacer zoom o moverlo. Al pulsar la chincheta del mapa se muestra una etiqueta con el título y un enlace que abre la ubicación en Google Maps o Apple Maps según el sistema operativo del dispositivo.

A continuación se muestra el panel *Acciones*, organizado en botones. El botón principal permite apuntarse al evento o, si el usuario ya está apuntado, eliminar su inscripción; este botón queda deshabilitado en eventos finalizados. Bajo él hay dos filas de botones secundarios: *Asistentes*, que abre una ventana emergente con la lista de personas apuntadas (cada nombre es pulsable y abre el perfil correspondiente); *Abrir mapa*, que lanza la aplicación de mapas externa con la dirección del evento ya cargada; *Guardar* (o *Quitar de guardados*), que añade el evento al listado de eventos guardados y alimenta el sistema de recomendaciones; y *Compartir* (o *Invitar* en eventos privados), que abre el menú de compartir nativo del dispositivo con un mensaje que incluye el título, fecha, dirección, categoría, precio y enlace al evento. En el caso de eventos privados, este botón solo está disponible para el creador, y al pulsarlo se genera el enlace de acceso compartible. Por último, el botón *Valorar* (o *Editar valoración* si ya existe una) abre una ventana emergente con cinco estrellas pulsables para indicar la puntuación y un campo de texto opcional para escribir un comentario; al pulsar *Enviar* la valoración se guarda y la lista de opiniones del propio evento se actualiza automáticamente.

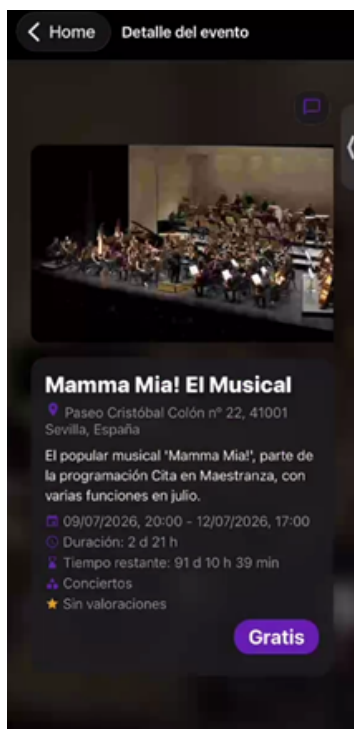
Bajo el panel de acciones se muestra la sección *Opiniones*, donde aparecen las valoraciones de otros usuarios con su foto, nombre, fecha, número de estrellas y comentario. Al pulsar sobre el nombre o la foto se accede al perfil público del autor de la opinión.

En la esquina superior derecha de la pantalla aparece un icono de bocadillo que abre el chat del evento, accesible a todos los participantes. El chat se despliega como un panel deslizante sobre el detalle, con el historial de mensajes en orden cronológico. Cada mensaje muestra el avatar del autor, su nombre y la hora; al pulsar sobre el avatar o el nombre se abre su perfil, y al pulsar el icono de sobre situado junto al mensaje se inicia un chat privado. Para enviar un mensaje se escribe en el campo inferior y se pulsa *Enviar*; junto al campo hay dos botones para adjuntar una

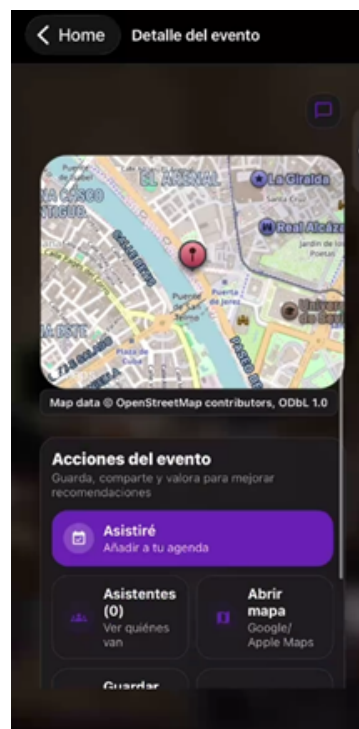
imagen desde la galería o tomar una foto directamente con la cámara. Al pulsar sobre una imagen recibida se abre a pantalla completa con posibilidad de hacer zoom con dos dedos. Manteniendo pulsado un mensaje propio aparece la opción de eliminarlo. Si el usuario tiene rol de moderador o administrador, en lugar del icono de bacadillo se muestra también un icono de lápiz junto a él, que abre la pantalla de edición moderada del evento.

#### Aspectos a tener en cuenta:

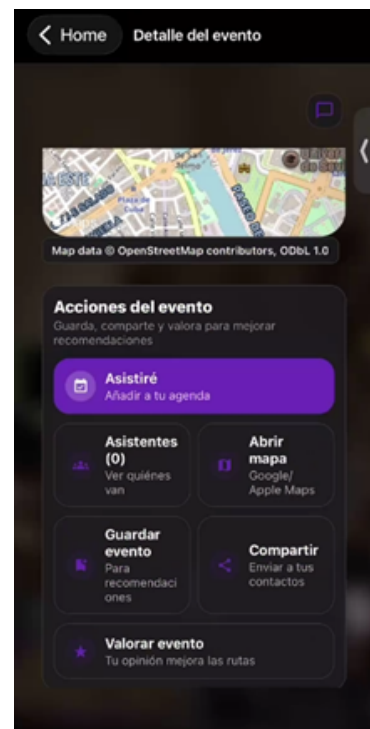
- Algunas acciones solo están disponibles en eventos activos (inscribirse) y públicos (compartir).
- La interacción con el evento influye en futuras recomendaciones.
- Desde asistentes, perfiles del chat y perfiles de usuario puede iniciarse mensajería privada.



(a) Detalles del evento

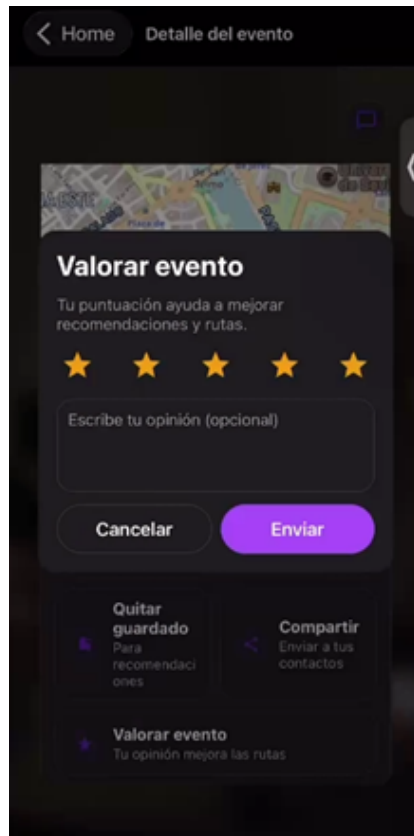


(b) Detalles del evento 2

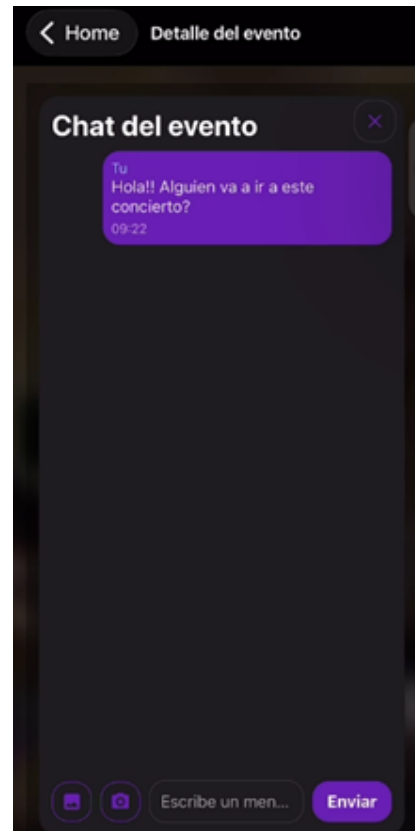


(c) Acciones del evento

Figura D.6: Pantalla del evento



(a) Valorar evento



(b) Chat del evento

Figura D.7: Interacción dentro del evento

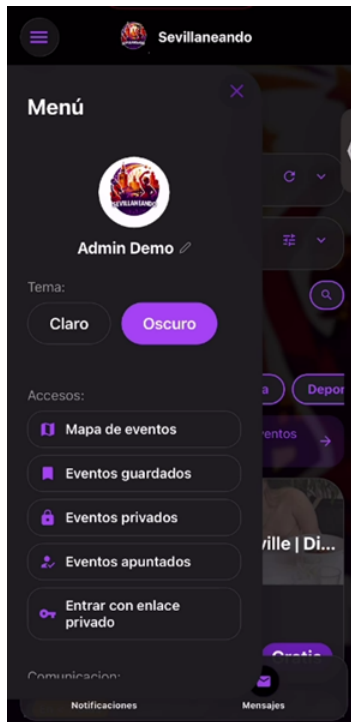
#### D.4.6. Listados de eventos

La aplicación organiza determinados eventos en listados personalizados accesibles desde el menú lateral. Cada listado responde a un tipo concreto de interacción previa del usuario y se actualiza automáticamente cada vez que se entra en él.

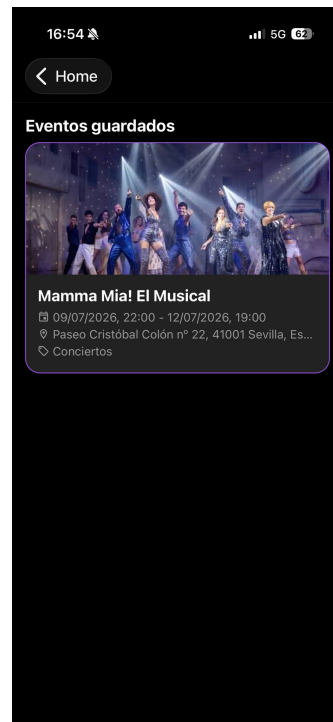
Para consultar los eventos guardados, el usuario debe abrir el menú lateral y pulsar *Eventos guardados*. La pantalla muestra el listado completo de los eventos que el usuario ha marcado pulsando el botón *Guardar* desde el panel de acciones del detalle del evento. Cada elemento se presenta como una tarjeta con la imagen, el título, la fecha, la dirección y la categoría, y al pulsar sobre ella se abre el detalle correspondiente. Para retirar un evento del listado basta con volver a su detalle y pulsar nuevamente el botón, que ahora aparece como *Quitar de guardados*.

Los eventos a los que el usuario se ha apuntado se consultan desde la opción *Eventos apuntados* del menú lateral. En esta pantalla se muestran únicamente las inscripciones activas, ordenadas por fecha. La inscripción y la cancelación se realizan siempre desde el botón principal del panel de acciones del detalle del evento.

El calendario es accesible desde la opción *Calendario* del menú lateral y muestra una vista mensual en la que los días con eventos a los que el usuario está apuntado aparecen marcados con un punto morado. Pulsando sobre un día concreto se selecciona y, bajo el calendario, se despliega la lista de eventos cuyo inicio coincide con esa fecha. Cada elemento de la lista incluye la hora de comienzo y el título, y al pulsarlo se abre el detalle del evento. Los controles de cabecera del calendario permiten avanzar y retroceder entre meses.

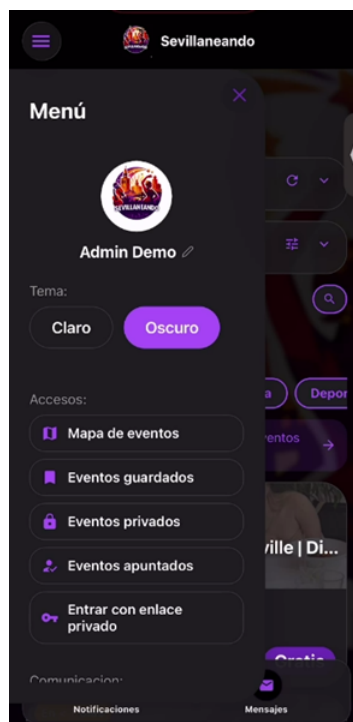


(a) Menú lateral

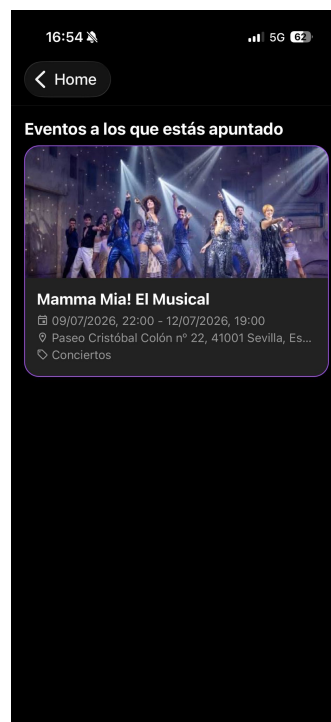


(b) Eventos guardados

Figura D.8: Listado de eventos guardados

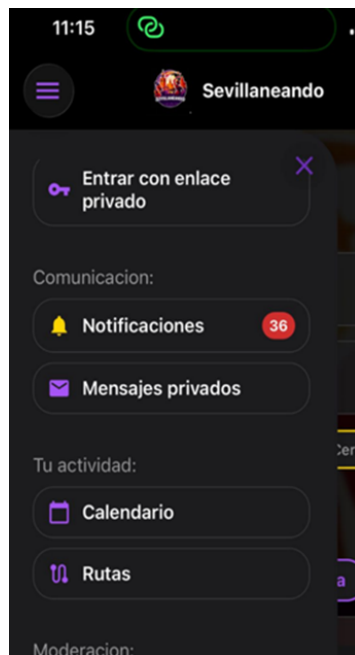


(a) Menú lateral



(b) Eventos apuntados

Figura D.9: Listado de eventos apuntados



(a) Menú lateral



(b) Calendario

Figura D.10: Calendario de eventos

### D.4.7. Gestión de administrador (rol de administrador)

El rol de administrador dispone de funcionalidades específicas para mantener la organización global del sistema. Tras iniciar sesión con una cuenta de administrador, en el menú lateral aparece la opción *Administración*, que abre el panel principal de gestión, así como una entrada adicional para la gestión de categorías.

El panel de administración se organiza en tres pestañas que pueden alternarse pulsando la barra superior. La pestaña *Estadísticas* es la que se muestra por defecto y presenta una serie de tarjetas con los principales indicadores del sistema: el número total de usuarios registrados, el total de eventos, los eventos aprobados, pendientes de moderación y rechazados, y, en un bloque diferenciado, el reparto entre eventos importados automáticamente por scraping y eventos creados manualmente por los propios usuarios. Esta información se actualiza al volver a entrar en el panel.

La pestaña *Usuarios* muestra la lista completa de cuentas con su foto, nombre, correo y rol actual. Al pulsar sobre un usuario se abre una ventana emergente desde la que el administrador puede asignar uno de los tres roles disponibles, *Usuario*, *Moderador* o *Administrador*, pulsando el botón correspondiente, o eliminar la cuenta por completo mediante el botón rojo de borrado. Ambas acciones piden confirmación antes de aplicarse, y al completarse, el listado se actualiza automáticamente con el nuevo estado.

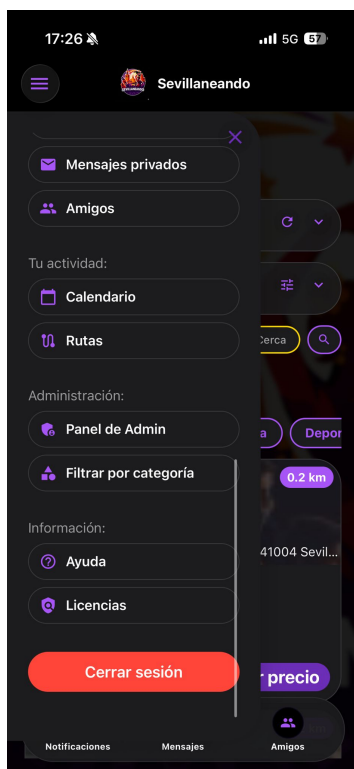
La pestaña *Scraping* contiene el botón *Restablecer eventos importados*. Al pulsarlo se solicita confirmación porque la operación borra todos los eventos previamente importados y vuelve a ejecutar los scrapers de las fuentes configuradas. La operación puede tardar varios minutos y al finalizar se muestra un resumen con el número de eventos eliminados y el número de eventos recuperados. Es recomendable utilizar esta funcionalidad solo cuando se detectan inconsistencias claras en el catálogo externo, ya que durante su ejecución la página principal puede mostrar resultados parciales.

La gestión de categorías se realiza desde una pantalla independiente accesible también desde el menú lateral. En ella se listan todas las categorías existentes con su nombre y descripción. Mediante el botón flotante con el símbolo + se abre un formulario para crear una nueva categoría rellenando estos dos campos, y pulsando sobre una categoría existente se abre el mismo formulario en modo edición. Cada categoría incluye también un botón de eliminación; los eventos asociados a una categoría borrada se reasignan automáticamente a la categoría *Otros* para no perderlos.

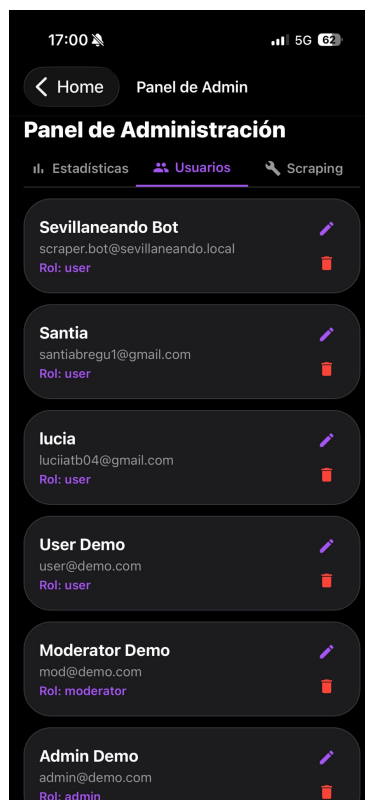
#### Credenciales de prueba:

- Email: admin@demo.com
- Contraseña: administrador

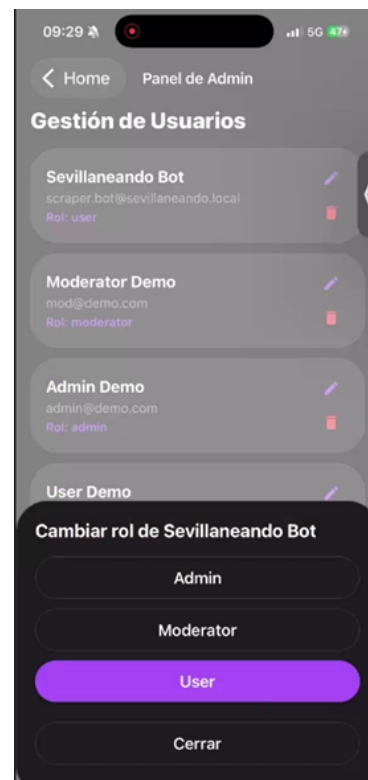




(a) Menú de administrador



(b) Gestión de usuarios

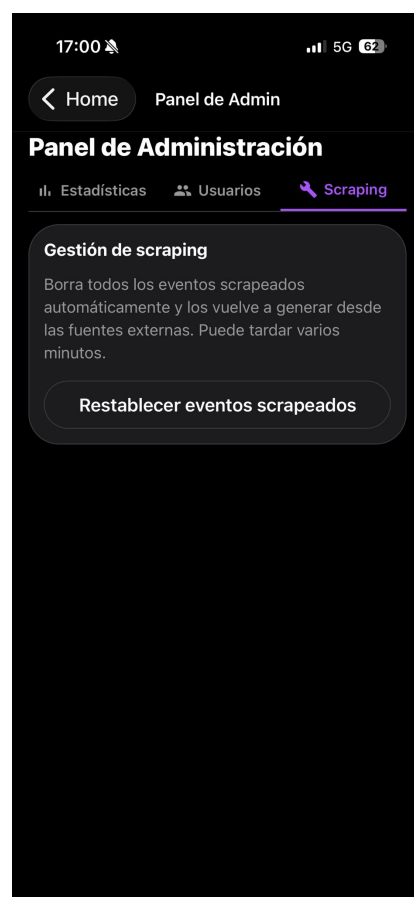


(c) Editar usuario

Figura D.11: Funcionalidades de administración



(a) Estadísticas

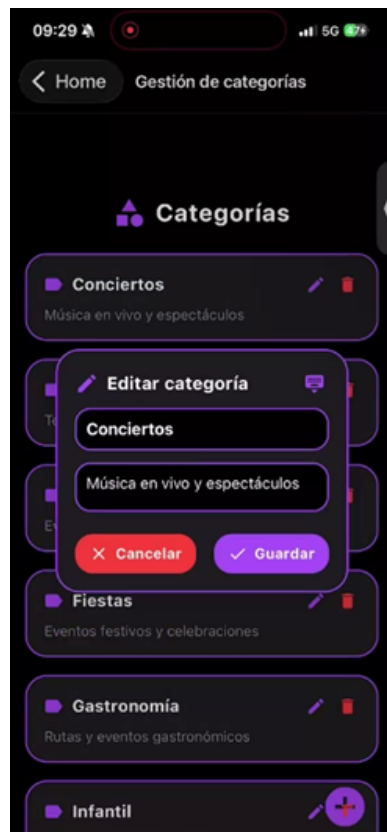


(b) Scraping

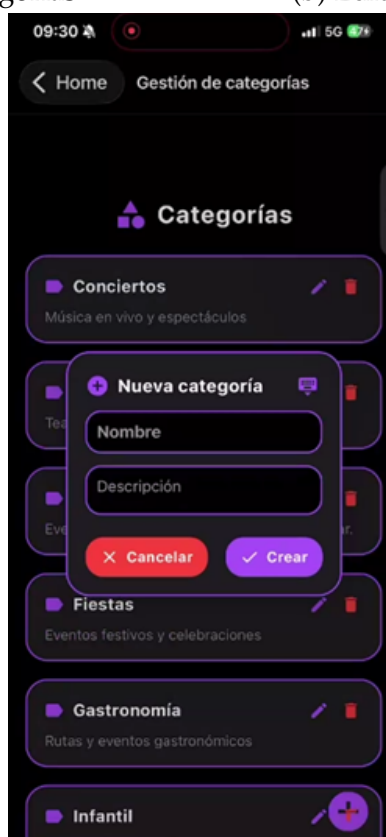
Figura D.12: Panel de estadísticas y scraping



(a) Categorías



(b) Editar categoría



(c) Crear categoría

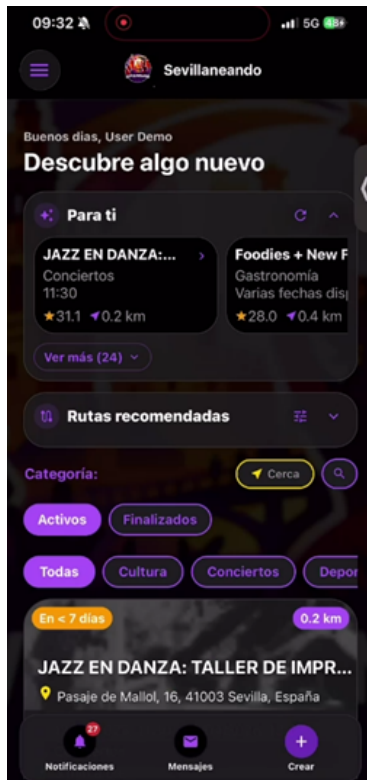
Figura D.13: Gestión de categorías

#### D.4.8. Recomendaciones y rutas recomendadas (para todos los roles)

La página principal incorpora dos secciones personalizadas situadas por encima del listado general: *Recomendaciones* y *Rutas recomendadas*. Estas propuestas se generan a partir de la distancia del usuario a cada evento, sus categorías de interés, sus interacciones previas (eventos vistos, guardados, apuntados, compartidos o valorados) y la valoración media global de cada evento. Por defecto ambas secciones aparecen replegadas con un encabezado; al pulsar sobre el encabezado se despliegan y se muestran las propuestas como un carrusel horizontal de tarjetas.

En la sección *Recomendaciones* se muestran inicialmente unos pocos eventos sugeridos; bajo el carrusel, el botón *Ver todos* amplía la lista al conjunto completo, mientras que *Ver menos* la vuelve a compactar. Al pulsar una tarjeta de evento recomendado se abre directamente la pantalla de detalle, donde el usuario puede interactuar con normalidad (apuntarse, guardar, valorar, etc.); cada interacción retroalimenta el sistema y afecta a las recomendaciones posteriores.

En la sección *Rutas recomendadas* se proponen agrupaciones de varios eventos cercanos entre sí, ordenados según un recorrido óptimo. Cada tarjeta indica el título, la fecha, la puntuación media de los eventos que la integran y, en su caso, un porcentaje de calidad y el tipo de ruta. Pulsando sobre una tarjeta se abre la previsualización de la ruta con el itinerario completo. Junto a la cabecera de la sección aparece un icono de ajustes que abre una ventana emergente con la configuración de generación de rutas: el usuario puede elegir una o varias estrategias (*Equilibrada*, *Caminable* o *Puntuación*), ajustar mediante botones + y – el número de rutas que se desean obtener y el número máximo de eventos por ruta. Tras aplicar los cambios, las rutas se recalculan automáticamente con los nuevos parámetros.



(a) Recomendaciones



(b) Rutas recomendadas



(c) Detalles de la ruta

Figura D.14: Recomendaciones y rutas

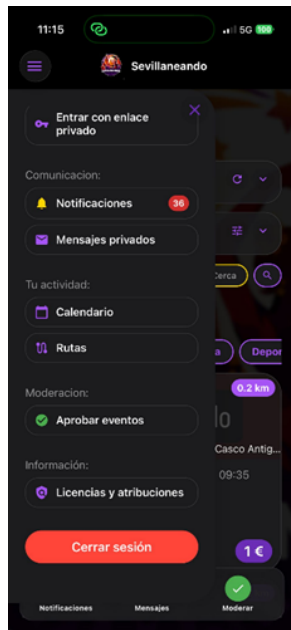
#### D.4.9. Rutas creadas por usuarios (para todos los roles)

Además de las rutas generadas automáticamente por el sistema, la aplicación permite consultar y crear rutas manuales propuestas por la propia comunidad. Para acceder a ellas, el usuario debe abrir el menú lateral y pulsar la opción *Rutas*, que muestra el listado completo de rutas existentes con su título, autor y puntuación media.

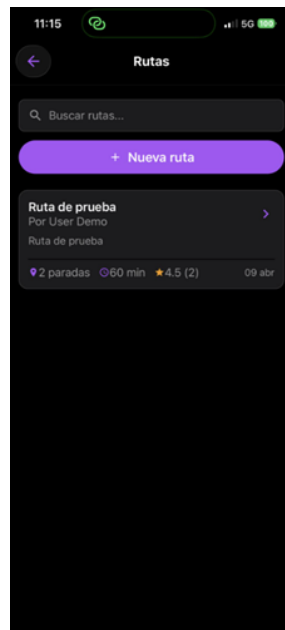
Al pulsar una ruta del listado se accede a su pantalla de detalle, donde se muestra el mapa con el recorrido completo trazado entre los puntos de interés, la lista ordenada de eventos que la integran y la sección de valoraciones. Cualquier usuario puede dejar su propia valoración, lo que contribuye a ordenar las rutas más recomendadas.

Para crear una nueva ruta, el usuario debe pulsar el botón de creación situado en la pantalla del listado. Esto abre un formulario en el que primero se introduce un título y una descripción que ayude a otros usuarios a entender la temática del recorrido. A continuación, debe indicar una duración estimada en minutos para el recorrido completo, valor que se utiliza para acotar la temporización entre paradas. La sección *Eventos disponibles* muestra el catálogo de eventos del sistema, y al pulsar sobre uno este se añade a la lista *Orden de visita*, donde se mantiene la secuencia de paradas en el orden de selección; al volver a pulsar sobre un evento ya seleccionado se retira de la ruta. A medida que se añaden eventos, el mapa superior actualiza los marcadores y traza la línea del recorrido entre ellos. Si dos eventos comparten exactamente las mismas coordenadas, sus marcadores se desplazan ligeramente para seguir siendo visibles.

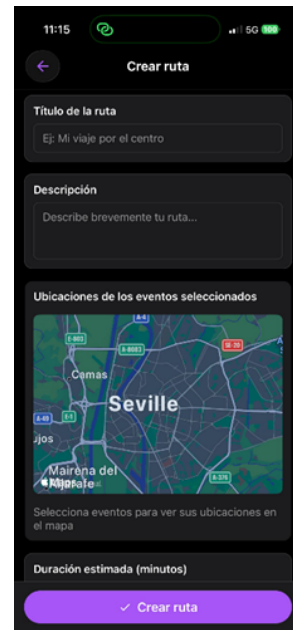
Antes de confirmar, el usuario debe asegurarse de que todos los eventos seleccionados disponen de ubicación válida; de lo contrario, la aplicación muestra un mensaje de error al intentar guardar. Pulsando el botón final del formulario se crea la ruta y se ofrece la opción de abrir directamente su detalle, donde ya queda visible para el resto de la comunidad.



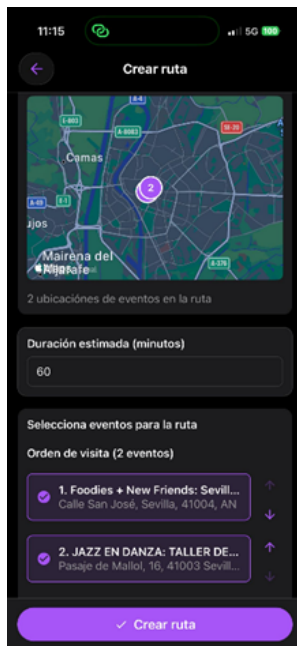
(a) Menú lateral



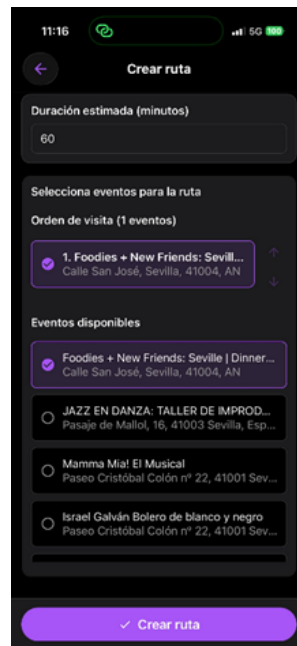
(b) Listado de rutas



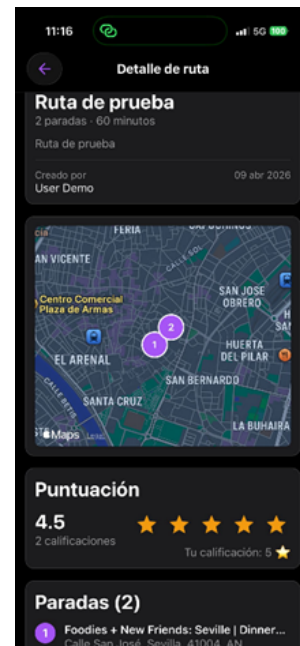
(c) Crear ruta 1



(d) Crear ruta 2



(e) Crear ruta 3



(f) Detalles de ruta

Figura D.15: Rutas creadas por usuarios

#### D.4.10. Creación de eventos (rol de usuario)

Los usuarios pueden iniciar la creación de un evento desde el botón *Crear evento* de la barra inferior de la página principal o desde el menú lateral. Al pulsarlo se abre un formulario extenso que debe recorrerse de arriba abajo. En primer lugar se introducen el título y la descripción del evento mediante dos campos de texto; la descripción admite varios párrafos. A continuación se definen la fecha de inicio y la fecha de fin pulsando los campos correspondientes, lo que abre un calendario y un selector de hora para fijar el momento exacto.

El siguiente bloque corresponde a la ubicación. El usuario puede escribir una dirección o el nombre de un lugar en el campo de búsqueda y pulsar la lupa para localizarlo automáticamente, o bien tocar directamente sobre el mapa para situar el marcador donde desee. La dirección elegida se muestra debajo del mapa. Más abajo se selecciona la categoría desde un desplegable con todas las categorías disponibles en el sistema.

El precio puede expresarse de tres formas distintas. Si se deja el campo principal vacío y los campos mínimo y máximo también, el evento se considera gratuito. Si se rellena solo el campo principal, se publica con ese precio fijo. Si se rellenan los campos *Precio mínimo* y *Precio máximo*, el evento mostrará un rango. La aplicación valida que el mínimo sea inferior al máximo y que no se combinen un precio fijo con un rango.

A continuación se encuentra la sección de recurrencia, marcada como opcional. Pulsando el botón de recurrencia se elige entre las opciones *Diaria*, *Semanal*, *Quincenal* o *Mensual*, y aparece un campo adicional para indicar la fecha límite hasta la que se repite el evento. Si no se desea recurrencia, basta con dejar este bloque sin tocar.

Más abajo se gestionan las imágenes del evento. El usuario puede subir varias fotos desde la galería del dispositivo, hasta un límite indicado por la aplicación. La primera imagen subida se marca por defecto como portada, pero puede cambiarse pulsando la opción *Elegir como portada* sobre cualquiera de las imágenes. También es posible eliminar imágenes individualmente.

Finalmente, en la parte inferior se sitúa el interruptor *Evento privado*. Si se deja desactivado, al pulsar el botón de creación el evento se envía a moderación y permanece oculto hasta su aprobación. Si se activa, el evento se publica directamente como privado sin pasar por moderación y, al terminar la creación, la aplicación abre una ventana emergente con el enlace de acceso generado para que el creador pueda copiarlo y compartirlo con sus invitados.

##### **Aspectos a tener en cuenta:**

- Es recomendable completar todos los campos relevantes del formulario.
- En eventos privados se genera un enlace de acceso compartible al que se tendrá que acceder para que sea visible.
- En eventos públicos se requiere moderación antes de ser visibles para el resto.



05:49 ★ 93

[Home](#) [Crear evento](#)

## Crear Evento

Los campos marcados como obligatorios deben completarse para enviar el evento.

- **Título** *Obligatorio*
- **Descripción** *Obligatorio*
- Fecha de inicio** *Opcional*  
 Si no la indicas, el evento se publicará sin fecha y podrás añadirla más tarde.
- Fecha de fin** *Opcional*
- Privado** *Opcional*  
☒
- **Dirección o lugar** *Obligatorio*

(a) Crear evento

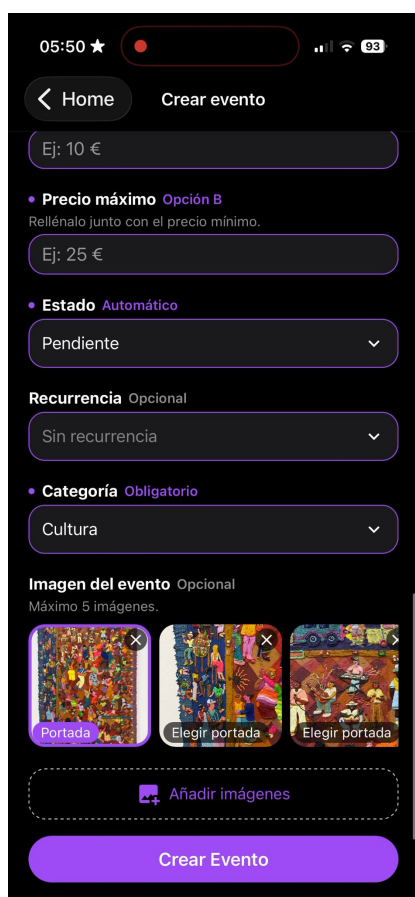
05:50 ★ 93

[Home](#) [Crear evento](#)

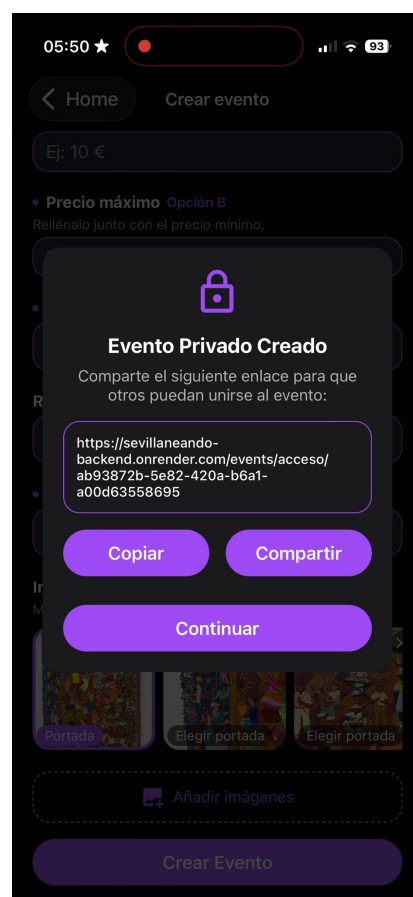
- **Ubicación en el mapa** *Obligatorio*  
  
 Map data © OpenStreetMap contributors, ODbL 1.0  
 Lat: 37.391453, Lng: -5.990947
- **Precio fijo o rango** *Obligatorio*  
 Elige una sola forma de precio.
- **Precio fijo** *Opción A*
- **Precio mínimo** *Opción B*  
 Rellénalo junto con el precio máximo.
- **Precio máximo** *Opción B*  
 Rellénalo junto con el precio mínimo.
- **Estado** *Automático*

(b) Crear evento 2

Figura D.16: Creación de eventos



(a) Crear evento 3



(b) Enlace privado

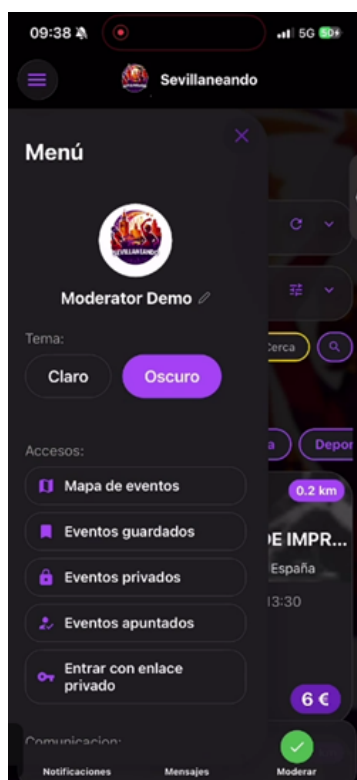
Figura D.17: Creación de eventos

### D.4.11. Eventos privados (todos los roles)

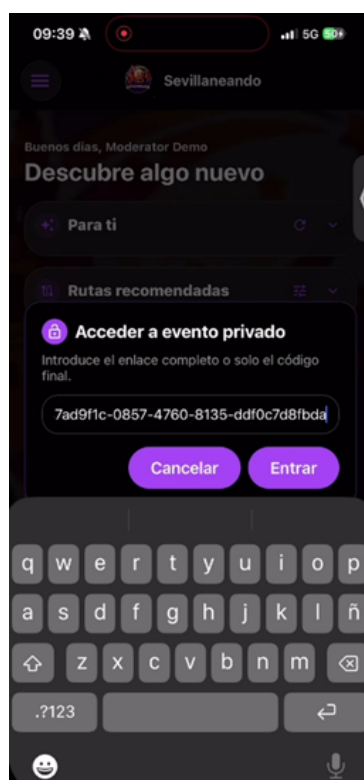
Los eventos privados están pensados para contextos en los que el acceso no debe ser abierto a toda la comunidad. Para participar en uno, el usuario necesita el enlace de acceso que le haya facilitado el creador. Este enlace puede recibirse a través de cualquier canal externo, como mensajería instantánea, redes sociales o correo electrónico.

Si el dispositivo tiene instalada la aplicación y se ha recibido el enlace en formato `sevillaneando://acceso/...`, basta con pulsarlo para que la aplicación se abra directamente en la pantalla del evento privado. Si el enlace es una dirección web, el usuario debe abrir manualmente la aplicación, entrar en el menú lateral y pulsar *Entrar con enlace privado*; en la ventana emergente que aparece debe pegar o escribir el código de acceso recibido y pulsar *Acceder*.

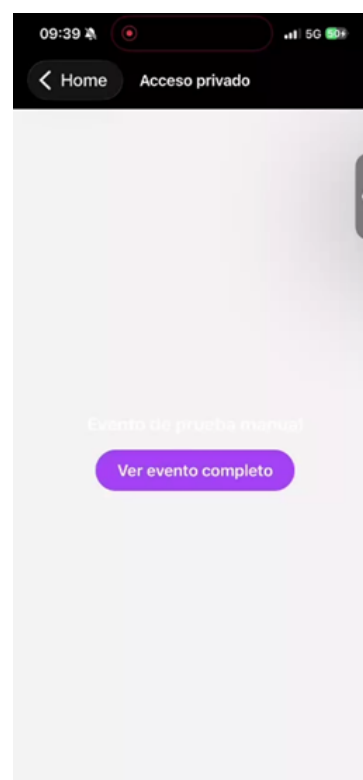
La aplicación carga entonces una pantalla intermedia que muestra el título del evento y dos botones, *Volver* y *Ver completo*. Al pulsar *Ver completo* se abre la pantalla de detalle del evento con las mismas funciones que un evento público: inscripción, asistentes, valoraciones y chat. La aplicación recuerda el enlace al que se ha accedido para que el evento siga apareciendo en el listado de eventos privados del usuario aunque cierre y vuelva a abrir la aplicación. No obstante, este tipo de eventos mantiene restricciones específicas en cuanto a su difusión: el botón *Compartir* solo está disponible para el creador, de modo que cualquier otro participante no puede invitar a terceros.



(a) Menú lateral

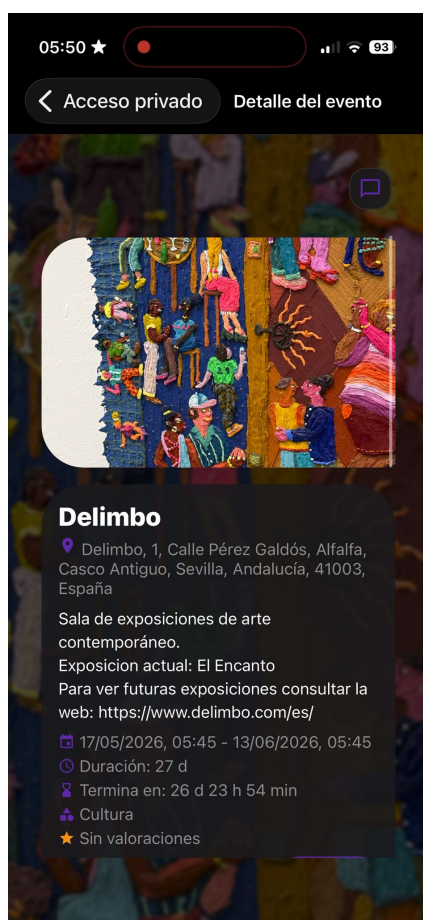


(b) Acceso privado

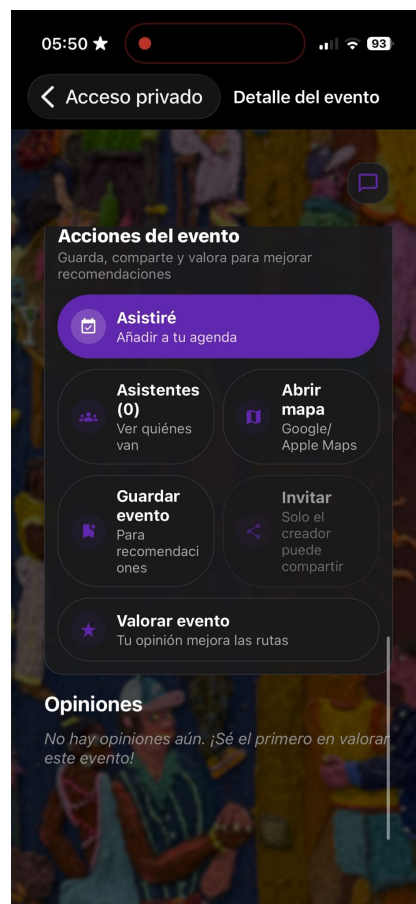


(c) Ver evento privado

Figura D.18: Acceso a evento privado



(a) Evento privado



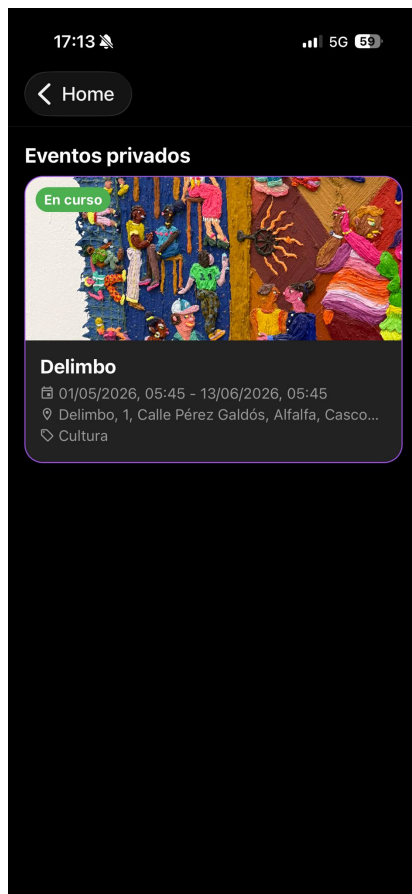
(b) Evento privado 2

Figura D.19: Consulta de evento privado

#### D.4.12. Gestión de eventos privados (todos los roles)

Todos los eventos privados a los que el usuario tiene acceso, ya sea como creador o como invitado, se agrupan en una misma pantalla accesible desde la opción *Eventos privados* del menú lateral. En esta pantalla se combinan los eventos privados creados por el propio usuario con aquellos a los que se ha accedido mediante un enlace previamente, cuyas referencias se conservan en el dispositivo para que sigan apareciendo en visitas posteriores. Cada evento se presenta como una tarjeta con su imagen, título y fecha, y al pulsarla se abre el detalle correspondiente.

En el caso del creador, el detalle del evento privado incluye el botón *Invitar* en el panel de acciones; al pulsarlo se vuelve a generar el enlace de acceso y se abre el menú nativo de compartir del dispositivo, lo que permite enviar la invitación por la vía deseada. Para los demás participantes, este botón aparece deshabilitado y la aplicación recuerda que solo el creador puede invitar a nuevos usuarios.



(a) Eventos privados



(b) Evento privado

Figura D.20: Gestión de eventos privados

#### D.4.13. Mis eventos (rol de usuario)

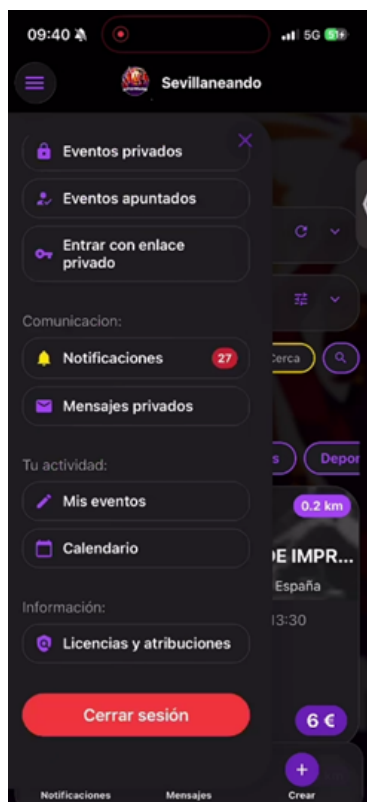
La sección *Mis eventos*, accesible desde el menú lateral, reúne todos los eventos que el usuario ha creado. La pantalla muestra los eventos privados, los públicos ya aprobados y los públicos rechazados; los eventos públicos pendientes de moderación no aparecen en este listado, pero la propia pantalla lo recuerda mediante un aviso superior que indica cuántos eventos están actualmente esperando aprobación.

Cada evento se presenta como una tarjeta con la imagen, el título, la fecha y un indicador del estado. Al pulsar una tarjeta se accede a la pantalla de edición del evento, donde aparecen los mismos campos que en la pantalla de creación con los valores actuales precargados. El usuario puede modificar cualquier dato (título, descripción, fechas, ubicación, categoría, precio, recurrencia o imágenes) y guardar los cambios pulsando el botón principal. La propia pantalla de edición incluye, en su parte inferior, un botón rojo para eliminar el evento de forma definitiva, que requiere confirmación.

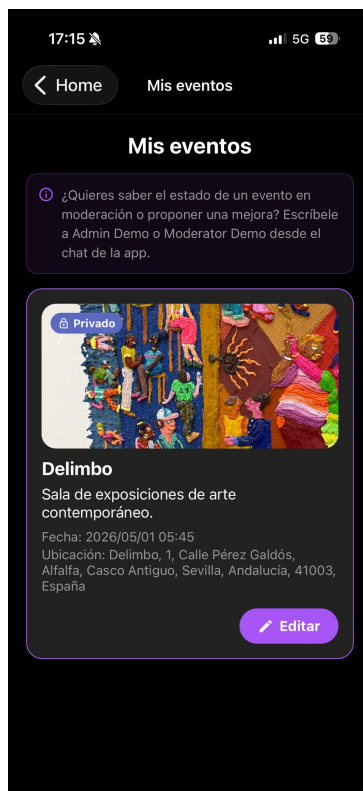
El comportamiento tras la edición depende del tipo de evento y del cambio realizado. Si se edita un evento privado, este sigue siendo visible para los participantes que ya tenían el enlace y aparece actualizado tanto para ellos como en *Mis eventos*. Si un evento público se modifica, vuelve a moderación y deja temporalmente de aparecer en el listado general hasta su nueva aprobación. Si se cambia un evento de público a privado, deja de ser visible para todos los usuarios excepto en *Mis eventos*, y el acceso queda restringido al enlace privado. Si se cambia de privado a público, el evento se envía a moderación, deja de ser visible mediante el enlace anterior y solo volverá a publicarse de forma general si se aprueba.

##### **Aspectos a tener en cuenta:**

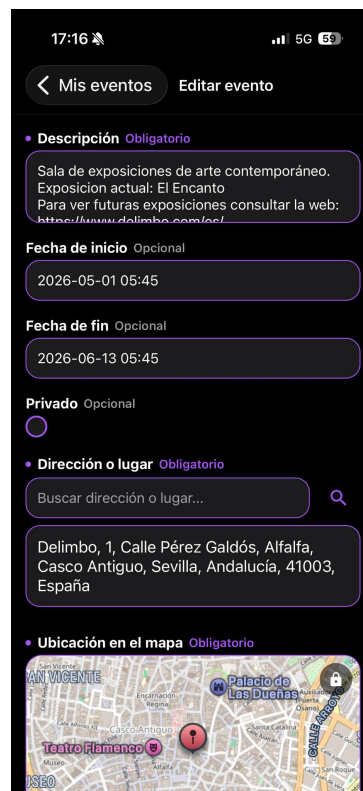
- Cuando se edita un evento privado se mantiene visible para participantes que accedieron mediante el enlace privado y en *Mis eventos*.
- Cuando un evento pasa de público a privado deja de ser visible para todos pero se mantiene visible en *Mis eventos*. Para que sea visible habría que acceder mediante el enlace privado.
- Cuando un evento pasa de privado a público se va a moderación, deja de ser visible para las personas que habían accedido mediante el enlace y pasaría a ser visible para todos si se aprueba.
- Cuando se edita un evento público vuelve a moderación antes de mostrarse de nuevo en el listado general.
- Desde la pantalla de edición puede eliminarse el evento definitivamente mediante el botón correspondiente.



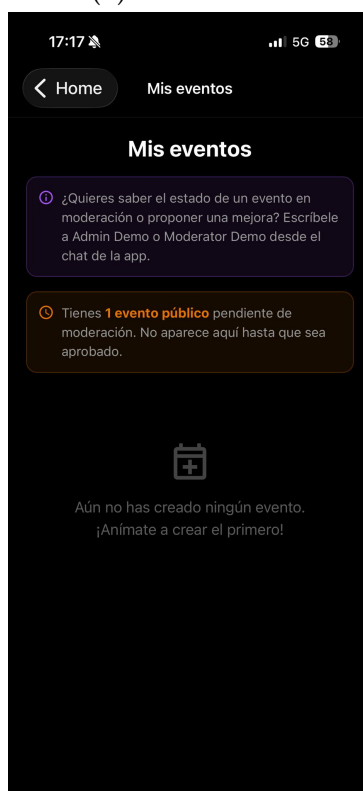
(a) Menú lateral



(b) Mis eventos



(c) Editar evento



(d) Evento en moderación

Figura D.21: Eventos creados por el usuario

#### D.4.14. Moderación (rol de moderador)

Los usuarios con rol de moderador disponen, en la barra inferior de la página principal y en el menú lateral, de un acceso adicional a la pantalla *Moderación*. Esta pantalla se organiza en dos pestañas que pueden alternarse pulsando su título: *Pendientes* y *Públicos aprobados*.

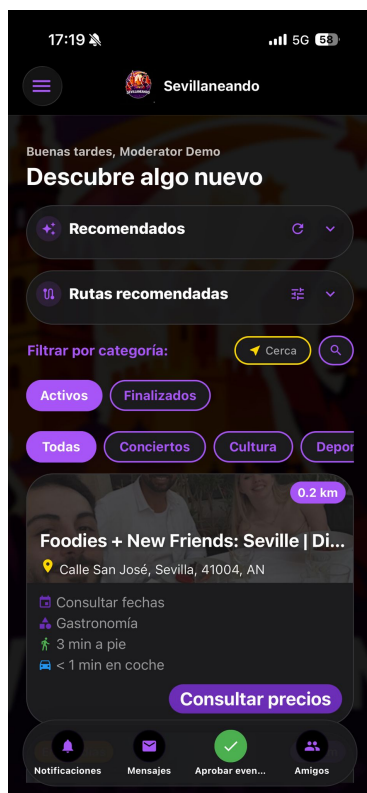
En la pestaña *Pendientes* se muestran los eventos públicos que aún no han sido validados, ordenados por fecha de creación. Cada tarjeta incluye la imagen, el título, las fechas, la categoría y el creador. Al pulsar la tarjeta se abre una pantalla de revisión del evento donde el moderador puede consultar todos los datos y, si es necesario, editarlos antes de tomar una decisión. En esa misma pantalla aparecen los botones *Aprobar* y *Rechazar*: el primero publica el evento de manera inmediata y lo hace visible para toda la comunidad; el segundo lo marca como rechazado y notifica al creador. Tras cualquiera de las dos acciones la lista de pendientes se actualiza automáticamente.

La pestaña *Públicos aprobados* muestra todos los eventos públicos que ya están publicados. El moderador puede pulsar sobre cualquiera de ellos para abrir la pantalla de edición y modificar los datos del evento, e incluso eliminarlo en caso de que sea necesario. Esta acción no afecta a los eventos privados de la plataforma, que quedan fuera del alcance de moderación.

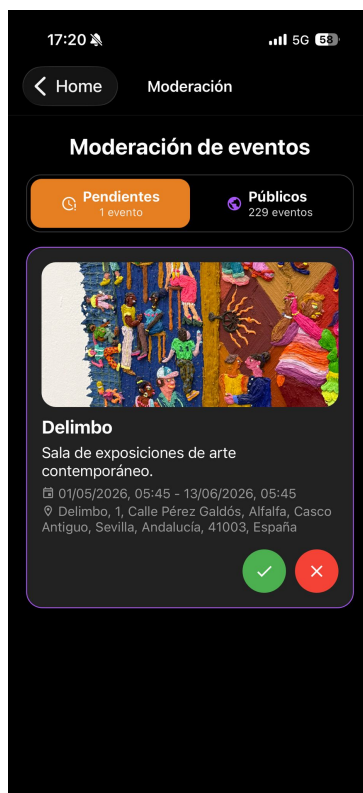
##### **Credenciales de prueba:**

- Email: mod@demo.com
- Contraseña: moderator

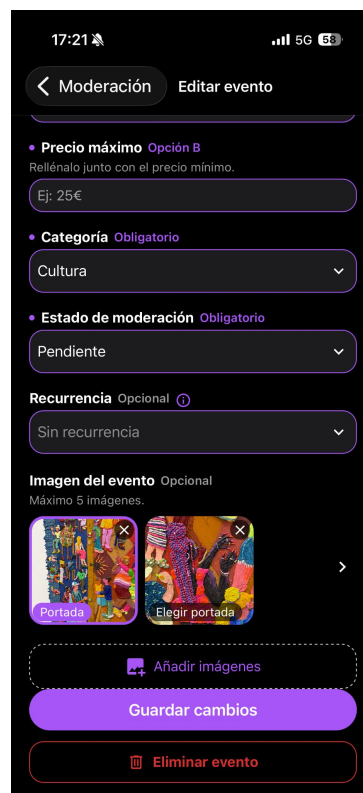




(a) Página principal



(b) Moderación



(c) Editar evento

Figura D.22: Moderación de eventos

#### **D.4.15. Notificaciones (todos los roles)**

El acceso a las notificaciones se realiza desde el icono de campana situado en la barra inferior de la página principal, que muestra un indicador numérico con el total de notificaciones aún no leídas. Al pulsarlo se abre la pantalla *Notificaciones*, donde aparecen listadas en orden cronológico inverso. Las notificaciones pueden referirse a la aprobación o el rechazo de un evento creado por el usuario, recordatorios previos a la celebración de eventos a los que está apuntado, avisos sobre eventos cercanos según su ubicación o nuevos seguidores.

Cada notificación incluye un icono representativo del tipo, un mensaje descriptivo y la fecha. Las notificaciones aún no leídas se distinguen visualmente del resto. Al pulsar una notificación esta queda marcada automáticamente como leída y, en función de su contenido, la aplicación abre el destino asociado: las notificaciones de seguidores llevan al perfil público de la persona que ha empezado a seguir al usuario, y las notificaciones relacionadas con un evento abren su pantalla de detalle. En la cabecera de la pantalla se incluye también un botón para marcar todas las notificaciones como leídas de una sola vez, lo que actualiza el indicador de la página principal sin necesidad de abrir cada una. Las notificaciones pueden eliminarse individualmente mediante el botón correspondiente que aparece en cada elemento.

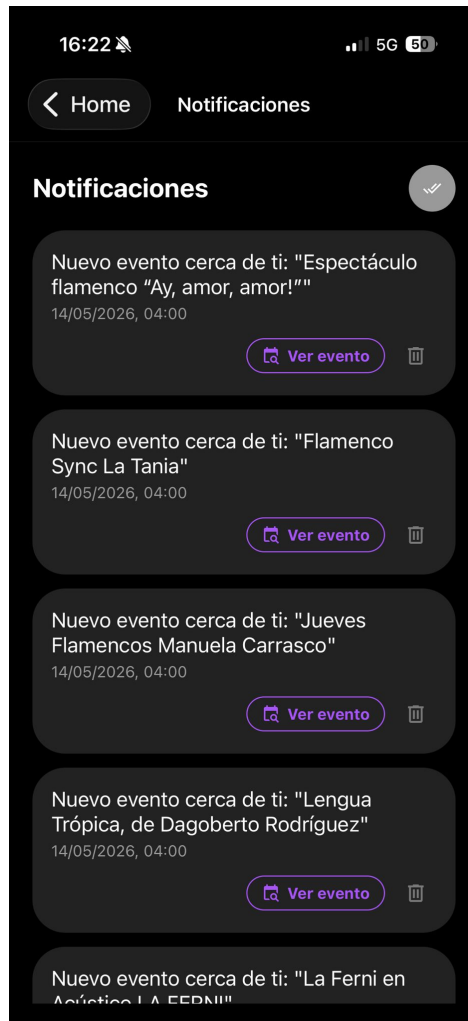


Figura D.23: Notificaciones

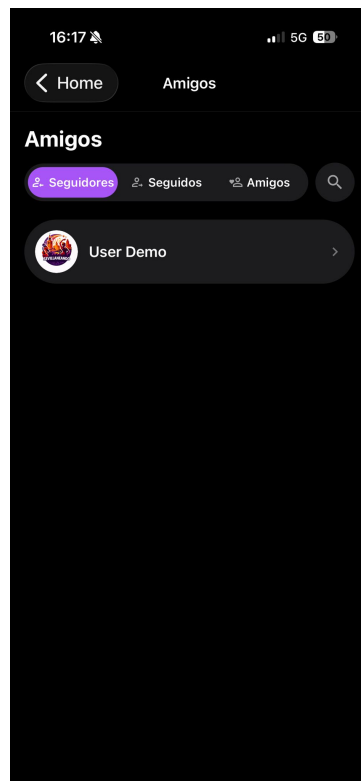
#### D.4.16. Amistad y mensajería privada (todos los roles)

La aplicación incorpora un módulo social que combina relaciones de amistad con mensajería privada. Para acceder se pulsa el icono de amigos en la barra inferior de la página principal, lo que abre una pantalla con cuatro pestañas en la cabecera. La pestaña *Seguidores* muestra todas las personas que siguen al usuario; la pestaña *Seguidos* muestra las personas a las que el usuario sigue; la pestaña *Amigos* reúne las relaciones mutuas, es decir, los usuarios que se siguen entre sí; y la pestaña *Buscar* incluye un campo de texto para localizar nuevos perfiles por nombre.

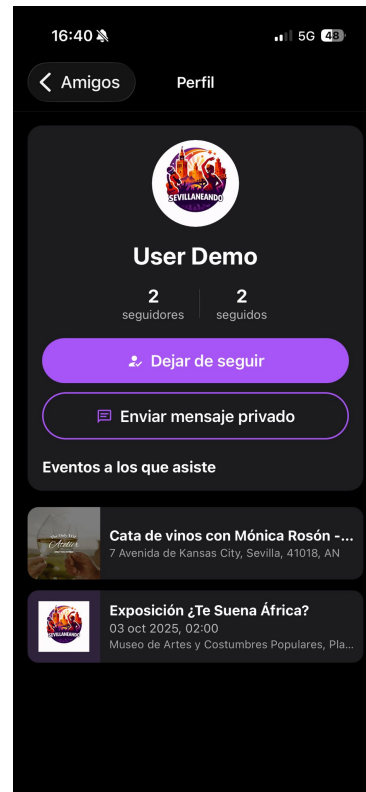
En cualquiera de los listados, cada elemento muestra el avatar y el nombre del usuario. Al pulsar sobre un usuario se abre su perfil público, donde se muestran sus datos básicos, el número de seguidores y seguidos, los eventos a los que asiste y los botones de acción. El botón *Seguir* (o *Dejar de seguir* si ya existe la relación) cambia su estado al pulsarlo. El botón de mensaje inicia una conversación privada en la pantalla *Chat directo*. Los contadores de seguidores y seguidos del perfil son pulsables: al tocarlos se abre una pantalla independiente con la lista completa correspondiente a ese usuario, lo que permite descubrir nuevos perfiles a través de las conexiones de los demás.

La mensajería privada puede iniciarse también desde el chat de un evento, manteniendo pulsado un mensaje ajeno o pulsando el icono de sobre que aparece junto al mensaje, así como desde la lista de asistentes de un evento. En la pantalla de chat directo se pueden enviar mensajes de texto e imágenes mediante los iconos de galería y cámara situados junto al campo de escritura. Al pulsar sobre una imagen recibida se abre a pantalla completa con posibilidad de ampliarla con dos dedos. Manteniendo pulsado un mensaje propio aparece la opción de eliminarlo.

Todas las conversaciones privadas se conservan y son accesibles posteriormente desde el icono de mensajes de la barra inferior de la página principal. La pantalla *Mensajes* lista las conversaciones existentes con la última actividad de cada una, y al pulsar sobre una se reanuda el chat con todo el historial.

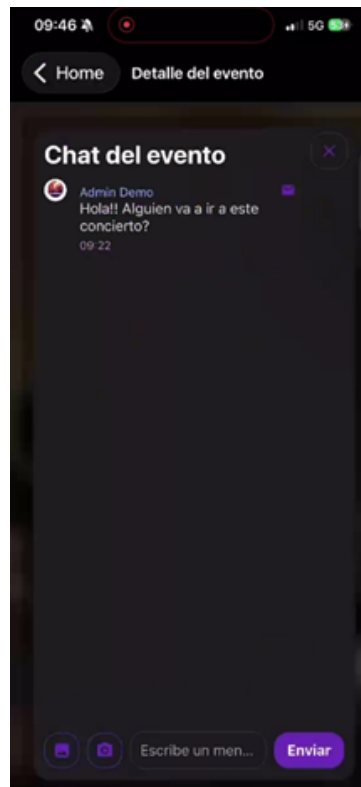


(a) Módulo de amistad

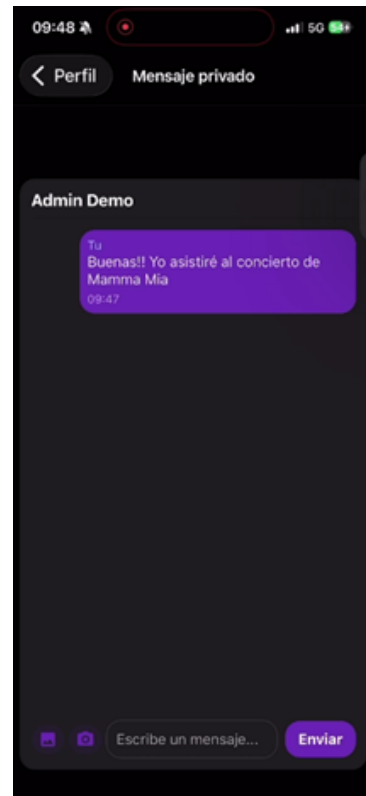


(b) Ver perfil

Figura D.24: Amistad y perfil de usuario

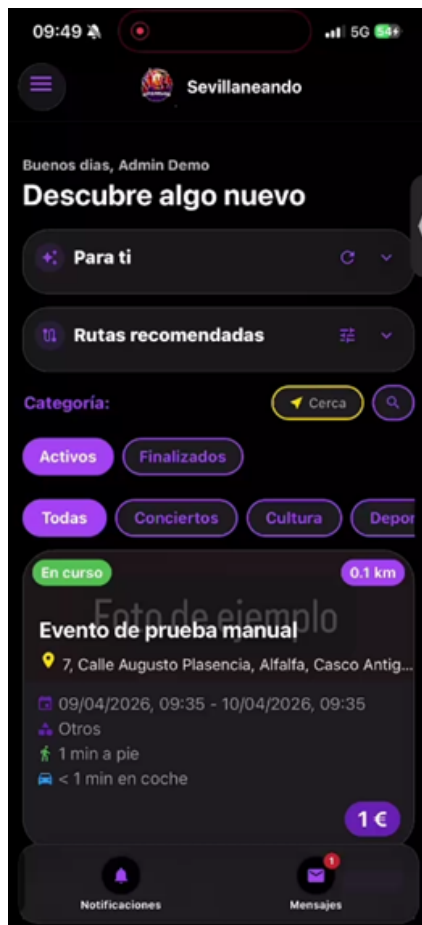


(a) Chat del evento

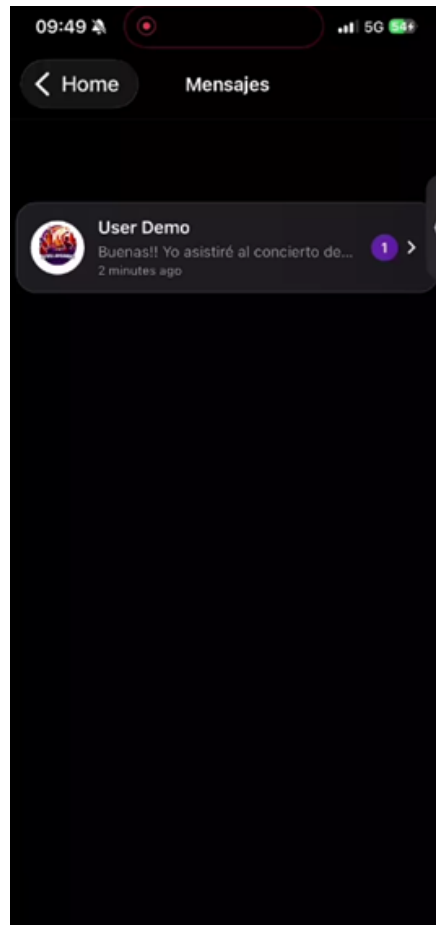


(b) Chat privado

Figura D.25: Mensajes privados



(a) Página principal



(b) Mensajes

Figura D.26: Consulta de mensajes privados

### D.4.17. Eventos finalizados

Los eventos cuyo intervalo de fechas ya ha pasado se consideran finalizados y la aplicación los trata de forma diferenciada. En la página principal aparecen únicamente al activar la pestaña *Eventos finalizados*, situada junto a *Eventos activos*, y siguen siendo accesibles desde los listados personales del usuario (guardados, apuntados o creados) en los que ya estuvieran presentes.

Al abrir el detalle de un evento finalizado, la información se muestra con normalidad pero el botón principal del panel de acciones aparece deshabilitado con la etiqueta *Finalizado*, impidiendo nuevas inscripciones. El usuario que ya estaba apuntado mantiene su asistencia registrada en el historial. El resto de acciones siguen activas en la medida en que tienen sentido: pueden consultarse los asistentes, abrir la ubicación en mapas externos, leer las opiniones y dejar una valoración del evento. Esta distinción permite preservar la utilidad informativa del sistema sin confundir al usuario sobre la disponibilidad real de cada actividad.

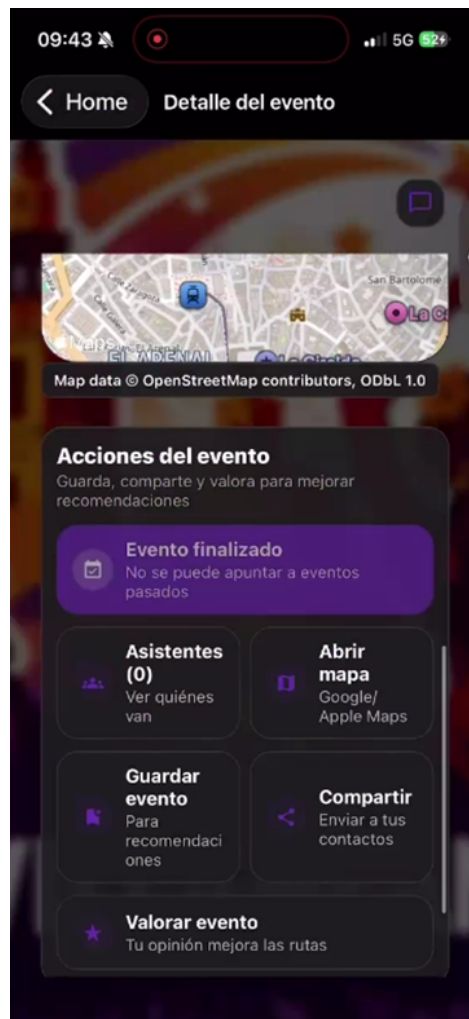


Figura D.27: Evento finalizado



## D.4.18. Guía de uso integrada

La aplicación incorpora una guía de uso accesible directamente desde el menú lateral, en el apartado *Información* → *Guía de uso*. Esta guía está pensada para resolver dudas frecuentes sin necesidad de consultar documentación externa.

La guía se presenta en formato de acordeón: cada sección puede desplegarse de forma independiente para mostrar su contenido. Las secciones disponibles son las siguientes:

- **Crear un evento.** Explica el proceso de creación de eventos, incluyendo los campos obligatorios y opcionales.
- **Precio de un evento.** Aclara las opciones de precio disponibles: gratuito, precio fijo o rango de precios.
- **Recurrencia.** Describe cómo crear eventos que se repiten periódicamente.
- **Eventos privados y enlace de acceso.** Explica qué es un evento privado, cómo se comparte mediante enlace y que no pasa por moderación.
- **Moderación de eventos.** Informa sobre el proceso de revisión de los eventos públicos antes de su publicación.
- **Guardar eventos.** Detalla cómo guardar eventos de interés para consultarlos más adelante.
- **Apuntarse a un evento.** Describe el proceso de inscripción a un evento y dónde encontrar los eventos a los que el usuario está apuntado.
- **Calendario de eventos.** Explica la vista de calendario y cómo navegar entre fechas.
- **Mapa de eventos.** Indica cómo funciona el mapa y la importancia de tener la ubicación del perfil configurada.
- **Categorías.** Explica para qué sirven las categorías y cómo pueden reorganizarse.
- **Rutas recomendadas.** Describe la funcionalidad de rutas que agrupa eventos cercanos entre sí.
- **Perfil e intereses.** Explica cómo configurar el perfil y los intereses, que influyen en las recomendaciones.
- **Amigos y seguidores.** Detalla las relaciones sociales disponibles en la aplicación.
- **Valorar un evento.** Informa sobre cómo dejar una reseña o valoración una vez finalizado el evento.
- **Chat del evento.** Explica el chat grupal asociado a cada evento y las condiciones de acceso.



Figura D.28: Guía de uso integrada en la aplicación

## D.5. Grafo de navegabilidad

Con el fin de complementar la guía de uso, se recomienda incluir un grafo de navegabilidad que muestre las principales pantallas de la aplicación y las rutas de transición más habituales entre ellas. Este diagrama ayuda a comprender la estructura general del sistema y la manera en que el usuario se desplaza por sus distintos módulos.

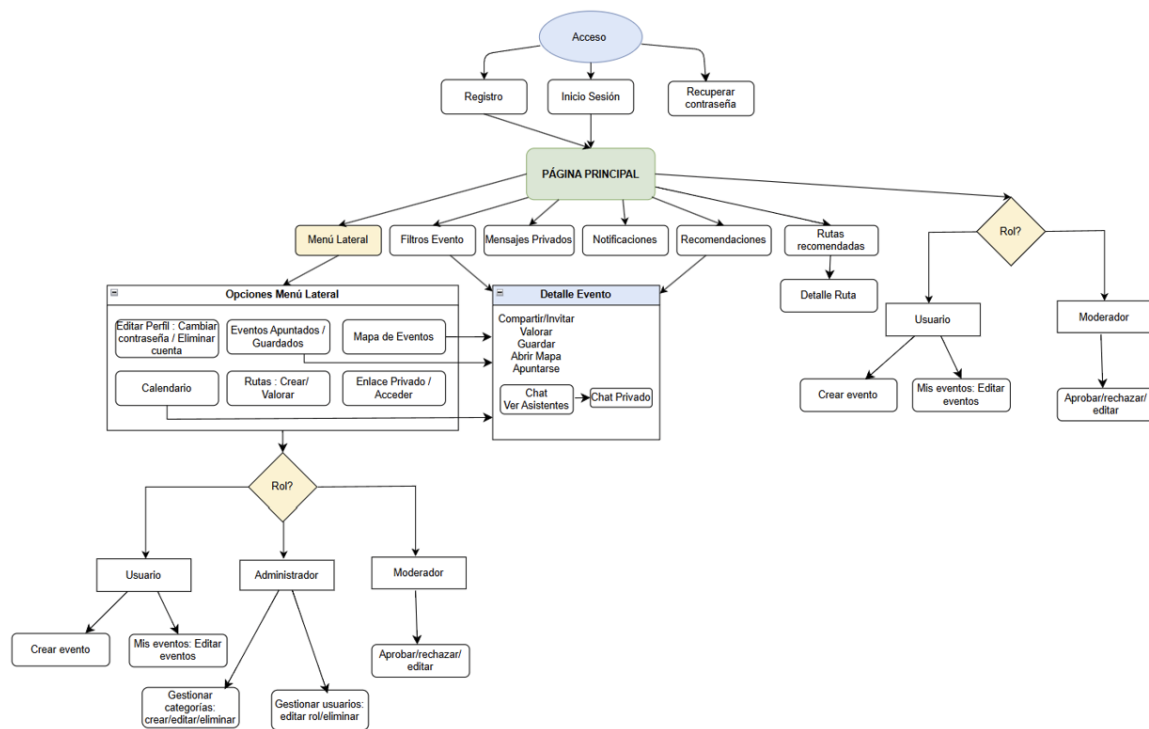


Figura D.29: Grafo de navegabilidad de la aplicación

## D.6. Errores frecuentes

A continuación se recogen algunos problemas habituales que pueden producirse durante el uso de la aplicación, junto con una posible explicación:

- **No aparecen eventos en el mapa.** Puede deberse a que el usuario no ha definido una ubicación en el perfil.
- **No llegan notificaciones de eventos cercanos.** Puede ser necesario revisar los permisos de ubicación.
- **No es posible apuntarse a un evento.** Es posible que el evento se encuentre finalizado o no admita nuevas inscripciones.
- **No se puede compartir un evento privado libremente.** En este tipo de eventos la capacidad de invitación queda restringida al creador.

- **Las recomendaciones no resultan relevantes.** El sistema mejora sus propuestas a medida que el usuario interactúa con la aplicación.
- **No se visualizan correctamente algunas funciones basadas en proximidad.** Debe comprobarse que la ubicación del perfil está configurada correctamente.

## E. Reporte de horas

---

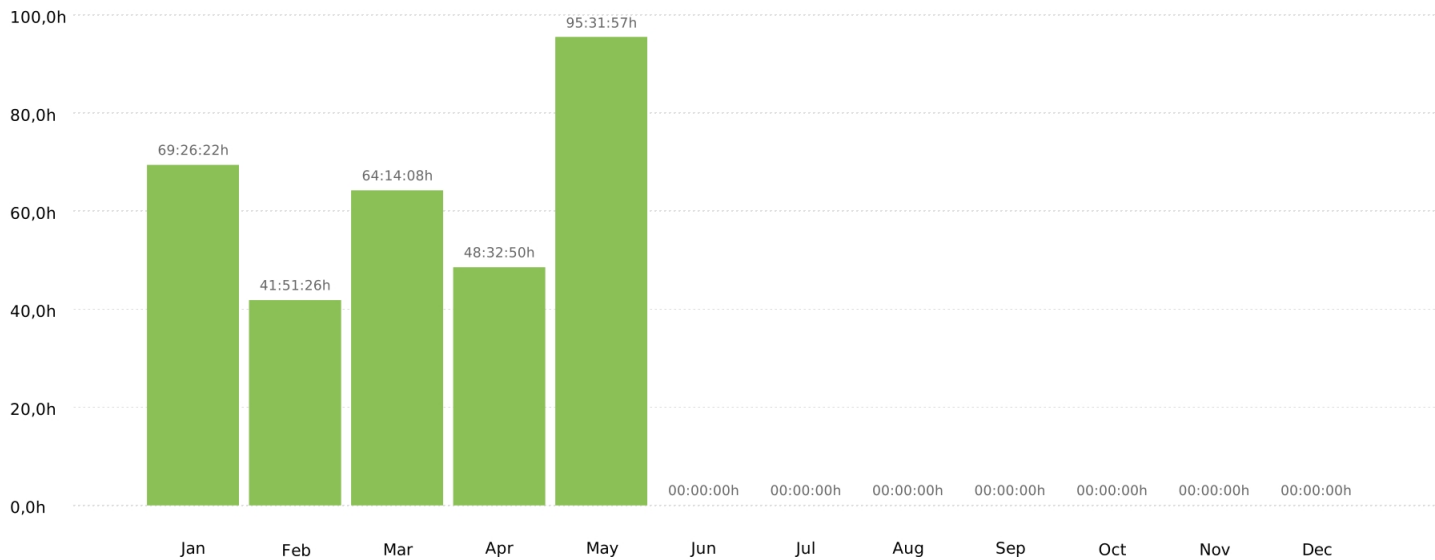
Este anexo recoge el reporte de horas exportado desde Clockify, utilizado como evidencia del seguimiento temporal realizado durante el desarrollo del proyecto. El informe se ha empleado como referencia para contrastar la estimación presupuestaria por fases presentada en el Capítulo [2](#), sin trasladar sus horas exactas de forma literal al presupuesto.

# Summary report

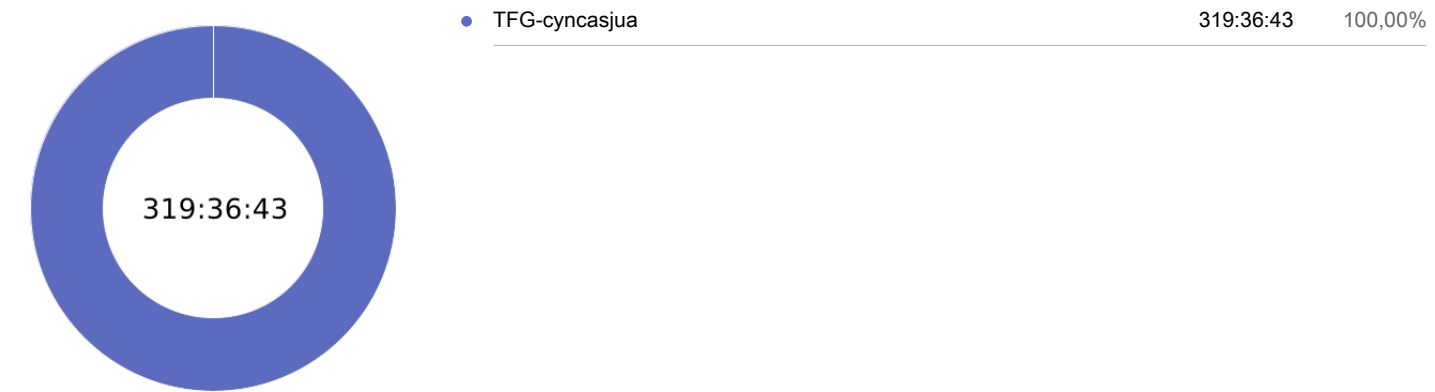
01/01/2026 - 31/12/2026



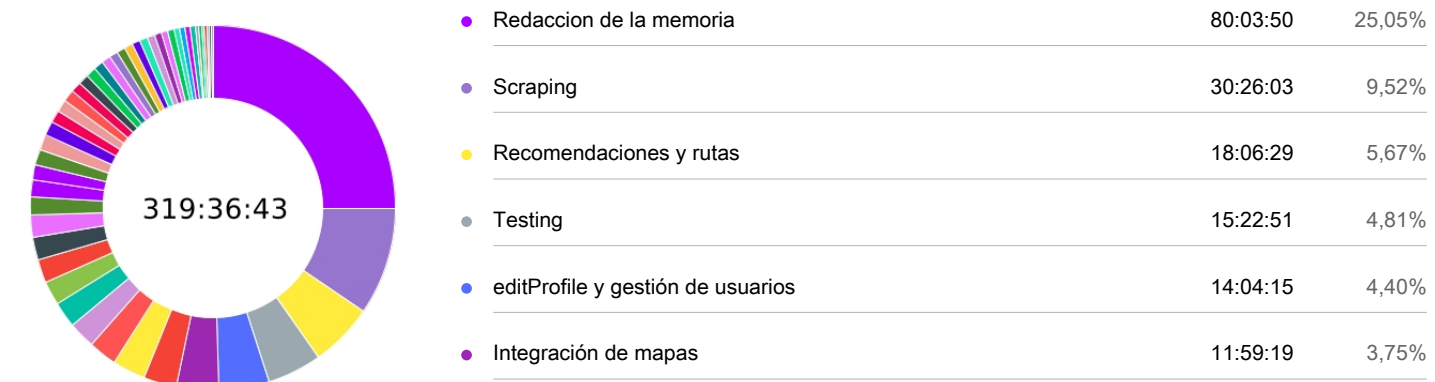
Total: 319:36:43    Billable: 319:36:43    Amount: 0,00 USD



## Project



## Description



|                                                                          |          |       |
|--------------------------------------------------------------------------|----------|-------|
| ● Mejorar interfaz                                                       | 09:29:23 | 2,97% |
| ● Chat de eventos                                                        | 09:10:33 | 2,87% |
| ● Descargar, probar todas las tecnologías y mockups                      | 08:00:54 | 2,51% |
| ● Crear evento, aprobar/rechazar evento, notificacion                    | 07:40:11 | 2,40% |
| ● Despliegue de la app                                                   | 07:14:12 | 2,26% |
| ● login                                                                  | 06:47:03 | 2,12% |
| ● Guía de estilo: eslint y prettier                                      | 06:37:01 | 2,07% |
| ● Manuales de usuario, despliegue y desarrollo                           | 06:21:11 | 1,99% |
| ● Edicion de eventos propios                                             | 06:16:22 | 1,96% |
| ● Revision final (pruebas) y arreglos                                    | 05:12:37 | 1,63% |
| ● Chat individual                                                        | 04:49:12 | 1,51% |
| ● Arreglo de errores al probar casos de uso (pruebas de aceptación)      | 04:19:15 | 1,35% |
| ● Listado de eventos y detalles                                          | 04:18:21 | 1,35% |
| ● Eventos privados                                                       | 04:12:00 | 1,31% |
| ● Editar evento moderador                                                | 04:02:32 | 1,26% |
| ● Actualizar documentacion                                               | 03:37:48 | 1,14% |
| ● Modulo de amistad y arreglos                                           | 03:23:24 | 1,06% |
| ● Subida de varias imágenes para un evento                               | 03:18:41 | 1,04% |
| ● Restricciones en backend                                               | 03:08:38 | 0,98% |
| ● Documentos finales (anexos finales y ajustes finales de documentacion) | 02:53:32 | 0,90% |
| ● Precio variable                                                        | 02:53:20 | 0,90% |
| ● Workflows y errores al desplegar                                       | 02:44:39 | 0,86% |
| ● Notificaciones de recordatorios y eventos cercanos                     | 02:34:46 | 0,81% |
| ● Arreglo de sockets                                                     | 02:24:12 | 0,75% |
| ● Arreglo de chats                                                       | 02:23:48 | 0,75% |
| ● Entidades                                                              | 02:23:20 | 0,75% |
| ● Arreglos de scrapeo de fechas                                          | 02:19:07 | 0,73% |

|                                                                |          |       |
|----------------------------------------------------------------|----------|-------|
| ● Icono de notificaciones y chats con numero de notificaciones | 02:18:10 | 0,72% |
| ● Video demo                                                   | 02:04:56 | 0,65% |
| ● integración de mapas                                         | 02:00:34 | 0,63% |
| ● Crear categorías                                             | 01:51:13 | 0,58% |
| ● Calendario de eventos                                        | 01:44:25 | 0,54% |
| ● Alistarse a eventos                                          | 01:41:52 | 0,53% |
| ● Buscador                                                     | 01:30:33 | 0,47% |
| ● Subida de fotos en cloudinary                                | 01:26:41 | 0,45% |
| ● Borrar y actualizar categorías                               | 01:21:54 | 0,43% |
| ● Corrección de docs de la reunion3                            | 00:56:16 | 0,29% |
| ● Acta de reunión 4                                            | 00:49:03 | 0,26% |
| ● menu y herramientas                                          | 00:47:49 | 0,25% |
| ● Acta de reunión 5                                            | 00:45:04 | 0,24% |
| ● Reunión 4                                                    | 00:37:34 | 0,20% |
| ● Reunion 5                                                    | 00:32:54 | 0,17% |
| ● modo oscuro                                                  | 00:28:56 | 0,15% |

| Project / Description                             | Duration  | Amount   |
|---------------------------------------------------|-----------|----------|
| TFG-cyncasjua                                     | 319:36:43 | 0,00 USD |
| Redaccion de la memoria                           | 80:03:50  | 0,00 USD |
| Scraping                                          | 30:26:03  | 0,00 USD |
| Recomendaciones y rutas                           | 18:06:29  | 0,00 USD |
| Testing                                           | 15:22:51  | 0,00 USD |
| editProfile y gestión de usuarios                 | 14:04:15  | 0,00 USD |
| Integración de mapas                              | 11:59:19  | 0,00 USD |
| Mejorar interfaz                                  | 09:29:23  | 0,00 USD |
| Chat de eventos                                   | 09:10:33  | 0,00 USD |
| Descargar, probar todas las tecnologías y mockups | 08:00:54  | 0,00 USD |



|                                                                        |          |          |
|------------------------------------------------------------------------|----------|----------|
| Crear evento, aprobar/rechazar evento, notificacion                    | 07:40:11 | 0,00 USD |
| Despliegue de la app                                                   | 07:14:12 | 0,00 USD |
| login                                                                  | 06:47:03 | 0,00 USD |
| Guía de estilo: eslint y prettier                                      | 06:37:01 | 0,00 USD |
| Manuales de usuario, despliegue y desarrollo                           | 06:21:11 | 0,00 USD |
| Edicion de eventos propios                                             | 06:16:22 | 0,00 USD |
| Revision final (pruebas) y arreglos                                    | 05:12:37 | 0,00 USD |
| Chat individual                                                        | 04:49:12 | 0,00 USD |
| Arreglo de errores al probar casos de uso (pruebas de aceptación)      | 04:19:15 | 0,00 USD |
| Listado de eventos y detalles                                          | 04:18:21 | 0,00 USD |
| Eventos privados                                                       | 04:12:00 | 0,00 USD |
| Editar evento moderador                                                | 04:02:32 | 0,00 USD |
| Actualizar documentacion                                               | 03:37:48 | 0,00 USD |
| Modulo de amistad y arreglos                                           | 03:23:24 | 0,00 USD |
| Subida de varias imágenes para un evento                               | 03:18:41 | 0,00 USD |
| Restricciones en backend                                               | 03:08:38 | 0,00 USD |
| Documentos finales (anexos finales y ajustes finales de documentacion) | 02:53:32 | 0,00 USD |
| Precio variable                                                        | 02:53:20 | 0,00 USD |
| Workflows y errores al desplegar                                       | 02:44:39 | 0,00 USD |
| Notificaciones de recordatorios y eventos cercanos                     | 02:34:46 | 0,00 USD |
| Arreglo de sockets                                                     | 02:24:12 | 0,00 USD |
| Arreglo de chats                                                       | 02:23:48 | 0,00 USD |
| Entidades                                                              | 02:23:20 | 0,00 USD |
| Arreglos de scrapeo de fechas                                          | 02:19:07 | 0,00 USD |
| Icono de notificaciones y chats con numero de notificaciones           | 02:18:10 | 0,00 USD |
| Video demo                                                             | 02:04:56 | 0,00 USD |

|                                   |          |          |
|-----------------------------------|----------|----------|
| integración de mapas              | 02:00:34 | 0,00 USD |
| Crear categorías                  | 01:51:13 | 0,00 USD |
| Calendario de eventos             | 01:44:25 | 0,00 USD |
| Alistarse a eventos               | 01:41:52 | 0,00 USD |
| Buscador                          | 01:30:33 | 0,00 USD |
| Subida de fotos en clouinary      | 01:26:41 | 0,00 USD |
| Borrar y actualizar categorías    | 01:21:54 | 0,00 USD |
| Corrección de docs de la reunion3 | 00:56:16 | 0,00 USD |
| Acta de reunión 4                 | 00:49:03 | 0,00 USD |
| menu y herramientas               | 00:47:49 | 0,00 USD |
| Acta de reunión 5                 | 00:45:04 | 0,00 USD |
| Reunión 4                         | 00:37:34 | 0,00 USD |
| Reunion 5                         | 00:32:54 | 0,00 USD |
| modo oscuro                       | 00:28:56 | 0,00 USD |

## **F. Informes de avance y cambios**

---

Este anexo resume la evolución del proyecto a lo largo de sus principales fases de trabajo. Los informes recogen el avance funcional, las decisiones tomadas y los cambios relevantes respecto a la planificación inicial.

### **F.1. Informe de avance 1: definición del alcance y análisis inicial**

#### **F.1.1. Trabajo realizado**

Durante la fase inicial se definió el problema que motiva el proyecto: la dispersión de la información sobre eventos y actividades en Sevilla. Se analizó el contexto de uso, se estudiaron soluciones existentes y se delimitó el alcance de *Sevillaneando* como aplicación móvil participativa orientada al descubrimiento de eventos.

También se identificaron los actores principales del sistema, los objetivos del proyecto y los objetivos funcionales de la aplicación.

#### **F.1.2. Cambios y decisiones**

Se decidió enfocar el producto en eventos locales y emergentes, diferenciándolo de directorios generales mediante funcionalidades sociales, eventos privados, recomendaciones y rutas de eventos.

### **F.2. Informe de avance 2: requisitos, planificación y diseño conceptual**

#### **F.2.1. Trabajo realizado**

En esta fase se elaboró el documento de requisitos, incluyendo requisitos de información, requisitos funcionales, requisitos no funcionales y reglas de negocio. Además, se preparó la planificación del proyecto mediante fases, sprints, estimación de esfuerzo, EDT y diagrama temporal.

Se diseñaron los primeros mockups y el modelo conceptual del sistema, sentando la base para el diseño técnico posterior.

## **F.2.2. Cambios y decisiones**

Se adoptó una metodología ágil adaptada a un proyecto unipersonal, con seguimiento mediante backlog, sprints y registro de horas. La estimación inicial se ajustó a una dedicación aproximada de 300 horas, conforme a la carga esperada del TFG; el seguimiento final registró un esfuerzo real cercano a 320 horas.

## **F.3. Informe de avance 3: arquitectura e implementación base**

### **F.3.1. Trabajo realizado**

Se definió la arquitectura cliente-servidor del sistema, separando la aplicación móvil, el backend y la base de datos. Se implementó la estructura inicial del backend con NestJS, la aplicación móvil con React Native y Expo, y la persistencia con PostgreSQL y PostGIS.

Durante esta fase se desarrollaron las funcionalidades básicas de autenticación, gestión de usuarios, eventos, categorías y consulta de datos.

### **F.3.2. Cambios y decisiones**

Se consolidó el uso de TypeScript en todo el sistema para mantener coherencia entre frontend y backend. También se decidió utilizar servicios externos como Firebase Authentication y Cloudinary para reducir complejidad propia en autenticación y almacenamiento de imágenes.

## **F.4. Informe de avance 4: funcionalidades sociales, mapas y recomendaciones**

### **F.4.1. Trabajo realizado**

Se incorporaron funcionalidades avanzadas: mapa de eventos, geolocalización, filtrado por distancia, chat de evento, mensajería privada, módulo de amistad basado en seguimiento, valoraciones, recomendaciones personalizadas y rutas de eventos.

El sistema empezó a comportarse como una plataforma social completa, no solo como un catálogo de eventos.

## **F.4.2. Cambios y decisiones**

El desarrollo del sistema de rutas y recomendaciones requirió ajustar el alcance de algunas funcionalidades secundarias. Se priorizaron las funcionalidades centrales de descubrimiento, participación y personalización frente a extras inicialmente previstos.

## **F.5. Informe de avance 5: scraping, moderación y despliegue**

### **F.5.1. Trabajo realizado**

Se integró el scraping de eventos desde fuentes externas, incluyendo procesamiento de datos, geocodificación y control de duplicados. También se desarrolló el flujo de moderación de eventos públicos, con estados de aprobación y rechazo, y se preparó el despliegue del backend y la distribución de la aplicación móvil.

Además, se documentó el proceso mediante manuales de desarrollo, despliegue y usuario.

### **F.5.2. Cambios y decisiones**

La complejidad del scraping fue superior a la prevista debido a la variabilidad de las fuentes externas, la extracción de direcciones, el tratamiento de fechas y los límites de peticiones. Como consecuencia, se descartó la implementación final de funcionalidades opcionales como la generación de carteles con IA y la moderación automática de imágenes mediante NSFWJS.

## **F.6. Informe de avance 6: pruebas, validación y cierre**

### **F.6.1. Trabajo realizado**

En la fase final se ejecutaron pruebas unitarias, pruebas funcionales y pruebas de aceptación sobre los flujos principales de la aplicación. También se recogieron observaciones de usuarios piloto y se corrigieron incidencias detectadas durante la validación.

Se completó la memoria, se prepararon los anexos y se revisó la coherencia documental del trabajo.

### **F.6.2. Cambios y decisiones**

Las pruebas con usuarios piloto permitieron detectar incidencias específicas en Android, especialmente relacionadas con el mapa y la disposición visual de algunos elementos. Estas correcciones introdujeron una desviación de esfuerzo, pero mejoraron la estabilidad de la aplicación en un entorno multiplataforma.

### **F.7. Resumen de cambios relevantes**

- Incorporación de CI/CD con GitHub Actions para automatizar comprobaciones y despliegues.
- Ajuste del alcance por la complejidad del scraping y del sistema de rutas.
- Descarte justificado de funcionalidades opcionales no prioritarias.
- Refuerzo de pruebas en Android tras detectar incidencias en dispositivos reales.
- Ampliación de la documentación final con manuales, glosario, actas e informes de avance y cambios.

## G. Declaración de autoría

---

La declaración oficial de autoría de este Trabajo de Fin de Grado, firmada por la autora y el tutor, se entrega como documento independiente conforme al modelo proporcionado por la Escuela Técnica Superior de Ingeniería Informática de la Universidad de Sevilla.

# H. Usuarios piloto

---

## H.1. Introducción

Durante la fase final del desarrollo se llevó a cabo una validación de la aplicación con usuarios piloto reales. El objetivo fue obtener retroalimentación directa de personas que no habían participado en el desarrollo, para identificar problemas de usabilidad, ajustes visuales necesarios, nuevas ideas de funcionalidad y comportamientos inesperados de la interfaz en condiciones de uso real.

La distribución se realizó mediante **Expo Go**, que permite ejecutar la aplicación sin publicarla en las tiendas oficiales. Durante las primeras pruebas se compartió un enlace de Expo con un grupo de usuarios voluntarios de distintos perfiles y con diferentes dispositivos.

Posteriormente, Expo modificó el comportamiento de carga de proyectos en Expo Go: desde el 12 de mayo de 2026, las actualizaciones publicadas mediante EAS Update solo pueden abrirse si la cuenta autenticada es propietaria del proyecto o pertenece a la organización correspondiente [16]. Por ello, para pruebas abiertas posteriores y demostraciones con evaluadores externos, la estrategia se ajustó a Expo Go con túnel (`npx expo start -tunnel`), manteniendo el backend desplegado en Render como entorno de datos.

## H.2. Perfil de los usuarios

Participaron un total de **11 usuarios piloto**, seleccionados entre conocidos y contactos de la autora, buscando diversidad en cuanto a:

- **Dispositivo:** 7 usuarios con iPhone (iOS) y 3 usuarios con distintos modelos de Android.
- **Perfil tecnológico:** desde personas habituadas al uso de aplicaciones móviles hasta perfiles con menor experiencia digital.
- **Interés en el dominio:** personas con interés en el ocio y la cultura de Sevilla, perfil objetivo de la aplicación.

## H.3. Metodología de recogida de feedback

El feedback se recogió a través de conversaciones directas y mensajes, sin un cuestionario estructurado. Se pidió a los 11 participantes que exploraran la



aplicación libremente y que comunicaran cualquier problema, sugerencia o impresión. Se tomaron notas de los comentarios más recurrentes y se agruparon por categoría.

Para estructurar mínimamente la sesión, se indicó a cada usuario que completase los siguientes flujos principales antes de explorar libremente el resto de la aplicación:

1. Registro e inicio de sesión.
2. Consulta del listado de eventos y acceso al detalle de un evento.
3. Creación de un evento propio (público o privado).
4. Uso del mapa interactivo de eventos cercanos.
5. Envío de un mensaje en el chat de un evento.

La tabla [H.1](#) recoge el número de usuarios que completaron cada flujo satisfactoriamente.

Tabla H.1: Completitud de flujos principales por los usuarios piloto (n = 11)

| Flujo                         | Completado | %     |
|-------------------------------|------------|-------|
| Registro e inicio de sesión   | 11 / 11    | 100 % |
| Consulta de eventos y detalle | 11 / 11    | 100 % |
| Creación de un evento         | 11 / 11    | 100 % |
| Uso del mapa interactivo      | 11 / 11    | 100 % |
| Chat de evento                | 11 / 11    | 100 % |

Los 11 usuarios completaron los cinco flujos. Las incidencias detectadas durante la realización de estos flujos (principalmente en dispositivos Android) se recogen en la sección siguiente.

## H.4. Resultados e incidencias detectadas

### H.4.1. Problemas de compatibilidad con Android

La incidencia más significativa detectada durante las pruebas piloto fue la **incompatibilidad de varios elementos de la interfaz en dispositivos Android**, reportada por los 3 usuarios con dispositivo Android (100% de los usuarios Android). Esta incidencia no había sido detectada durante el desarrollo, que se realizó íntegramente sobre iOS.

Los principales problemas observados en Android fueron:

- **Mapa no visible** (3/3 usuarios Android): el componente de mapa (OpenStreetMap) no se renderizaba en la pantalla de mapa de eventos en ningún dispositivo Android, mostrando una pantalla en blanco. El problema

estaba relacionado con la configuración del proveedor de tiles en la compilación de Expo Go para Android.

- **Descuadres de pantalla** (2/3 usuarios Android): varios elementos de la interfaz (modales, cabeceras, botones flotantes y disposición de formularios) presentaban desajustes de posicionamiento en Android, causados por diferencias en el comportamiento del *safe area* y la gestión del teclado virtual respecto a iOS.

Estos problemas fueron corregidos en una iteración adicional de la fase de cierre del proyecto. Supusieron una desviación no prevista en la planificación, documentada en el capítulo 2 (sección de desviaciones) y en el capítulo 7 (lecciones aprendidas).

#### H.4.2. Sugerencias de mejora visual y de interacción

Los usuarios piloto aportaron numerosas sugerencias de mejora relacionadas con aspectos visuales y de comodidad en la interacción. Las más recurrentes fueron:

- **Ajustes en el tamaño y contraste de algunos textos:** en determinadas pantallas, el texto informativo secundario resultaba difícil de leer, especialmente en modo oscuro. Se realizaron ajustes de contraste y tamaño en los componentes afectados.
- **Mejoras en el flujo de creación de eventos:** algunos usuarios encontraron el formulario de creación de eventos extenso. Se reorganizaron los campos para que los datos más importantes (título, categoría, fecha y ubicación) aparecieran en primer lugar.
- **Navegación entre secciones:** varios usuarios indicaron que no encontraban intuitivamente algunas secciones (como “Mis eventos” o “Guardados”). Se revisó la estructura del menú lateral para mejorar la accesibilidad de estas pantallas.
- **Feedback visual en acciones:** en algunas acciones (apuntarse a un evento, guardar, enviar mensaje) no había suficiente retroalimentación visual inmediata. Se añadieron indicadores de carga y mensajes de confirmación en los casos más relevantes.

#### H.4.3. Ideas de nuevas funcionalidades

Los usuarios piloto también propusieron funcionalidades que consideraban interesantes y que no estaban disponibles en la versión distribuida. La más mencionada fue:

- **Módulo avanzado de amigos** (mencionado por 4/11 usuarios): varios usuarios valoraron positivamente el módulo de amistad existente, que permite buscar usuarios, entrar en sus perfiles, seguir o dejar de seguir, enviar mensajes privados y ver los eventos a los que asisten. Como mejora

futura, echaron en falta una gestión de contactos todavía más completa: distinguir amistades cercanas, recibir sugerencias de planes basadas en personas de confianza y poder enviar invitaciones directas a eventos. Esta funcionalidad ampliaría la dimensión social existente. Se ha incluido en la sección de trabajo futuro del capítulo de conclusiones.

## H.5. Valoración general de los usuarios

La valoración global de los 11 usuarios piloto fue muy positiva, sin quejas funcionales. Los aspectos destacados de forma unánime o mayoritaria fueron:

- **Intención de descarga** (11/11 usuarios): todos los usuarios indicaron que se descargarían la aplicación si estuviera disponible en las tiendas oficiales y que la utilizarían en su día a día para descubrir planes en Sevilla.
- **Estética e interfaz** (11/11 usuarios): la interfaz fue valorada muy positivamente de forma unánime, con mención expresa a la calidad visual, la coherencia de las pantallas, el modo oscuro y la comodidad de uso general.
- **Mapa interactivo de eventos** (mencionado por 11/11 usuarios): el mapa de eventos fue el aspecto más destacado por todos los usuarios, que lo señalaron como la funcionalidad que más les gustó y que consideran más diferenciadora respecto a otras aplicaciones similares.
- **Sistema de rutas de eventos** (mencionado por 11/11 usuarios): las rutas recomendadas y la posibilidad de crear rutas propias fueron también señaladas por todos los usuarios como una funcionalidad original que no han visto en otras aplicaciones de ocio, y que consideran especialmente útil para planificar una salida con varias actividades.
- **Módulo social como diferenciador** (mencionado por 8/11 usuarios): el chat de evento, los mensajes privados y el módulo de amistad fueron valorados como novedosos y útiles respecto a otras aplicaciones de eventos.

Ninguno de los 11 usuarios reportó quejas sobre el funcionamiento de la aplicación ni problemas durante la realización de los flujos principales. Las únicas incidencias detectadas fueron las de compatibilidad con Android descritas en la sección anterior, que afectaron a la visualización pero no impidieron completar ningún flujo.

## H.6. Conclusiones del proceso piloto

El proceso de validación con usuarios piloto fue valioso en dos dimensiones. Por un lado, permitió detectar y corregir la incidencia crítica de compatibilidad con Android antes de la entrega final. Por otro, proporcionó retroalimentación cualitativa sobre usabilidad e ideas de nuevas funcionalidades que han sido incorporadas al trabajo futuro del proyecto.

La experiencia confirma que las pruebas con usuarios reales en dispositivos reales son un complemento imprescindible de las pruebas técnicas automatizadas: los problemas de compatibilidad multiplataforma y los ajustes de usabilidad difícilmente se detectan sin pasar la aplicación por manos de personas ajenas al desarrollo.

# I. Actas de reunión

---

Las actas recogidas en este anexo reflejan el estado del proyecto y las decisiones discutidas en el momento de cada reunión. Algunas decisiones técnicas iniciales evolucionaron durante el desarrollo, por lo que la solución final válida es la descrita en los capítulos de diseño, implementación y despliegue.

## Acta de Reunión 1

### Información General

- **Fecha:** 14 de julio de 2025
- **Hora:** 11:00 a.m.
- **Duración:** 30 minutos
- **Lugar:** Escuela Técnica Superior de Ingeniería Informática

#### Participantes:

- José Antonio Parejo Maestre (Tutor)
- Cynthia Castaño Juan (Estudiante)

### Objetivo de la reunión

Presentar y proponer al tutor el tema del Trabajo de Fin de Grado (TFG) y discutir las dimensiones principales del proyecto (alcance funcional, complejidad técnica y viabilidad en el tiempo).

### Temas tratados

#### 1. Propuesta del tema del TFG

La estudiante propone como tema del TFG el desarrollo de una aplicación para la gestión y consulta de eventos culturales y de ocio, centrada inicialmente en una ciudad concreta.

El tutor valora positivamente la propuesta y confirma que encaja con los objetivos formativos del grado y con su área de tutoría.

## 2. Definición de las dimensiones del proyecto

Se discuten las dimensiones principales del TFG:

- **Dimensión funcional:** alcance inicial de funcionalidades (búsqueda y visualización de eventos, fichas de detalle, posibles opciones futuras como mapa, reseñas o recomendaciones).
- **Dimensión técnica:** tecnologías potenciales a utilizar (aplicación móvil o web, arquitectura general, posibles servicios externos).
- **Dimensión temporal:** viabilidad del proyecto dentro del calendario del TFG, estimando el esfuerzo necesario y la necesidad de acotar el alcance para garantizar su finalización.

## 3. Viabilidad y acotación inicial

Se comenta la importancia de mantener un alcance realista para asegurar que el proyecto pueda completarse con la calidad requerida dentro del plazo establecido.

Se acuerda que en reuniones posteriores se detallará mejor el backlog y la planificación, una vez formalizado el tema.

## Acuerdos y decisiones

1. El tutor acepta de forma preliminar el tema propuesto para el TFG, pendiente de la formalización administrativa correspondiente.
2. Se acuerda acotar el proyecto a un conjunto de funcionalidades básicas para garantizar su viabilidad (definición detallada en siguientes reuniones).
3. La estudiante elaborará una primera propuesta de descripción del TFG, incluyendo objetivos, alcance y dimensiones del proyecto, para compartir con el tutor.

## Tareas asignadas

### Estudiante (Cynthia Castaño Juan)

- Redactar un breve documento de propuesta del TFG (título provisional, objetivos, justificación y alcance inicial).
- Enviar la propuesta al tutor para revisión antes de la siguiente reunión.

### Tutor (José Antonio Parejo Maestre)

- Revisar la propuesta de tema y alcance del TFG.
- Orientar sobre posibles ajustes en las dimensiones (funcional, técnica y temporal) en la próxima reunión.

## Próxima reunión

- **Objetivo:** Revisar y, en su caso, validar la propuesta escrita del TFG (tema, objetivos y alcance) y definir los siguientes pasos de planificación.
- **Fecha:** Por determinar.

**Elaborado por:** Cynthia Castaño Juan

**Fecha:** 3 de enero de 2026

# Acta de Reunión 2

## Información General

- **Fecha:** 25 de septiembre de 2025
- **Hora:** 10:00 a.m.
- **Duración:** 30 minutos
- **Lugar:** Reunión online (videollamada)

### Participantes:

- José Antonio Parejo Maestre (Tutor)
- Cynthia Castaño Juan (Estudiante)

## Objetivo de la reunión

Revisar y discutir el documento de Anteproyecto de TFG “Aplicación móvil de eventos emergentes en Sevilla, ‘Sevillaneando’”, contrastando la descripción, objetivos y tecnologías iniciales, y acordar una ampliación y concreción del alcance funcional del proyecto.

## Temas tratados

### 1. Revisión del anteproyecto

Se revisa la descripción del proyecto, centrada en una aplicación móvil para promover eventos culturales y de ocio emergentes en Sevilla, incluyendo también eventos convencionales.

El tutor considera adecuada la motivación y confirma que la propuesta encaja con el ámbito de Ingeniería del Software y con la línea de proyectos de aplicaciones móviles.

### 2. Alcance funcional y funcionalidades iniciales

Se comentan los objetivos del anteproyecto, que incluyen: listado de eventos con información detallada, posibilidad de que los usuarios añadan eventos sujetos a moderación, integración de mapa, subida de imágenes en vivo, reseñas y valoraciones, recomendaciones personalizadas y notificaciones dentro de la aplicación.

A partir de estos objetivos, se acuerda estructurar el alcance en bloques funcionales (búsqueda/listado, detalle de evento, propuesta y moderación, mapa,



interacción social, recomendaciones y notificaciones), para facilitar su futura transformación en historias de usuario.

### 3. Matización del alcance

Durante la reunión se decide:

- Confirmar el uso de un sistema de moderación de contenido para fotos en vivo, basado en un modelo propio con NSFW.js/TensorFlow.js integrado en el backend, considerándolas funcionalidades avanzadas pero incluidas en el TFG.

### 4. Validación de tecnologías iniciales

Se revisan las decisiones tecnológicas recogidas en el anteproyecto: React Native para el frontend móvil, backend con Node.js y NestJS, PostgreSQL con PostGIS para datos geoespaciales, Firebase Authentication para la autenticación, Cloudinary para la gestión de imágenes y OpenStreetMap integrado mediante react-native-maps para mapas.

El tutor valida estas decisiones como técnicamente coherentes y adecuadas para el tipo de aplicación propuesta, recomendando documentar claramente las razones de elección en la memoria del TFG.

## Acuerdos y decisiones

1. El documento de Anteproyecto queda aceptado como base oficial para el desarrollo del TFG “Sevillaneando”, con la descripción, objetivos y tecnologías recogidas en él.
2. Se amplía y concreta el alcance funcional, fijando como parte del proyecto:
  - Listado y detalle de eventos con información completa.
  - Alta de eventos por usuarios con flujo de moderación.
  - Integración de mapa con búsqueda por proximidad.
  - Subida de imágenes en vivo con moderación automática.
  - Sistema de reseñas y valoraciones.
  - Módulo de recomendaciones personalizadas y notificaciones dentro de la aplicación.
3. Se acuerda utilizar este alcance como base para derivar el backlog inicial de épicas e historias de usuario.

## Tareas asignadas

### Estudiante (Cynthia Castaño Juan)

- Actualizar el anteproyecto, si es necesario, incorporando claramente el listado de funcionalidades acordadas.
- Elaborar un primer backlog de historias de usuario alineado con los objetivos y tecnologías del anteproyecto.

### Tutor (José Antonio Parejo Maestre)

- Revisar el backlog inicial y proponer ajustes de alcance o prioridades en la siguiente reunión.

## Próxima reunión

- **Objetivo:** Revisar el backlog inicial de historias de usuario, validar la coherencia entre alcance, funcionalidades y tecnologías, y esbozar una primera planificación por sprints.
- **Fecha:** Por determinar (a concretar por correo tras la entrega del backlog).

**Elaborado por:** Cynthia Castaño Juan

**Fecha:** 3 de enero de 2026

# Acta de Reunión 3

## Información General

- **Fecha:** 10 de octubre de 2025
- **Hora:** 9:58 a.m.
- **Duración:** 30 minutos 13 segundos

### Participantes:

- José Antonio Parejo Maestre (Tutor)
- Cynthia Castaño Juan (Estudiante)

## Objetivo de la reunión

Revisión del backlog del proyecto, validación del modelo conceptual de datos, definición de la planificación inicial y establecimiento de objetivos para el siguiente sprint.

## Temas tratados

### 1. Aprobación del Product Backlog

Se revisó y aprobó el backlog inicial presentado. Se acordó que el backlog es un documento vivo que se irá modificando y perfilando durante todo el proceso de desarrollo. Se considera suficiente contar con una primera versión de historias básicas bien revisadas para arrancar la fase de desarrollo.

### 2. Reorganización de la planificación

Se acordó reorganizar la estructura de fases:

- **Fase 1:** Incluye tareas 1.1, 1.2, 1.3 y la creación del modelo conceptual de datos.
- **Fase 2:** El diseño de arquitectura (tarea 1.4) pasa a la fase 2 (2.01).

Se confirmó el stack tecnológico: React Native para el frontend y backend con Node.js (Next.js).

### 3. Estimación de tareas

Se definió utilizar la técnica de sizing con tallas de camiseta (XS, S, M, L, XL) para estimar el esfuerzo de las historias de usuario y tareas relevantes. Este método utiliza medidas relativas en lugar de horas fijas.

#### 4. Revisión del modelo conceptual (UML)

Se revisó el diagrama de clases del modelo conceptual con las siguientes entidades principales:

##### **Entidad Evento:**

- Atributos: ID, título, descripción, fecha/hora inicio, fecha/hora fin, ubicación (coordenadas), precio, categorías, estado (pendiente/aprobado/rechazado), creador, imágenes, reseñas.
- Se aclaró que fecha inicio/fin son instantes en el tiempo, no solo fechas.
- Las imágenes son referencias URL almacenadas en el servidor.

##### **Entidad Mapa:**

- Se acordó añadir campos adicionales: nivel de zoom, centro (latitud/longitud) o esquinas (punto inicio/fin).
- Esto permite controlar la visualización de eventos relacionados en una zona específica.

##### **Nueva Entidad: Ruta**

- Incluye: conjunto de coordenadas, secuencia de eventos, temporización.
- Permite planificar rutas por múltiples eventos en un día o periodo concreto.

##### **Otras entidades:**

- Categorías, Imágenes, Reseñas.
- Usuario con tipo de usuario (incluyendo Moderador).

##### **Entidad Recomendación:**

- Un conjunto de eventos recomendados automáticamente a un usuario específico.
- Se acordó añadir un flag para indicar si el usuario ya ha visto la recomendación.
- Requiere definir un algoritmo de generación basado en criterio de similaridad (inicialmente por ubicación).

##### **Entidad Notificación:**

- Debe relacionarse con recomendaciones y eventos guardados.
- Requiere mecanismos de configuración para usuarios.
- Debe incluir información del evento específico.

#### 5. Metodología de desarrollo

##### **Control de versiones:**

- Git con política de ramas: Feature Branch (3 niveles: main/develop/feature).
- Mensajes de commit siguiendo Conventional Commits.

#### **Gestión del proyecto:**

- Se utilizará una adaptación de Scrum para TFG:
  - No se realizan daily standups.
  - Las reuniones de review y planning se fusionan en una sola sesión.
  - Se mantiene estructura de sprints con objetivos definidos.

Se acordó completar la sección de metodología en el documento correspondiente.

### **Decisiones tomadas**

1. El backlog inicial queda aprobado y se actualizará durante el desarrollo.
2. Se incorpora la entidad “Ruta” al modelo conceptual.
3. Se añaden campos de visualización (zoom, centro) a la entidad “Mapa”.
4. Se utilizará sizing con tallas de camiseta para estimación.

### **Tareas asignadas**

#### **Estudiante (Cynthia):**

- Desarrollar Producto Mínimo Viable (MVP) con sistema de login/logout y visualización de mapa en pantalla.
- Actualizar el modelo conceptual incorporando las modificaciones discutidas.
- Completar la sección de metodología en el documento de gestión.
- Crear repositorio del proyecto con la política de ramas acordada.
- Aplicar estimaciones con sizing a las historias y tareas.
- Conceder acceso al tutor del sistema de seguimiento de horas del proyecto.

#### **Tutor (José Antonio):**

- Realizar adjudicación oficial del TFG antes de mayo de 2026.

### **Próxima reunión**

Objetivos para el siguiente sprint:

1. Presentar MVP funcional con login/logout y visualización de mapa.

2. Mostrar modelo conceptual actualizado.
3. Backlog actualizado con historias de notificaciones y recomendaciones.
4. Documento de metodología completado.
5. Repositorio creado y configurado.

**Fecha:** Por determinar

**Elaborado por:** Cynthia Castaño Juan

**Fecha:** 2 de enero de 2026

# Acta de Reunión 4

## Información General

- **Fecha:** 9 de enero de 2026
- **Hora:** 13:25
- **Duración:** 32 minutos 41 segundos

### Participantes:

- José Antonio Parejo Maestre (Tutor)
- Cynthia Castaño Juan (Estudiante)

## Objetivo de la reunión

Revisión del estado del proyecto, validación del modelo conceptual, definición de prioridades del backlog y planificación del siguiente sprint.

## Temas tratados

### 1. Revisión del modelo conceptual

Se revisó el modelo conceptual actualizado, especialmente las entidades relacionadas con eventos, imágenes, usuarios y reseñas. Se aclaró que:

- Una imagen pertenece a un único usuario (el que la sube), aunque pueda estar asociada a múltiples eventos.
- Un evento puede tener múltiples imágenes.
- Las relaciones entre eventos, reseñas y usuarios quedan correctamente definidas.
- No es necesario vincular rutas directamente a eventos, ya que la recomendación por proximidad se obtiene mediante la estructura geográfica.

Se confirmó que el sistema utilizará coordenadas geográficas (latitud y longitud) para representar ubicaciones.

### 2. Sistema de mapas

Se acordó utilizar OpenStreetMap en lugar de Google Maps debido a sus menores restricciones y mayor facilidad de integración.

Se almacenará la ubicación mediante un GeoPoint (latitud, longitud) para permitir cálculos de distancia y filtrado geográfico.

Esta decisión debe documentarse como una decisión de diseño.

### 3. Estado actual del proyecto

Se confirmó que actualmente están implementadas las siguientes funcionalidades:

- Registro y gestión de usuarios.
- Tres tipos de usuario: Usuario, Moderador y Administrador.
- Edición de perfil, cambio de contraseña y eliminación de cuenta.
- Creación de eventos por usuarios.
- Moderación de eventos por moderadores.
- Visualización de eventos en mapa.
- Gestión de categorías (por organizadores).
- Notificaciones cuando un evento es aprobado o rechazado.

Se indicó que la edición de eventos por moderadores aún está pendiente.

### 4. Backlog y priorización

Se acordó sustituir la priorización Alta / Media / Baja por el modelo MoSCoW (Must, Should, Could, Won't), ya que proporciona mayor claridad sobre qué funcionalidades son esenciales y cuáles son opcionales.

### 5. Metodología y documentación

Se confirmó que ya están definidos: estimación por tallas de camiseta, políticas de ramas, políticas de versionado, política de commits y planificación tipo Gantt.

Se detectó como pendiente añadir una guía de estilo de codificación, incluyendo reglas de formato, convenciones y configuración del entorno para autoformateo.

### 6. Funcionalidades opcionales propuestas

**Generación automática de carteles:** uso de una API de IA para generar carteles a partir de la descripción de los eventos, que se asociarán automáticamente al evento.

**Gestión avanzada de imágenes:**

- Posibilidad de subir varias imágenes por evento.
- Límite de almacenamiento por usuario (ej. 15 MB).



- Proceso batch que calcule el uso y bloquee subidas al superar el límite.

**Sistema de chat:** implementación de chat entre usuarios mediante WebSockets, con una primera fase en tiempo real y una segunda con persistencia en base de datos.

## 7. Búsqueda y visualización de eventos

Se acordó:

- En la vista de mapa, mostrar eventos cercanos usando filtros por distancia geográfica.
- En la vista de búsqueda, permitir búsquedas más precisas por nombre, categoría o fecha.

## 8. Planificación del proyecto

Se acordó que el siguiente sprint tendrá una duración de 4 semanas.

Las prioridades serán: mejora de la búsqueda y visualización geográfica, gestión de imágenes de eventos, edición de eventos por moderadores, gestión de categorías e inicio del sistema de chat.

Las funcionalidades de recomendaciones y scraping se dejan para sprints posteriores.

## Decisiones tomadas

- Se adopta OpenStreetMap como sistema de mapas.
- Se utilizará GeoPoint (latitud/longitud) para ubicaciones.
- Se cambia la priorización del backlog a MoSCoW.
- Se incluyen como opcionales la generación de carteles, el chat y la gestión avanzada de imágenes.
- El sprint tendrá una duración de 4 semanas.

## Tareas asignadas

**Estudiante (Cynthia):**

- Actualizar el backlog usando MoSCoW.
- Añadir la guía de estilo de codificación a la metodología.
- Implementar mejoras en búsqueda y visualización de eventos.
- Implementar edición de eventos por moderadores.

- Avanzar en la gestión de imágenes.
- Iniciar el sistema de chat.

**Tutor (José Antonio):**

- Revisar los avances en la siguiente reunión.
- Validar las decisiones de diseño y priorización.

## **Próxima reunión**

Objetivos para el siguiente sprint:

- Sistema de eventos con búsqueda geográfica funcional.
- Gestión de imágenes por evento.
- Edición de eventos por moderadores.
- Backlog priorizado con MoSCoW.
- Metodología actualizada con guía de estilo.
- Restricciones en el backend.

**Fecha:** Por determinar

**Elaborado por:** Cynthia Castaño Juan

**Fecha:** 9 de enero de 2026

# Acta de Reunión 5

## Información General

- **Fecha:** 17 de febrero de 2026
- **Hora:** 08:14
- **Duración:** 31 minutos 43 segundos

### Participantes:

- José Antonio Parejo Maestre (Tutor)
- Cynthia Castaño Juan (Estudiante)

## Objetivo de la reunión

Revisión del progreso del sprint anterior (mejoras de búsqueda, gestión de imágenes, edición por moderadores y chat), validación de la metodología de documentación y planificación del nuevo sprint enfocado en scraping.

## Temas tratados

### 1. Estado actual del proyecto y sprint finalizado

Se validó la implementación de las funcionalidades acordadas en el sprint de cuatro semanas:

- **Gestión de Categorías:** Finalizada. Solo los administradores pueden crearlas, aunque los usuarios pueden asociarlas a sus eventos.
- **Edición de Eventos por Moderadores:** Finalizada. Los moderadores pueden editar campos y aceptar o rechazar eventos.
- **Gestión de Imágenes:** Implementada la subida de una imagen de portada. Se discutió la migración a una tabla adicional para permitir múltiples imágenes por evento.
- **Búsqueda y Visualización Geográfica:** Implementado el mapa con filtros de proximidad codificados por colores (verde, amarillo, rojo) y enlace directo a aplicaciones externas de mapas, además de filtrados fuera del mapa interactivo por proximidad.

### 2. Sistema de chat

Se presentó el sistema de chat en vivo utilizando WebSockets.

- **Optimización de recursos:** El tutor advirtió sobre el consumo de memoria si los sockets permanecen abiertos indefinidamente.
- **Decisión técnica:** Se implementará una tarea programada (tipo cron) en el servidor para cerrar conexiones de eventos finalizados o chats inactivos tras un tiempo configurable.

### 3. Metodología y documentación

- **Guía de estilo:** Se confirmó la creación de una guía de estilo en formato Markdown y la inclusión de scripts de linting (ESLint/Prettier) para automatizar el cumplimiento de reglas en el backend y frontend.

### 4. Planificación del nuevo sprint

Se definieron las prioridades para las próximas semanas:

- **Scraping:** Extracción de datos de eventos de sitios clave (Turismo, Universidad, etc.).
- **Gestión de precios:** Se acordó añadir campos opcionales de “precio máximo” y “precio mínimo” para manejar eventos con costes variables o gratuitos.
- **Gestión de fotos:** Permitir múltiples imágenes por evento.
- **Gestión de chats:** Cerrar los sockets cuando se sale del chat y cuando se acaba el evento o tras un cierto tiempo.

### Decisiones tomadas

- Se utilizará una tabla adicional en la base de datos para la colección de imágenes de eventos.
- Se implementará el cierre automático de websockets por inactividad o fin de evento para proteger el servidor.
- La memoria oficial se redactará en  $\text{\LaTeX}$  utilizando la plantilla de la Escuela.
- Se incorpora el uso de IA (Gemini) para facilitar el proceso de scraping y estructuración de datos.

### Tareas asignadas

**Estudiante (Cynthia):**

- Migrar la lógica de imágenes a una colección/tabla dedicada.
- Implementar la lógica de cierre de chats en el backend.
- Implementar intervalo de precio.

- Preparar el primer borrador (índice y metodología) en Overleaf e invitar al tutor.
- Desarrollar los scrapers para fuentes de eventos institucionales.
- Actualizar el backlog y el diagrama UML.

**Tutor (José Antonio):**

- Revisar los cambios y la estructura de la memoria en la siguiente sesión.

## **Próxima reunión**

- **Objetivos:** Revisión de los datos extraídos mediante scraping, validación del primer borrador de la memoria.
- **Fecha:** Por determinar.

**Elaborado por:** Cynthia Castaño Juan

**Fecha:** 17 de febrero de 2026

# Acta de Reunión 6

## Información General

- **Fecha:** 27 de marzo de 2026
- **Hora:** 16:30
- **Duración:** 32 minutos

### Participantes:

- José Antonio Parejo Maestre (Tutor)
- Cynthia Castaño Juan (Estudiante)

## Objetivo de la reunión

Revisión del estado actual del proyecto tras el sprint en curso, definición del último sprint de desarrollo centrado en rutas de eventos y orientación sobre la fase de redacción de la memoria.

## Temas tratados

### 1. Estado actual del proyecto

Se revisaron las funcionalidades desarrolladas durante el sprint actual y el cómputo total de horas invertidas:

- Horas actuales: 160 horas.
- Se confirma que queda un único sprint de desarrollo.

### 2. Funcionalidades revisadas

Se repasaron funcionalidades implementadas o en progreso:

- Edición de eventos (públicos y privados).
- Calendario.
- Buscador.
- Notificaciones de eventos y recordatorios.
- Gestión de imágenes (posibilidad de múltiples fotos).
- Precio variable en eventos.
- Gestión de cierre de sockets.

Se discutieron mejoras opcionales: configuración del intervalo de notificaciones y mejora en gestión de imágenes (cropping).

### **3. Planificación del último sprint**

Se define como objetivo principal la implementación de rutas de eventos, incluyendo seguimiento de rutas dentro de la aplicación.

Como funcionalidades opcionales: mejoras en notificaciones y ajustes en funcionalidades existentes.

Se acuerda priorizar el objetivo principal y ajustar el alcance si el tiempo es limitado.

### **4. Organización del tiempo y entrega**

- Se recomienda realizar un sprint aproximado de 3 semanas, ajustado por la feria (21–26 de abril).
- Próxima reunión propuesta: 17 de abril de 2026.

### **5. Memoria y documentación**

Se enfatiza la importancia de comenzar la redacción de la memoria. Se identifican los tres manuales del TFG: manual de usuario, manual de desarrollo y manual de despliegue.

### **6. Testing y validación**

- El testing de aceptación se realizará en paralelo con la redacción de la memoria.
- Se utilizarán estas pruebas para documentar la validación del sistema.

### **7. Dudas técnicas**

**Eventos pasados:** Se propone un toggle para mostrar/ocultar eventos pasados, o bien separarlos en una categoría/pestaña distinta.

#### **Despliegue de la aplicación:**

- En iOS requiere cuenta de pago → se documentará en la memoria.
- Para pruebas: uso de emuladores Android y posibilidad de usar alternativas más ligeras que Android Studio.

## Decisiones tomadas

- Priorizar la implementación de rutas de eventos en el último sprint.
- Iniciar la redacción intensiva de la memoria.
- Realizar el testing junto con la documentación.
- Mantener flexibilidad en funcionalidades opcionales según el tiempo disponible.

## Tareas asignadas

### Estudiante (Cynthia):

- Implementar el sistema de rutas de eventos.
- Avanzar en la redacción de la memoria.
- Elaborar los manuales (usuario, desarrollo y despliegue).
- Realizar pruebas de validación del sistema.

### Tutor (José Antonio):

- Revisar el progreso del sprint y la memoria en la próxima reunión.
- Proporcionar orientación sobre herramientas (emuladores, despliegue).

## Próxima reunión

- **Fecha:** 17 de abril de 2026.
- **Objetivos:** Revisión del sistema de rutas, evaluación del estado de la memoria y decisión sobre cierre de desarrollo o continuación.

**Elaborado por:** Cynthia Castaño Juan

**Fecha:** 27 de marzo de 2026



# Acta de Reunión 7

## Información General

- **Fecha:** 17 de abril de 2026

- **Hora:** 09:31

- **Duración:** 42 minutos

### **Participantes:**

- José Antonio Parejo Maestre (Tutor)

- Cynthia Castaño Juan (Estudiante)

## Objetivo de la reunión

Revisión del estado de la memoria del TFG, análisis de los manuales (usuario, desarrollo y despliegue), validación de diagramas de arquitectura y definición de tareas para el siguiente sprint centrado en pruebas, refactorización y cierre de la documentación.

## Temas tratados

### 1. Estado actual del proyecto

Se revisa el progreso general del proyecto:

- Horas estimadas actuales: entre 220 y 230 horas.
- Se confirma que el proyecto entra en fase final, con aproximadamente un mes hasta la entrega.

### 2. Revisión de manuales

Se analizan los tres manuales del sistema (usuario, desarrollo y despliegue).

#### **Observaciones principales:**

- El contenido es correcto y completo, pero demasiado esquemático.
- Se recomienda transformar el contenido en texto narrativo (más descriptivo y orientado a usuario).
- El manual de usuario debe centrarse en las necesidades del usuario y no en la estructura de la interfaz.
- Es necesario ampliar la introducción para dar más contexto.

**Mejoras propuestas:**

- Añadir descripciones detalladas de pantallas, campos y validaciones.
- Incluir un grafo de navegabilidad de la aplicación.
- Incorporar una sección de errores frecuentes.

**3. Estructura de la memoria**

- Se recomienda estructurar los manuales como capítulos independientes dentro de la memoria.
- Se valida el uso de la EDT como base de planificación y se recomienda mantenerla simple (análisis, desarrollo y cierre).
- El presupuesto puede calcularse sobre estimación (300 horas) o sobre horas reales.

**4. Diagramas de arquitectura**

Se revisan los diagramas realizados (vista de datos, vista de componentes/despliegue y diagramas de relaciones).

**Observaciones:**

- El contenido es correcto, pero algunos diagramas no siguen la notación estándar UML.
- Se recomienda usar diagramas UML formales (clases, componentes, despliegue), mantener coherencia en la representación y mejorar la claridad visual.

**5. Desarrollo y entrega**

- Fecha de entrega: 20 de mayo de 2026.
- Se debe entregar: memoria y código fuente.
- El despliegue puede realizarse posteriormente de cara a la presentación.
- Es necesario tener una versión funcional completa del sistema.

**6. Testing y validación**

Se establece como prioridad alta definir un plan de pruebas completo que incluya: historias de usuario a validar, datos de entrada, resultados esperados y resultados obtenidos. Se recomienda realizar capturas durante las pruebas para usarlas como evidencias en la memoria.

## 7. Próximo sprint

Se define el objetivo del siguiente sprint (3 semanas):

- Finalizar la refactorización del sistema.
- Diseñar y ejecutar el plan de pruebas.
- Completar la sección de validación.
- Mejorar y cerrar las secciones de la memoria.

## Decisiones tomadas

- Priorizar la realización del plan de pruebas y validación del sistema.
- Reescribir los manuales en formato narrativo y no esquemático.
- Mejorar los diagramas siguiendo estándares UML.
- Considerar el desarrollo funcional prácticamente finalizado.
- Enfocar el trabajo restante en documentación y validación.

## Tareas asignadas

### Estudiante (Cynthia):

- Finalizar la refactorización del sistema.
- Elaborar el plan de pruebas.
- Ejecutar las pruebas y documentar resultados.
- Reescribir los manuales (usuario, desarrollo y despliegue).
- Mejorar diagramas de arquitectura.
- Completar y pulir la memoria.

### Tutor (José Antonio):

- Revisar avances de la memoria.
- Validar el plan de pruebas.
- Proporcionar feedback sobre la documentación.

## Próxima reunión

- **Fecha:** 5 de mayo de 2026.
- **Objetivos:** Revisión del plan de pruebas y resultados, validación de la refactorización y evaluación del estado final de la memoria.

**Elaborado por:** Cynthia Castaño Juan

**Fecha:** 17 de abril de 2026

# Acta de Reunión 8

## Información General

- **Fecha:** 5 de mayo de 2026
- **Hora:** 10:31
- **Duración:** 35 minutos

### Participantes:

- José Antonio Parejo Maestre (Tutor)
- Cynthia Castaño Juan (Estudiante)

## Objetivo de la reunión

Revisión detallada del borrador completo de la memoria del TFG, identificación de mejoras estructurales y de contenido, validación de aspectos metodológicos y definición de ajustes finales antes de la entrega.

## Temas tratados

### 1. Estado actual del proyecto

- La memoria está completada en una primera versión y se encuentra en fase de revisión y mejora.
- Es necesario refinar contenido, especialmente a nivel de redacción y estructura.

### 2. Revisión del resumen e introducción

#### Observaciones:

- El resumen está demasiado centrado en el producto; debe incluir también información del proyecto (contexto académico, metodología, tecnologías, etc.).
- Se recomienda mantener una extensión breve (aprox. 3/4 de página) e incluir versión en inglés (abstract).
- La introducción puede repetir parcialmente el resumen, pero debe reforzar la motivación con datos económicos del dominio y añadir contexto que justifique la relevancia del proyecto.

### 3. Análisis y requisitos

#### Observaciones:

- Se solicita añadir un apartado específico de actores del sistema (tipos de usuarios y diagrama de actores).
- Se deben añadir mockups como parte del análisis.
- Los requisitos funcionales deben reestructurarse en formato estándar: “Como [usuario], quiero [funcionalidad], para [objetivo]”.

### 4. Objetivos del proyecto

Se detecta mezcla de objetivos del sistema y objetivos del proyecto (aprendizaje, gestión, competencias). Se recomienda separarlos claramente y usarlos posteriormente en conclusiones para justificar resultados.

### 5. Metodología

La sección actual mezcla distintos niveles. Se recomienda dividir en subsecciones: metodología de gestión del proyecto, metodología de desarrollo (sprints) y metodología técnica (gestión de código, commits, repositorio, etc.).

### 6. Planificación y fases

- Añadir una fase 0 (definición del alcance).
- Renombrar última fase a “Despliegue y cierre del proyecto”.
- Incluir diagramas (Gantt u otros) y aportar datos cuantitativos (horas totales).

### 7. Presupuesto

El coste actual es demasiado bajo y debe recalcularse utilizando datos oficiales, incluyendo: coste base por hora, costes sociales (~30 %) y costes indirectos (infraestructura). El presupuesto final debe incluir margen de contingencia (10–15 %) y beneficio industrial (~20 %).

### 8. Diseño del sistema

- Se solicita añadir un diagrama conceptual independiente del tecnológico.
- Se debe crear un diagrama conceptual (sin detalles técnicos) junto al diagrama de diseño ya existente.
- Añadir sección de patrones de diseño y justificación de tecnologías utilizadas.

## 9. Diagramas y modelado

Se recomienda incluir un diagrama de clases conceptual y diagramas UML claros, manteniendo coherencia en la representación.

## 10. Despliegue y uso real

- Backend desplegado; frontend desplegado sin publicación en stores (no es obligatorio publicar).
- Se recomienda añadir datos de uso, análisis de accesos y automatizaciones (cron jobs, scraping, etc.).

## 11. Anexos

Se identifican elementos a completar antes de la entrega: actas de reuniones, reporte de horas, resultados de pruebas y uso por usuarios piloto. Se recomienda incluir feedback de usuarios y evidencias del sistema en uso.

## 12. Entrega final

Elementos necesarios para la entrega:

- Código fuente (organizado).
- Aplicación funcional.
- Vídeo demostrativo (~3 minutos).
- Documentación completa.

## Decisiones tomadas

- Reestructurar la memoria para mejorar claridad y separación de conceptos.
- Incluir actores, mockups y diagramas conceptuales.
- Recalcular completamente el presupuesto.
- Añadir métricas, datos reales y anexos pendientes de completar en ese momento.
- Reforzar la introducción con contexto económico.

## Tareas asignadas

**Estudiante (Cynthia):**

- Revisar y mejorar el resumen e introducción.
- Separar objetivos del sistema y del proyecto.
- Añadir actores, mockups y diagramas conceptuales.
- Reestructurar la metodología.
- Recalcular el presupuesto.
- Completar anexos (actas, reporte de horas, pruebas y feedback de usuarios).
- Preparar vídeo demostrativo.
- Finalizar documentación y presentación.

**Tutor (José Antonio):**

- Revisar el documento completo.
- Proporcionar feedback adicional.
- Validar mejoras realizadas.

## **Próxima reunión**

- **Fecha estimada:** en torno al 13 de mayo de 2026.
- **Objetivos:** Revisión final de la memoria, validación de correcciones y preparación de la entrega definitiva.

**Elaborado por:** Cynthia Castaño Juan

**Fecha:** 6 de mayo de 2026